快速使用手册



ELFBOARD

ELF 2(RK3588)开发板快速使用手册

ARM Cortex-A55 ARM Cortex-A76

Embedded Learning Fans Board

Instruction Manual

Rev. 1.0

2025/11/09

Forlinx Embedded Technology Co. Ltd. www.elfboard.com

前言

本手册以使用户快速熟悉产品，了解接口功能和测试方法为目的，主要讲述了开发板接口功能的测试，烧写镜像的方法，以及使用过程中出现的一些问题如何排查等。在测试过程中，对一些命令进行了注释，方便用户理解，以实用够用为主。

本手册一共分为六部分：

- 第一部分产品的整体概述，简单介绍了开发板资源、内核源码相关驱动路径；
- 第二部分产品的快速开机启动，可采用串口登录和网络登录两种方式；
- 第三部分产品的功能介绍，包括不同系统下命令行功能测试及界面功能测试；
- 第四部分产品的系统烧录介绍，包括整体系统镜像烧录和单独更新部分系统镜像；
- 第五部分产品开发所用环境的介绍，包括开发环境的搭建方法，以及搭建好环境的使用方法。
- 第六部分产品开中所涉及到的源码编译方法介绍，包括系统源码编译和提供的应用源码编译方法。

本手册中一些符号及格式的相关说明：

表现形式	含义
⚠	注意或者是需要特别关注的信息，一定要仔细阅读
🕮	对测试章节做的相关说明
🗁	表示相关路径
灰底蓝色	指在命令行输入的命令，需要手动输入
灰底黑色字体	输入命令后的串口输出信息
灰底黑色加粗	串口输出信息中的关键信息
//	对输入指令或输出信息的解释内容
用户名@主机名	root@elf2-buildroot: 开发板串口登录账户信息 elf@elf2-buildroot：开发板远程登录账户信息 elf@ubuntu:~$：开发环境 Linux 账户信息 用户可通过该信息确定功能操作的环境

例：插入 TF 卡后，通过 ls 指令查看挂载目录。

```
root@elf2-buildroot:~# ls /mnt                    //列出/mnt 目录下的文件
external_sd  sdcard  storage  udisk  usb_storage
```

- root@elf2-buildroot：用户名为 root，主机名为 elf2-buildroot，表示在开发板上使用 root 用户进行操作。
- // ：对指令操作或打印信息的解释内容，不需要输入。
- ls /mnt：灰底蓝色字体，表示需要手动输入的相关命令。
- external_sd **sdcard** storage udisk usb_storage：灰底黑色字体为输入命令后的输出信息，加粗字体为关键信息，在此处表示 TF 卡的挂载目录。

更新记录

日期	手册版本	更新内容
2024.11.20	V1.0	初版
2026.03.09	V1.1	1.Desktop 系统增加 WiFi、蓝牙测试章节 2.NPU 驱动升级为 0.9.8 3.调整分区，交换 userdata 和 rootfs 分区

目录

前言 - 2 -
更新记录 - 3 -
目录 - 4 -
第一章 ELF 2 开发板初体验 - 9 -
1.1. 资源介绍 - 9 -
1.2. CPU/GPU/NPU 频率说明 - 10 -
1.3. Linux 5.10.209 系统软件资源介绍 - 11 -
1.4. eMMC 存储器分区表 - 11 -
第二章 快速开机启动 - 12 -
2.1. 开机前的准备 - 12 -
2.2. 串口登录方式 - 12 -
2.2.1. 调试串口驱动安装 - 12 -
2.2.2. 串口连接设置 - 12 -
2.2.3. 串口登录常见问题 - 16 -
2.3. 网络登录方式 - 16 -
2.3.1. 网络连接测试 - 16 -
2.3.2. SSH 服务器 - 17 -
2.3.3. SFTP - 18 -
2.4. 屏幕显示 - 20 -
2.5. 系统关闭 - 20 -
第三章 Buildroot 文件系统-板卡接口功能测试 - 22 -
3.1. 主板命令行功能测试 - 22 -
3.1.1. TF 卡测试 - 22 -
3.1.2. USB 接口测试 - 23 -
3.1.2.1. U 盘测试 - 23 -
3.1.2.2. USB 鼠标测试 - 25 -
3.1.2.3. USB 转网络测试 - 25 -
3.1.3. 有线网络测试 - 28 -
3.1.3.1. IPV4 基本命令测试 - 28 -
3.1.3.2. IPV6 测试 - 31 -
3.1.3.3. 以太网配置 - 32 -
3.1.4. WiFi 测试 - 33 -
3.1.4.1. STA 模式 - 33 -
3.1.4.2. AP 模式 - 34 -
3.1.5. 蓝牙测试 - 35 -
3.1.6. 看门狗测试 - 39 -
3.1.7. RTC 时钟驱动测试 - 39 -
3.1.8. 放/录音测试 - 40 -
3.1.8.1. HDMI 播放声音 - 40 -
3.1.8.2. 耳机播放声音 - 41 -
3.1.8.3. SPKOUT 播放声音 - 43 -
3.1.8.4. MIC 输入 - 44 -

3.1.9. LCD 屏测试 - 45 -
3.1.10. SQLite3 测试 - 46 -
3.1.11. 调频测试 - 47 -
3.1.12. 温度测试 - 48 -
3.1.13. GPIO 脚本测试 - 48 -
3.1.14. GPIO 命令行测试 - 48 -
3.1.15. PWM 测试 - 49 -
3.1.16. UART 测试 - 50 -
3.1.17. ADC 测试 - 54 -
3.1.18. SPI 测试 - 54 -
3.1.19. TYPE-C 测试 - 55 -
3.1.20. PCIE 测试 - 57 -
3.1.21. RKNPU 测试 - 58 -
3.1.22. DDR 测试 - 59 -
3.1.23. eMMC 测试 - 63 -
3.2. Qt 5.15.10 桌面功能测试 - 63 -
3.2.1. 界面功能简介 - 63 -
3.2.2. 硬件解码体验 - 64 -
3.2.3. OpenGL 测试 - 65 -
3.2.4. 播放音乐测试 - 65 -
3.2.5. WiFi 测试 - 66 -
3.2.6. 网络配置测试 - 68 -
3.2.7. Ping 测试 - 69 -
3.2.8. 浏览器测试 - 70 -
3.2.9. 看门狗测试 - 71 -
3.2.10. 按键测试 - 72 -
3.2.11. RTC 测试 - 73 -
3.2.12. 同显异显 - 74 -
3.2.13. UART 测试 - 74 -
3.2.14. SPI 测试 - 79 -
3.2.15. 数据库测试 - 80 -
3.2.16. 背光测试 - 81 -
3.2.17. Web 服务 - 81 -
3.2.18. CPU 频率配置测试 - 82 -
3.3. 多媒体测试 - 83 -
3.3.1. 音频和视频播放体验 - 84 -
3.3.1.1. 使用 gst-play 播放器播放视频和音频 - 84 -
3.3.1.2. 使用 gst-launch 播放视频 - 85 -
3.3.1.3. 使用 gst-launch 播放音频 - 87 -
3.3.1.4. 使用 gst-launch 播放视频和音频 - 88 -
3.3.2. 视频硬编码 - 90 -
3.3.2.1. 视频硬编码 H.264 - 90 -
3.3.2.2. 视频硬编码 H.265 - 91 -
3.3.2.3. JPEG 硬编码 - 92 -

3.3.3. 视频硬解码 - 94 -
3.3.3.1. 解码并播放 H264 格式视频 - 94 -
3.3.3.2. 解码并播放 H264 格式视频带音频 - 95 -
3.3.3.3. 解码并播放 H265 格式视频 - 96 -
3.3.3.4. 解码并播放 H265 格式视频带音频 - 97 -
3.3.3.5. 解码并播放 VP9 格式视频 - 98 -
3.3.3.6. 解码并播放 VP9 格式视频带音频 - 99 -
3.3.3.7. 解码并播放 VP8 格式视频 - 100 -
3.3.3.8. 解码并播放 VP8 格式视频带音频 - 101 -
3.3.4. 摄像头测试 - 102 -
3.3.4.1. UVC Camera 测试 - 103 -
3.3.4.2. OV13855 测试 - 108 -
第四章 Desktop 文件系统-板卡接口功能测试 - 115 -
4.1. 主板命令行功能测试 - 115 -
4.1.1. TF 卡测试 - 115 -
4.1.2. USB 接口测试 - 116 -
4.1.2.1. U 盘测试 - 116 -
4.1.2.2. USB 鼠标测试 - 117 -
4.1.2.3. USB 转网络测试 - 117 -
4.1.3. 有线网络测试 - 119 -
4.1.3.1. IPV4 基本命令测试 - 119 -
4.1.3.2. IPV6 测试 - 121 -
4.1.3.3. 以太网配置 - 123 -
4.1.4. WiFi 测试 - 123 -
4.1.4.1. STA 模式 - 124 -
4.1.4.2. AP 模式 - 125 -
4.1.5. 蓝牙测试 - 126 -
4.1.6. 看门狗测试 - 132 -
4.1.7. RTC 时钟驱动测试 - 132 -
4.1.8. 放/录音测试 - 133 -
4.1.8.1. HDMI 播放声音 - 133 -
4.1.8.2. 耳机播放声音 - 134 -
4.1.8.3. SPKOUT 播放声音 - 136 -
4.1.8.4. MIC 输入 - 138 -
4.1.9. LCD 屏测试 - 139 -
4.1.10. 调频测试 - 139 -
4.1.11. 温度测试 - 140 -
4.1.12. GPIO 命令行测试 - 140 -
4.1.13. PWM 测试 - 141 -
4.1.14. UART 测试 - 142 -
4.1.15. ADC 测试 - 146 -
4.1.16. SPI 测试 - 146 -
4.1.17. TYPE-C 测试 - 147 -
4.1.18. PCIE 测试 - 148 -

4.1.19. eMMC 测试 - 149 -
4.2. 多媒体测试 - 150 -
4.2.1. 音频和视频播放体验 - 150 -
4.2.1.1. 用 gst-play 播放器播放视频和音频 - 150 -
4.2.1.2. 使用 gst-launch 播放视频 - 151 -
4.2.1.3. 使用 gst-launch 播放音频 - 153 -
4.2.1.4. 使用 gst-launch 播放视频和音频 - 154 -
4.2.2. 视频硬编码 - 155 -
4.2.2.1. 视频硬编码 H.264 - 155 -
4.2.2.2. 视频硬编码 H.265 - 157 -
4.2.2.3. JPEG 硬编码 - 158 -
4.2.3. 视频硬解码 - 160 -
4.2.3.1. 解码并播放 H264 格式视频 - 160 -
4.2.3.2. 解码并播放 H264 格式视频带音频 - 161 -
4.2.3.3. 解码并播放 H265 格式视频 - 162 -
4.2.3.4. 解码并播放 H265 格式视频带音频 - 163 -
4.2.3.5. 解码并播放 VP9 格式视频 - 164 -
4.2.3.6. 解码并播放 VP9 格式视频带音频 - 165 -
4.2.3.7. 解码并播放 VP8 格式视频 - 166 -
4.2.3.8. 解码并播放 VP8 格式视频带音频 - 167 -
4.2.4. 摄像头测试 - 169 -
4.2.4.1. UVC Camera 测试 - 169 -
4.2.4.2. OV13855 测试 - 174 -
第五章 烧写系统 - 181 -
5.1. 烧写所需镜像 - 181 -
5.2. OTG 烧写系统 - 181 -
5.2.1. OTG 驱动安装 - 181 -
5.2.2. OTG 烧写测试 - 182 -
5.2.2.1. RKDevTool 烧写测试 - 182 -
5.2.2.2. FactoryTool 烧写测试 - 184 -
5.3. TF 卡烧写系统 - 186 -
5.4. 单独更新镜像 - 190 -
5.4.1. 单独更新内核 - 190 -
5.4.2. 单独更新 uboot - 192 -
5.4.3. 单独更新文件系统 - 192 -
5.4.3.1. 更新 buildroot 系统 - 192 -
5.4.3.2. 更新 Desktop 系统 - 192 -
5.4.4. 修改 buildroot 文件系统的方法 - 192 -
5.4.5. 修改 Desktop 文件系统的方法 - 193 -
5.4.6. 单独更新 logo - 193 -
第六章 开发环境搭建 - 2 -
6.1. VMware 虚拟机软件安装 - 2 -
6.1.1. VMware 软件的下载与购买 - 2 -
6.1.2. VMware 软件的安装 - 3 -

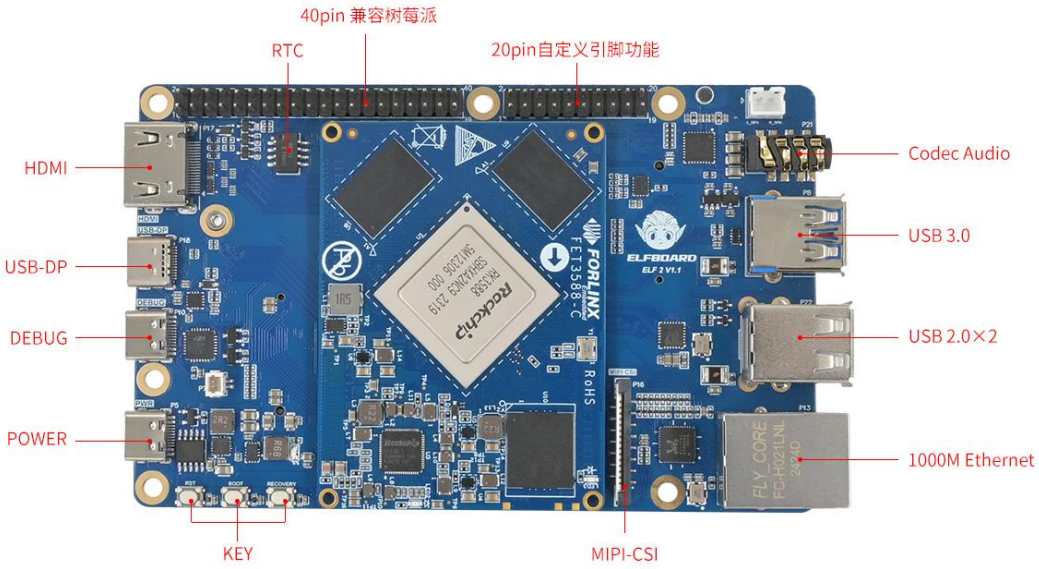
6.2. 加载已有 ubuntu 开发环境 - 7 -
6.3. 搭建新的 ubuntu 开发环境 - 10 -
6.3.1. ubuntu 系统搭建 - 10 -
6.3.1.1. 创建 Ubuntu 虚拟机 - 10 -
6.3.1.2. 系统安装 - 25 -
6.3.1.3. VMware Tools 安装 - 39 -
6.3.1.4. Ubuntu 网络设置 - 46 -
6.3.2. 必要库安装 - 52 -
6.3.3. 交叉编译链安装 - 52 -
6.3.4. Qt Creator 安装及配置 - 53 -
6.3.4.1. Qt Creator 安装 - 53 -
6.3.4.2. Qt Creator 环境配置 - 63 -
第七章 相关代码编译 - 66 -
7.1. 编译前准备 - 66 -
7.1.1. 环境说明 - 66 -
7.1.2. 拷贝源码 - 66 -
7.2. 源码编译 - 66 -
7.2.1. SDK 源码全编译 - 66 -
7.2.2. SDK 源码单独编译 - 68 -
7.2.2.1. 单独编译内核 - 68 -
7.2.2.2. 单独编译 uboot - 68 -
7.2.2.3. 单独编译 buildroot - 69 -
7.2.2.4. 打包镜像文件 - 70 -
7.2.3. 编译并运行命令行应用 - 70 -
7.2.3.1. Buildroot 系统编译并运行命令行应用 - 70 -
7.2.3.2. Desktop 系统编译并运行命令行应用 - 70 -
7.2.4. 编译并运行 QT 应用程序 - 71 -
7.2.4.1. Buildroot 系统编译并运行 QT 应用 - 71 -
7.2.4.2. Desktop 系统编译并运行 QT 应用 - 74 -
声明 - 76 -
更多帮助 - 77 -

第一章 ELF 2 开发板初体验

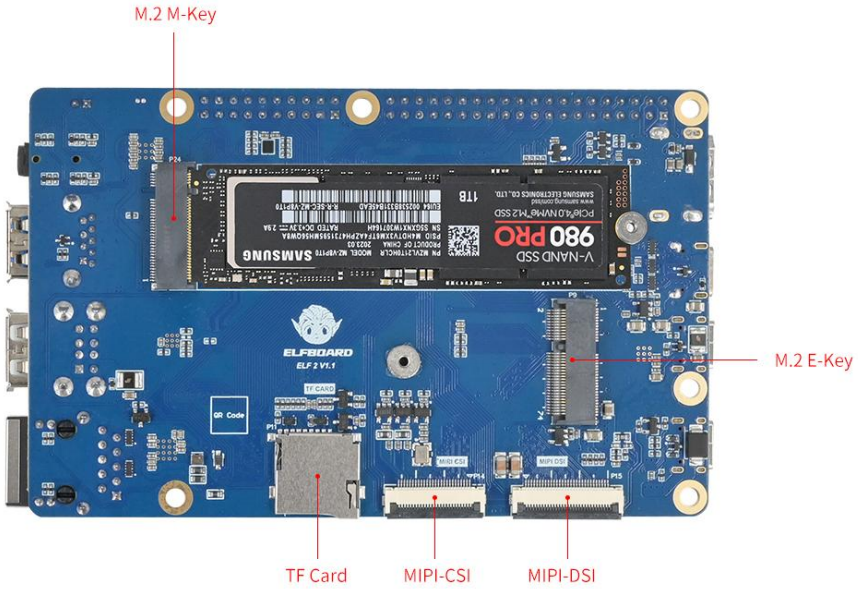
1.1. 资源介绍

ELF 2开发板卡核心板和底板采用接插件的连接方式，它是基于ARM64架构的低功耗高性能处理器，包括4核Cortex-A55和4核Conrtex-A76以及独立的NEON协处理器和神经网络加上处理器NPU，可应用于计算机、手机、个人移动互联网，数字多媒体设备。

1.正面：



2.背面：



📖 **请阅读：**

本软件手册中不再对硬件参数进行叙述，在参考本手册进行软件开发前请阅读 ELF 2 开发板资料包\01-教程文档\进阶篇之-ELF 2 开发板硬件设计教程，以了解产品的硬件配置信息。
，这样有助于您对本产品的使用。

1.2. CPU/GPU/NPU 频率说明

依据《Rockchip RK3588 Datasheet V1.7-20231117.pdf》手册 Table 3-2 Recommended operating conditions。

最大 CPU A76 频率	2.2-2.4 GHz
最大 CPU A55 频率	1.8 GHz
最大 GPU 频率	1 GHz
最大 NPU 频率	1 GHz

1.3. Linux 5.10.209 系统软件资源介绍

设备	驱动程序源代码在内核中的位置	设备名
LCD 背光驱动	drivers/video/backlight/pwm_bl.c	/sys/class/backlight
USB 接口 U 盘	drivers/usb/storage/	
USB 鼠标	drivers/hid/usbhid/	/dev/input/eventX
USB 转网卡	drivers/net/usb/r8152.c	ethX
以太网	drivers/net/ethernet/stmicro/stmmac	
SD/micro TF 卡驱动	drivers/mmc/host/dw_mmc-rockchip.c	/dev/mmcblk1pX
EMMC 驱动	drivers/mmc/host/dw_mmc-rockchip.c	/dev/mmcblk0pX
OV13855	drivers/media/i2c/ov13855.c	/dev/videoX
LCD 控制器	drivers/gpu/drm/rockchip/rockchip_drm_vop.c	
MIPI CSI	drivers/phy/rockchip/phy-rockchip-mipi-rx.c	
MIPI DSI	drivers/phy/rockchip/phy-rockchip-inno-mipi-dphy.c	
LCD 触摸驱动	drivers/input/touchscreen/gt9xx/* drivers/input/touchscreen/edt-ft5x06.c	/dev/input/eventX
RTC 实时时钟驱动	drivers/rtc/rtc-rx8010.c drivers/rtc/rtc-pcf8563.c	/dev/rtc0
串口	drivers/tty/serial/8250/8250_dw.c	/dev/ttySX
按键驱动	drivers/input/keyboard/adc-keys.c	/dev/input/eventX
LED	drivers/leds/leds-gpio.c	
I2S	sound/soc/rockchip/rockchip_i2s.c	
音频驱动	sound/soc/codecs/nau8822.c	/dev/snd/
PMIC	drivers/mfd/rk806.c drivers/regulator/rk860x-regulator.c	
PCIE	drivers/pci/controller/pcie-rockchip.c	
看门狗	drivers/watchdog/dw_wdt.c	
SPI	drivers/spi/spi-rockchip.c	
PWM	drivers/video/backlight/pwm_bl.c	
WiFi	drivers/net/wireless/intel/iwlwifi/	wlP4p65s0
蓝牙	drivers/bluetooth/	hci0

1.4. eMMC 存储器分区表

下面表格是Linux操作系统的eMMC存储器分区信息（计算时一个sector大小为512 Bytes）。

分区索引	名称	分区起始地址（sector）	大小（sector）	内容
N/A	security	0x00000000	0x00004000	MiniLoaderAll.bin
1	uboot	0x00004000	0x00002000	uboot.img
2	misc	0x00006000	0x00002000	misc.img
3	boot	0x00008000	0x00020000	boot.img
4	recovery	0x00028000	0x00040000	recovery.img
5	rootfs	0x00078000	0x01c00000	rootfs.img
6	oem	0x01c78000	0x00040000	oem.img
7	userdata	0x01cb8000		userdata.img

第二章 快速开机启动

ELF 2 出厂默认烧录的是 buildroot 文件系统镜像，以该镜像为例进行说明。

2.1. 开机前的准备

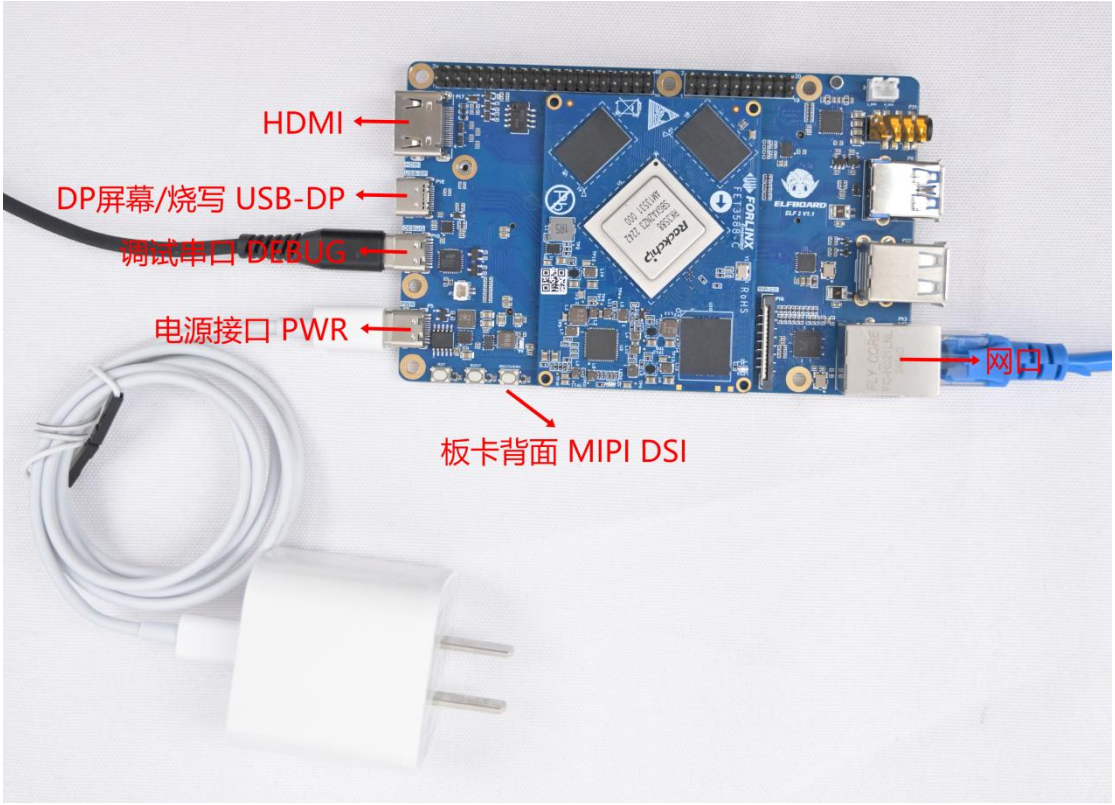
ELF 2 开发板有串口登录和网络登录两种系统登录方式，系统开机前硬件准备：

- 12V 3A DC 电源线。
- 调试串口线（串口登录使用）。

开发板上的调试串口为 Type-C USB 插孔，用户可以使用 USB 转 Type-C 线连接开发板和 PC 机，以便查看开发板状态。

- 网线（网络登录使用）。

屏幕，根据开发板接口连接屏幕（不需要显示的可以不接）。



2.2. 串口登录方式

2.2.1. 调试串口驱动安装

ELF 2 板卡调试串口使用的是 Type-C 接口，板载 USB 转 UART 芯片，无需客户购买 USB 转串口调试工具，使用极其简单方便。

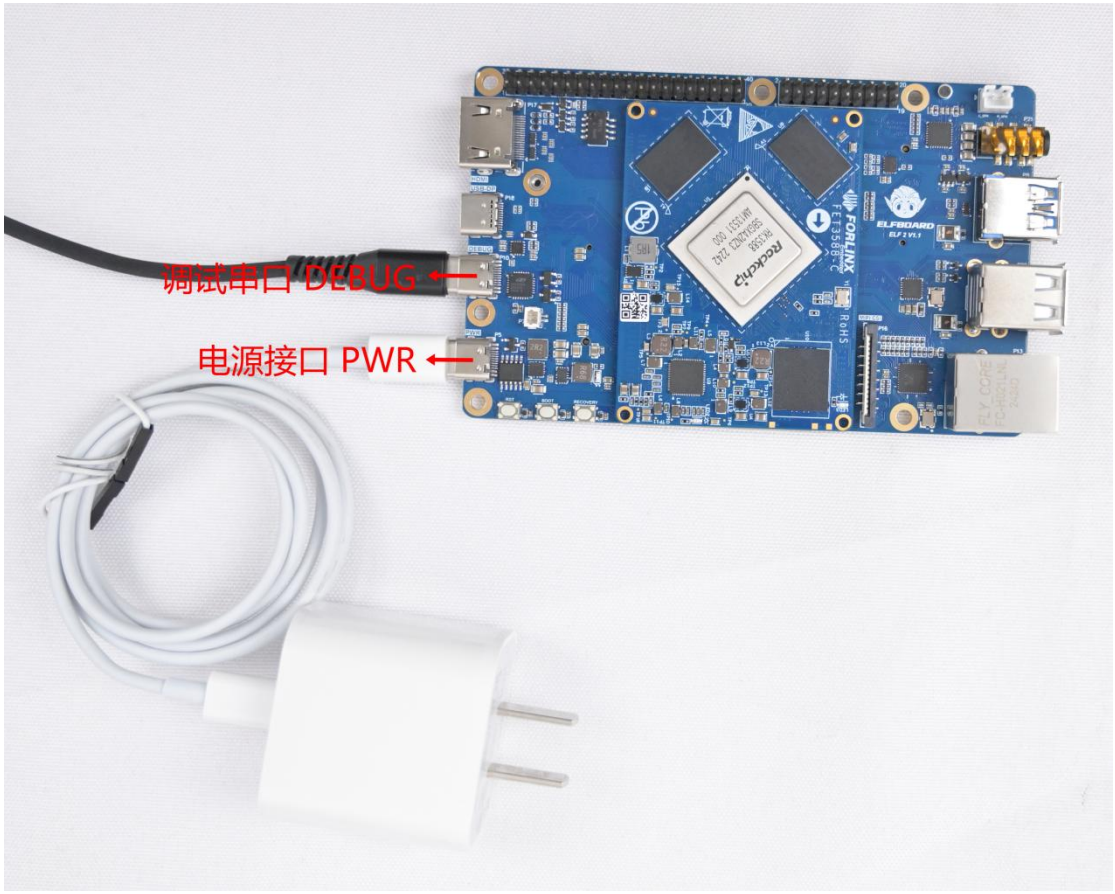
安装驱动请使用 ELF 2 开发板资料包\05-常用工具\00-串口调试工具\CP210x_VCP_Windows_XP_Vista.zip 进行安装。

解压完成后选择与自己计算机对应架构的应用程序进行安装。

2.2.2. 串口连接设置

使用 Type-C 线连接 ELF 2 板卡的 DEBUG 接口与 PC 机的 USB 接口，电源适配器连接 ELF 2 板卡

的 PWR 接口，硬件连接如图：



📖 说明：

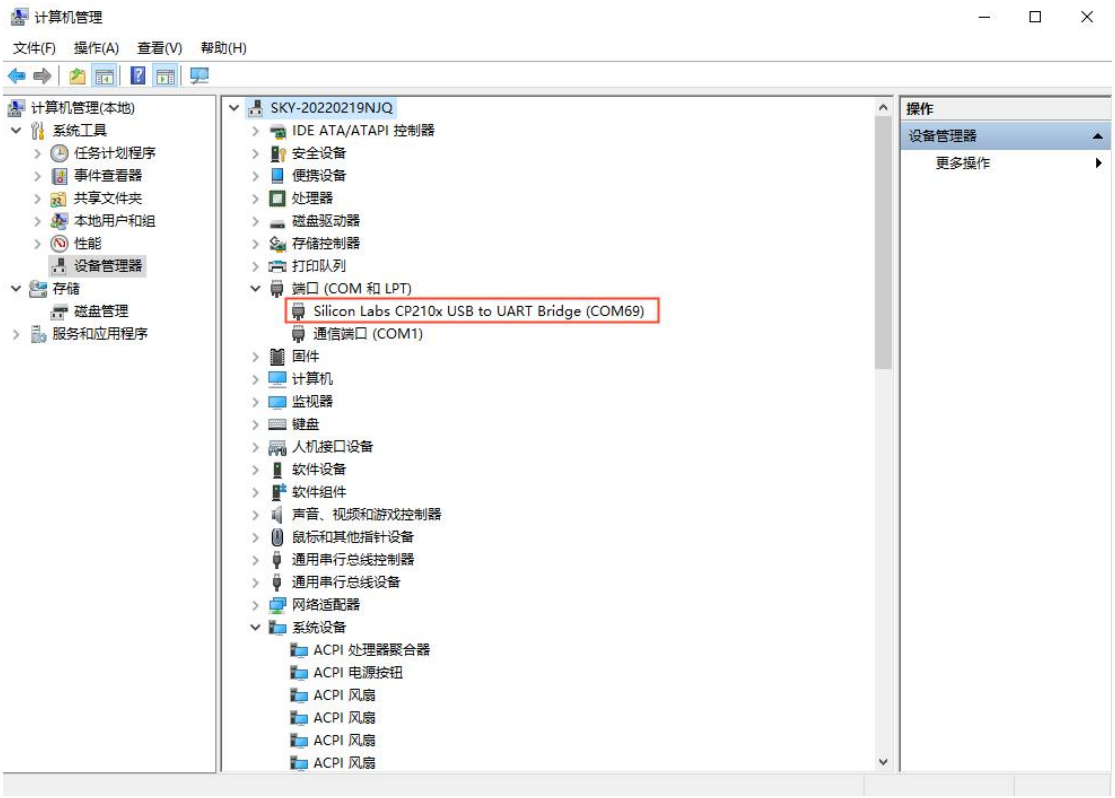
- 串口设置：波特率 115200、数据位 8、停止位 1、无校验、无流控。
- 串口终端登录 elf 用户，密码 elf。root 用户，密码 root。
- 软件需求：PC 端 Windows 系统需要安装超级终端软件，超级终端软件有多种，可自行使用自己熟悉的串口终端软件。

以下我们以 putty 终端软件为例介绍串口的登录方式。

1.打开串口工具，设置串口工具参数。

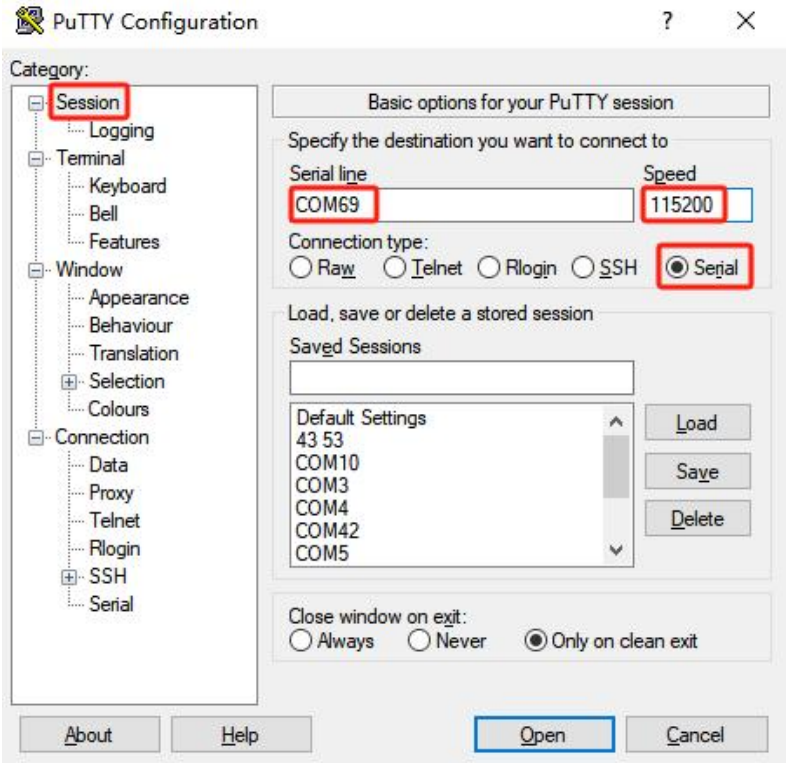
1）右击桌面“计算机--->管理--->设备管理器”。

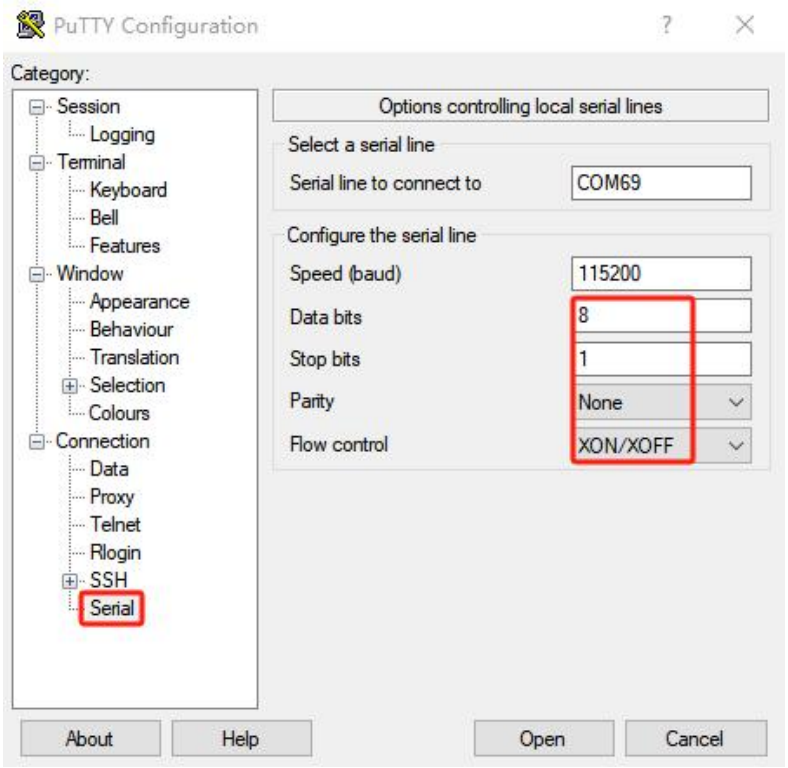
2）单击端口查看串口信息，我的串口被识别为 COM69，不同的 PC 机或者使用不同的 USB 接口，实际识别到的串口信息也会不同，请根据实际显示串口信息进行调整。



⚠**注意**：putty 安装包所在路径：ELF 2 开发板资料包\05-常用工具\00-串口调试工具/putty-64bit-0.71-installer.msi，也可以使用其他串口工具。

2.打开 putty，点击左侧 Session，选择 Serial，Serial line 根据识别到的 COM 口号设置，Speed 为 115200。再点击左侧 Serial，Data bits 为 8，Stop bits 为 1，Parity 选择 None，Flow control 选择 XON/XOFF。最后点击 Open 打开串口。





3.开发板上电，串口会有打印信息输出。

串口终端登录 elf 用户，密码 elf。root 用户，密码 root。

```
Welcome to ELF 2 Board
elf2-buildroot login: elf
Password:
elf@elf2-buildroot:~$
```

4.查看内核版本信息。

```
root@elf2-buildroot:~# cat /proc/version
Linux version 5.10.209 (elf@ubuntu) (aarch64-none-linux-gnu-gcc (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29)) 10.3.1 20210621, GNU ld (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29)) 2.36.1.20210621) #1 SMP Sun Sep 29 16:42:41 CST 2024
```

查看内核和 cpu 信息：

```
root@elf2-buildroot:~# uname -a
Linux elf2-buildroot 5.10.209 #12 SMP Tue Sep 24 15:28:26 CST 2024 aarch64 GNU/Linux
```

查看操作系统信息：

```
root@elf2-buildroot:~# cat /etc/issue
Welcome to ELF 2 Board
```

查看环境变量信息：

```
root@elf2-buildroot:~# env
SHELL=/bin/bash
GST_V4L2_PREFERRED_FOURCC=NV12:YU12:NV16:YUY2
GST_VIDEO_CONVERT_PREFERRED_FORMAT=NV12:NV16:I420:YUY2
CHROMIUM_FLAGS=--enable-wayland-ime
```

```
GST_V4L2_USE_LIBV4L2=1
GST_INSPECT_NO_COLORS=1
PULSE_HOME=/userdata/.pulse
EDITOR=/bin/vi
GST_DEBUG_NO_COLOR=1
PWD=/root
LOGNAME=root
HOME=/root
LANG=en_US.UTF-8
WESTON_DRM_PRIMARY=HDMI-A-1
ADB_TCP_PORT=5555
WAYLANDSINK_FORCE_DMABUF=1
GST_V4L2SRC_DEFAULT_DEVICE=/dev/video-camera0
QT_QPA_PLATFORM=wayland
USB_FW_VERSION=0x0310
TERM=vt100
USER=root
AUTOAUDIOSINK_PREFERRED=pulsesink
ADBD_SHELL=/bin/bash
GST_V4L2SRC_RK_DEVICES=_mainpath:_selfpath:_bypass:_scale
WESTON_DRM_MIRROR=1
SHLVL=1
USB_FUNCS=adb
WESTON_DISABLE_ATOMIC=1
USB_MANUFACTURER=Rockchip
USB_PRODUCT=rk3xxx
XDG_RUNTIME_DIR=/var/run
USB_VENDOR_ID=0x2207
PLAYBIN2_PREFERRED_AUDIOSINK=pulsesink
PATH=/usr/bin:/usr/sbin
GST_V4L2SRC_MAX_RESOLUTION=3840x2160
GST_VIDEO_DECODER_QOS=0
_=/usr/bin/env
```

2.2.3. 串口登录常见问题

电脑端口没有串口的可以通过 USB 转串口线与开发板连接，使用 USB 转串口线接需要安装对应的驱动程序。

建议使用质量好的串口线以避免出现乱码情况。

2.3. 网络登录方式

2.3.1. 网络连接测试

🕮 说明：

- 出厂时 buildroot 文件系统网卡默认配置为静态 IP，IP 地址为 192.168.0.232，更改静态 IP 的方

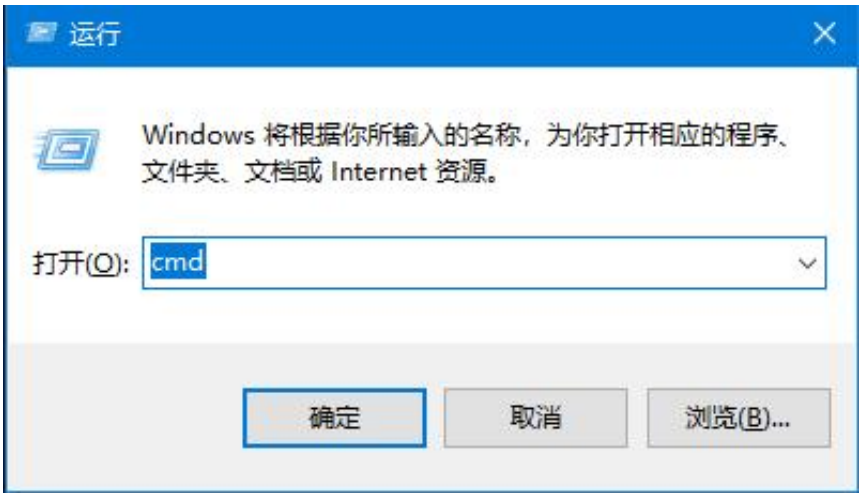
法可参考“以太网配置”测试章节。desktop 文件系统网卡默认配置为通过 DHCP 自动获取 IP 地址，配置静态 IP 的方法可参考“以太网配置”。

- 测试时电脑和开发板需要在同一网段。

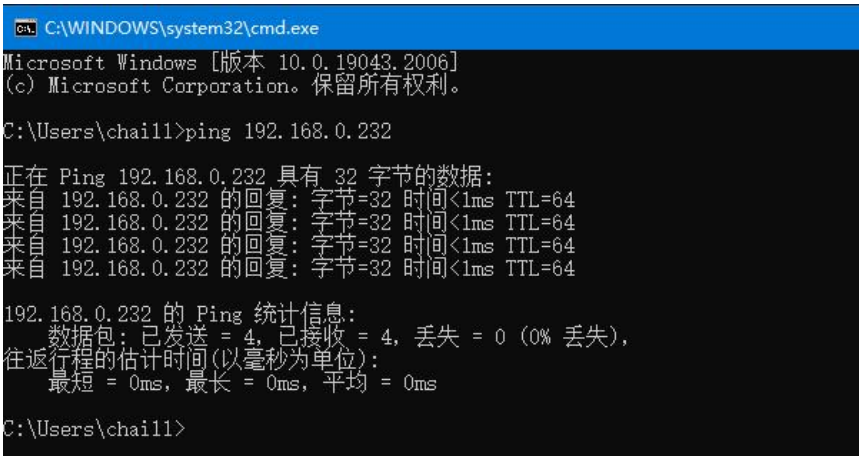
在进行网络登录前，需要先确保电脑和开发板直连的网络连接正常，可通过 ping 指令测试电脑和开发板的连接状态。具体方法操作如下：

1.将开发板的 eth0 和电脑通过网线连接，开发板上电，内核启动后核心板上会有蓝色心跳灯闪烁，与电脑连接的网卡在正常启动后快速闪烁，此时可以测试网络连接。

2.**关闭电脑防火墙（此处不作关闭防火墙的介绍，电脑通用操作）**，Win+R 打开电脑的运行命令，使用 cmd 打开电脑管理员界面。



3、使用 ping 指令测试电脑和 ELF 2 的网络连接状态。



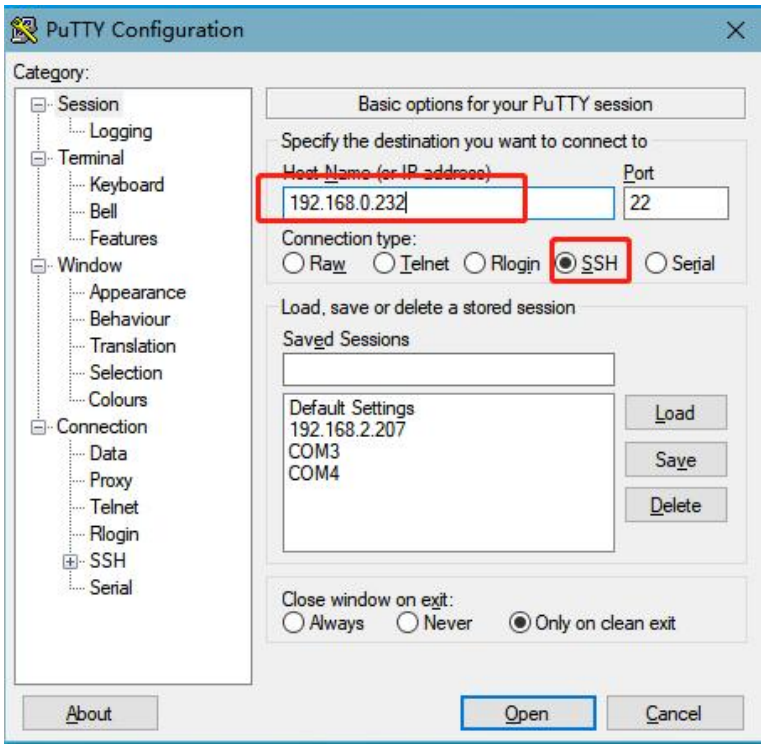
有数据返回，网络连接正常。

2.3.2. SSH 服务器

🕮 说明：

- 出厂时网卡默认配置为静态 IP，IP 地址为 192.168.0.232，更改静态 IP 的方法可参考“以太网配置”测试章节。
- 登录为 elf 用户，密码 elf。root 用户，密码 root。

1.选择 SSH 登录，输入开发板 IP，默认端口号 22，点击 Open 打开。



2.出现如下对话框，点击“是”进入登录界面。



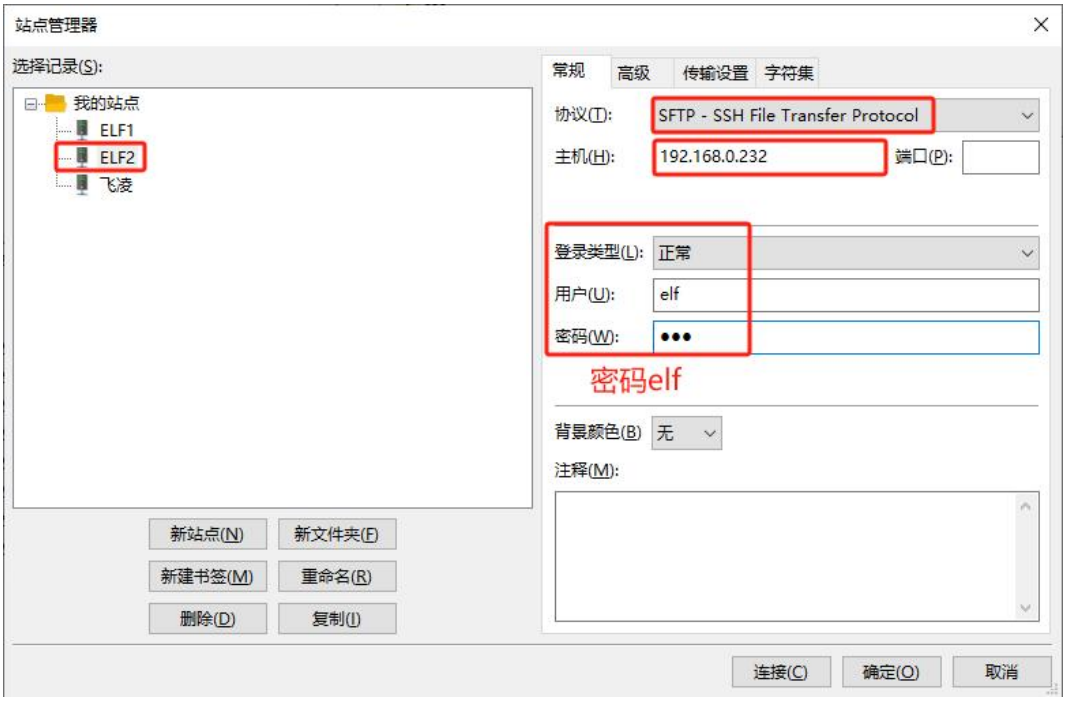

```
Login as：elf
elf@192.168.0.232's password:                    //按照提示输入开发板 elf 账户的密码 elf
elf@elf2-buildroot:~$
```

2.3.3. SFTP

ELF 2 开发板支持 SFTP 服务并且启动时已自动开启，设置好 IP 地址后就可以作为一台 SFTP 服务器。下面介绍如何利用 FTP 工具进行文件传输。

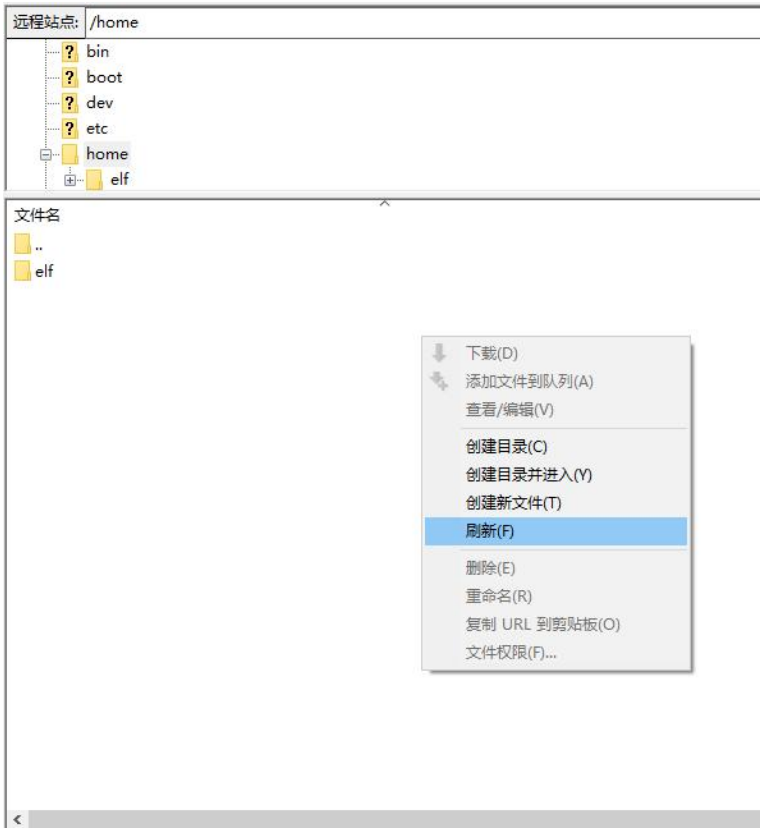
在 Windows 上安装好 FileZilla 工具，用户名和密码均为 elf。

打开 FileZilla 工具，点击文件->站点管理器，按下图配置，点击连接。



登录成功后便可以进行上传下载操作。

⚠**注意：**对开发板文件系统进行操作后，SFTP 可能会出现不同步的现象，右键刷新即可解决。

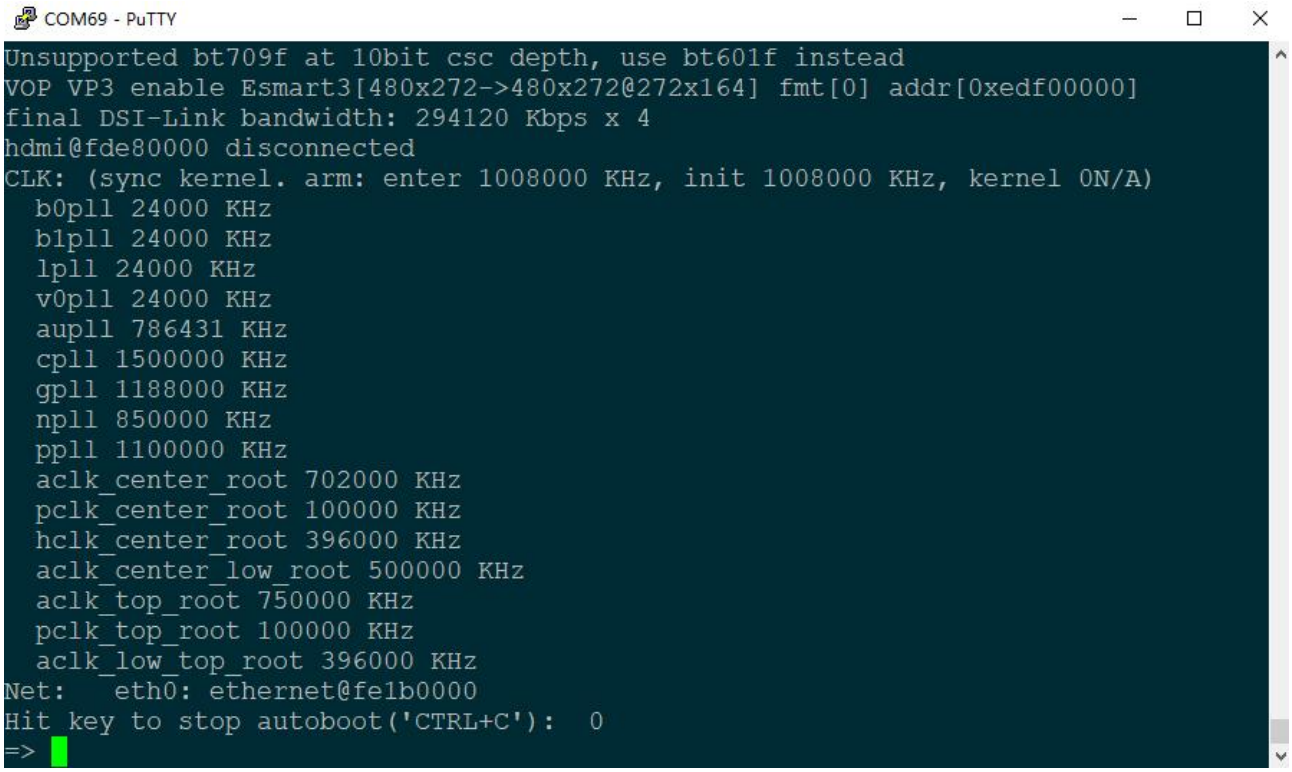


2.4. 屏幕显示

ELF2 板卡默认支持 HDMI、DP 和 MIPI 屏幕（1024x600），用户只需插入屏幕即可实现显示功能。

板卡的显示模式分为同显和异显。用户可以在 U-Boot 阶段通过设置 disp_type 属性值来选择显示模式。默认情况下，板卡配置为同显，disp_type 属性值为 sync。要配置为异显，只需将 disp_type 属性值更改为 async，具体方法如下：

1.打开串口调试终端，ELF 2 上电后按 CTRL+C 进入 uboot 命令行：



2. 板卡默认配置为同显，查看 disp_type 的属性值。

🕮 说明：

在同显模式下，如果同时插入三个屏幕，将遵循优先级顺序：HDMI > MIPI > DP。这意味着如果三种屏幕都插入，则三个屏幕将采用 HDMI 的分辨率。如果仅插入 MIPI 和 DP 屏幕，则两个屏幕都会使用 MIPI 的分辨率。

```
Hit key to stop autoboot('CTRL+C'):  0
=> print disp_type
disp_type=sync
```

3.要配置为异显，只需将 disp_type 属性值更改为 async。

```
Hit key to stop autoboot('CTRL+C'):  0
=> print disp_type
disp_type=sync
=> setenv disp_type async
=> saveenv
Saving Environment to ENV_BLK...
Writing to mmc(0)... done
=>
```

2.5. 系统关闭

一般情况下直接关闭电源即可，如果有数据存储、功能使用等操作，操作过程中不要随意断电，以防

出现文件不可逆损坏，只能重新烧写固件。未确保数据完全写入，可输入 sync 命令完成数据同步后再关闭电源。

⚠**注意：**用户依据核心板设计的产品，若在使用中出现意外掉电导致系统异常关闭的现象，可在设计中加入掉电保护等措施。

第三章 Buildroot 文件系统-板卡接口功能测试

本章测试基于 Buildroot 文件系统开展，该系统为板卡出厂默认配置。

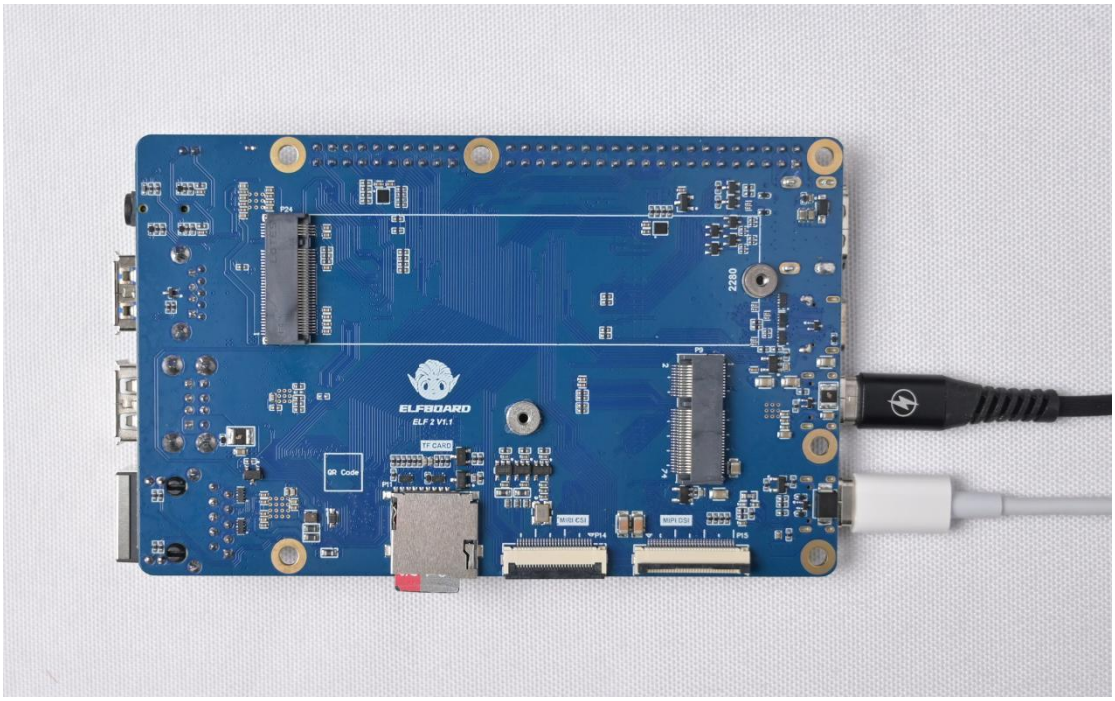
镜像文件路径：ELF 2 开发板资料包\03-镜像\00-eMMC 镜像\buildroot 镜像。

3.1. 主板命令行功能测试

ELF 2 内置了丰富的命令行工具可供用户使用。

3.1.1. TF 卡测试

TF 卡与 ELF 2 硬件连接如图：

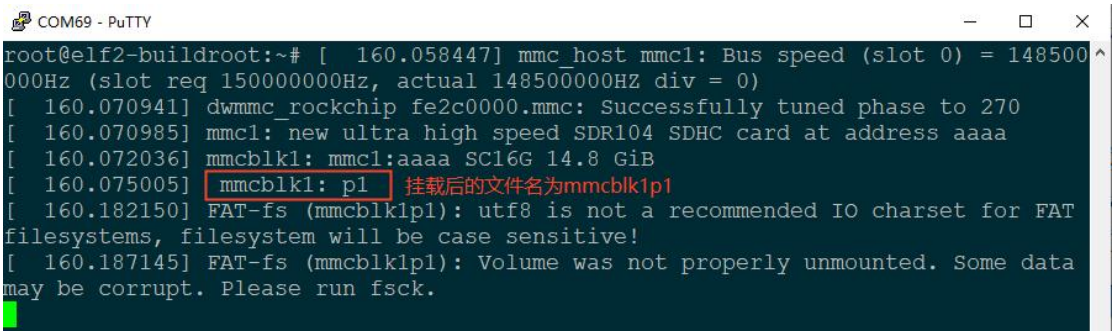


📖 说明：

- TF 卡挂载目录为/mnt/sdcard，支持热插拔，终端会打印关于 TF 卡的信息。
- 不同的 TF 卡显示信息可能会有差别，我们测试采用的是闪迪 16G 的 TF 卡。
- 使用 TF 卡测试前需将其格式化成 FAT32 格式。

TF 卡插入 ELF 2 的 TF 卡槽后，系统会自动检查并挂载 TF 卡，挂载成功后，可对 TF 卡进行读写操作。

1.插上 16G TF 卡，正常挂载后从打印信息可以看到 TF 卡挂载后的设备名，打印信息如下：



2.查看挂载目录：

```
root@elf2-buildroot:~# mount | grep "mmcblk1p1"
```

```
/dev/mmcblk1p1 on /mnt/sdcard type vfat
(rw,nodev,noexec,noatime,nodiratime,fmask=0022,dmask=0022,codepage=936,iochar
set=utf8,shortname=mixed,errors=remount-ro)
```

3.写入测试：

```
root@elf2-buildroot:~# dd if=/dev/zero of=/mnt/sdcard/test bs=1M count=500
conv=fsync
500+0 records in
500+0 records out
524288000 bytes (524 MB, 500 MiB) copied, 16.7985 s, 31.2 MB/s
```

4.读取测试：

⚠**注意：**为确保数据准确，请重启开发板后测试读取速度。

```
root@elf2-buildroot:~# dd if=/mnt/sdcard/test of=/dev/null bs=1M
500+0 records in
500+0 records out
524288000 bytes (524 MB, 500 MiB) copied, 9.77849 s, 53.6 MB/s
```

5.往 TF 卡中写入文件并查看，命令如下：

```
root@elf2-buildroot:~# echo 1 > /mnt/sdcard/test.txt      //写 1 到 test.txt 文件中
root@elf2-buildroot:~# sync                               //文件同步
root@elf2-buildroot:~# cat /mnt/sdcard/test.txt           //读取 TF 卡中 test.txt 文件
1                                                         //会读到刚才写入的 1
```

6.TF 卡使用完成后，在弹出 TF 卡前，需要使用 umount 卸载 TF 卡：

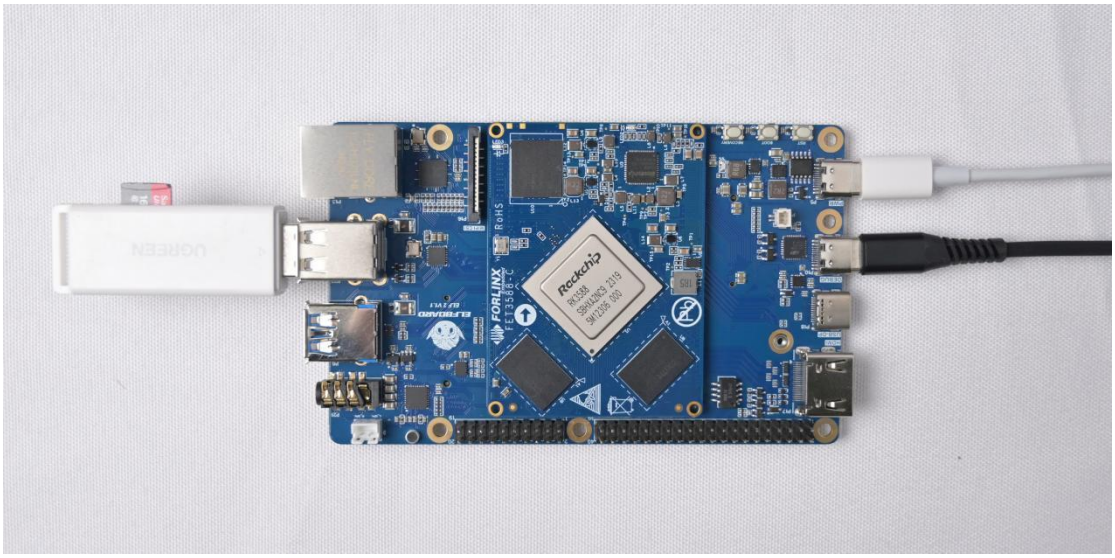
```
root@elf2-buildroot:~# umount /mnt/sdcard
```

⚠**注意：**退出 TF 卡挂载路径后再插拔 TF 卡。

3.1.2. USB 接口测试

3.1.2.1. U 盘测试

U 盘与 ELF 2 硬件连接如图：



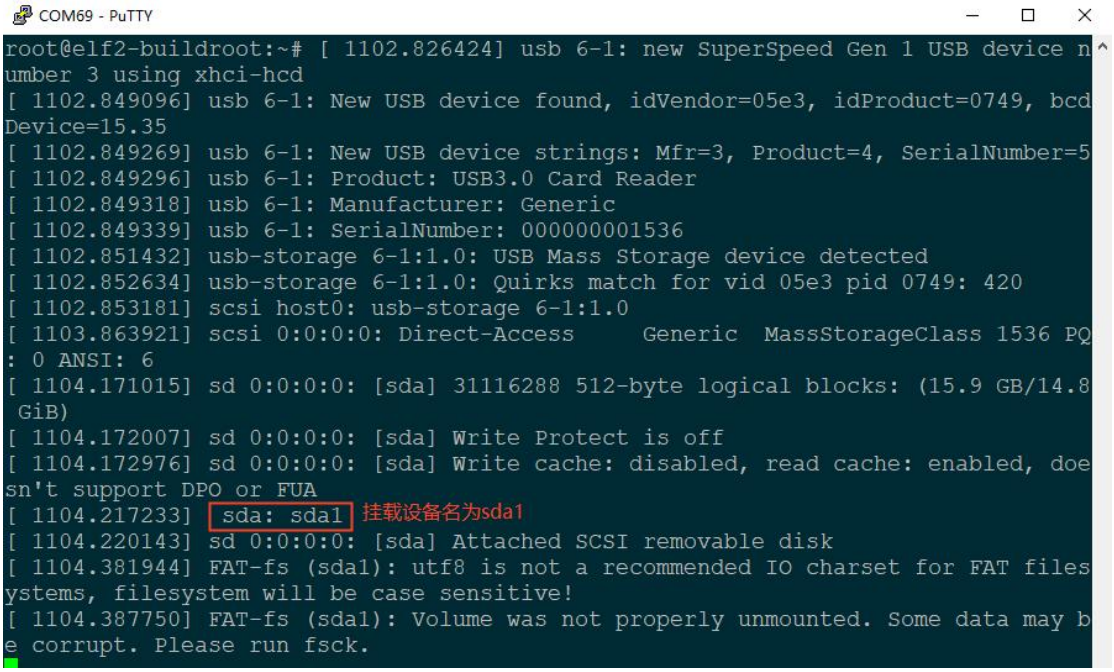
🕮 说明：

- 支持 USB 鼠标，USB 键盘，U 盘设备的热插拔。
- 本节测试所用的 U 盘大小为 32G。

- U 盘挂载目录为/mnt/udisk。
- 使用 U 盘测试前需将其格式化成 FAT32 格式。

ELF 2 支持两个 USB2.0 接口和一个 USB3.0 接口。用户可以在任何一个板载 USB HOST 接口上连接 USB 鼠标、 USB 键盘、U 盘等设备，并支持以上设备的热插拔。 这里用挂载 U 盘为例进行演示，目前 U 盘测试支持到 32G，32G 以上并未测试。

1.开发板启动后，插入 U 盘到开发板的 USB HOST 接口，同时终端会打印关于 U 盘的信息，由于存在很多种 U 盘，显示的信息可能会有差别，插入 U 盘，打印信息如下：



2.查看挂载目录：

```
root@elf2-buildroot:~# mount | grep "sda1"
/dev/sda1 on /mnt/udisk type vfat
(rw,nodev,noexec,noatime,nodiratime,fmask=0022,dmask=0022,codepage=936,iochar
set=utf8,shortname=mixed,errors=remount-ro)
```

可以看到/mnt/udisk为USB存储设备的挂载路径。

3.写入测试，写入速度受限于具体的存储设备：

```
root@elf2-buildroot:~# dd if=/dev/zero of=/mnt/udisk/test bs=1M count=500 conv=fsync
500+0 records in
500+0 records out
524288000 bytes (524 MB, 500 MiB) copied, 39.3141 s, 13.3 MB/s
```

4.读取测试：

⚠**注意：**为确保数据准确，请重启开发板后测试读取速度。

```
root@elf2-buildroot:~# dd if=/mnt/udisk/test of=/dev/null bs=1M iflag=direct
500+0 records in
500+0 records out
524288000 bytes (524 MB, 500 MiB) copied, 19.9524 s, 26.3 MB/s
```

5.U盘使用完成后，在拔出U盘前，需要使用umount卸载：

```
root@elf2-buildroot:~# umount /mnt/udisk/
```

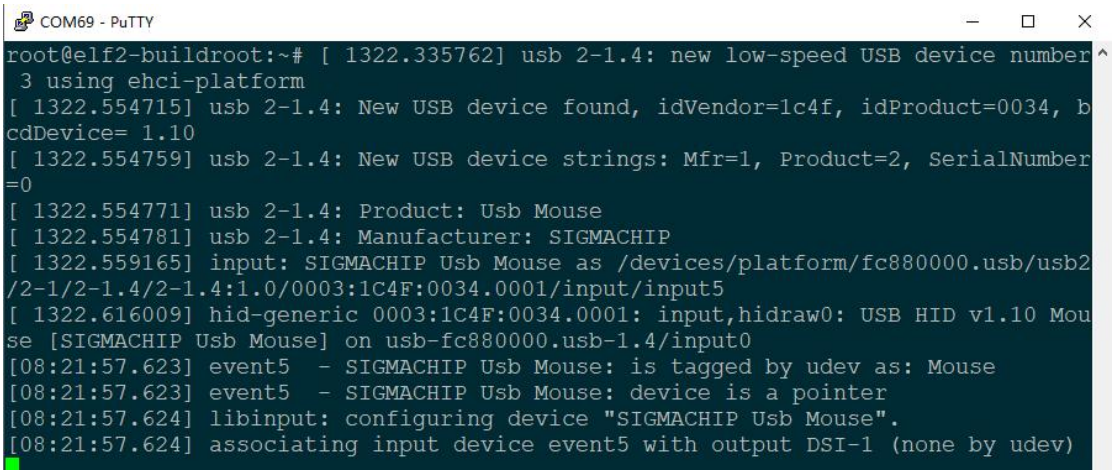
⚠**注意：**退出 U 盘挂载路径后再插拔 U 盘。

3.1.2.2. USB 鼠标测试

鼠标与 ELF 2 硬件连接如图：



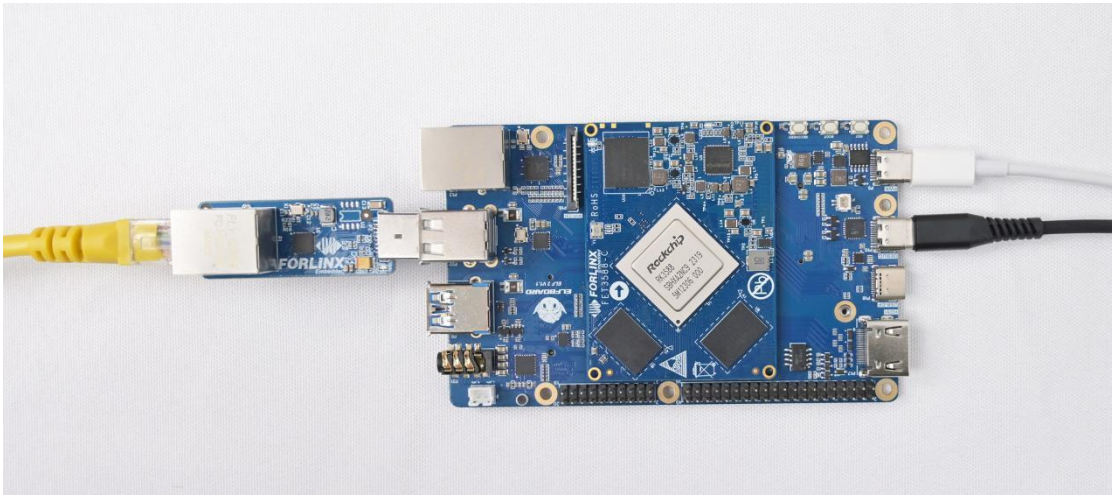
将 USB 鼠标接入 ELF 2 板卡的 USB 接口，串口终端的打印信息如下：



此时在屏幕上出现箭头光标，鼠标已可正常使用。

3.1.2.3. USB 转网络测试

USB 转网卡模块型号为 RTL8153BI-CG，与 ELF 2 硬件连接如图：



将 USB 转网卡模块插入 ELF 2 开发板的 USB 接口，终端会有如下打印信息：

```
root@elf2-buildroot:~# [    29.279882] usb 2-1.4: new high-speed USB device number 5 using ehci-platform
[    29.479331] usb 2-1.4: New USB device found, idVendor=0bda, idProduct=8153, bcdDevice=31.00
[    29.479428] usb 2-1.4: New USB device strings: Mfr=1, Product=2, SerialNumber=6
[    29.479453] usb 2-1.4: Product: USB 10/100/1000 LAN
[    29.479474] usb 2-1.4: Manufacturer: Realtek
[    29.479495] usb 2-1.4: SerialNumber: 000000000000
[    29.663249] usb 2-1.4: reset high-speed USB device number 5 using ehci-platform
[    29.870614] r8152 2-1.4:1.0: Direct firmware load for rtl_nic/rtl8153b-2.fw failed with error -2
[    29.870653] r8152 2-1.4:1.0: unable to load firmware patch rtl_nic/rtl8153b-2.fw (-2)
[    29.870912] r8152 2-1.4:1.0 (unnamed net_device) (uninitialized): Invalid ether addr 00:00:00:00:00:00
[    29.870929] r8152 2-1.4:1.0 (unnamed net_device) (uninitialized): Random ether addr 42:b5:54:dd:37:be
[    29.911964] r8152 2-1.4:1.0 eth1: v1.11.11
```

使用如下命令查看是否生成网卡设备：

```
root@elf2-buildroot:~# ifconfig -a
can0      Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
          NOARP  MTU:16  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:10
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:100
```

```
can1      Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
          NOARP  MTU:16  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:10
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:101

eth0      Link encap:Ethernet  HWaddr D6:EA:EA:91:8F:9A
          inet addr:10.10.19.35  Bcast:10.10.19.255  Mask:255.255.255.0
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:133

eth1      Link encap:Ethernet  HWaddr 42:B5:54:DD:37:BE
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:44 errors:0 dropped:0 overruns:0 frame:0
          TX packets:44 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:7513 (7.3 KiB)  TX bytes:7513 (7.3 KiB)

wlP4p65s0 Link encap:Ethernet  HWaddr C4:75:AB:2E:BF:8F
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

可看到生成网卡 eth1 成功，现在使用 ifconfig 命令设置此网卡 IP：

```
root@elf2-buildroot:~# ifconfig eth1 10.10.19.38
root@elf2-buildroot:~# ifconfig eth1
eth1      Link encap:Ethernet  HWaddr 46:A1:B9:3C:D2:3C
```

```
          inet addr:10.10.19.38  Bcast:10.255.255.255  Mask:255.0.0.0
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

使用网线连接模块与 PC 机：

```
root@elf2-buildroot:~# [ 1397.478083] IPv6: ADDRCONF(NETDEV_CHANGE): eth
1: link becomes ready
[ 1397.479563] r8152 2-1.4:1.0 eth1: carrier on
```

使用 ping 命令 ping PC 机测试(这里设置 PC 机 IP 为 10.10.19.34)：

```
root@elf2-buildroot:~# ping 10.10.19.34 -I eth1 -c 5
PING 10.10.19.34 (10.10.19.34) from 10.10.19.38 eth1: 56(84) bytes of data.
64 bytes from 10.10.19.34: icmp_seq=1 ttl=128 time=0.443 ms
64 bytes from 10.10.19.34: icmp_seq=2 ttl=128 time=0.533 ms
64 bytes from 10.10.19.34: icmp_seq=3 ttl=128 time=0.458 ms
64 bytes from 10.10.19.34: icmp_seq=4 ttl=128 time=0.508 ms
64 bytes from 10.10.19.34: icmp_seq=5 ttl=128 time=0.759 ms

--- 10.10.19.34 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4042ms
rtt min/avg/max/mdev = 0.443/0.540/0.759/0.114 ms
```

3.1.3. 有线网络测试

3.1.3.1. IPV4 基本命令测试

📖 说明：

- 本节示例在本公司的网络环境下进行测试，实际使用中网络环境会有差异，请用户按照自己实际网络环境进行配置。
- ELF 2 有 eth0 一路网卡，开机默认 eth0 IP 为 192.168.0.232，用户可以根据下面的指令自行设置 IP。

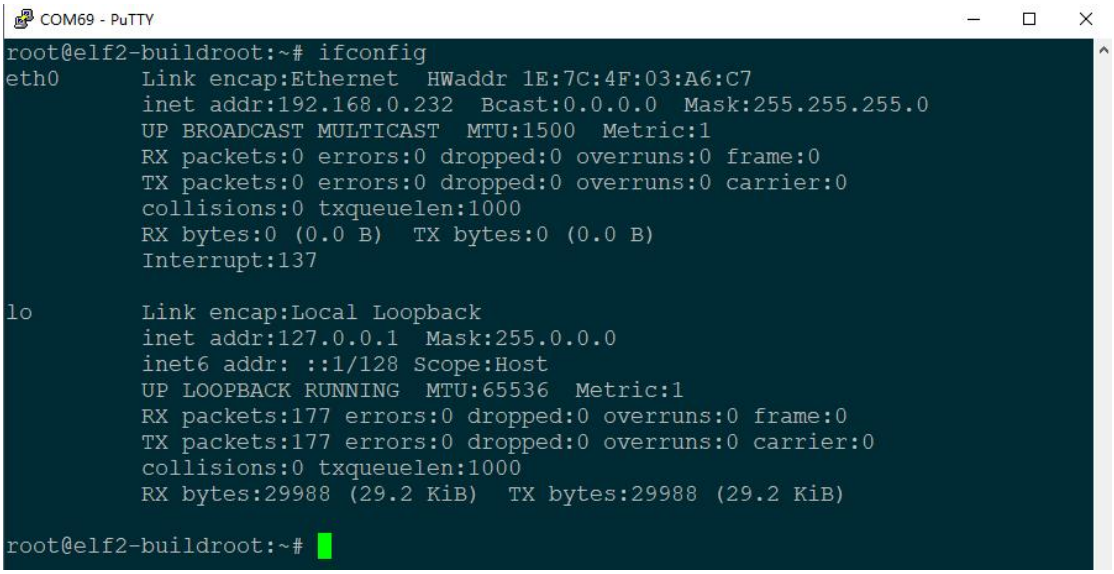
1.在 Linux 系统下，使用 ifconfig 命令可以显示或配置网络设备，使用 ethtool 查询及设置网卡参数。

```
root@elf2-buildroot:~# ifconfig
```

📖 说明：

- ‘ifconfig’是 Linux 系统下的一个命令，用于显示和配置网络接口的信息。它可以显示已经配置的网络接口的状态，包括 IP 地址、MAC 地址、网络掩码、广播地址、传输速率等信息。使用‘ifconfig’命令时，可以不带参数，这样会列出系统中所有已经配置的网络接口的状态和信息。也可以使用‘ifconfig’命令加上网络接口的名称，例如‘ifconfig eth0’，这样会显示指定网络接口的详细信息。用户可以使用该命令修改网络接口的参数，例如设置 IP 地址、子网掩码、网关等。

打印信息如下：



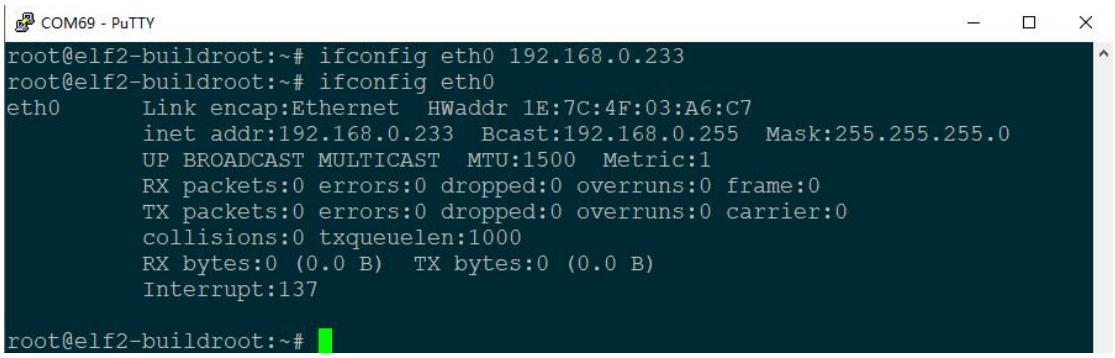
2.设置 IP 地址，将 eth0 IP 设置为 192.168.0.233。

📖 说明：

- 此方法修改 IP 仅在终端中临时生效。

```
root@elf2-buildroot:~# ifconfig eth0 192.168.0.233          //设置 IP
root@elf2-buildroot:~# ifconfig eth0                        //查看设置后网络状况
```

打印信息如下：

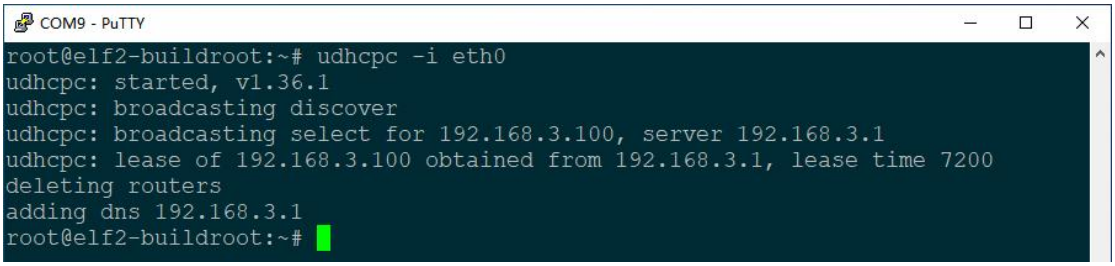


inet addr:192.168.0.233 可以看出 IP 设置成功。

3.动态分配 IP 地址。

如果您的 ELF 2 与路由器连接，且路由器支持 DHCP 自动分配 IP 地址，可以在终端里面输入命令：

```
root@elf2-buildroot:~# udhcpc -i eth0
```



用来动态获取 IP 地址，“-i”参数用来指定网卡名称，ELF 2 有线网络的网卡名称为 eth0。/etc/resolv.conf 文件中有 dns 服务器信息会被自动添加。

```
root@elf2-buildroot:~# vi /etc/resolv.conf
```



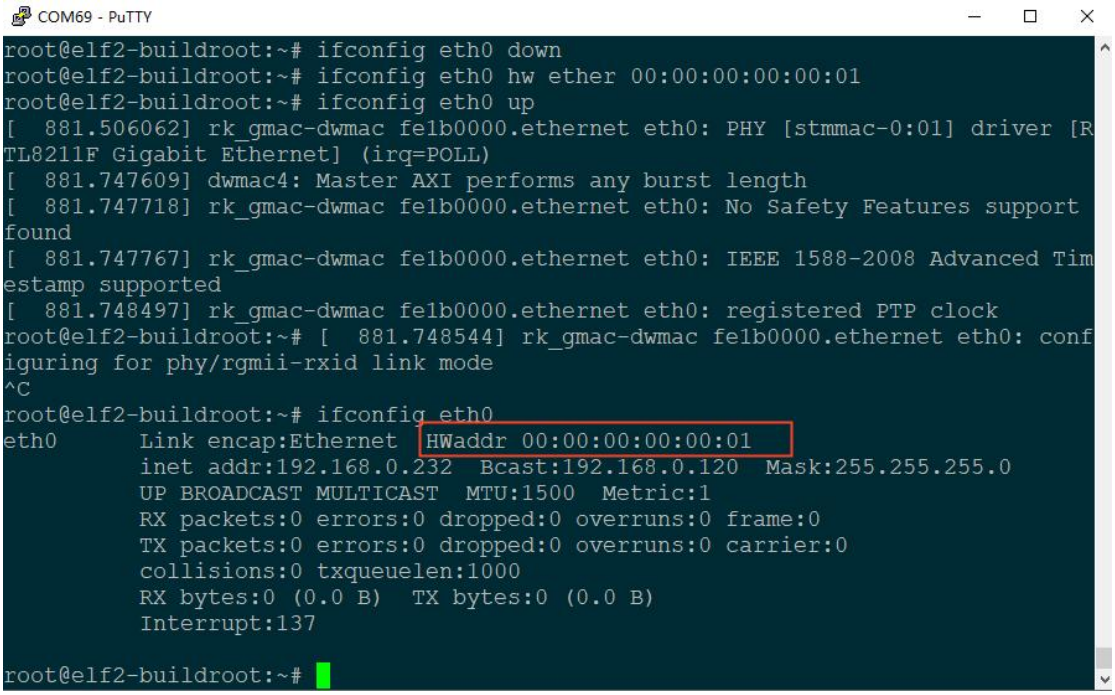
4.修改 mac 地址。

说明：

- 以下设置方法仅在终端中临时生效。

```
root@elf2-buildroot:~# ifconfig eth0 down
root@elf2-buildroot:~# ifconfig eth0 hw ether 00:00:00:00:00:01 //eth0 mac 设为 00:00:00:00:00:01
root@elf2-buildroot:~# ifconfig eth0 up
root@elf2-buildroot:~# ifconfig
```

打印信息如下：



HWaddr 00:00:00:00:00:01 可以看出 mac 设置成功。

5.设置子网掩码。

```
root@elf2-buildroot:~# ifconfig eth0 netmask 255.255.255.0 //设置 eth0 子网掩码为 255.255.255.0
```

6.设置广播地址。

```
root@elf2-buildroot:~# ifconfig eth0 broadcast 192.168.0.120 //eth0 广播地址设 192.168.0.120
```

7.添加/删除默认网关。

添加默认网关：

```
root@elf2-buildroot:~# route add default gw 192.168.0.1 //添加 eth0 默认网关
```

删除默认网关：

```
root@elf2-buildroot:~# route del default gw 192.168.0.1 //删除 eth0 默认网关
```

8.关闭/开启网卡。

关闭 eth0 网卡：

```
root@elf2-buildroot:~# ifconfig eth0 down //关闭 eth0 网卡
```

开启 eth0 网卡：

```
root@elf2-buildroot:~# ifconfig eth0 up                    //开启 eth0 网卡
```

3.1.3.2. IPV6 测试

1.设置 IPV6 地址。

```
root@elf2-buildroot:~# ip -6 addr add 2001:250:4000:2000::50/64 dev eth0    //设置 IPV6 地址
```

📖 说明：

- 在 Linux 系统中，可以使用‘ip’命令来管理网络接口和配置网络参数。命令中的‘-6’选项用于指定要添加的是 IPv6 地址，‘addr’表示要添加 IP 地址，add 表示添加 IP 地址，后面跟着要添加的 IPv6 地址‘2001:250:4000:2000::50/64’，‘/64’表示网络前缀长度，添加 IP 地址的网络接口名称‘dev eth1’，这里指定的网络接口是‘eth0’。

2.配置电脑 ipv6 地址。

打开控制面板->网络和 Internet->网络和共享中心->更改适配器设置，出现如下界面：



右击以太网，选择属性，取消勾选 ipv4，勾选 ipv6。



双击“Internet 协议版本 6（TCP/IPV6）”，参考下图修改：



将 ELF 2 板卡和电脑用网线直连，使用 ping6 命令测试如下：

```
root@elf2-buildroot:~# ping6 2001:250:4000:2000::49
PING 2001:250:4000:2000::49(2001:250:4000:2000::49) 56 data bytes
64 bytes from 2001:250:4000:2000::49: icmp_seq=1 ttl=128 time=1.43 ms
64 bytes from 2001:250:4000:2000::49: icmp_seq=2 ttl=128 time=0.399 ms
64 bytes from 2001:250:4000:2000::49: icmp_seq=3 ttl=128 time=0.501 ms
^C
--- 2001:250:4000:2000::49 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 0.399/0.640/1.432/0.398 ms
```

3.1.3.3. 以太网配置

📖 说明：

参考“ IPv4 基本命令测试”章节，可以在命令行中设置板卡的 IP 地址、网关等信息。请注意，这种设置仅为临时生效。如需使其永久生效，需修改相关配置文件。

ELF 2板载一个千兆网卡。

创建一个eth0配置文件，配置文件的路径为：/etc/network/interface，设置动态IP的配置文件内容为：

```
auto eth0
iface eth0 inet dhcp
```

设置静态配置IP，以下以设置eth0 IP为192.168.0.232为例：

```
auto eth0
iface eth0 inet static
address 192.168.0.232
netmask 255.255.255.0
gateway 192.168.0.1
```

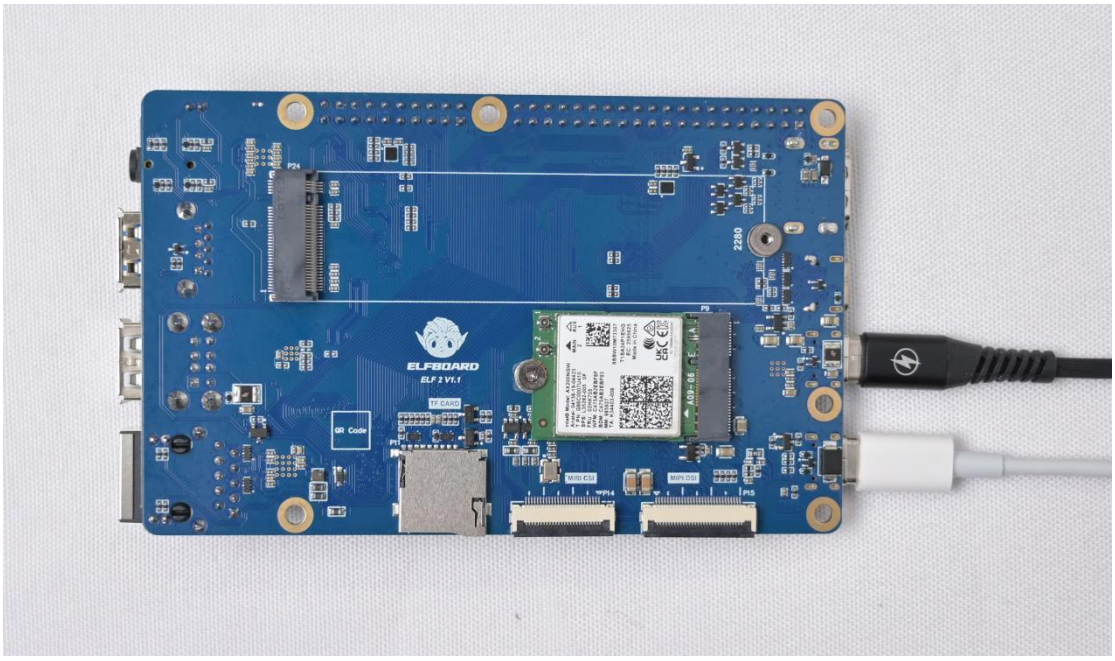
参数	含义
iface	用于指定需要固定 IP 的网卡
address	用于指定需要固定的 IP 地址
netmask	用于设置子网掩码
gateway	用于指定网关

设置完后使用sync文件同步指令，重启开发板或者重启服务，配置生效。

```
root@elf2-buildroot:~# ifdown -a
root@elf2-buildroot:~# ifup -a
```

3.1.4. WiFi 测试

wifi&bt 模块与 ELF 2 硬件连接如图：



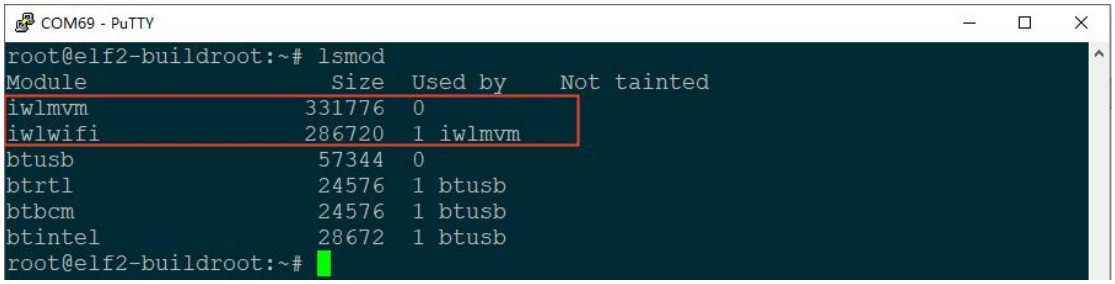
说明：

- 由于网络环境的不同，所以您在做本实验时，请根据实际情况进行设置。

ELF 2 集成了 wifi&bt 模块 CF-AX200-M，WiFi 模块支持 STA 和 AP 两种模式，以下分别对这两种模式进行操作说明。

ELF 2 上电，启动 Linux 系统，先使用 lsmod 查看模块加载状态：

```
root@elf2-buildroot:~# lsmod
```



3.1.4.1. STA 模式

该模式即作为一个站点，连接到无线网络中。以下测试中，路由器采用 wpa 加密方式，连接的 WiFi 热点名称为：forlinx，密码为：03123102650。由于网络环境的不同，用户在进行本次测试时，请根据实际情况进行设置：

1.以 CF-AX200-M 模块为例，开发板终端中输入如下命令：

```
root@elf2-buildroot:~# cmddemo_wifi.sh -i wlP4p65s0 -s forlinx -p 03123102650
```

命令中相关参数含义如下：

参数	含义
-i	不同 WiFi 模块所用参数不同，指定 WiFi 设备名称

-s	连接的实际 WiFi 热点名称
-p	后接参数 password 指要连接的实际 WiFi 热点的密码 如果当前热点没有密码，-p 后参数写 NONE

串口打印如下：

```
wifi wlP4p65s0
ssid forlinx
pasw 03123102650
waiting...
try to connect again...
[ 227.266555] wlP4p65s0: authenticate with fa:cd:55:73:9c:c0
[ 227.271416] wlP4p65s0: send auth to fa:cd:55:73:9c:c0 (try 1/3)
[ 227.299078] wlP4p65s0: authenticated
[ 227.305721] wlP4p65s0: associate with fa:cd:55:73:9c:c0 (try 1/3)
[ 227.409213] wlP4p65s0: associate with fa:cd:55:73:9c:c0 (try 2/3)
[ 227.512547] wlP4p65s0: associate with fa:cd:55:73:9c:c0 (try 3/3)
[ 227.528117] wlP4p65s0: RX AssocResp from fa:cd:55:73:9c:c0 (capab=0x11 st
atus=0 aid=65)
[ 227.534494] wlP4p65s0: associated
[ 227.675596] IPv6: ADDRCONF(NETDEV_CHANGE): wlP4p65s0: link becomes
ready
udhcpc: started, v1.36.1
udhcpc: broadcasting discover
udhcpc: broadcasting select for 10.10.20.119, server 10.10.23.254
udhcpc: lease of 10.10.20.119 obtained from 10.10.23.254, lease time 1800
deleting routers
adding dns 119.29.29.29
adding dns 113.313.313.314
Finshed!
```

2.检查是否能 ping 外网，在终端中输入如下命令：

```
root@elf2-buildroot:~# ping www.elfboard.com -c 3
PING www.elfboard.com (211.149.236.219) 56(84) bytes of data.
64 bytes from 211.149.236.219 (211.149.236.219): icmp_seq=1 ttl=53 time=79.1 ms
64 bytes from 211.149.236.219 (211.149.236.219): icmp_seq=2 ttl=53 time=78.2 ms
64 bytes from 211.149.236.219 (211.149.236.219): icmp_seq=3 ttl=53 time=76.1 ms

--- www.elfboard.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 76.075/77.786/79.079/1.261 ms
```

3.1.4.2. AP 模式

即接入点模式，是一种无线网络工作模式，在这种模式下，设备可以作为一个无线接入点，允许其他无线设备连接到它所提供的网络。

📖 说明：

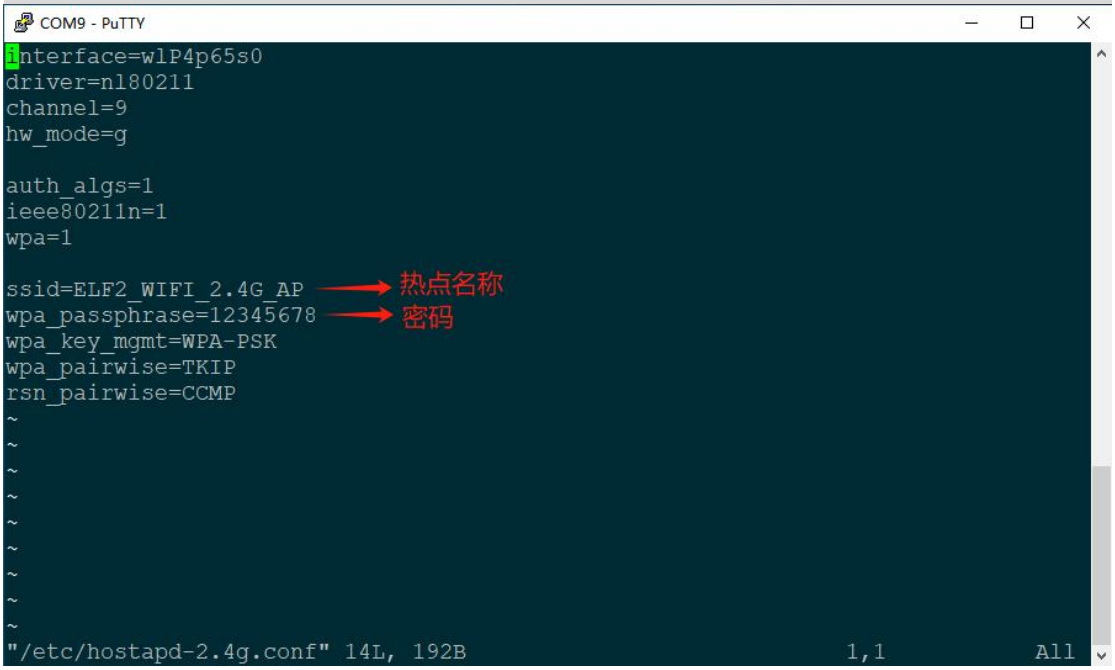
- 进行该项测试前需要保证千兆网卡 eth0 连网，并且网络正常；

WiFi 热点名称：ELF2_WIFI_2.4G_AP

密码：12345678

热点名称和密码可通过/etc/hostapd-2.4g.conf 文件查看。

```
root@elf2-buildroot:~# vi /etc/hostapd-2.4g.conf
```



执行脚本启动 hostapd 服务，将设备配置为无线接入点：

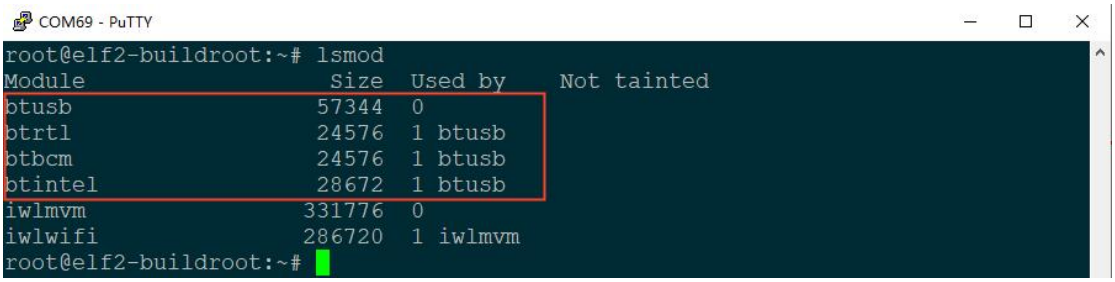
```
root@elf2-buildroot:~# cmddemo_hostapd.sh
killall: hostapd: no process killed
Stopping dnsmasq: [ 1575.722716] wlP4p65s0: deauthenticating from
9e:2b:a6:85:b4:af by local choice (Reason: 3=DEAUTH_LEAVING)
FAIL
Starting dnsmasq: OK
root@elf2-buildroot:~# [ 1577.029564] IPv6: ADDRCONF(NETDEV_CHANGE):
wlP4p65s0: link becomes ready
wlP4p65s0: interface state UNINITIALIZED->ENABLED
wlP4p65s0: AP-ENABLED
```

3.1.5. 蓝牙测试

ELF 2 开发板中的 CF-AX200-M 模块，集成了蓝牙功能，本节将演示如何使用蓝牙实现手机与开发板之间的数据传输。

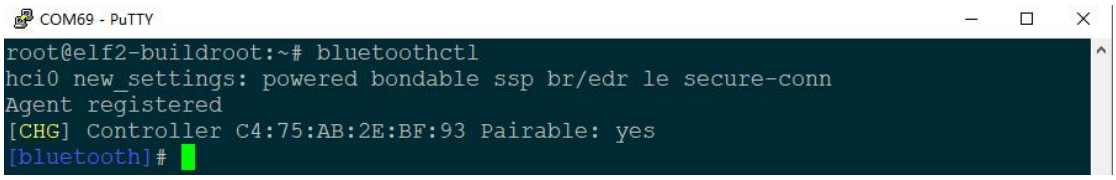
1.ELF 2 上电，启动 Linux 系统，先使用 lsmod 查看模块加载状态。

```
root@elf2-buildroot:~# lsmod
```



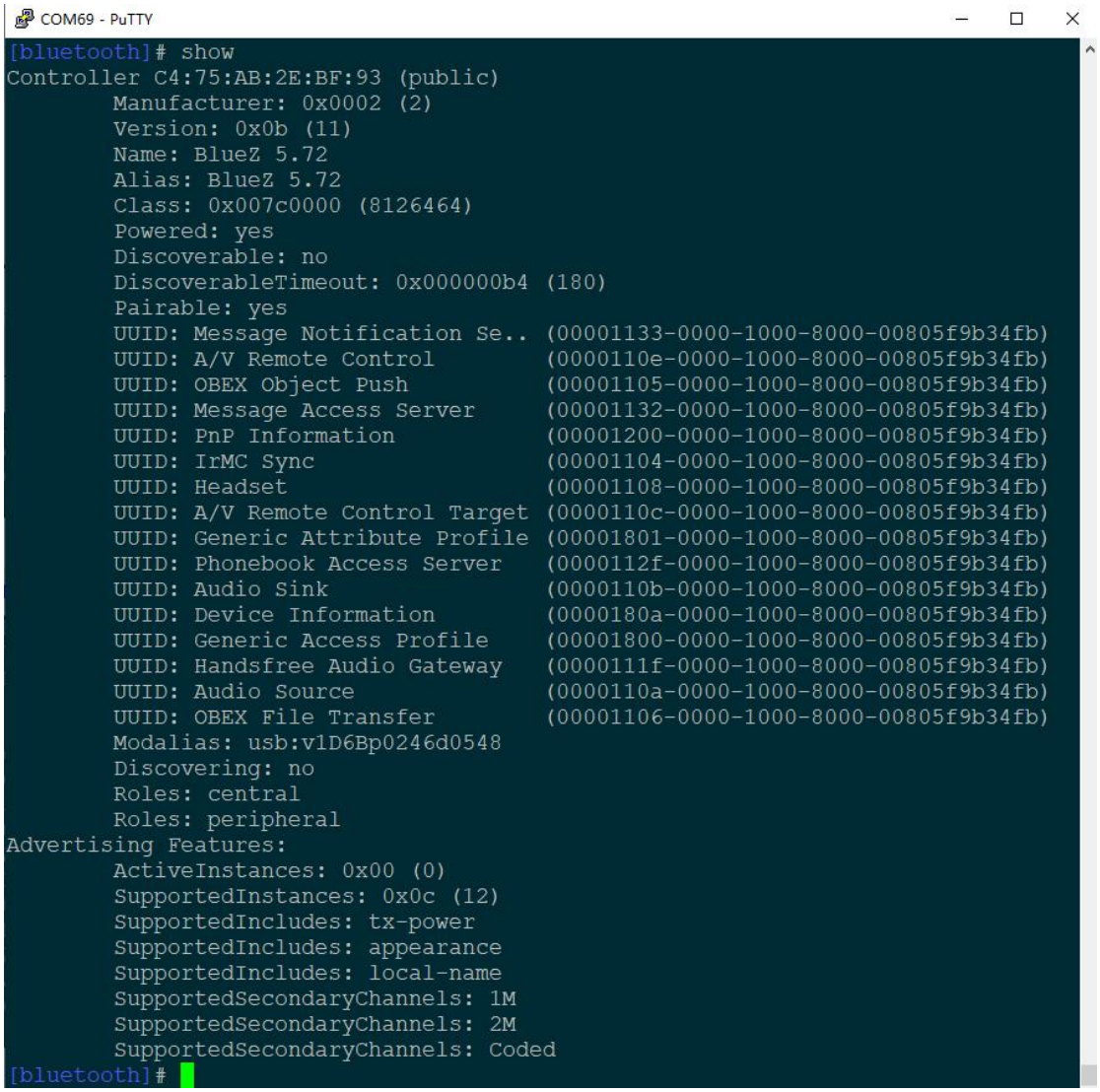
2.使用 bluez 命令行工具 bluetoothctl 配置蓝牙。

```
root@elf2-buildroot:~# bluetoothctl
```



查看当前设备信息：

```
[bluetooth]# show
```



依次执行以下命令对蓝牙设备控制器进行配置：

```
[ bluetooth ]# power on                                    //启动蓝牙设备
Changing power on succeeded
[ bluetooth ]# pairable on                                 //设置为配对模式
Changing pairable on succeeded
[ bluetooth ]# discoverable on                             //设置为可发现模式
```

```
hci0 new_settings: powered connectable bondable ssp br/edr le secure-conn
hci0 new_settings: powered connectable discoverable bondable ssp br/edr le secur
e-conn
Changing discoverable on succeeded
[CHG] Controller C4:75:AB:2E:BF:93 Discoverable: yes
```

```
[ bluetooth ]# agent on                                    //启动代理
Agent registered
[ bluetooth ]# default-agent                               //设置当前代理为默认代理
Default agent request successful
```

3.开发板被动配对。

此时打开手机蓝牙搜索，会出现一个“BlueZ 5.72”的设备，选择配对。



同时开发板上打印信息如下，输入 yes：

```
[agent] Confirm passkey 474108 (yes/no): yes
hci0 new_link_key 90:09:DF:E6:57:39 type 0x08 pin_len 0 store_hint 1
hci0 device_flags_changed: 90:09:DF:E6:57:39 (BR/EDR)
        supp: 0x00000001    curr: 0x00000000
[CHG] Device 90:09:DF:E6:57:39 Bonded: yes
[CHG] Device 90:09:DF:E6:57:39 UUIDs: 0000110c-0000-1000-8000-00805f9b34fb
[CHG] Device 90:09:DF:E6:57:39 UUIDs: 0000110e-0000-1000-8000-00805f9b34fb
[CHG] Device 90:09:DF:E6:57:39 Modalias: bluetooth:v0006p0001d0A00
[CHG] Device 90:09:DF:E6:57:39 UUIDs: 00001000-0000-1000-8000-00805f9b34fb
[CHG] Device 90:09:DF:E6:57:39 UUIDs: 0000110a-0000-1000-8000-00805f9b34fb
[CHG] Device 90:09:DF:E6:57:39 UUIDs: 0000110b-0000-1000-8000-00805f9b34fb
[CHG] Device 90:09:DF:E6:57:39 UUIDs: 0000110c-0000-1000-8000-00805f9b34fb
[CHG] Device 90:09:DF:E6:57:39 UUIDs: 0000110e-0000-1000-8000-00805f9b34fb
[CHG] Device 90:09:DF:E6:57:39 UUIDs: 00001115-0000-1000-8000-00805f9b34fb
[CHG] Device 90:09:DF:E6:57:39 UUIDs: 0000111e-0000-1000-8000-00805f9b34fb
[CHG] Device 90:09:DF:E6:57:39 UUIDs: 0000111f-0000-1000-8000-00805f9b34fb
[CHG] Device 90:09:DF:E6:57:39 UUIDs: 00001200-0000-1000-8000-00805f9b34fb
[CHG] Device 90:09:DF:E6:57:39 UUIDs: c7f94713-891e-496a-a0e7-983a0946126e
[CHG] Device 90:09:DF:E6:57:39 ServicesResolved: yes
[CHG] Device 90:09:DF:E6:57:39 Paired: yes
Authorize service
[agent] Authorize service 0000110e-0000-1000-8000-00805f9b34fb (yes/no): yes
Authorize service
```

查看和移除连接设备：

```
[bluetooth]# devices     //查看连接的蓝牙设备
Device 90:09:DF:E6:57:39 TEST
```

```
[bluetooth]# remove 90:09:DF:E6:57:39      //移除设备
```

4.开发板主动配对。

除了被动配对，也可以在开发板终端发送主动配对的请求。

```
[bluetooth]# scan on          //搜索可被发现
SetDiscoveryFilter success
hci0 type 7 discovering on
Discovery started
[CHG] Controller C4:75:AB:2E:BF:93 Discovering: yes
[NEW] Device 35:50:CD:F3:84:A1 35-50-CD-F3-84-A1
[NEW] Device 53:38:D7:51:19:8B 53-38-D7-51-19-8B
[NEW] Device 43:C4:E1:EE:17:8E 43-C4-E1-EE-17-8E
[NEW] Device 05:A5:CB:BB:CC:01 05-A5-CB-BB-CC-01
[bluetooth]# scan off                                    //停止搜索
[bluetooth]# pair 90:09:DF:E6:57:39                      //配对蓝牙
Attempting to pair with 90:09:DF:E6:57:39
hci0 device_flags_changed: 90:09:DF:E6:57:39 (BR/EDR)
        supp: 0x00000001    curr: 0x00000000
hci0 90:09:DF:E6:57:39 type BR/EDR connected eir_len 16
[CHG] Device 90:09:DF:E6:57:39 Connected: yes      //口令确认
```

5.开发板接收文件。

配对成功后，在手机端，可以使用蓝牙发送文件至 ELF 2 中。



接收到的文件保存在/tmp 目录。

6.开发板发送文件。

ELF 2 可以发送文件至手机端，测试方法如下：

```
root@elf2-buildroot:~# cmddemo_obexctl.sh     //开启 obexctl
[NEW] Client /org/bluez/obex
[obex]# connect B8:27:C5:DC:A7:92          //连接需要通讯的蓝牙的 MAC
Attempting to connect to B8:27:C5:DC:A7:92
[NEW] Session /org/bluez/obex/client/session0 [default]
[NEW] ObjectPush /org/bluez/obex/client/session0
Connection successful
[B8:27:C5:DC:A7:92]# send /userdata/media/audio/piano2-CoolEdit.mp3   //发送文件
```

手机将收到传入文件请求，点击接受，进行文件传输。

3.1.6. 看门狗测试

看门狗是嵌入式系统中经常用到的功能，ELF 2中看门狗的设备节点为/dev/watchdog。

参数说明：

参数	含义	说明
settimeout	开启看门狗，设置复位时间，不喂狗	复位时间需大于 2s
keepalive	开启看门狗，设置复位时间，定时 2s 喂狗	复位时间需大于 2s

⚠**注意：**测试时以上两个参数只能使用一个，可通过 ps 查看进程，保证只有一个 cmddemo_wdt 进程。

1.开启看门狗，设置复位时间测试。

执行 60s 复位命令，（可设置最小复位时间间隔为 2s），60s 后 ELF 2 复位：

```
root@elf2-buildroot:~# cmddemo_wdt /dev/watchdog settimeout 60 & //执行 60s 复位命令
[1] 1385
```

🕮 说明：

- 在该命令中，‘cmddemo_wdt’表示要使用 cmddemo_wdt 工具程序；‘/dev/watchdog’表示硬件看门狗的设备文件路径；‘settimeout 60’表示设置超时时间为 60 秒；‘&’表示将 cmddemo_wdt 命令放到后台运行。当用户执行‘cmddemo_wdt /dev/watchdog settimeout 60 &’命令时，它会使用 cmddemo_wdt 工具程序来设置硬件看门狗的超时时间为 60 秒。

等待 60s 后可以看到系统重启。

2.cmddemo_wdt的参数keepalive为设置复位时间，并进行2s定时喂狗，10为设置的复位时间为10s。

```
root@elf2-buildroot:~# cmddemo_wdt /dev/watchdog keepalive 10 &
[1] 2242
```

如果 killall 关闭 cmddemo_wdt 应用在后台的运行，则会造成系统在 10s 内不去喂狗而超时重启。

```
root@elf2-buildroot:~# killall cmddemo_wdt
[   252.231320] watchdog: watchdog0: watchdog did not stop!
```

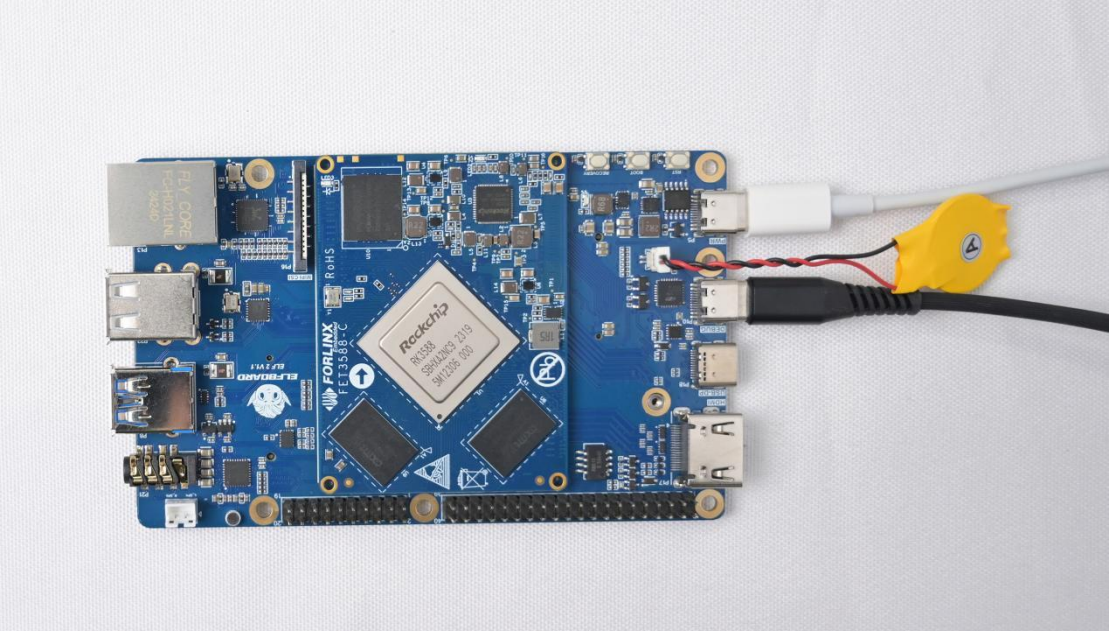
🕮 说明：

- 在该命令中，‘killall’表示终止具有指定名称的进程；‘cmddemo_wdt’表示要终止的进程名称。当用户执行‘killall cmddemo_wdt’命令时，它会终止系统中所有名为 cmddemo_wdt 的进程。

3.1.7. RTC 时钟驱动测试

⚠**注意：**请确保板子上已经安装了电池，并且电池电压正常，请注意，禁止对提供的电池进行充电。

ELF 2 板卡使用的 RTC 芯片为 RX8010，电池与 ELF 2 硬件连接如图：



RTC测试，主要通过使用date和hwclock工具设置软、硬件时间，测试当板子断电再上电的时候，软件时钟读取RTC时钟是否同步。

时间设置。

```
root@elf2-buildroot:~# date -s "2022-12-12 17:23:00"   //设置软件时间
Mon Dec 12 17:23:00 CST 2022
root@elf2-buildroot:~# hwclock -wu                     //将软件时间同步到硬件时间
root@elf2-buildroot:~# hwclock -r                      //显示硬件时间
Mon Dec 12 17:23:15 2022   0.000000 seconds
```

然后给板子断电再上电，进入系统后读取系统时间，可以看到时间已经同步。

```
root@elf2-buildroot:~# date
Mon Dec 12 17:24:17 CST 2022
```

3.1.8. 放/录音测试

ELF 2 使用 NAU88C22YG 芯片，提供了 1 路标准 3.5mm 音频插座，1 个 XH2.0-2P 白色插座，可驱动 8Ω 喇叭，最高输出功率为 1W，在进行放音测试前，请将准备好的耳机插入听筒接口，或将喇叭插入底板上的对应插槽上进行测试。

⚠**注意：**在播放音频时，当耳机和喇叭同时连接，默认声音会从耳机输出。而在录音时，如果耳机和板载麦克风同时存在，录音将会通过耳机进行。

3.1.8.1. HDMI 播放声音

用 HDMI 线将 ELF 2 开发板的 HDMI 接口与显示器的 HDMI 接口连接起来，可以正常显示桌面。

1.列出系统中可用的音频播放硬件设备。

```
root@elf2-buildroot:~# aplay -l
**** List of PLAYBACK Hardware Devices ****
card 0: rockchipdp0 [rockchip,dp0], device 0: rockchip,dp0 spdif-hifi-0 [rockchip,dp
0 spdif-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 1: rockchipnau8822 [rockchip-nau8822], device 0: dailink-multicodecs nau882
```

```
2-hifi-0 [dailink-multicodecs nau8822-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 2: rockchiphdmi0 [rockchip-hdmi0], device 0: rockchip-hdmi0 i2s-hifi-0 [rockch
ip-hdmi0 i2s-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
```

相关参数含义如下，alpay 更多使用方法可通过 aplay --help 查看。

-l	列出所有声卡和数字音频设备
-L	列出设备名称
-t	文件类型（voc、wav、raw 或 au）
-D	指定声卡设备

使用 aplay -l 列出所有声卡和数字音频设备：

第一个音频卡（card 0）名为 rockchipdp0，具有一个高保真的音频输出设备（device 0），并且该设备有一个可用的子设备。

第二个音频卡（card 1）为 rockchipnau8822，同样具有一个高保真输出设备（device 0），并且也有一个可用的子设备。

第三个音频卡（card 2）为 rockchiphdmi0，同样具有一个高保真输出设备（device 0），并且也有一个可用的子设备。

2.下面使用 rockchiphdmi0 播放指定的 MP3 文件。将耳机插入显示器的音频接口，然后在命令行中执行以下命令，以听到播放的音频。

```
root@elf2-buildroot:~# gst-play-1.0 /userdata/media/audio/piano2-CoolEdit.mp3
--audiosink="alsasink device=plughw:2,0"
```

gst-play-1.0	GStreamer 是一个强大的多媒体框架，它提供了一套完整的工具、库和插件，支持音频、视频的处理和播放。gst-play-1.0 是 GStreamer 的一个命令行工具，用于播放多媒体文件。
/userdata/media/audio/piano2-CoolEdit.mp3	播放的音频文件的路径。此处是一个 MP3 格式的音频文件，文件名为 piano2-CoolEdit.mp3
--audiosink="alsasink device=plughw:2,0"	"alsasink device=plughw:2,0" 表示使用 ALSA 作为音频输出，并指定了具体的设备。plughw:2,0 是 ALSA 的设备名称，2 是声卡的索引，0 是设备的索引。它告诉系统使用 card 2 声卡的 device 0 设备进行输出。

3.1.8.2. 耳机播放声音

将耳机插入 ELF 2 开发板的音频插座上，然后在命令行中执行以下命令，以听到播放的音频。

耳机与 ELF 2 硬件连接如图：

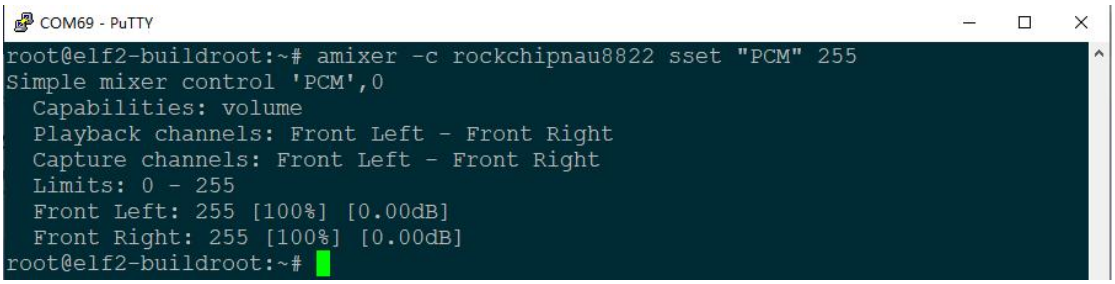


1.查询音频参数

```
root@elf2-buildroot:~# amixer -c rockchipnau8822
```

2.设置 PCM 参数

```
root@elf2-buildroot:~# amixer -c rockchipnau8822 sset "PCM" 255
```

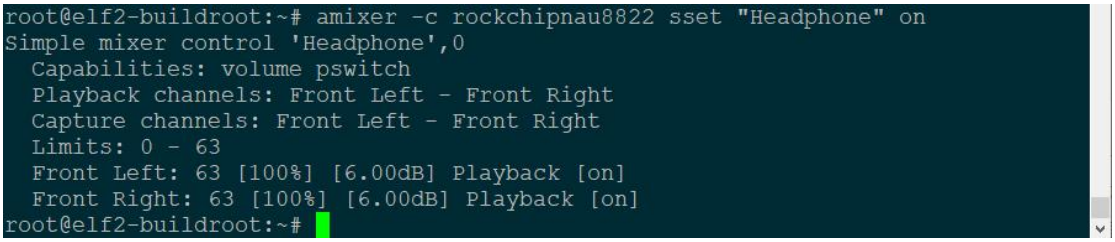


说明：

该行代码是一个命令行命令，用于设置音量控制。amixer 是用于音频混合和控制的命令行工具；-c 后面跟着的是声卡设备 rockchipnau8822 。如果有多个音频设备，使用此选项可以明确指定要控制的设备。sset 是 amixer 的一个参数，用于设置特定控制的音量级别；"PCM"是要设置音量的目标，即 脉冲编码调制（PCM）；255 是要设置的音量级别，可以是 0 到 255 之间的任意整数值，表示音量的百分比。在这里，将 PCM 的音量设置为最大值 255。

3.打开 Speaker

```
root@elf2-buildroot:~# amixer -c rockchipnau8822 sset "Headphone" on
```



说明：

该行代码是一个命令行命令，用于控制耳机的开关状态。amixer 是用于音频混合和控制的命令行工具； -c 后面跟着的是声卡设备 rockchipnau8822 。如果有多个音频设备，使用此选项可以明确指定要控制的设备。sset 是 amixer 的一个参数，用于设置特定控制的属性；"Headphoner"是要设置的目标，即耳机；on 表示要将耳机打开。这意味着耳机将开始输出音频信号。

4.设置耳机音量

```
root@elf2-buildroot:~# amixer -c rockchipnau8822 sset "Headphone" 63
```

```
root@elf2-buildroot:~# amixer -c rockchipnau8822 sset "Headphone" 63
Simple mixer control 'Headphone',0
  Capabilities: volume pswitch
  Playback channels: Front Left - Front Right
  Capture channels: Front Left - Front Right
  Limits: 0 - 63
  Front Left: 63 [100%] [6.00dB] Playback [on]
  Front Right: 63 [100%] [6.00dB] Playback [on]
```

说明：

该行代码是一个命令行命令，用于设置耳机的音量级别。amixer 是用于音频混合和控制的命令行工具；sset 是 amixer 的一个参数，用于设置特定控制的音量级别；"Headphone"是要设置的目标，即耳机；63 是要设置的音量级别，表示将耳机的音量设置为 63。该值可以是 0 到 63 之间的任意整数值，表示音量的百分比。在这里，将耳机的音量设置为 100%。

5.使用耳机播放音频

```
root@elf2-buildroot:~# gst-play-1.0 /userdata/media/audio/piano2-CoolEdit.mp3
--audiosink="alsasink device=plughw:1,0"
```

3.1.8.3. SPKOUT 播放声音

将喇叭插入 ELF 2 开发板的音频插座上，然后在命令行中执行以下命令，以听到播放的音频。喇叭与 ELF 2 硬件连接如图：

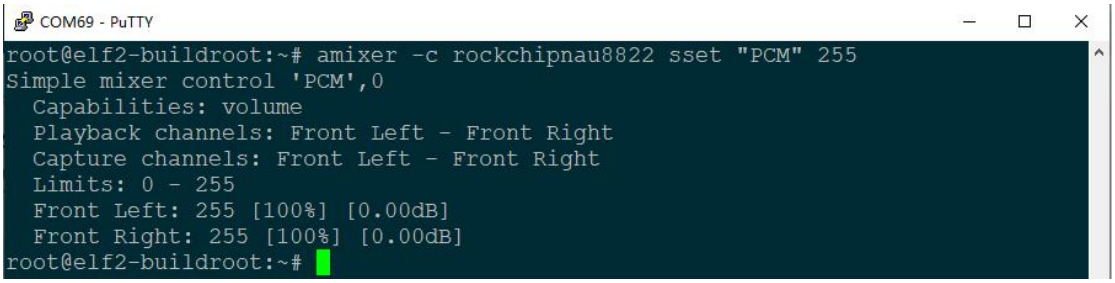


1.查询音频参数

```
root@elf2-buildroot:~# amixer -c rockchipnau8822
```

2.设置 PCM 参数

```
root@elf2-buildroot:~# amixer -c rockchipnau8822 sset "PCM" 255
```



说明：

该行代码是一个命令行命令，用于设置音量控制。amixer 是用于音频混合和控制的命令行工具；-c 后面跟着的是声卡设备 rockchipnau8822 。如果有多个音频设备，使用此选项可以明确指定要控制的设备。sset 是 amixer 的一个参数，用于设置特定控制的音量级别；"PCM"是要设置音量的目标，即脉冲编码调制（PCM）；255 是要设置的音量级别，可以是 0 到 255 之间的任意整数值，表示音量的百分比。在这里，将 PCM 的音量设置为最大值 255。

3.打开 Speaker

```
root@elf2-buildroot:~# amixer -c rockchipnau8822 sset "Speaker" on
```

```
root@elf2-buildroot:~# amixer -c rockchipnau8822 sset "Speaker" on
Simple mixer control 'Speaker',0
  Capabilities: volume pswitch
  Playback channels: Front Left - Front Right
  Capture channels: Front Left - Front Right
  Limits: 0 - 63
  Front Left: 63 [100%] [6.00dB] Playback [on]
  Front Right: 63 [100%] [6.00dB] Playback [on]
root@elf2-buildroot:~#
```

说明：

该行代码是一个命令行命令，用于控制喇叭的开关状态。amixer 是用于音频混合和控制的命令行工具； -c 后面跟着的是声卡设备 rockchipnau8822 。如果有多个音频设备，使用此选项可以明确指定要控制的设备。sset 是 amixer 的一个参数，用于设置特定控制的属性；"Speaker"是要设置的目标，即喇叭；on 表示要将喇叭打开。这意味着喇叭将开始输出音频信号。

4.设置喇叭音量

```
root@elf2-buildroot:~# amixer -c rockchipnau8822 sset "Speaker" 63
```

```
root@elf2-buildroot:~# amixer -c rockchipnau8822 sset "Speaker" 63
Simple mixer control 'Speaker',0
  Capabilities: volume pswitch
  Playback channels: Front Left - Front Right
  Capture channels: Front Left - Front Right
  Limits: 0 - 63
  Front Left: 63 [100%] [6.00dB] Playback [on]
  Front Right: 63 [100%] [6.00dB] Playback [on]
root@elf2-buildroot:~#
```

说明：

该行代码是一个命令行命令，用于设置喇叭的音量级别。amixer 是用于音频混合和控制的命令行工具；sset 是 amixer 的一个参数，用于设置特定控制的音量级别；"Speaker"是要设置的目标，即喇叭；63 是要设置的音量级别，表示将喇叭的音量设置为 63。该值可以是 0 到 63 之间的任意整数值，表示音量的百分比。在这里，将喇叭的音量设置为 100%。

5.使用喇叭播放音频

```
root@elf2-buildroot:~# gst-play-1.0 /userdata/media/audio/piano2-CoolEdit.mp3
--audiosink="alsasink device=plughw:1,0"
```

3.1.8.4. MIC 输入

1.查看系统中可用的音频捕获（录音）硬件设备。

```
root@elf2-buildroot:~# arecord -l
**** List of CAPTURE Hardware Devices ****
card 1: rockchipnau8822 [rockchip-nau8822], device 0: dailink-multicodecs nau882
                                                    2-hifi-0 [dailink-multicodecs nau
                                                    8822-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
```

音频卡（card 0）名为 rockchipnau8822，它的设备编号为 0，支持高保真的音频捕获，并且有一个可用的子设备。

2.下面使用 rockchipnau8822 录制 3 秒的 CD 质量音频，并将其保存为名为 test1.wav 的文件。

```
root@elf2-buildroot:~# arecord -D hw:rockchipnau8822,0 -d 3 -f cd -t wav test1.wav
```

说明：

该行代码是一个命令行命令，arecord 是用于录制音频的命令行工具；-D 表示要使用的音频输入设备。hw:rockchipnau8822,0：这里的 hw 表示直接使用硬件设备，rockchipnau8822 是音频设备的名称，0 是设备的索引。这个参数用于选择具体的声卡设备。-d 3 指定录音将持续 3 秒。-f 指定录音的格式。 -t wav test1.wav 指定输出的音频文件格式为 WAV，录制的音频将被保存为 test1.wav。

3.使用 HDMI 播放采集声音

```
root@elf2-buildroot:~# aplay -D plughw:2,0 test1.wav
```

4.下面使用 rockchipnau8822 播放 test1.wav 文件。将耳机插入 ELF 2 开发板的音频插座上，然后在命令行中执行以下命令，以听到播放的音频。

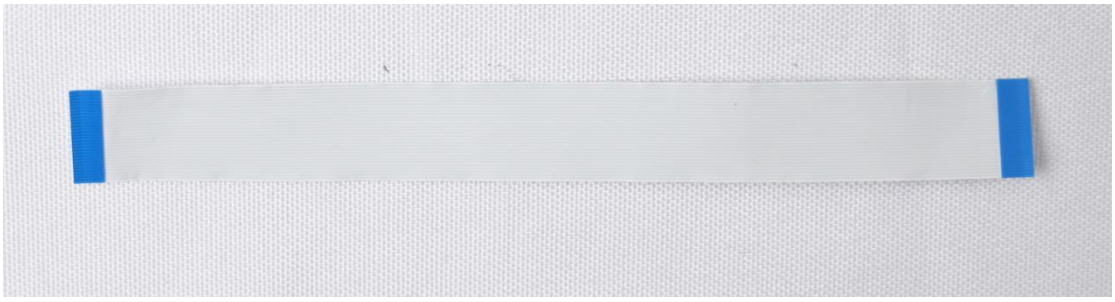
```
root@elf2-buildroot:~# aplay -D plughw:1,0 test1.wav
```

3.1.9. LCD 屏测试

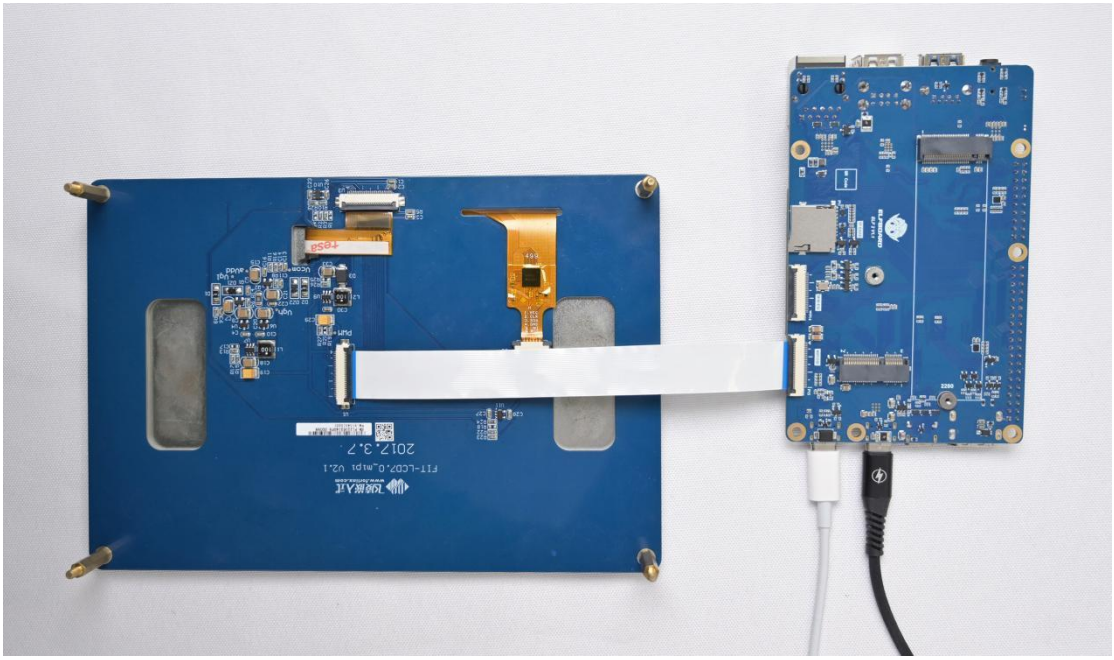
背光的亮度设置范围为（0--255），255表示亮度最高，0表示关闭背光亮度。在MIPI DSI上接上mipi屏幕后，上电启动。进入系统后在终端输入如下命令进行背光测试。

1.屏幕连接

用 FPC 排线进行连接，下图是同向 FPC 排线（蓝色在同一侧）图片，本节测试使用同向 FPC 排线进行连接。



LCD 屏幕与 ELF 2 硬件连接如图：



1.查看支持背光型号。

```
root@elf2-buildroot:~# ls /sys/class/backlight/
backlight-dsi0    显示当前支持屏背光型号
```

下面以dsi0为例：

2.查看当前屏幕背光值。

```
root@elf2-buildroot:~# cat /sys/class/backlight/backlight-dsi0/brightness
200                                                    //当前背光值为 200
```

3.背光熄灭。

```
root@elf2-buildroot:~# echo 0 > /sys/class/backlight/backlight-dsi0/brightness
```

4.LCD 背光亮起。

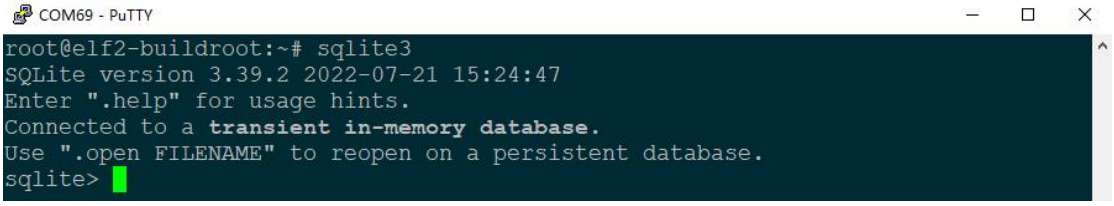
```
root@elf2-buildroot:~# echo 125 > /sys/class/backlight/backlight-dsi0/brightness
```

3.1.10. SQLite3 测试

SQLite3 是一款轻型的数据库，是遵守 ACID 的关系型数据库管理系统，占用资源低。

ELF 2 移植的是 3.21.0 版本的 SQLite3。

```
root@elf2-buildroot:~#  sqlite3
```



测试 SQLite 软件。

```
sqlite>  create  table  tbl1  (one  varchar(10),  two  smallint);          //创建表 tbl1
```

📖 说明：

- 这是一个用于创建数据库表的 SQL 语句，‘create table’：这是用于创建数据库表的关键字；‘tbl1’：这是要创建的数据库表的名称，可以自定义；‘(one varchar(10), two smallint)’：这是数据库表中的列定义部分，其中，one 是一个长度为 10 的字符串类型，two 是一个 16 位整数类型。执行这个 SQL 语句会在当前的数据库中创建一个名为 tbl1 的表，该表包括两列，分别为 one 和 two，

其中 one 是一个长度为 10 的字符串类型，two 是一个 16 位整数类型。

```
sqlite> insert into tbl1 values('hello!',10);                    //tbl1 表内插入数据 hello!|1
0
```

说明：

- 这是一个用于向数据库表中插入数据的 SQL 语句，‘insert into’：这是用于向数据库表中插入数据的关键字；‘tbl1’：这是要插入数据的数据库表的名称；‘values('hello!',10)’：这是要插入的数据部分，其中，‘hello!’是要插入到 one 列中的字符串数据，10 是要插入到 two 列中的整数数据。执行这个 SQL 语句会向名为 tbl1 的数据库表中插入一行数据，该行数据包括一个字符串类型数据‘hello!’和一个 16 位整数类型数据 10。

```
sqlite> insert into tbl1 values('goodbye', 20);                  //tbl1 表内插入数据 goodbye|
20
```

说明：

- 这是一个用于向数据库表中插入数据的 SQL 语句，‘insert into’：这是用于向数据库表中插入数据的关键字；tbl1：这是要插入数据的数据库表的名称；‘values('goodbye', 20)’：这是要插入的数据部分，其中，‘goodbye’是要插入到 one 列中的字符串数据，20 是要插入到 two 列中的整数数据。执行这个 SQL 语句会向名为 tbl1 的数据库表中插入一行数据，该行数据包括一个字符串类型数据‘goodbye’和一个 16 位整数类型数据 20。

```
sqlite> select * from tbl1;                                      //查询表 tbl1 中内容
```

说明：

- 这是一个用于查询数据库表中所有数据的 SQL 语句，‘select’：这是用于查询数据库表中数据的关键字；‘*’：这是表示查询所有列的通配符。‘from tbl1’：这是要查询数据的数据库表的名称。执行这个 SQL 语句会查询名为 tbl1 的数据库表中的所有数据，并将其返回。查询结果将包括所有列的数据，每行数据对应数据库表中的一行数据。

```
hello!|10
goodbye|20
```

退出数据库。

```
sqlite> .quit                                                    //退出数据库（或使用.exit 命令）
```

3.1.11. 调频测试

说明：四核 A55 分别是 cpu0、cpu1、cpu2、cpu3; 四核 A76 分别是 cpu5、cpu6、cpu7、cpu8。此过程以 cpu0 为例操作，实际过程 cpu1、cpu2、cpu3 会同时改变； 单独操作 cpu4、 cpu5、cpu6、cpu7 相互之间不会受到影响。

1.当前内核中支持的所有 cpufreq governor 类型。

```
root@elf2-buildroot:~# cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_g
overnors
interactive conservative ondemand userspace powersave performance schedutil
```

其中 userspace 表示用户模式，在此模式下允许其他用户程序调节 CPU 频率。

2.查看当前 CPU 支持的频率档位。

```
root@elf2-buildroot:~# cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_fr
equencies
408000 600000 816000 1008000 1200000 1416000 1608000 1800000
```

3.设置为用户模式，修改频率为 1800000。

```
root@elf2-buildroot:~# echo userspace > /sys/devices/system/cpu/cpu0/cpufreq/scali
ng_governor
root@elf2-buildroot:~# echo 1800000 > /sys/devices/system/cpu/cpu0/cpufreq/scalin
```

```
g_setspeed
```

4.查看修改后当前频率。

```
root@elf2-buildroot:~# cat /sys/devices/system/cpu/cpu0/cpufreq/cpuinfo_cur_freq
1800000
```

3.1.12. 温度测试

查看 CPU 当前温度值。

```
root@elf2-buildroot:~# cat /sys/class/thermal/thermal_zone0/temp
63769                                                    //温度值即为 63.7℃。
```

查看内核中 cpu 降频温度值。

```
root@elf2-buildroot:~# cat /sys/devices/virtual/thermal/thermal_zone0/trip_point_0_te
mp
75000                                                    //温度值 75℃
```

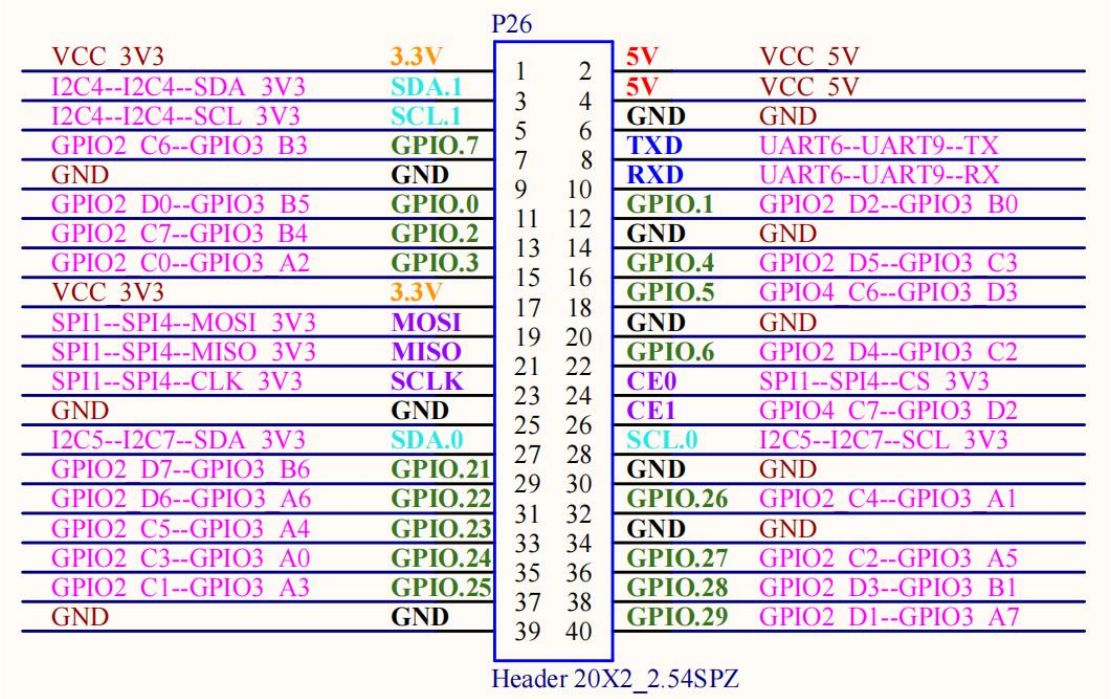
查看内核中 cpu 重启温度值。

```
root@elf2-buildroot:~# cat /sys/devices/virtual/thermal/thermal_zone0/trip_point_1_te
mp
85000                                                    //温度值 85℃
```

3.1.13. GPIO 脚本测试

⚠**注意：**GPIO引脚的电平范围为0~3.3V，严禁对此引脚接入超过3.3V的电压。

ELF 2 板卡底板原理图中引出扩展 IO 引脚，位于底板 P26。



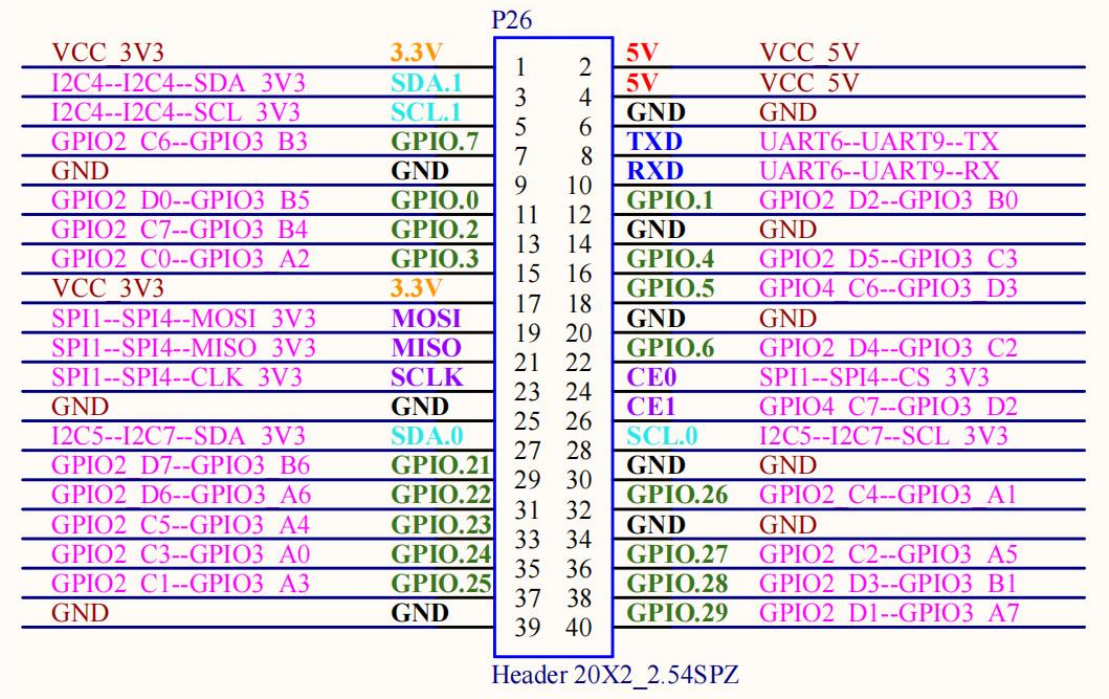
（原理图左侧是3576核心板引脚，右侧是3588核心板引脚）

以 GPIO3_B3 为例进行测试。

```
root@elf2-buildroot:~# cmddemo_gpio.sh GPIO3_B3 1     // GPIO3_B3 拉高
root@elf2-buildroot:~# cmddemo_gpio.sh GPIO3_B3 0     //GPIO3_B3 拉低
```

3.1.14. GPIO 命令行测试

⚠**注意：**GPIO引脚的电平范围为0~3.3V，严禁对此引脚接入超过3.3V的电压。



（原理图左侧是3576核心板引脚，右侧是3588核心板引脚）

GPIO 的命名规则为 GPIOn_xy 其中 x 的表现形式有 ABCD 四种。在 GPIO 计算编号的过程中，A 对应 1，B 对应 2，C 对应 3，D 对应 4。

计算公式为：

$$GPIO_{n_xy} = n \times 32 + （x - 1）\times 8 + y$$

以 GPIO3_B3 为例，计算其 GPIO 编号。

$$\text{GPIO3_B3} = 3 \times 32 + (2 - 1) \times 8 + 3 = 107$$

将 GPIO 引脚配置为输出模式后，在完成相关设置操作后，可使用万用表测量该引脚的电压状态，以验证 GPIO 输出模式是否配置成功。。

```
root@elf2-buildroot:~# echo 107 > /sys/class/gpio/export        //系统需要导出控制的
GPIO 引脚编号
root@elf2-buildroot:~# echo out > /sys/class/gpio/gpio107/direction //设置为输出
root@elf2-buildroot:~# echo 1 > /sys/class/gpio/gpio107/value //设置 IO 输出高电平
root@elf2-buildroot:~# echo 0 > /sys/class/gpio/gpio107/value //设置 IO 输出低电平
root@elf2-buildroot:~# echo 107 > /sys/class/gpio/unexport      //系统取消导出
```

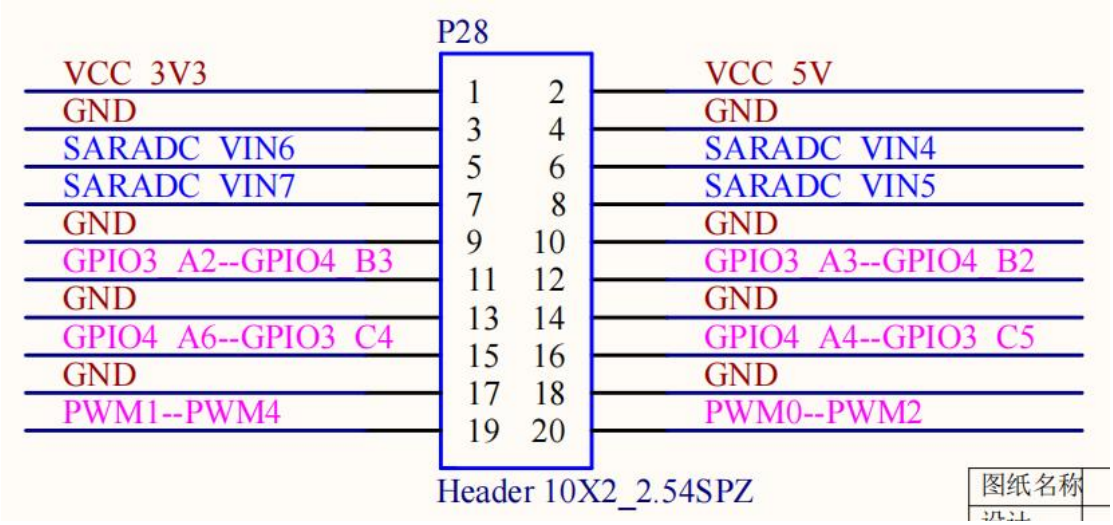
将 GPIO 引脚配置为输入模式后，通过向该引脚施加高电平或低电平信号，随后可以使用 cat 命令读取并查看该引脚的当前电平状态。

```
root@elf2-buildroot:~# echo 107 > /sys/class/gpio/export        //系统需要导出控制的
GPIO 引脚编号
root@elf2-buildroot:~# echo in > /sys/class/gpio/gpio107/direction //设置为输出
root@elf2-buildroot:~# cat /sys/class/gpio/gpio107/value          //查看引脚状态
root@elf2-buildroot:~# echo 107 > /sys/class/gpio/unexport      //系统取消导出
```

3.1.15. PWM 测试

⚠**注意：**PWM引脚的电平范围为0~3.3V，严禁对此引脚接入超过3.3V的电压。

ELF 2底板上引出2路PWM接口，位于底板P28。



启动开发板，查看 pwm 节点：

```
root@elf2-buildroot:~# ls /sys/class/pwm/
pwmchip0   pwmchip1   pwmchip2
```

由于 pwm 驱动中生成的节点是依次向下枚举的，所以 pwmchip0 节点对应的是设备树中的 pwm2，pwmchip1 节点对应的是设备树中的 pwm4，pwmchip2 节点对应的是设备树中的 pwm5。

下面以测试 pwm2 为例，设置 PWM 为输出：

```
root@elf2-buildroot:~# echo 0 > /sys/class/pwm/pwmchip0/export
```

设置 PWM 周期，单位为 ns。

```
root@elf2-buildroot:~# echo 1000000 > /sys/class/pwm/pwmchip0/pwm0/period
```

设置 PWM 占空比为 50%。

```
root@elf2-buildroot:~# echo 500000 > /sys/class/pwm/pwmchip0/pwm0/duty_cycle
```

使能 PWM 功能。

```
root@elf2-buildroot:~# echo 1 > /sys/class/pwm/pwmchip0/pwm0/enable
```

失能 PWM 功能。

```
root@elf2-buildroot:~# echo 0 > /sys/class/pwm/pwmchip0/pwm0/enable
```

3.1.16. UART 测试

⚠**注意**：UART引脚的电平范围为0~3.3V，严禁对此引脚接入超过3.3V的电压。

🗁 路径：ELF 2 开发板资料包\05-常用工具\04-测试工具\SerialTool1.4.0a_Win32_Setup.zip

ELF 2 板卡底板原理图中标示引出的 UART2、UART9 共 2 路串口，其中 UART2 为调试串口。UART9 在开发板中的默认设备名称为 ttyS9。

UART	设备节点	说明
UART2	/dev/ttyFIQ0	调试串口，不能直接用于该测试
UART9	/dev/ttyS9	TTL 电平，可用于测试

本次测试采用 UART9(ttyS9)，分别对应 PIN8，PIN10。通过开发板的 UART 和电脑串口工具软件之间的数据收发，来进行串口测试。

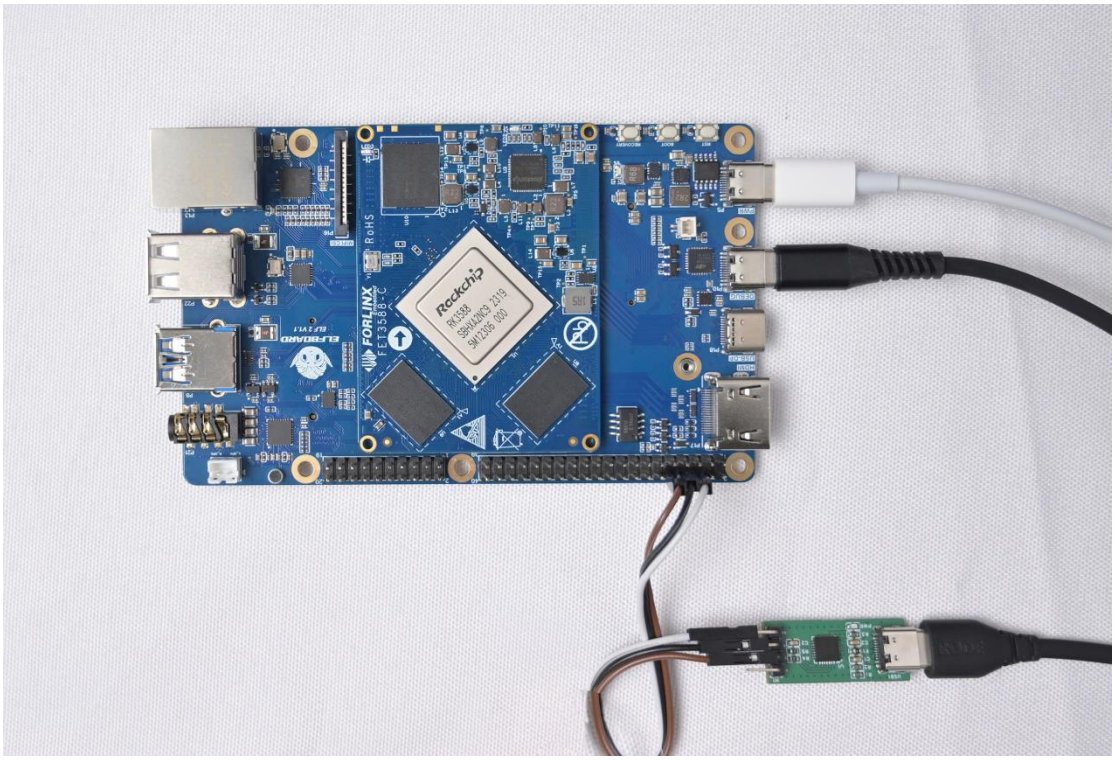
P26

VCC_3V3	3.3V	1	2	5V	VCC_5V
I2C4--I2C4--SDA_3V3	SDA.1	3	4	5V	VCC_5V
I2C4--I2C4--SCL_3V3	SCL.1	5	6	GND	GND
GPIO2_C6--GPIO3_B3	GPIO.7	7	8	TXD	UART6--UART9--TX
GND	GND	9	10	RXD	UART6--UART9--RX
GPIO2_D0--GPIO3_B5	GPIO.0	11	12	GPIO.1	GPIO2_D2--GPIO3_B0
GPIO2_C7--GPIO3_B4	GPIO.2	13	14	GND	GND
GPIO2_C0--GPIO3_A2	GPIO.3	15	16	GPIO.4	GPIO2_D5--GPIO3_C3
VCC_3V3	3.3V	17	18	GPIO.5	GPIO4_C6--GPIO3_D3
SPI1--SPI4--MOSI_3V3	MOSI	19	20	GND	GND
SPI1--SPI4--MISO_3V3	MISO	21	22	GPIO.6	GPIO2_D4--GPIO3_C2
SPI1--SPI4--CLK_3V3	SCLK	23	24	CE0	SPI1--SPI4--CS_3V3
GND	GND	25	26	CE1	GPIO4_C7--GPIO3_D2
I2C5--I2C7--SDA_3V3	SDA.0	27	28	SCL.0	I2C5--I2C7--SCL_3V3
GPIO2_D7--GPIO3_B6	GPIO.21	29	30	GND	GND
GPIO2_D6--GPIO3_A6	GPIO.22	31	32	GPIO.26	GPIO2_C4--GPIO3_A1
GPIO2_C5--GPIO3_A4	GPIO.23	33	34	GND	GND
GPIO2_C3--GPIO3_A0	GPIO.24	35	36	GPIO.27	GPIO2_C2--GPIO3_A5
GPIO2_C1--GPIO3_A3	GPIO.25	37	38	GPIO.28	GPIO2_D3--GPIO3_B1
GND	GND	39	40	GPIO.29	GPIO2_D1--GPIO3_A7

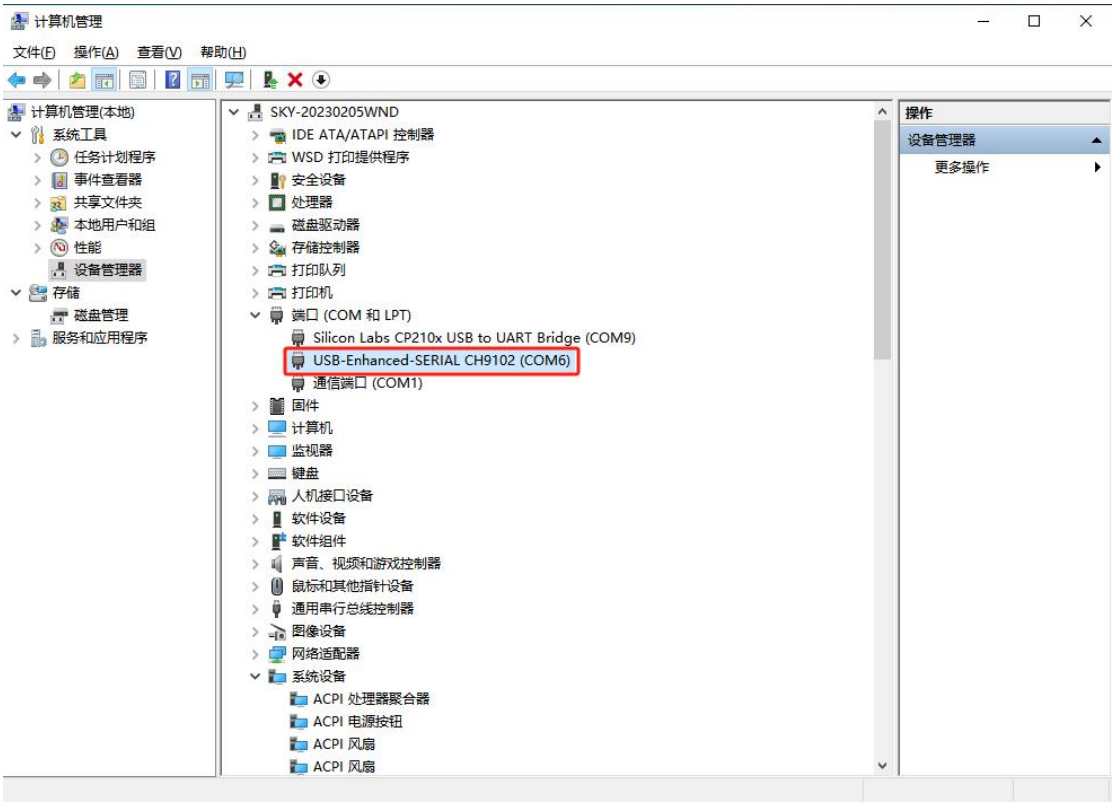
Header 20X2_2.54SPZ

（原理图左侧是3576核心板引脚，右侧是3588核心板引脚）

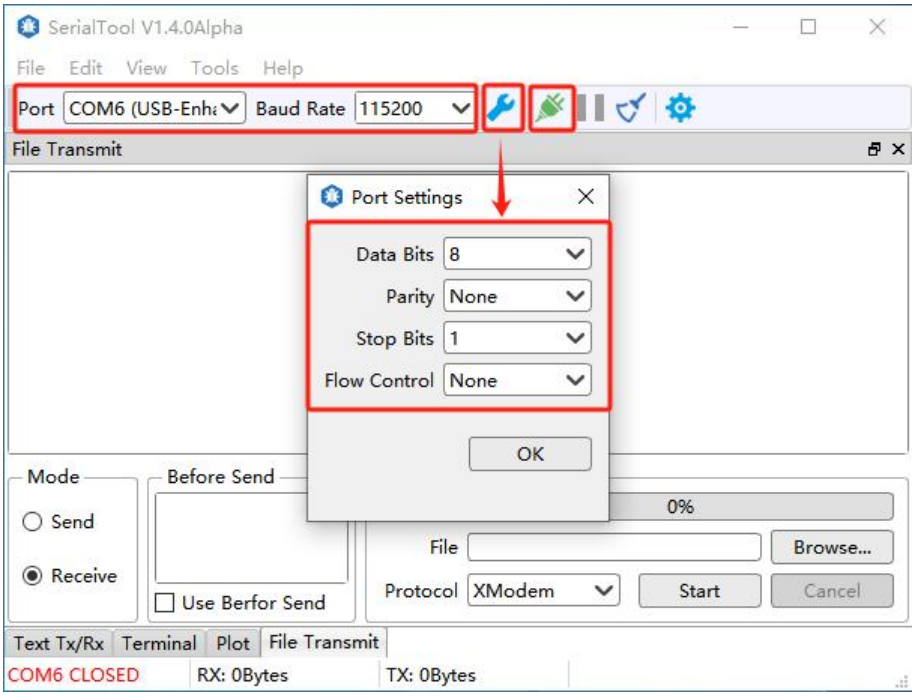
TTL 转 USB 模块与 ELF 2 硬件连接如图：



1.开发板和电脑通过 TTL 转 USB 模块连接好后，开发板上电，在电脑设备管理器查看识别为 COM6（用户以自己实际识别的 COM 口设置参数）。



2.打开电脑串口工具，选择端口 COM6、设置波特率 115200，点击蓝色端口设置图标进行设置，8 位数据位、1 位停止位、无校验、无流控制，点击 OK，点击绿色图标打开串口。



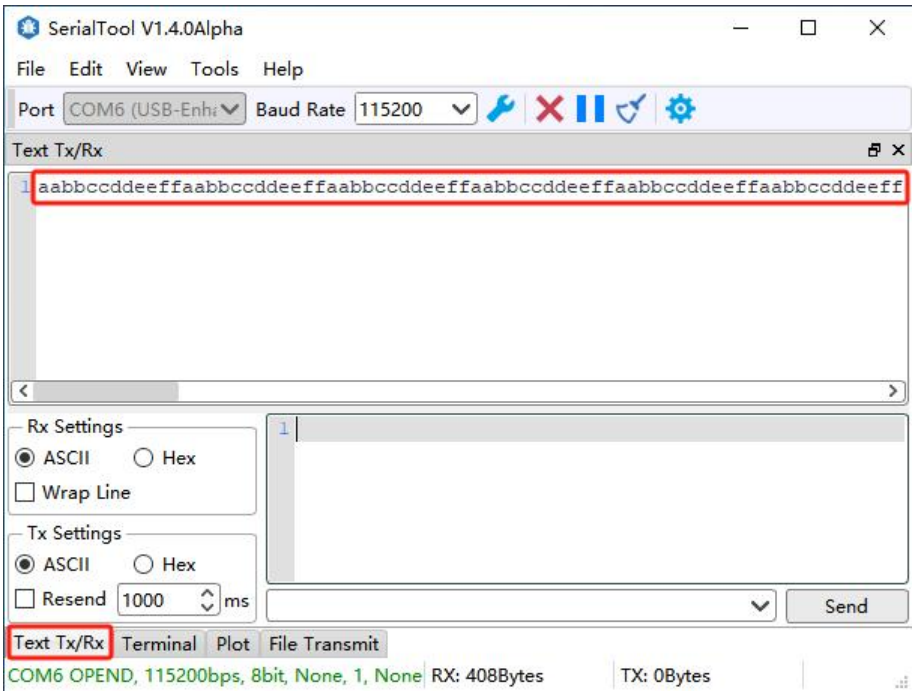
在开发板终端输入如下命令，测试程序将自动发送“aabbccddeeff”。

```
root@elf2-buildroot:~# cmddemo_serialport ttyS9 -o -b 115200 -t aabbccddeeff 1
baudrate=115200,data_bit=8,stop_bit=1,check='O'
set done!
[nwrite=12] aabbccddeeff          //发送的数据
```

说明：

‘cmddemo_serialport ttyS9 -o -b 115200 -t aabbccddeeff 1’这段代码是一个命令行 程序的命令行参数。cmddemo_serialport 程序的名称；ttyS9 表示要连接到 ttyS9 这个设备；-o 表示设置为奇校验；-b 115200 表示设置波特率为 115200；-t aabbccddeeff 表示要发送的数据为 aabbccddeeff；1 表示间隔 1 秒发送一次数据。

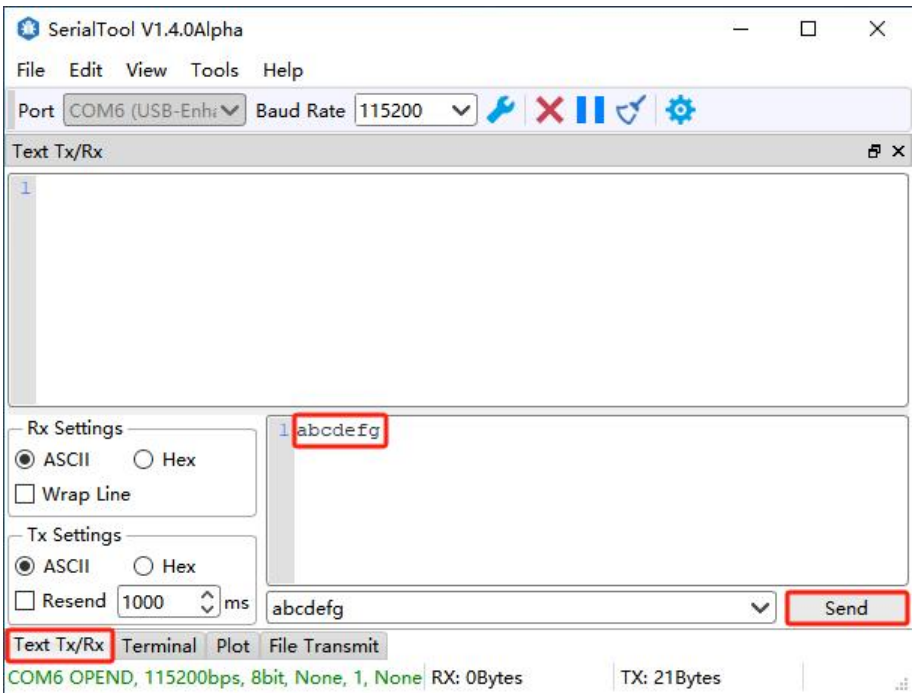
此时查看串口助手，接收到数据如下。



在开发板终端输入如下命令，开发板接收串口工具发来的数据。

```
root@elf2-buildroot:~# cmddemo_serialport ttyS9 -b 115200
```

电脑串口工具发送“abcdefg”。



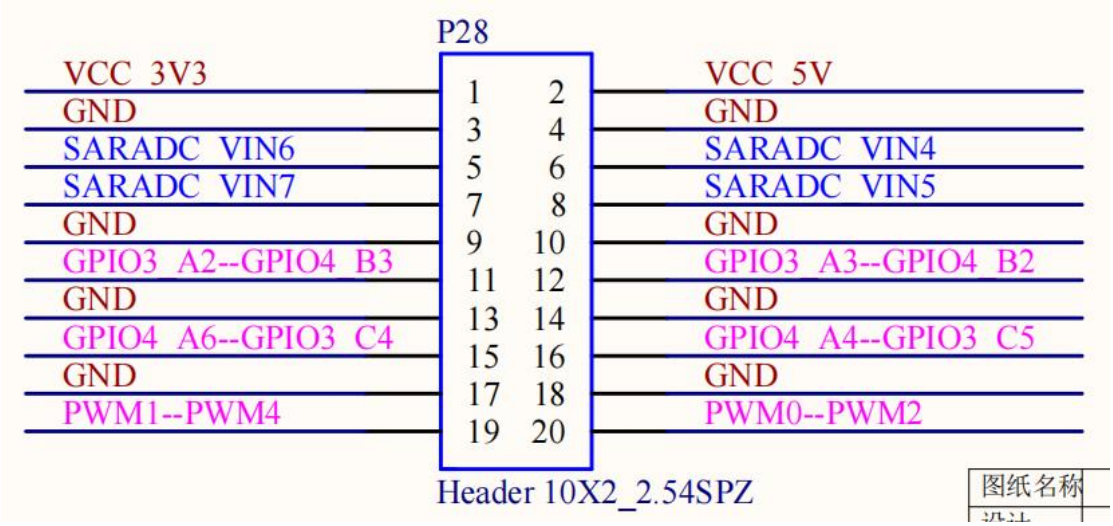
此时开发板接收到数据。

```
baudrate=115200,data_bit=8,stop_bit=1,check='N'
set done!
[nread=7] abcdefg                    //接收到的数据
```

3.1.17. ADC 测试

⚠**注意：**ADC引脚的电平范围为0~1.8V，严禁对此引脚接入超过1.8V的电压。

ELF 2开发板内部提供了8路ADC，ADC引脚硬件原理图如下。



（原理图左侧是3576核心板引脚，右侧是3588核心板引脚）

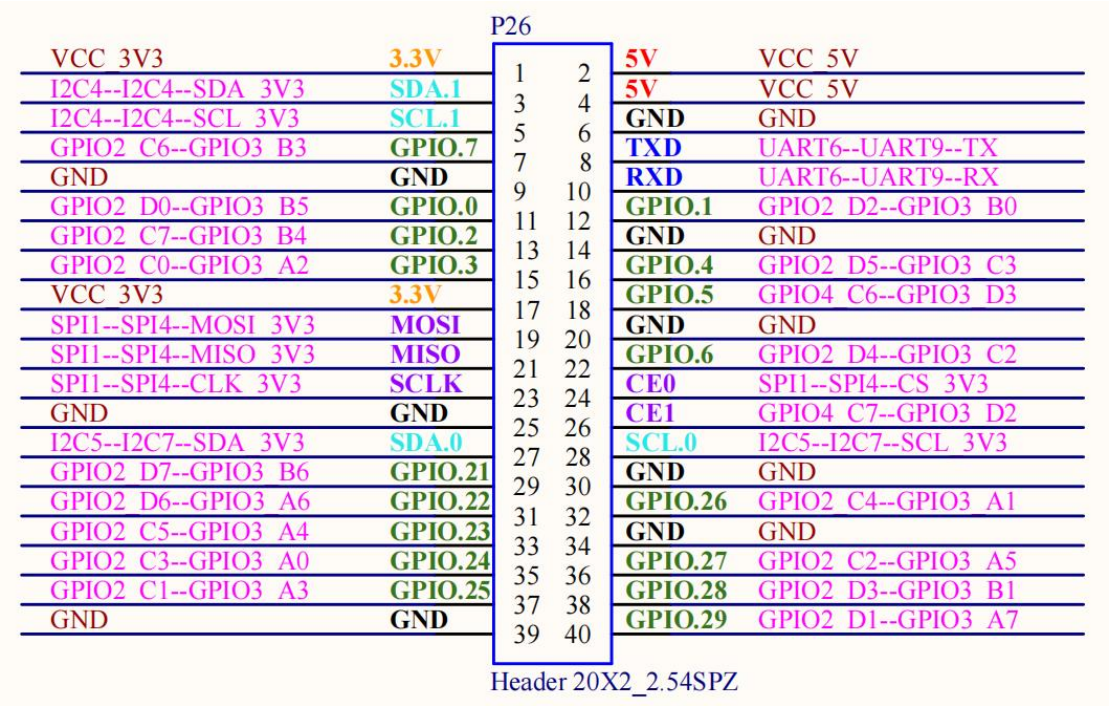
在P28的5引脚输入电压，当前芯片使用1.8V参考电压对应12位ADC最大值4096。

```
root@elf2-buildroot:~# cd /sys/bus/iio/devices/iio:device0
root@elf2-buildroot:/sys/bus/iio/devices/iio:device0# cat in_voltage6_raw
2813
```

3.1.18. SPI 测试

⚠**注意：**SPI引脚的电平范围为0~3.3V，严禁对此引脚接入超过3.3V的电压。

ELF 2 底板上引出 1 路 SPI 接口，默认软件上将其配置为 spidev 用于回环测试。测试时，请参考原理图短接 MOSI (PIN19)与 MISO (PIN21)，然后分别使用下面的命令进行测试。



（原理图左侧是3576核心板引脚，右侧是3588核心板引脚）

未短接 MOSI 与 MISO 时，执行测试指令：

```
root@elf2-buildroot:~# cmddemo_spi spidev4.0
[/dev/spidev4.0 nwrite=38] 0xff 0xff 0xff 0xff 0xff 0xff 0x40 0x00 0x00
0x00 0x00 0x95 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff
0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xde 0xad 0xbe 0xef 0xba
0xad 0xf0 0x0d
[/dev/spidev4.0 nread=38 ] 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff
0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff
0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff
0xff
```

短接 MOSI 与 MISO 时，执行测试指令：

```
root@elf2-buildroot:~# cmddemo_spi spidev4.0
[/dev/spidev4.0 nwrite=38] 0xff 0xff 0xff 0xff 0xff 0xff 0x40 0x00 0x00
0x00 0x00 0x95 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff
0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xde 0xad 0xbe 0xef 0xba
0xad 0xf0 0x0d
[/dev/spidev4.0 nread=38 ] 0xff 0xff 0xff 0xff 0xff 0xff 0x40 0x00 0x00
0x00 0x00 0x95 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff
0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xde 0xad 0xbe 0xef 0xba
0xad 0xf0 0x0d
 0xff
```

3.1.19. TYPE-C 测试

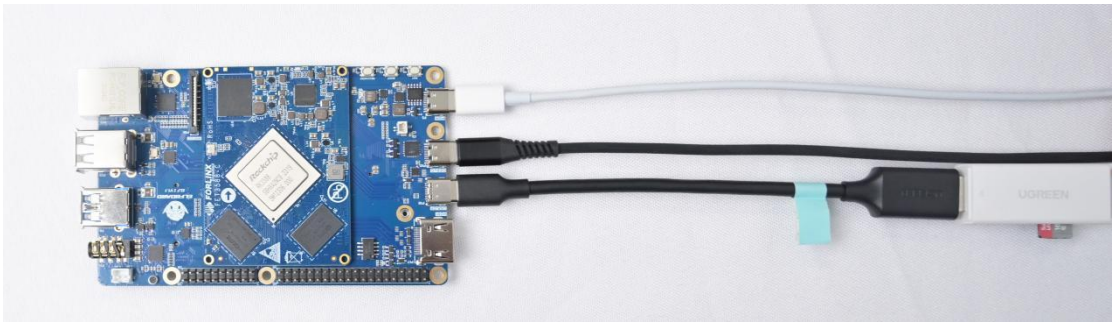
ELF 2 包含 1 个 TYPE-C 接口，USB-DP 的 HOST/DEVICE 模式自动识别。Device 模式可以用它来进行刷机、ADB 文件传输、调试，Host 模式可以插入普通的 USB 设备。

Device 模式：

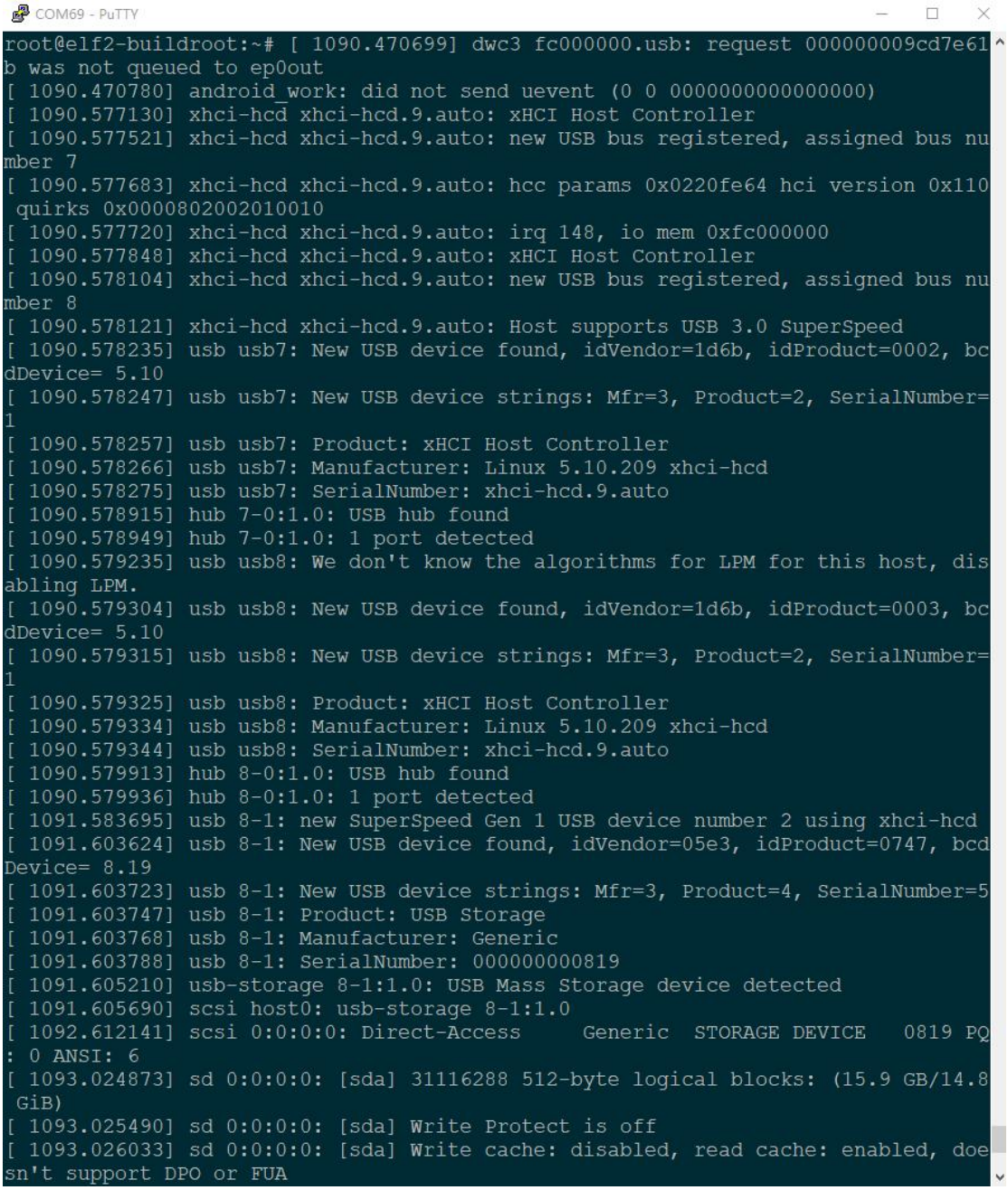


Host 模式：

OTG 转 USB 模块与 ELF 2 硬件连接如图：

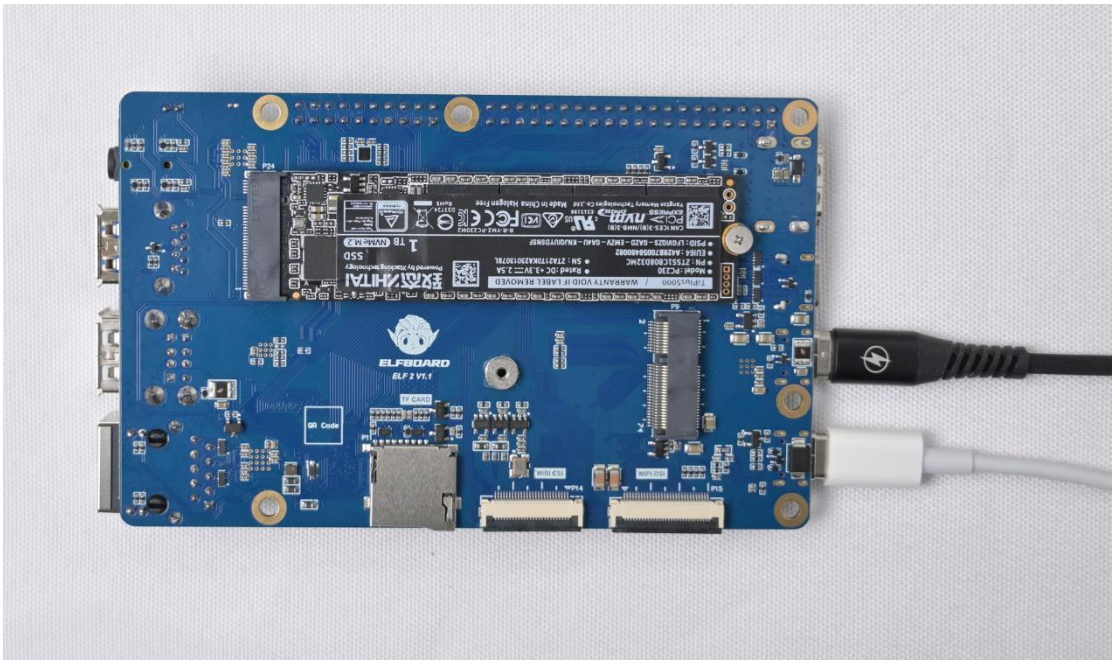


模块与 ELF 2 连接后的打印信息如图所示。



3.1.20. PCIE 测试

ELF 2 板卡有 1 个 PCIE2.0 接口，PCIE 模块与 ELF 2 硬件连接如图：



系统上电前将 PCIE 模块插入底板 PCIE 卡槽。上电后启动后，通过 lspci 可以看到对应设备枚举成功。

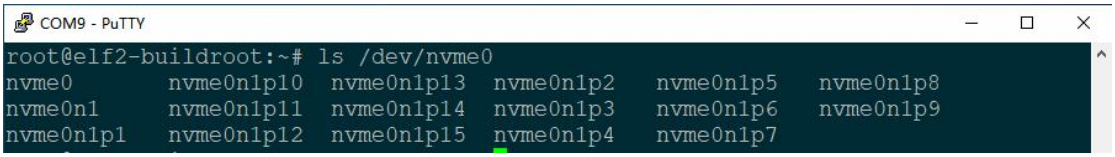
```
root@elf2-buildroot:~# lspci
```



由于 pcie 设备类型较多，有可能默认不被内核支持需自行添加编译设备对应的驱动程序。

以 PCIE 硬盘举例，linux 内核默认已经包含该驱动。插入 PCIE 硬盘，此时/dev 目录下有 nvme 设备。

```
root@elf2-buildroot:~# ls /dev/nvme0
```



3.1.21. RKNPU 测试

linux 文件系统里已有 rknpu2 例子，主要用于测试并评估深度学习模型的性能，下面以 mobilenet_v1.rknn 为例进行测试：

```
root@elf2-buildroot:~# /usr/bin/rknn_common_test /usr/share/model/RK3588/mobilenet_v1.rknn /usr/share/model/dog_224x224.jpg
rrknn_api/rknnrt version: 2.0.0b0 (35a6907d79@2024-03-24T10:31:14), driver version: 0.9.8
model input num: 1, output num: 1
input tensors:
  index=0, name=input, n_dims=4, dims=[1, 224, 224, 3], n_elems=150528, size=150528, fmt=NHWC, type=INT8, qnt_type=AFFINE, zp=0, scale=0.007812
output tensors:
  index=0, name=MobilenetV1/Predictions/Reshape_1, n_dims=2, dims=[1, 1001, 0, 0], n_elems=1001, size=2002, fmt=UNDEFINED, type=FP16, qnt_type=AFFINE, zp=0,
```

```
scale=1.000000
custom string:
Begin perf ...
    0: Elapse Time = 3.45ms, FPS = 289.77
---- Top5 ----
0.884766 - 156
0.054016 - 155
0.003677 - 205
0.002974 - 284
0.000189 - 285
```

/usr/bin/rknn_common_test 这是要执行的程序或脚本的路径，它是 RKNN 的测试工具/usr/share/model/RK3588/mobilenet_v1.rknn 这是需要加载的模型文件的路径，/usr/share/model/dog_224x224.jpg 这是要进行推理的输入图像的路径。

最后的打印信息是模型对输入图像的前五个预测结果。每行包含一个置信度（probability）和相应的类别索引：0.884766 - 156：第 156 类的置信度为 0.884766，说明模型认为输入图像最有可能属于第 156 类。其他类的置信度依次递减。根据 ImageNet 的分类，156 类通常代表“杂种犬”（Mutt）

这个输出显示了模型对一张 224x224 尺寸的狗的图片进行推理的结果。模型性能良好，耗时较短且预测结果显示模型能够有效地识别出输入图像的内容。

3.1.22. DDR 测试

通过不同的操作模式测试 L1 和 L2 缓存及主内存的带宽性能，评估、验证系统性能。

```
root@elf2-buildroot:~# cmddemo_memory_bandwidth.sh
L1 cache bandwidth rd test with # process
0.008192 53077.30
0.008192 53109.36
0.008192 30713.00
0.008192 53243.57
0.008192 26197.53
L2 cache bandwidth rd test
0.131072 42252.23
0.131072 42257.54
0.131072 42282.83
0.131072 30646.64
0.131072 19162.98
Main mem bandwidth rd test
52.43 17733.40
52.43 17982.78
52.43 17263.35
52.43 17567.03
52.43 17817.77
L1 cache bandwidth wr test with # process
0.008192 24231.81
0.008192 68753.47
0.008192 64734.99
```

```
0.008192 69563.44
0.008192 69366.16
L2 cache bandwidth wr test
0.131072 41123.04
0.131072 42969.36
0.131072 12731.06
0.131072 42018.18
0.131072 18280.54
Main mem bandwidth wr test
52.43 6598.97
52.43 7158.49
52.43 6817.79
52.43 6950.66
52.43 6872.30
L1 cache bandwidth rdwr test with # process
0.008192 34801.55
0.008192 18774.56
0.008192 34837.09
0.008192 34990.45
0.008192 20442.82
L2 cache bandwidth rdwr test
0.131072 22834.49
0.131072 20806.62
0.131072 24274.35
0.131072 24254.98
0.131072 24164.77
Main mem bandwidth rdwr test
52.43 8607.58
52.43 9198.04
52.43 9416.09
52.43 9335.61
52.43 9188.36
L1 cache bandwidth cp test with # process
0.008192 34832.65
0.008192 34856.86
0.008192 34969.59
0.008192 26067.97
0.008192 35024.26
L2 cache bandwidth cp test
0.131072 10263.12
0.131072 21817.55
0.131072 11734.61
0.131072 21205.44
0.131072 11917.79
```

```
Main mem bandwidth cp test
52.43 5272.94
52.43 5308.17
52.43 5679.03
52.43 5303.87
52.43 5708.09
L1 cache bandwidth frd test with # process
0.008192 10540.67
0.008192 10562.27
0.008192 3547.92
0.008192 10556.53
0.008192 9362.52
L2 cache bandwidth frd test
0.131072 10499.98
0.131072 10502.03
0.131072 5674.01
0.131072 10510.49
0.131072 10490.52
Main mem bandwidth frd test
52.43 9246.70
52.43 9768.73
52.43 9181.93
52.43 8817.49
52.43 8640.21
L1 cache bandwidth fwr test with # process
0.008192 29601.12
0.008192 29595.51
0.008192 29650.00
0.008192 29648.37
0.008192 29550.67
L2 cache bandwidth fwr test
0.131072 8407.58
0.131072 26596.74
0.131072 26565.13
0.131072 25834.96
0.131072 17250.02
Main mem bandwidth fwr test
52.43 3356.30
52.43 3346.02
52.43 3362.98
52.43 3354.37
52.43 3351.37
L1 cache bandwidth fcp test with # process
0.008192 6843.27
```

```
0.008192 6439.21
0.008192 7872.04
0.008192 8766.33
0.008192 8798.43
L2 cache bandwidth fcp test
0.131072 7967.30
0.131072 8700.10
0.131072 8698.26
0.131072 8695.49
0.131072 4814.99
Main mem bandwidth fcp test
52.43 8068.45
52.43 8161.39
52.43 8259.11
52.43 8279.97
52.43 8320.71
L1 cache bandwidth bzero test with # process
0.008192 68207.27
0.008192 68197.32
0.008192 68064.08
0.008192 68257.76
0.008192 60286.06
L2 cache bandwidth bzero test
0.131072 55157.59
0.131072 46893.40
0.131072 47043.90
0.131072 66726.66
0.131072 38708.91
Main mem bandwidth bzero test
52.43 26900.36
52.43 26951.06
52.43 25700.39
52.43 26794.96
52.43 27516.86
L1 cache bandwidth bcopy test with # process
0.008192 34491.54
0.008192 34578.02
0.008192 34430.14
0.008192 8066.89
0.008192 34365.06
L2 cache bandwidth bcopy test
0.131072 31745.49
0.131072 31762.77
0.131072 31700.25
```

```
0.131072 31797.41
0.131072 31793.75
Main mem bandwidth bcopy test
52.43 11155.06
52.43 11356.83
52.43 11303.38
52.43 11171.70
52.43 11289.58
```

在 cmddemo_memory_bandwidth.sh 脚本中，包含一系列操作选项（rd、wr、rdwr、cp、frd、fwr、fcp、bzero 和 bcopy），这些选项表示不同的内存访问模式。这些操作选项用于 L1 缓存带宽测试、L2 缓存带宽测试、主内存带宽测试。

3.1.23. eMMC 测试

下面简单测试 eMMC 的读写速度，以读写 ext4 文件系统为例。

1.写入测试：

```
root@elf2-buildroot:~# dd if=/dev/zero of=/test bs=1M count=500 conv=fsync
500+0 records in
500+0 records out
524288000 bytes (524 MB, 500 MiB) copied, 2.81849 s, 186 MB/s
```

说明：

dd：这是一个用于转换和复制文件的命令。

if=/dev/zero：指定输入文件（if）。/dev/zero 是一个特殊的设备文件，读取时会不断输出零字节。

of=/test：指定输出文件（of）。这里 /test 是将数据写入的目标文件。

bs=1M：设置块大小为 1MB，意味着每次读写 1MB 的数据。

count=500：指定读取的块数为 500。因此总共要写入的数据量为 500MB（500 x 1MB）。

conv=fsync：在完成写入后，强制刷新输出文件的缓存，以确保所有数据都写入存储设备。

2.读取测试：

⚠**注意：**为确保数据准确，请重启开发板后测试读取速度。

```
root@elf2-buildroot:~# dd if=/test of=/dev/null bs=1M
500+0 records in
500+0 records out
524288000 bytes (524 MB, 500 MiB) copied, 1.6132 s, 325 MB/s
```

说明：

if=/test：指定输入文件，这里 /test 是之前写入的文件，包含了 500MB 的零数据。

of=/dev/null：指定输出文件，/dev/null 是一个特殊的设备文件，写入到它的数据会被丢弃，因此可以用来测试读取速度而不实际保存数据。

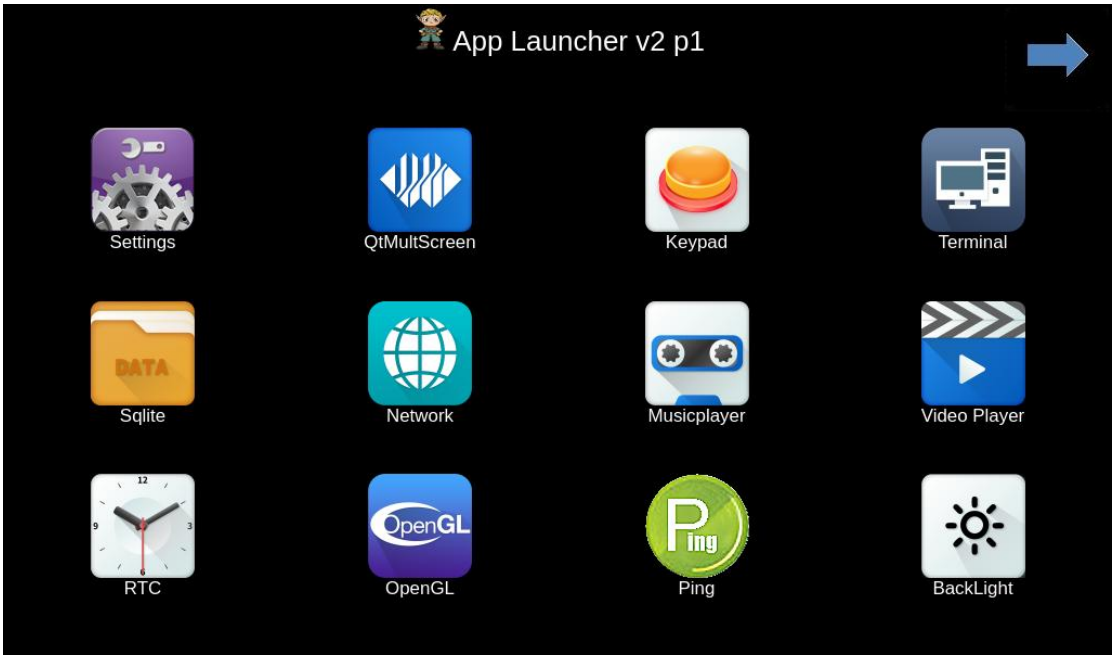
bs=1M：设置块大小为 1MB，意味着每次读取 1MB 的数据。

3.2. Qt 5.15.10 桌面功能测试

以下测试专门针对基于 Buildroot 构建的文件系统进行的验证。

3.2.1. 界面功能简介

开发板启动后桌面显示如下：

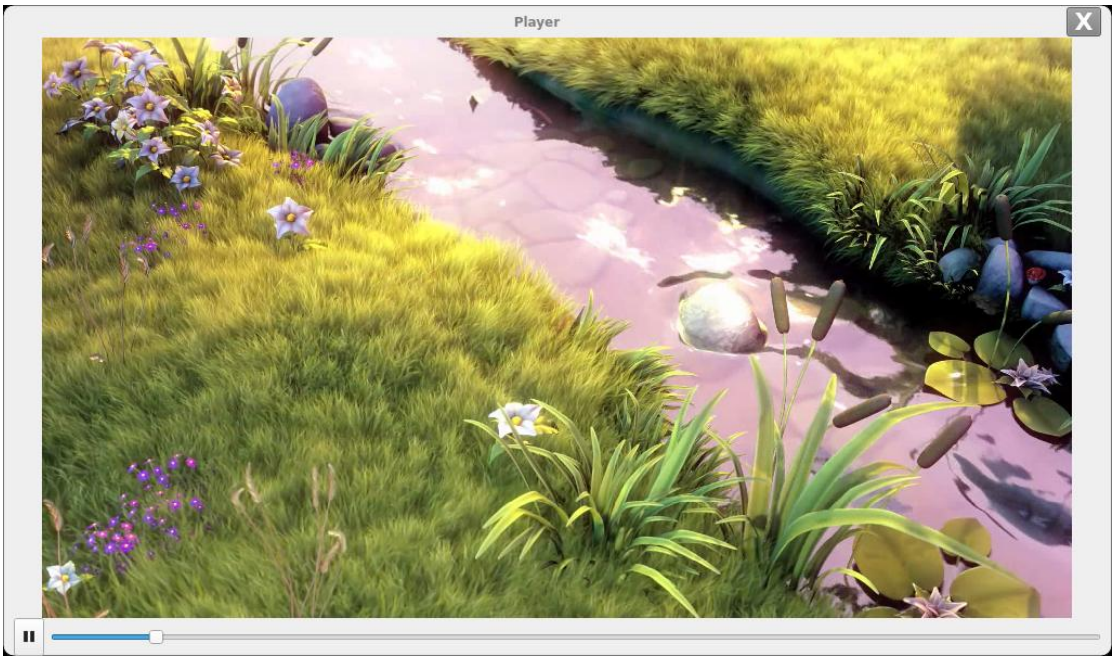


3.2.2. 硬件解码体验

点击桌面图标进入打开视频播放器 video player。



应用图标



⚠**注意：**目前使用 qt 播放视频存在卡顿问题，工程师后续会解决。

⚠**注意：**测试视频文件所在目录：/userdata/media/video/*.mp4。

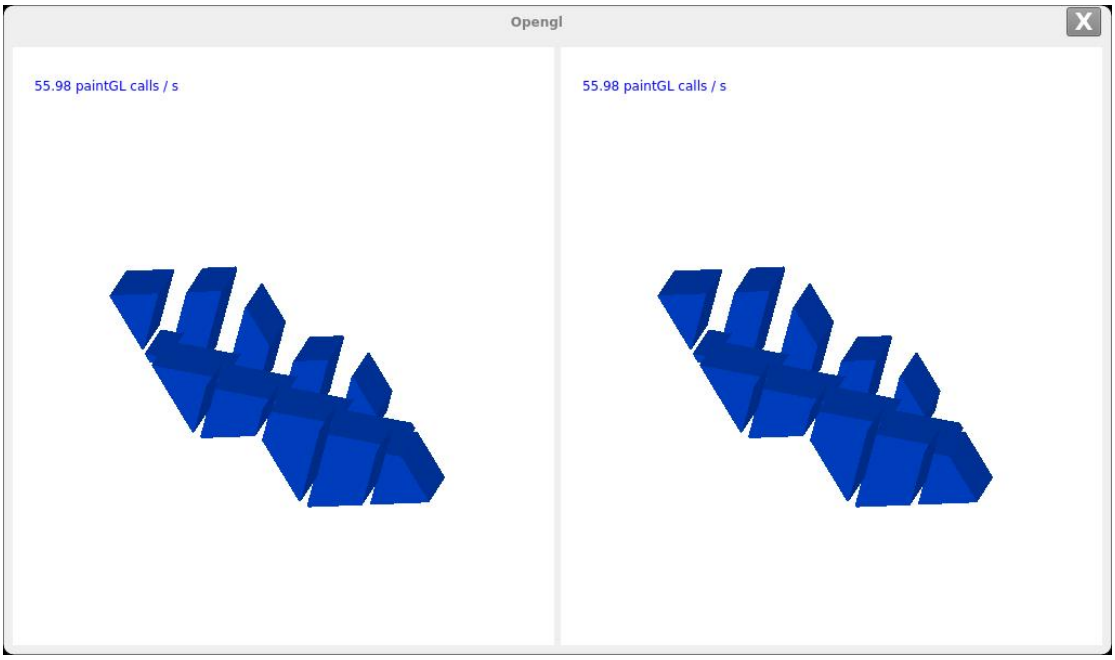
⚠**注意：**播放 8K 视频请只开一个显示屏进行测试。

3.2.3. OpenGL 测试

ELF 2 支持 OpenGL ES3.2，点击桌面图标进行 OpenGL 测试。



应用图标

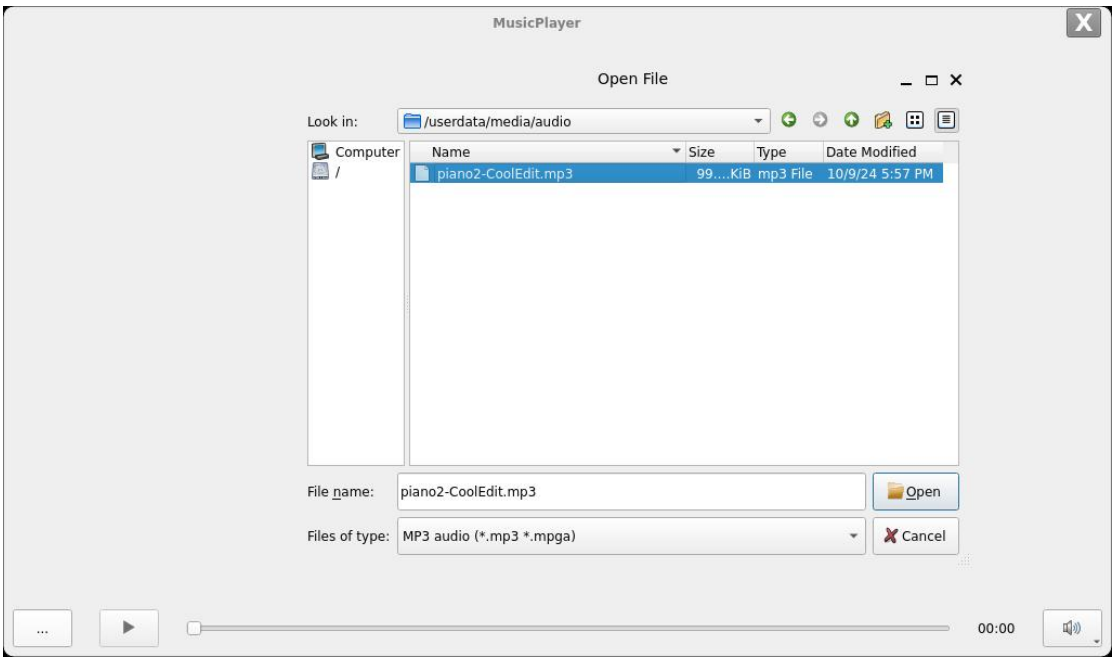


3.2.4. 播放音乐测试

“Musicplayer”是一款简单的音频测试应用，可用来测试声卡功能是否正常，也可用来作为一款简单的音频播放器。



应用图标



点击左下角三个点按钮，选择测试音频/userdata/media/audio/piano2-CoolEdit.mp3。



应用界面

⚠**注意：**默认声卡输出为 nau8822 输出，若要使用 HDMI 输出请在串口使用命令：

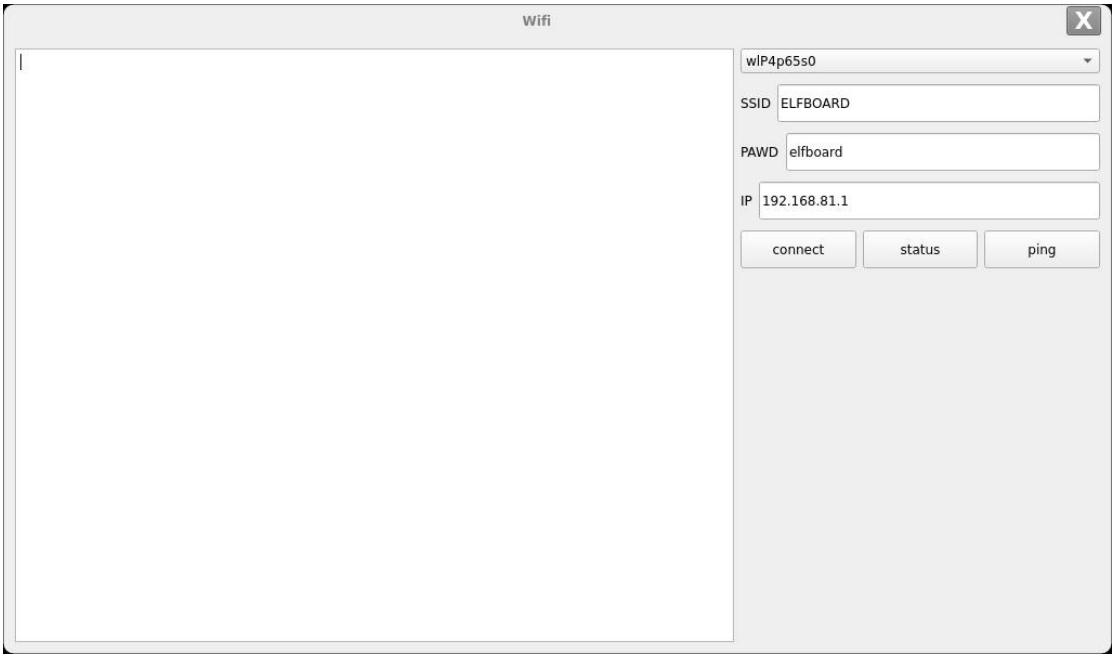
```
root@elf2-buildroot:~# gst-play-1.0 /userdata/media/piano2-CoolEdit.mp3 --audiosink="alsasink device= plughw:2,0"
```

3.2.5. WiFi 测试

“WiFi”是一款配置 WiFi 的工具，ELF 2 板卡默认板载 CF-AX200-M 模块。WiFi 模块在系统中会以 wlP4p65s0 节点的形式存在，此测试对应 wlP4p65s0（多设备时使用其它对应节点）。



应用图标

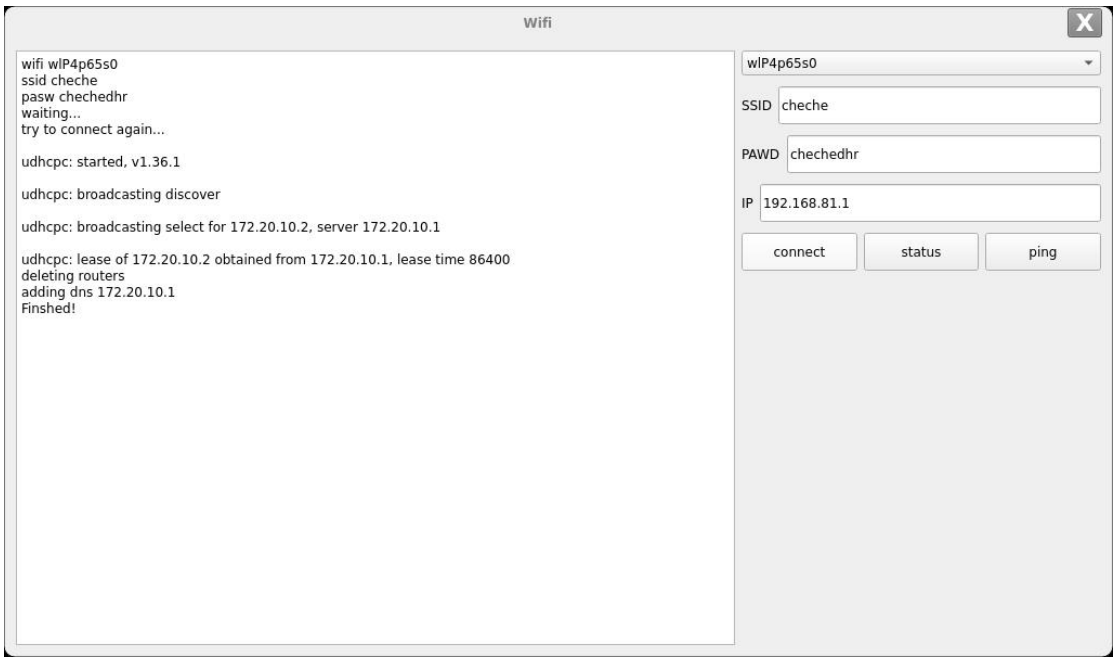


应用界面

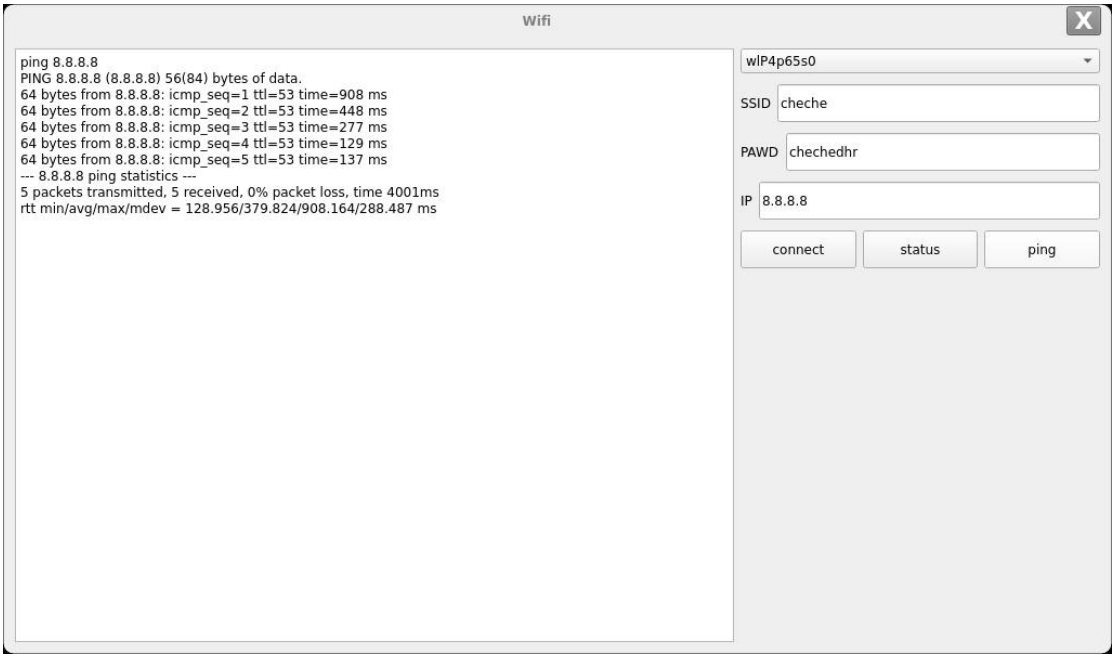
选中 wlP4p65s0，在 SSID 栏中输入需要使用 WiFi 连接的路由器名称，PAWD 栏输入路由器密码，点击 connect，即可通过 WiFi 连接路由器，在 IP 栏中输入有效 IP 后，点击 ping，可查看当前使用 WiFi 网络是否畅通。

下面以 CF-AX200-M 模块为例测试。

打开 WiFi 测试应用，输入正确的网络名称及密码，点击 connect，等待 5 秒后点击 status 查看连接窗台。



连接成功后可点击 ping 进行网络测试。



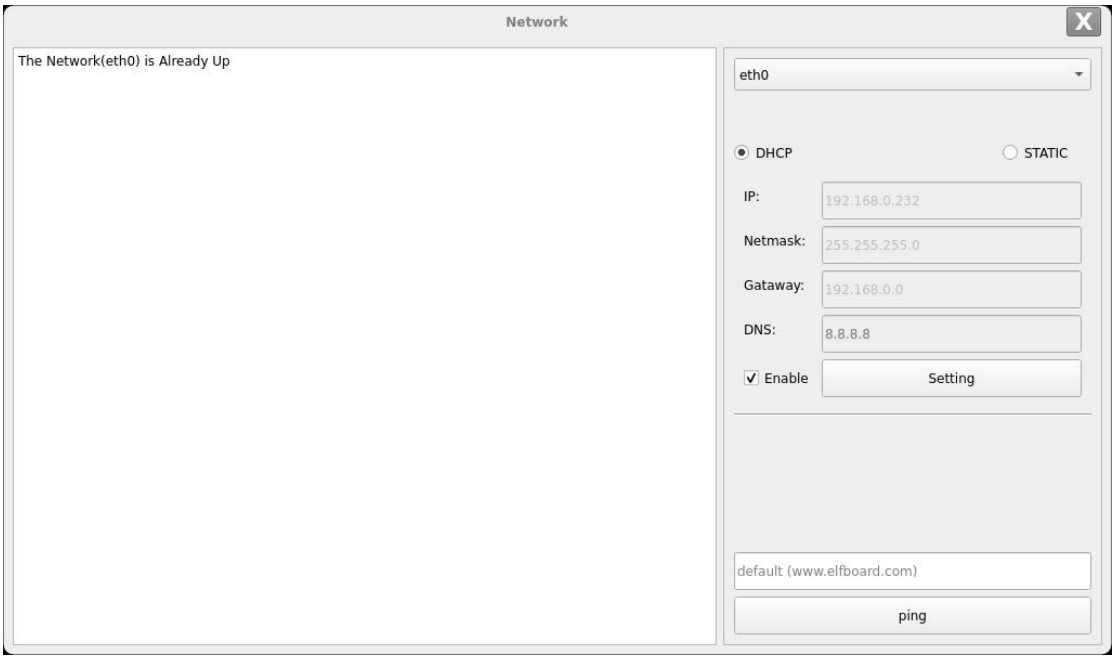
3.2.6. 网络配置测试

ELF 2 可通过“Network”网络配置应用来选择 dhcp 和 static 两种模式，static 模式可配置 IP 地址、子网掩码、网关、DNS。



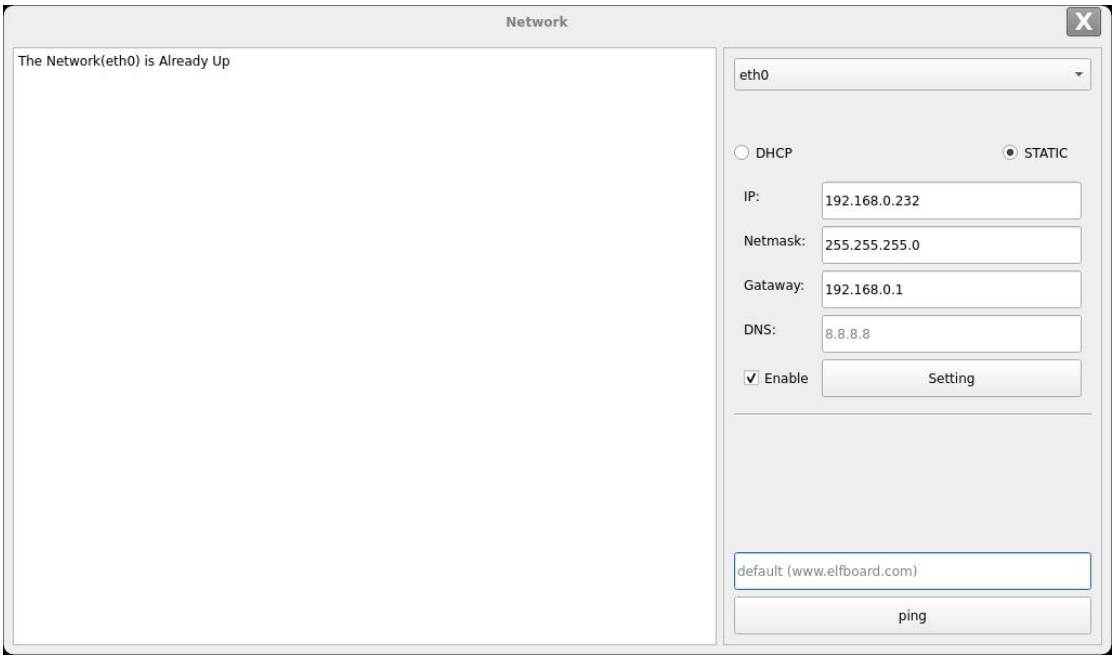
应用图标

DHCP 模式界面如下：



选中 DHCP，在 interface 中选中需要配置的网卡设备，点击界面下方的 Setting，即可重启网络并自动获取到 IP。

static 模式界面如下：



在 interface 中选中需要配置的网卡设备，在 IP 栏中输入要设定的 IP，Netmask 栏中输入子网掩码，Gataway 栏中输入网关，DNS 栏中输入 DNS。

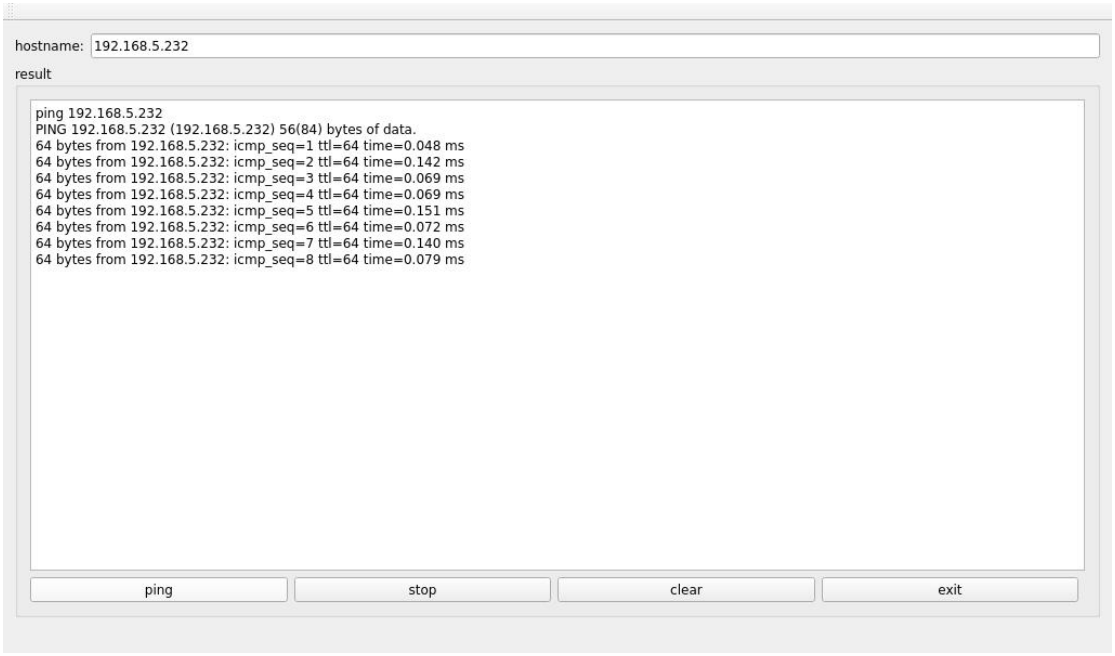
注：在 STATIC 模式下设置的 IP 等信息会被保存至系统的相关配置文件中，因此每次重启都会使用本次设置的网络信息；而在 DHCP 模式下配置的网络信息则不需注意这一点，每次重新启动都会动态分配一次 IP 地址。

3.2.7. Ping 测试

“Ping”是网络测试常用命令 ping 的界面版应用。



应用图标



应用界面

在 hostname 一栏中写入需要 ping 的目标 IP，点击 ping 按钮后，result 栏中会提示 ping 的结果，点击 stop 可停止 ping 测试，点击 clear 清空 result 中的信息。

3.2.8. 浏览器测试

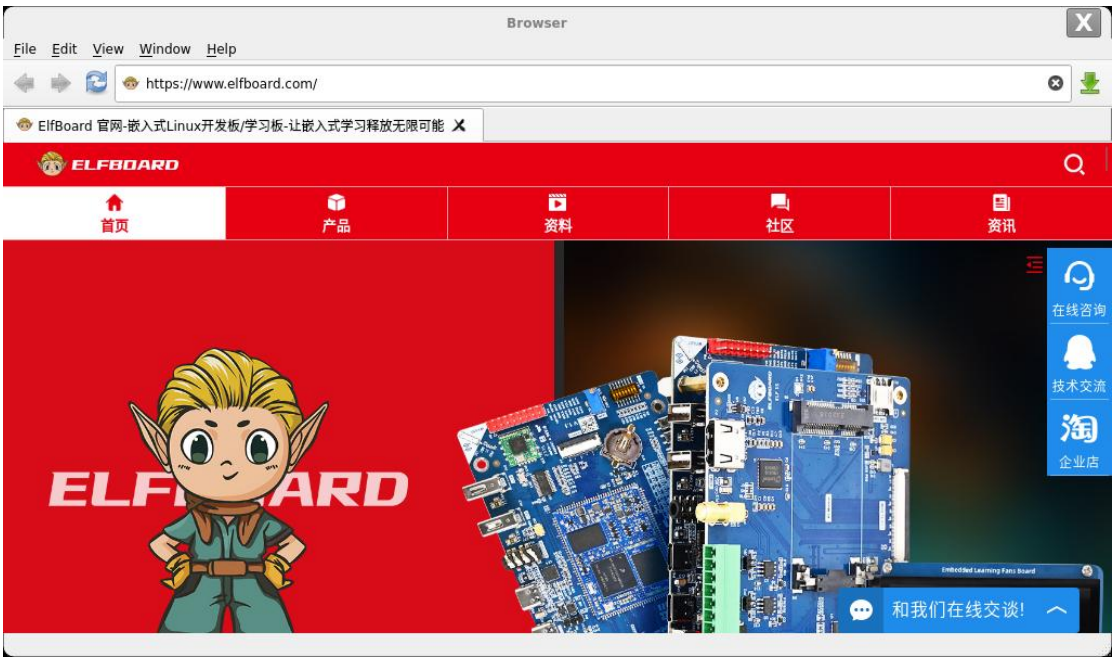
“DemoBrowser”是一款简单实用的网络浏览器，界面版应用。



应用图标

在使用时请保证网络通畅，访问外网前需保证 dns 可用，浏览器启动时默认访问 ElfBoard 官方网站。界面如下：

⚠**注意：**如果开发板的时间异常会导致证书问题。使用浏览器后不可以立即关闭电源或者在关闭电源前在命令行使用 sync 命令，否则可能导致浏览器异常退出，无法正常运行，只能重新烧录解决。



通过上方导航栏 File->Quit 退出该浏览器。

3.2.9. 看门狗测试

“WatchDog”是用来测试看门狗功能是否正常的应用。



应用图标



应用界面

点击 start，勾选 feed dog，此时看门狗功能会被启动，程序会进行喂狗操作，正常情况下系统不会重启；取消勾选 feed dog 时，看门狗功能会被启动，程序不进行喂狗操作，在打开看门狗约 5s 后，系统进入重启，说明看门狗功能正常。

3.2.10. 按键测试

“Keypad”用于测试板卡自带按键是否可用。



应用图标



应用界面

ELF 2 板卡默认将 1 个物理按键 RECOVERY 配置为音量+键。当按下开发板 RECOVERY 按键时，QT 测试应用中的按键变为蓝色，说明按键功能正常。

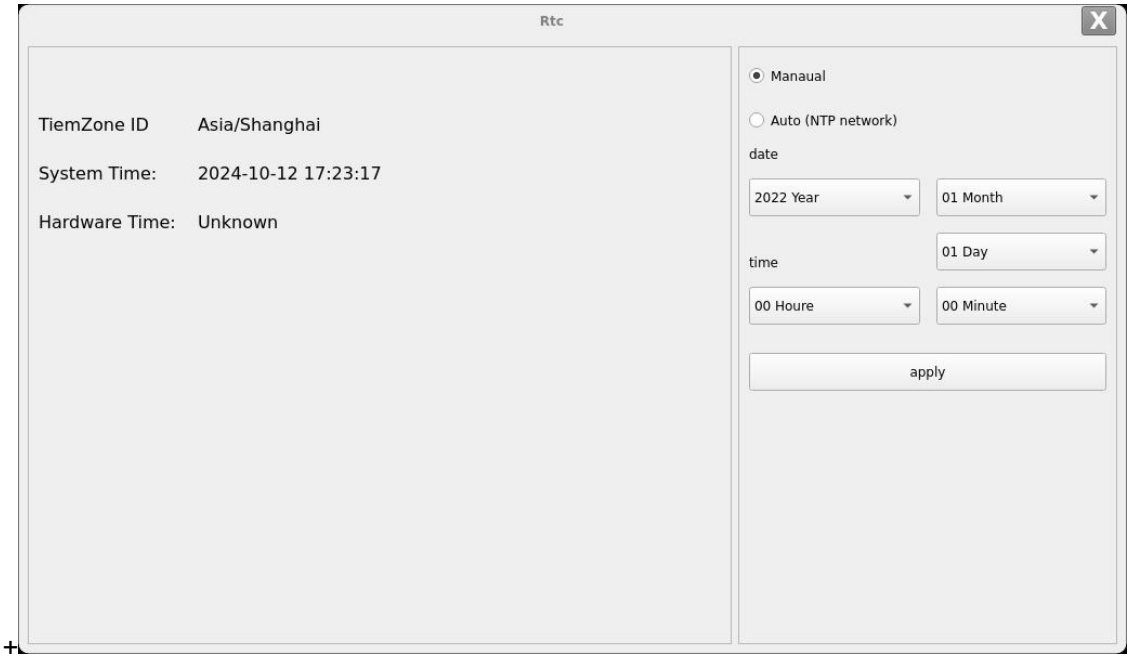


3.2.11. RTC 测试

通过“RTC”应用，可查看和设置当前的系统时间。



应用图标



应用界面

对时间进行设置，设置完成后点击 apply，即可完成设置。

在安装 RTC 备用电池的情况下，断电重启开发板以确认 RTC 时钟设置成功。

3.2.12. 同显异显

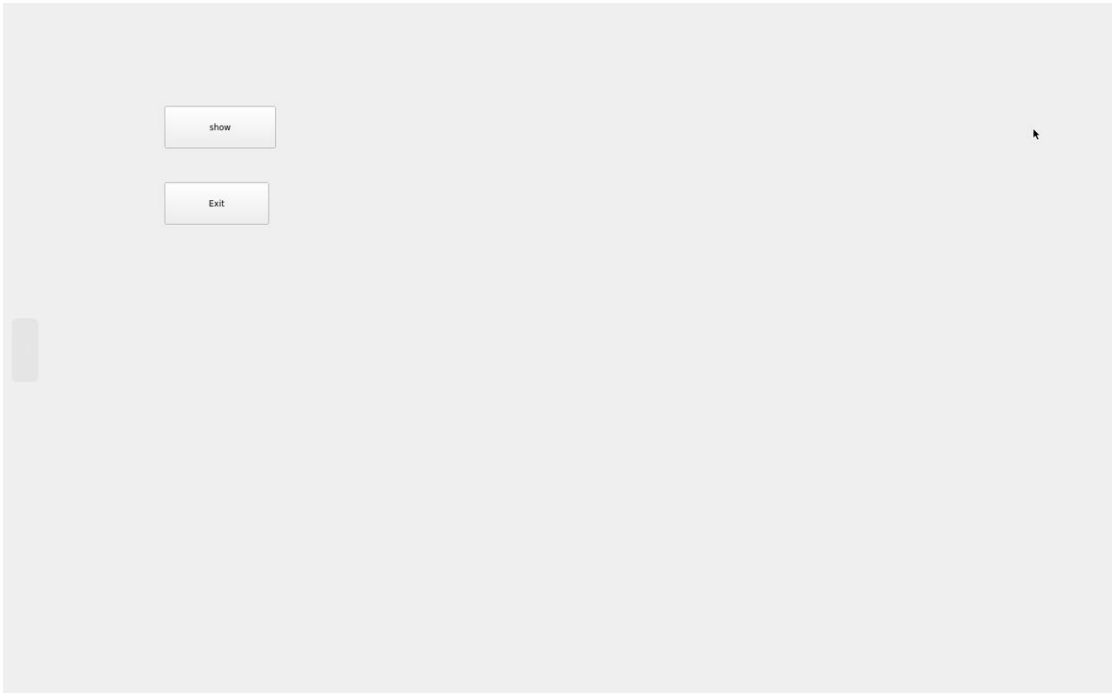
⚠**注意：**默认为同显；异显配置需要在 UBOOT 菜单将 disp_type 属性值更改成 async。

⚠**注意：**在异显情况下，配置多屏显示而没有接上显示器，QT 显示将会在相邻屏上显示，需要在 QT 里检测是否已经接屏。

点击桌面图标QtMultScreen用来测试多屏异显。



应用图标



点击“show”按钮之后将会在其它屏上弹出一个窗口，使用鼠标可以在其它屏上移动 QT 窗口。



3.2.13. UART 测试

路径：ELF 2 开发板资料包\05-常用工具\04-测试工具\SerialTool1.4.0a_Win32_Setup.zip

点击桌面图标可以使用它来测试ELF 2板载UART接口。

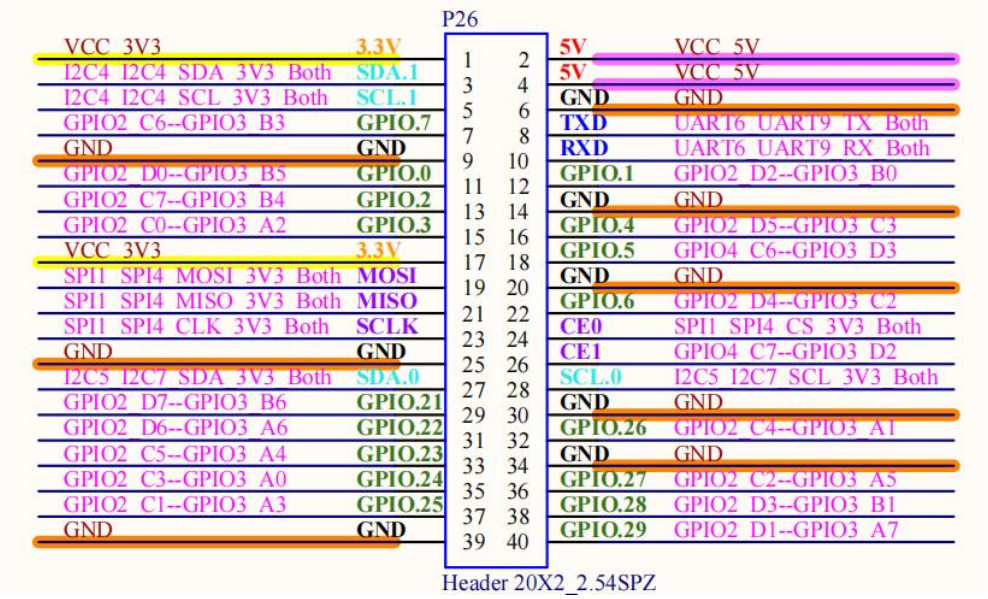


应用图标

ELF 2 板卡底板原理图中标示引出的 UART2、UART9 共 2 路串口，其中 UART2 为调试串口，UART9 在开发板中的默认设备名称为 ttyS9。

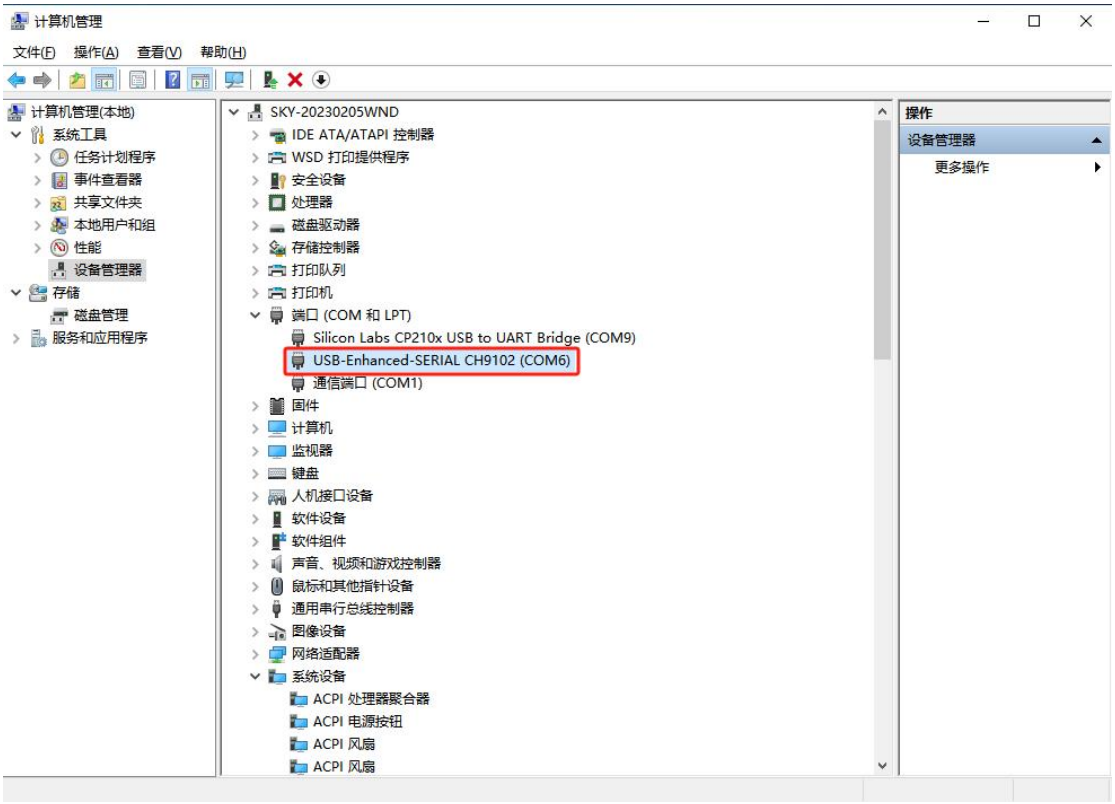
UART	设备节点	说明
UART2	/dev/ttyFIQ0	调试串口，不能直接用于该测试
UART9	/dev/ttyS9	TTL 电平，可用于测试

使用命令 qtdemo_terminal 可以打开 uart 的 qt 测试程序。本次测试采用 UART9(ttyS9)，通过开发板的 UART 和电脑串口工具软件之间的数据收发，来进行串口测试。

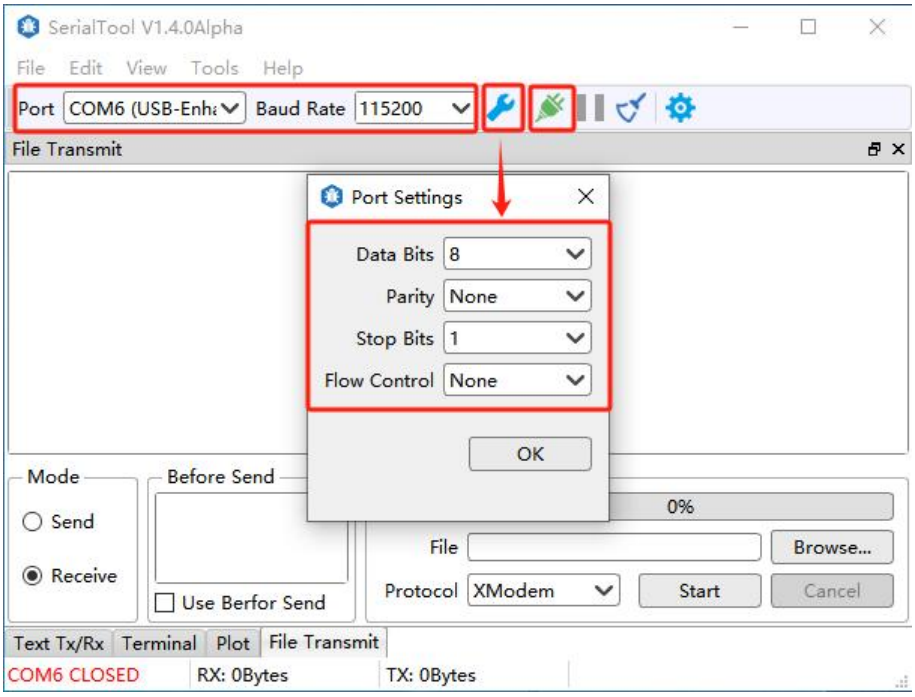


（原理图左侧是3576核心板引脚，右侧是3588核心板引脚）

1.开发板和电脑通过 TTL 转 USB 模块连接好后，给开发板上电，在电脑设备管理器查看识别为 COM74（用户以自己实际识别的 COM 口设置参数）



2.打开电脑串口工具，选择端口 COM6、设置波特率 115200，点击蓝色端口设置图标进行设置，8 位数据位、1 位停止位、无校验、无流控制，点击 OK，点击绿色图标打开串口。

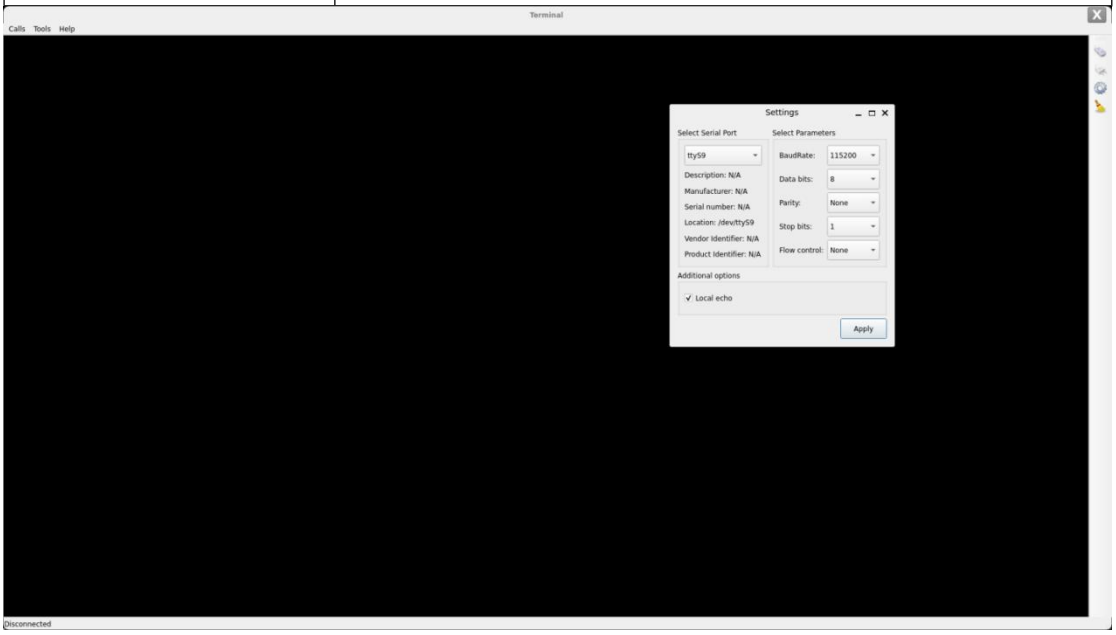


3.点击 UART 测试图标，进入如下界面，进行串口参数设置：



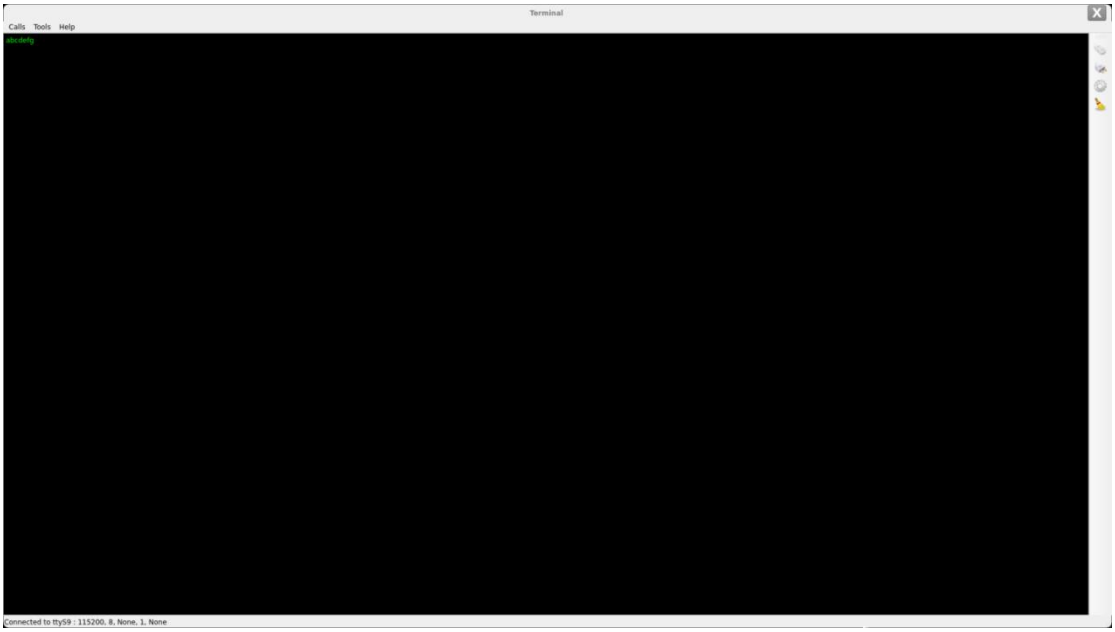
点击右上角  设置按钮，设置串口参数与电脑端串口工具参数一致，如下图：

相关参数	含义
Select Serial Port	设置串口（选择 UART9，即 ttyS9）
BaudRate	设置波特率（115200）
Data bits	设置数据位（8 位）
Parity	设置校验位（无校验）
Stop bits	设置停止位（1 位）
Flow control	设置流控（无流控）

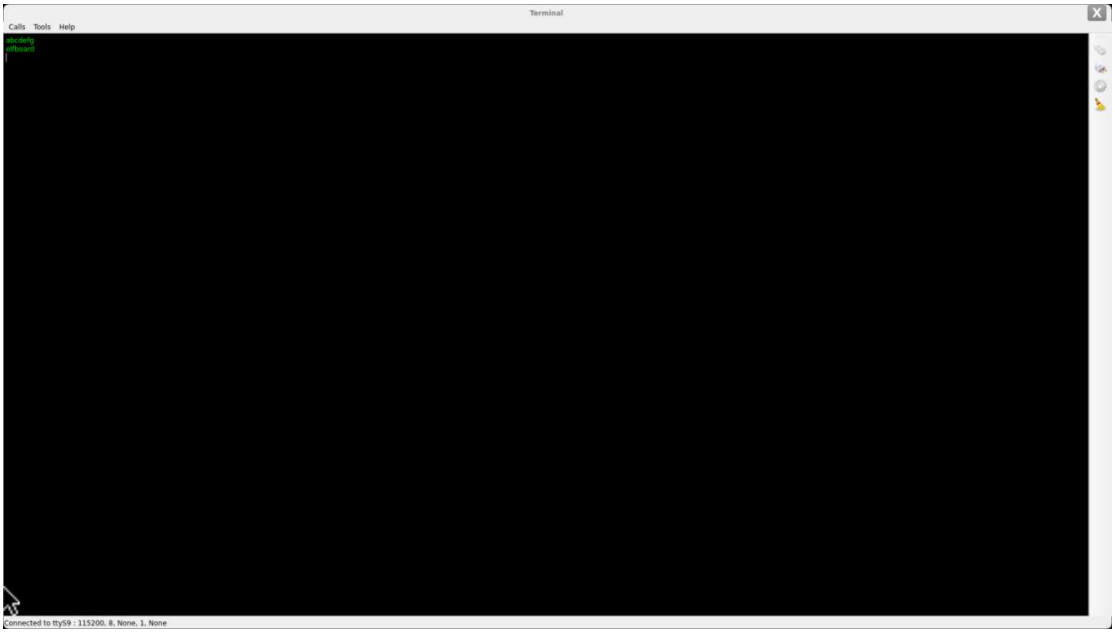


设置完串口参数后，点击 Apply 按钮，再点击右上角的  连接按钮，此时测试程序可以进行数据收发测试。

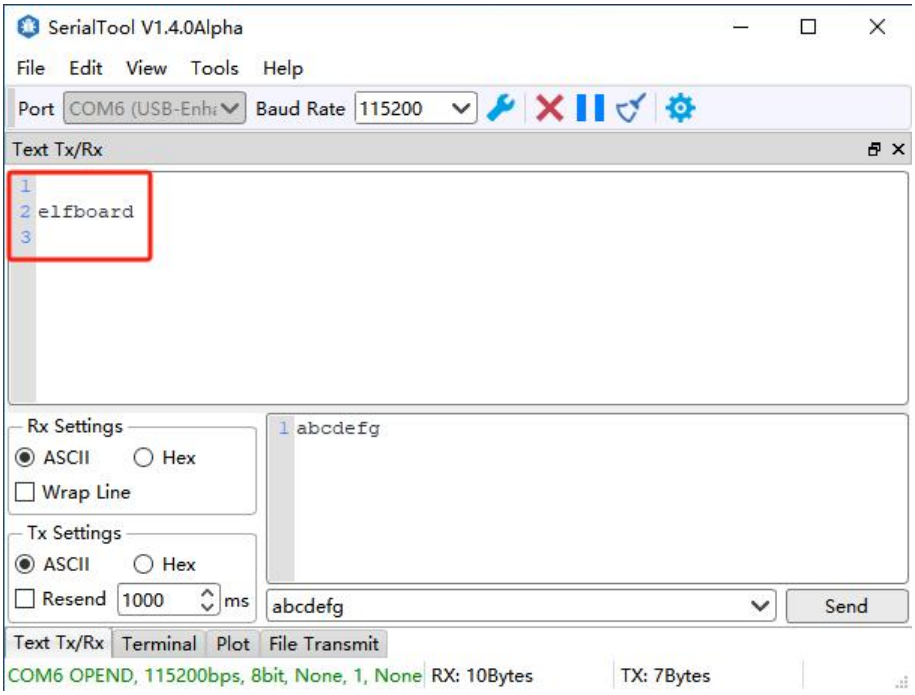
4.此时电脑端串口工具发送：“adcdefg”，测试界面会接收到数据：



点击测试界面会弹出软键盘，输入“elfboard”，按软键盘的回车，向电脑端串口工具发送数据：



电脑端串口工具接收到数据：



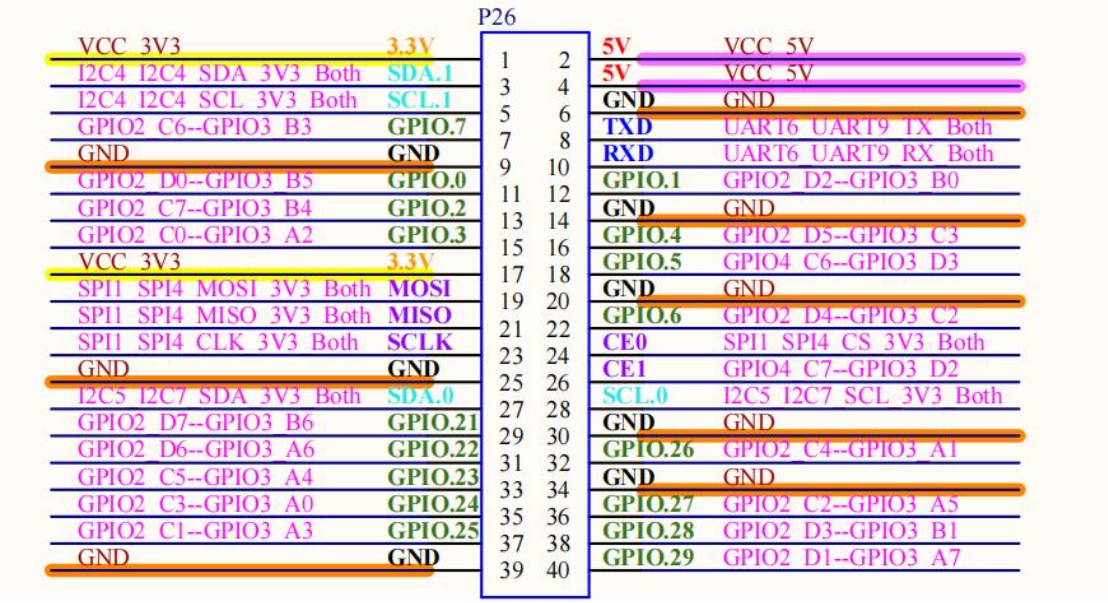
3.2.14. SPI 测试

点击桌面图标可以使用它来测试ELF 2板载spi接口。

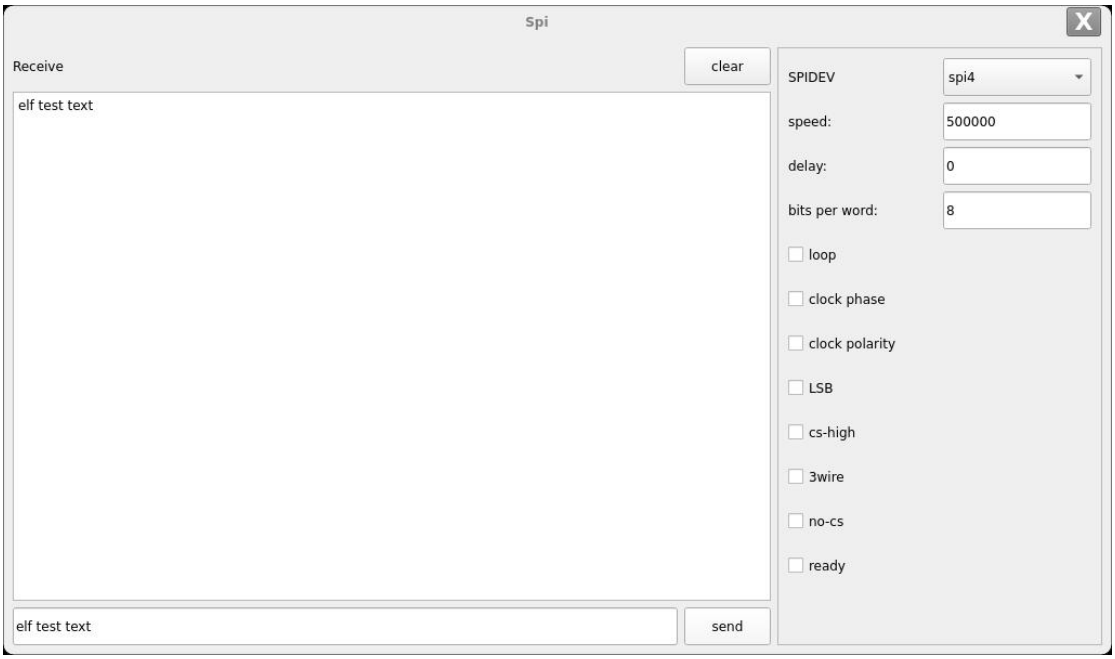


应用图标

ELF 2 底板上引出 1 路 SPI 接口，默认软件上将其配置为 spidev 用于回环测试。测试时，请参考原理图短接 MOSI (PIN19)与 MISO (PIN21)。



点击界面中的 send，Receive 会接收到数据，效果如下图所示：

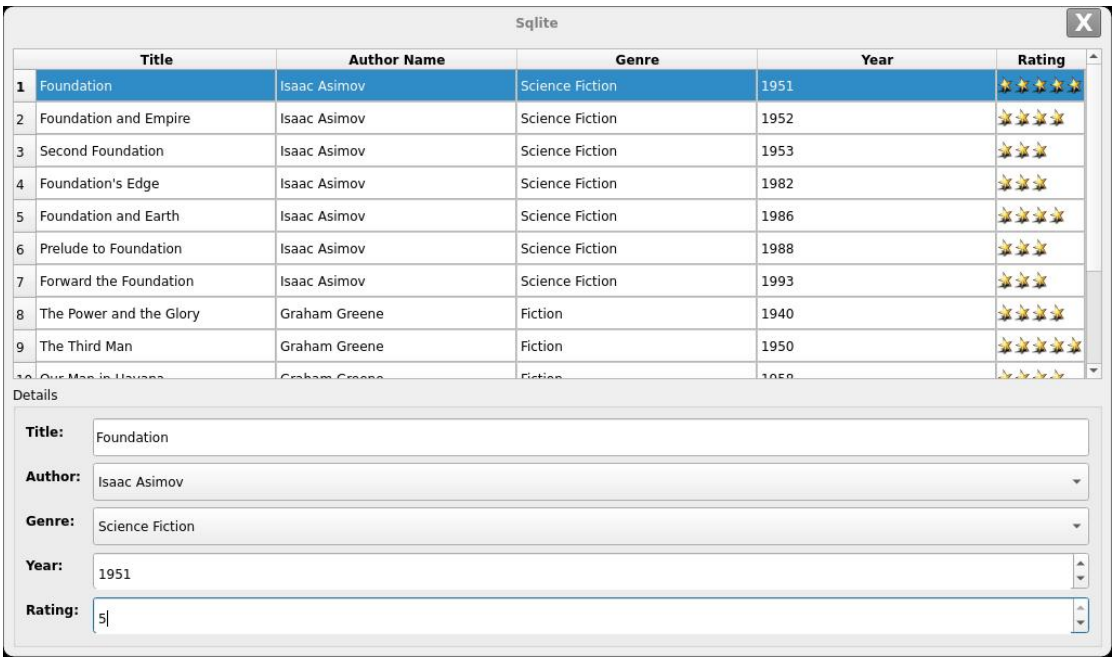


3.2.15. 数据库测试

点击桌面图标可以使用Sqlite测试数据库。



应用图标

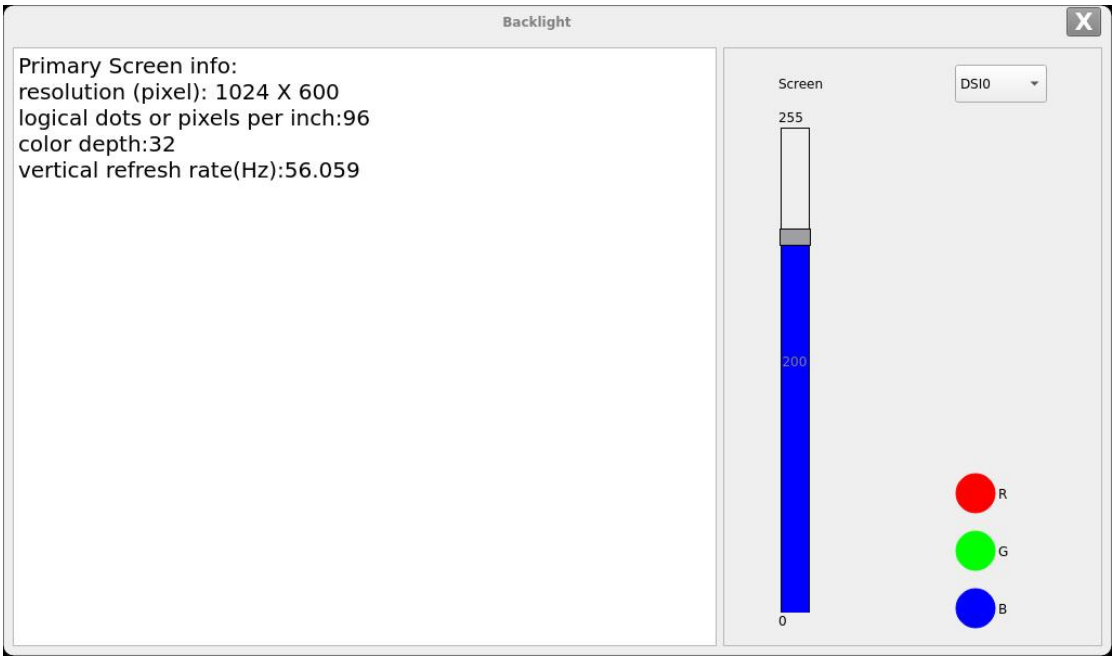


3.2.16. 背光测试

“BackLight”是 lcd 背光调整应用。



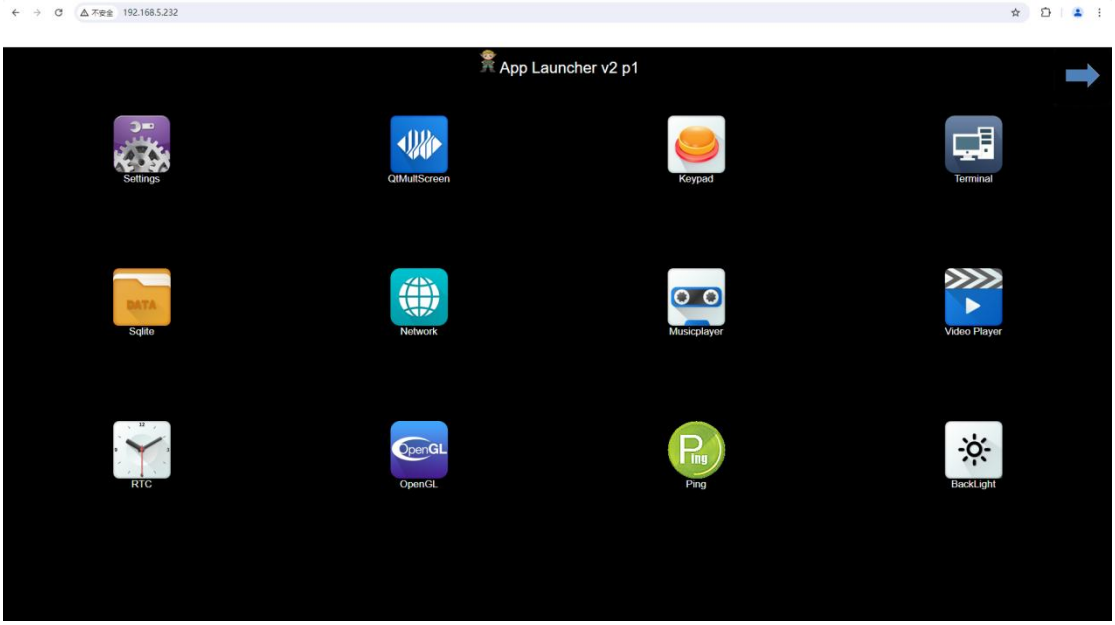
应用图标



拖动界面中的滑块即可设置 LCD 背光亮度，0 级为无背光，255 级为最高亮度。

3.2.17. Web 服务

ELF 2开发板预装了lighttpdweb服务器，并且系统启动时已经自动启动了lighttpd服务，在PC端的浏览器中输入开发板的IP地址即可浏览开发板webserver中的网页，如下图所示：



⚠**注意：**开发板的网络 IP 需要和 PC 机的网络 IP 在同一网段下才可以正常使用该功能，或者 PC 在开发板所处网络的子网下。

3.2.18. CPU 频率配置测试

⚠**注意：**当前界面只配置 A55 核，没有配置 A76 核调频，A76 核调频请参考调频测试章节。

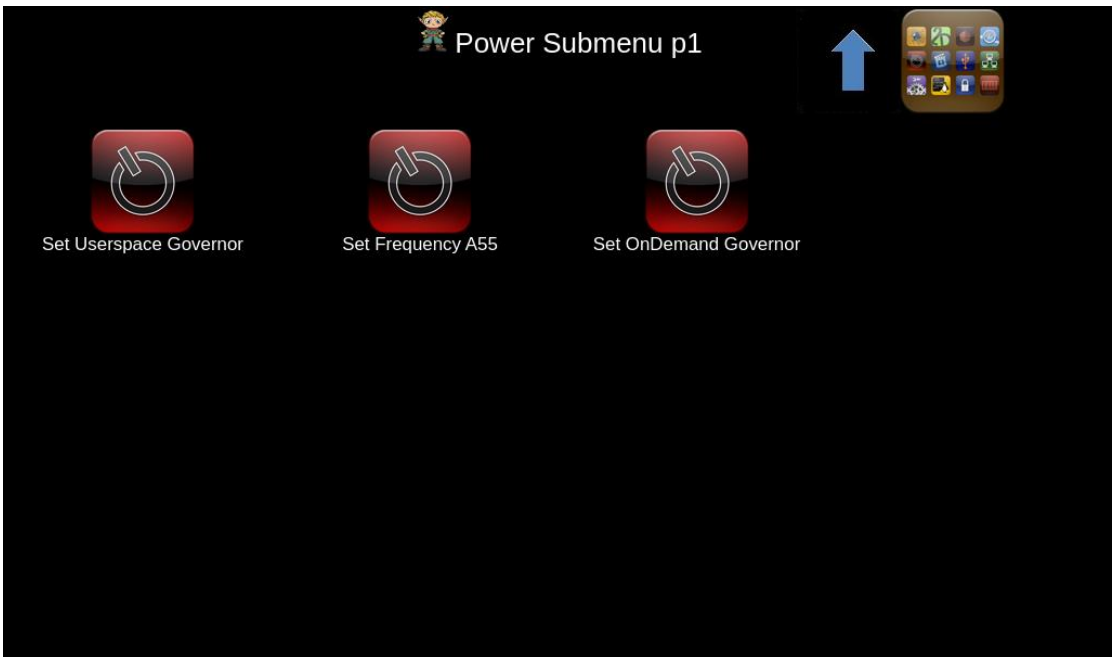
点击桌面图标进入下一级菜单：



应用图标

ELF 2 CPU 主频最大 1.8Ghz，默认情况下 CPU 会根据负载动态调整主频，也可以通过设置固定 CPU 主频率。

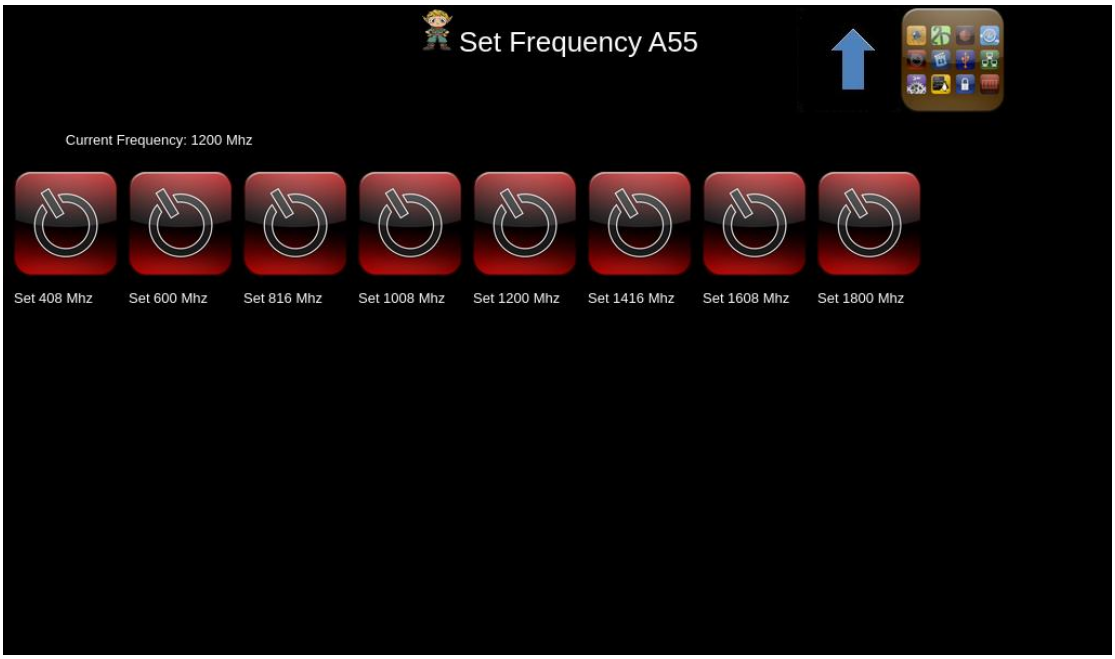
点击桌面 Power 图标进入 CPU 主频设置页面：



Set Userspace Governor：用户态设置主频。

Set Frequency A55：设置主频。

以设置主频频率为例，如果需要设置固定频率，请先点击 Set Userspace Governor，点击 RUN，再返回上图操作界面，点击 Set Frequency A55 进行设置。



根据需求选择对应的频率进行设置。

3.3. 多媒体测试

ELF 2 板卡音视频部分应用层软件采用的是 Gstreamer，支持硬件编解码。本节所有的示例均是基于 Gstreamer 命令行的形式。如果您需要带界面的播放器，您也可以使用 qt 的多媒体类，同样支持硬编解，可以参考 Qt 测试章节。

ELF 2 板卡内部有一个视频处理单元 VPU，支持以下格式的视频硬编解：

视频解码： H264，H265，VP8，VP9 等。

视频编码： H264、H.265，最大支持 8k@30fps。

ELF 2 板卡硬件编解码参数表：

	Format	Profile	Resolution	Frame rate
Video Decoder	H.265	main 10	7680x4320	60 fps
	H.264	main 10	7680x4320	30 fps
	VP9	Profile 0/2	7680x4320	60 fps
	VP8	version2	1920x1080	60 fps
	VC1		1920x1080	60 fps
	MPEG-2		1920x1080	60 fps
	MPEG-1		1920x1080	60 fps
	H.263		720x576	60 fps
Video Encoder	H.264	BP/MP/HP@level4.2	7680x4320	30 fps
	H.265	MP@level4.1	7680x4320	30 fps

3.3.1. 音频和视频播放体验

3.3.1.1. 使用 gst-play 播放器播放视频和音频

Gplay 是基于 GStreamer 实现的音视频播放器，能够自动根据硬件自动选择合适的插件进行音视频播放，运行也十分简单。

```
root@elf2-buildroot:~# gst-play-1.0 /userdata/media/video/1080p_60fps_h265-30S.mp4
//播放带声音视频文件，由耳机放音测试
Press 'k' to see a list of keyboard shortcuts.
Now playing /userdata/media/video/1080p_60fps_h265-30S.mp4
Redistribute latency...
Redistribute latency...
Redistribute latency...
0:00:30.0 / 0:00:30.0
Reached end of play list.
```

📖 说明：运行此命令将使用 GStreamer 播放一个 30 秒长的 H.265 编码的 1080p 60fps 视频文件。

gst-play-1.0	这是 GStreamer 的一个工具，主要用于播放音视频文件，1.0 表示这是 GStreamer 的 1.0 版本。
/userdata/media/video/1080p_60fps_h265-30S.mp4	播放的视频文件的路径。

gst-play-1.0 是一个功能强大的命令行工具，允许用户播放多媒体文件，并通过多种选项来控制播放行为和输出设备，下面是 gst-play-1.0 命令选项说明。

类别	选项/命令	描述
使用说明	gst-play-1.0 [OPTION...] FILE1|URI1 [FILE2|URI2] [FILE3|URI3] ...	gst-play-1.0：命令名称，表示调用 GStreamer 媒体播放器。
		[OPTION...]：可选参数，用于指定播放器的不同配置和行为（例如音量、输出设备

		等）。
		FILE1|URI1：第一个必需的参数，表示要播放的音频或视频文件的路径或 URI。
		[FILE2|URI2、[FILE3|URI3]：可选参数，可以指定一个或多个额外的文件或 URI，以便连续播放多个媒体项。
帮助选项	-h, --help	显示帮助信息，提供基本使用说明。
	--help-all	显示所有可用的帮助选项。
	--help-gst	显示 GStreamer 的选项和设置。
应用程序选项	-v, --verbose	启用详细模式，输出状态信息和属性通知，便于调试和监控播放进程。
	--flags	控制播放行为，设置 playbin 的 flags 属性。
	--version	打印 GStreamer 的版本信息并退出。
	--videosink	指定视频输出设备，默认使用 autovideosink。
	--audiosink	指定音频输出设备，默认使用 autoaudiosink。
	--gapless	启用无缝播放功能，避免播放列表中曲目之间出现间隙。
	--shuffle	随机播放播放列表中的曲目。
	--no-interactive	禁用键盘的交互控制，播放过程中将不会响应用户的键盘输入。
	--volume	设置播放音量。
	--playlist	指定一个播放列表文件，方便批量播放。
	-q, --quiet	不打印任何输出（除了错误信息），适合减少输出信息的场景。

3.3.1.2. 使用 gst-launch 播放视频

```
root@elf2-buildroot:~# gst-launch-1.0 filesrc location=
/userdata/media/video/1080p_60fps_h265-30S.mp4 ! qtdemux ! queue ! h265parse !
mppvideodec ! waylandsink
//仅播放视频
Setting pipeline to PAUSED ...
```

```
Pipeline is PREROLLING ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
0:00:01.0 / 0:00:30.0 (3.6 %)
```

🕮 说明：运行此命令将播放一个名为 1080p_60fps_h265-30S.mp4 的视频文件，使用 H.265 编码和 1080p 分辨率，帧率为 60fps，持续 30 秒。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/video/1080p_60fps_h265-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location= /userdata/media/video/1080p_60fps_h265-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
qtdemux	qtdemux 是一个 MP4 文件解复用器，它从 MP4 容器中提取出音频和视频流。
h265parse	h265parse 是一个 H.265 视频流解析器，负责解析 H.265 视频数据。
mppvideodec	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 H.265 流进行解码。
waylandsink	waylandsink 是一个视频输出元素，用于在 Wayland 显示服务器上显示视频。它将解码后的视频帧呈现到屏幕上。

gst-launch-1.0 是一个命令行工具，用于构建和运行媒体处理管道，下面是 gst-launch-1.0 命令选项说明。

类别	选项/命令	描述
使用说明	gst-launch-1.0 [OPTION...] PIPELINE-DESCRIPTION	gst-launch-1.0：命令的名称，用于启动 GStreamer 的管道处理工具。
		[OPTION...]：可选参数部分，允许用户在运行命令时指定特定的选项。
		PIPELINE-DESCRIPTION：管道描述部分，定义了数据流的元素、如何连接这些元素以及它们的属性
帮助选项	-h, --help	显示帮助信息，提供基本使用说明。

	--help-all	显示所有可用的帮助选项。
	--help-gst	显示 GStreamer 的选项和设置。
应用程序选项	-t, --tags	输出标签（也称为元数据）。
	-c, --toc	输出 TOC（章节和版本信息）。
	-v, --verbose	启用详细模式，输出状态信息和属性通知，便于调试和监控。
	-q, --quiet	禁用任何进度信息的打印。
	-m, --messages	输出消息。
	-X, --exclude=PROPERTY-NAME	如果启用了详细输出，指定不输出某个属性的状态信息（可多次使用）。
	-f, --no-fault	不安装故障处理程序。
	-e, --eos-on-shutdown	在关闭管道之前，强制对源发送 EOS（结束信号）。
	--version	打印版本信息并退出。

3.3.1.3. 使用 gst-launch 播放音频

```
root@elf2-buildroot:~# aplay -l
**** List of PLAYBACK Hardware Devices ****
card 0: rockchipdp0 [rockchip,dp0], device 0: rockchip,dp0 spdif-hifi-0 [rockchip,dp0 spdif-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 1: rockchipnau8822 [rockchip-nau8822], device 0: dailink-multicodecs nau8822-hifi-0 [dailink-multicodecs nau8822-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 2: rockchiphdmi0 [rockchip-hdmi0], device 0: rockchip-hdmi0 i2s-hifi-0 [rockchip-hdmi0 i2s-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
root@elf2-buildroot:~# gst-launch-1.0 filesrc location=/userdata/media/audio/piano2-CoolEdit.mp3 ! id3demux ! mpegaudioparse ! mpg123audiodec ! alsasink device=plughw:1,0
//仅播放音频，device=plughw:1,0 由耳机放音测试。
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
New clock: GstAudioSinkClock
handling interrupt.
```

```
Interrupt: Stopping pipeline ...
Execution ended after 0:00:02.665159268
Setting pipeline to PAUSED ...
Setting pipeline to READY ...
Setting pipeline to NULL ...
Freeing pipeline ...
```

🕮 说明：运行此命令将从指定的 MP3 文件中读取音频数据，解析并解码音频流，最终通过指定的 ALSA 音频设备播放出来。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/audio/piano2-CoolEdit.mp3	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/audio/piano2-CoolEdit.mp3 播放的音频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
id3demux	这是一个解复用器，用于解析 MP3 文件中的 ID3 标签（如标题、艺术家、专辑等）和音频数据。它将 MP3 文件中的音频流和元数据分开。
mpegaudioparse	这是一个解析器，用于解析 MPEG 音频流。它处理从 id3demux 传递过来的音频数据，为后续解码做准备。
mpg123audiodec	这是一个音频解码器，专门用于解码 MP3 音频流。它将经过解析的音频数据转换为原始 PCM 格式，以便播放。
alsasink device=plughw:1,0	alsasink 是一个音频输出元素，使用 ALSA（高级 Linux 声音架构）播放音频。
	device=plughw:1,0 指定了使用的音频设备，plughw:1,0 表示使用第一个硬件设备。可以根据系统的实际音频设备调整这个参数。

3.3.1.4. 使用 gst-launch 播放视频和音频

```
root@elf2-buildroot:~# gst-launch-1.0 filesrc location=
/userdata/media/video/1080p_60fps_h265-30S.mp4 ! qtdemux name=dec dec. !
queue ! h265parse ! mppvideodec ! waylandsink dec. ! queue ! decodebin ! alsasink
device=plughw:1,0
//播放带声音视频文件，device=plughw:1,0 由耳机放音测试
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Redistribute latency...
```

```
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstAudioSinkClock
^Chandling interrupt. (2.6 %)
```

🕮 说明：运行此命令将从指定的 MP4 文件中提取 H.265 编码的视频和音频流，进行解复用、解码后，通过 Wayland 显示服务器输出视频，并通过 ALSA 输出音频。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/video/1080p_60fps_h265-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/video/1080p_60fps_h265-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
qtdemux name=dec	qtdemux 是一个解复用器，用于解析 MP4 文件格式（QuickTime 格式）。
	name=dec 给解复用器命名为 dec，这样可以在后面引用它。
dec. ! queue ! h265parse ! mppvideodec ! waylandsink	dec.引用名为 dec 的解复用器的输出，处理视频流。
	queue 是一个队列元素，用于缓冲数据流，确保后续处理的流畅性。
	h265parse 是一个 H.265 视频流解析器，负责解析 H.265 视频数据。
	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 H.265 流进行解码。
	waylandsink 是一个视频输出元素，用于在 Wayland 显示服务器上显示视频。它将解码后的视频帧呈现到屏幕上。
dec. ! queue ! decodebin ! alsasink device=plughw:1,0	dec.再次引用 dec 解复用器的输出，这次处理音频流。
	queue 添加一个队列，用于音频流的处理。
	decodebin 自动选择合适的解码器来解码音频流。decodebin 可以处理多种音频格式，因此会选择适当的解码器。
	alsasink device=plughw:1,0 使用 ALSA 音频输出元素将解码后的音频播放出来，device=plughw:1,0 指定了使用的音频设备。

3.3.2. 视频硬编码

ELF 2 最大支持 8K@30fps/H.265 和 8K@30fps/H.264 视频编码。

3.3.2.1. 视频硬编码 H.264

```
root@elf2-buildroot:~# gst-launch-1.0 videotestsrc num-buffers=600 !
video/x-raw,framerate=30/1,width=7680,height=4320 ! mpph264enc ! h264parse !
mp4mux ! filesink location=test.mp4
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
Got EOS from element "pipeline0".
Execution ended after 0:00:53.676009268
Setting pipeline to NULL ...
Freeing pipeline ...
//查看/root 目录下生成 test.mp4 文件
root@elf2-buildroot:~# ls
test.mp4
```

📖 说明：运行此命令将生成一个 8K 分辨率、30 帧每秒的测试视频，编码为 H.264 格式，并将其封装为 MP4 文件，最后保存为 test.mp4。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。	
videotestsrc num-buffers=600	videotestsrc 是一个视频测试源元素，它生成一系列的测试视频帧。默认生成的测试视频通常为颜色渐变或其他模式。	
	num-buffers=600 指定生成 600 帧视频数据，达到这个数量后，视频源将停止输出。	
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。	
video/x-raw,framerate=30/1,width=7680,height=4320	这是一个 caps(能力)字符串，定义了视频流的格式和参数。	video/x-raw 表示原始视频数据格式。
		framerate=30/1 指定帧率为 30 帧每秒。
		width=7680,height=4320 指定视频分辨率为 7680x4320（即 8K 分辨率）。
mpph264enc	mpph264enc 是一个 H.264 编码器，用于将	

	原始视频流编码为 H.264 格式。
h264parse	h264parse 是一个 H.264 解析器，用于处理 H.264 编码流。
mp4mux	mp4mux 是一个 MP4 封装器，用于将 H.264 视频流封装成 MP4 文件格式。
filesink location=test.mp4	filesink 是一个文件输出元素，它将处理过的数据写入指定的文件。
	location=test.mp4 指定输出文件的路径和名称，生成的 MP4 文件将被命名为 test.mp4。

3.3.2.2. 视频硬编码 H.265

```
root@elf2-buildroot:~# gst-launch-1.0 videotestsrc num-buffers=600 !
video/x-raw,framerate=30/1,width=7680,height=4320 ! mpph265enc ! h265parse !
mp4mux ! filesink location=test.mp4
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
Got EOS from element "pipeline0".
Execution ended after 0:00:56.942676054
Setting pipeline to NULL ...
Freeing pipeline ...
//查看/root 目录下生成 test.mp4 文件
root@elf2-buildroot:~# ls
test.mp4
```

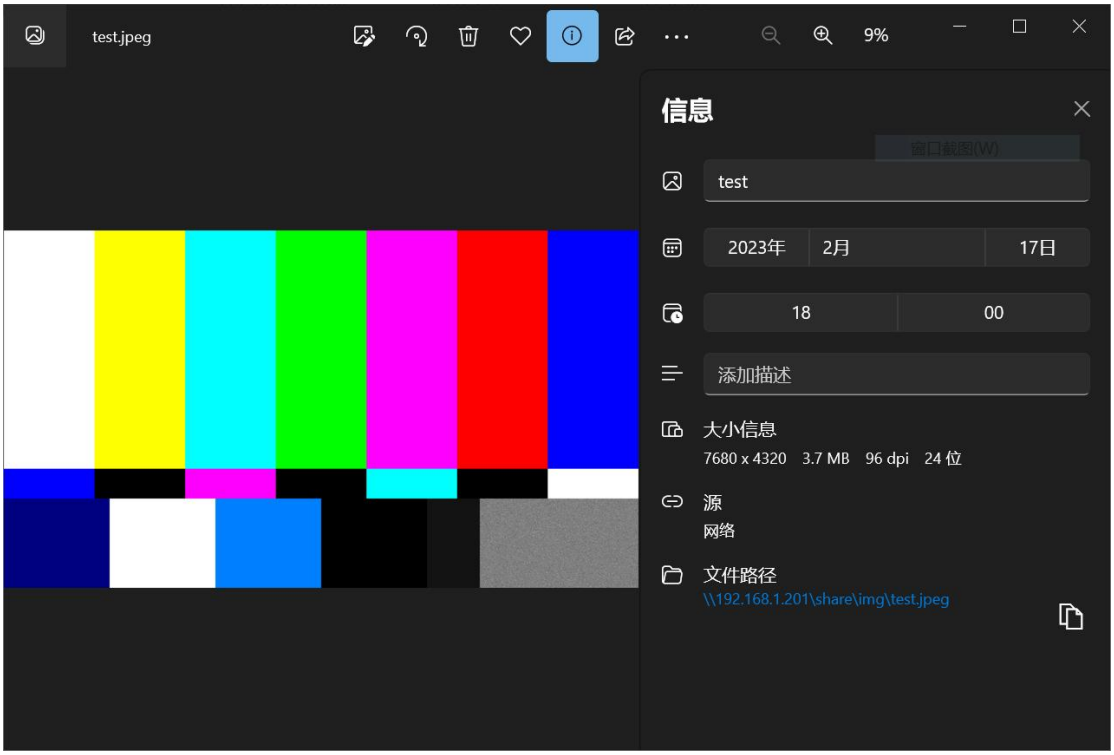
📖 说明：运行此命令将生成一个 8K 分辨率、30 帧每秒的测试视频，编码为 H.265 格式，并将其封装为 MP4 文件，最后保存为 test.mp4。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。	
videotestsrc num-buffers=600	videotestsrc 是一个视频测试源元素，它生成一系列的测试视频帧。默认生成的测试视频通常为颜色渐变或其他模式。	
	num-buffers=600 指定生成 600 帧视频数据，达到这个数量后，视频源将停止输出。	
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。	
video/x-raw,framerate=30/1,width=7680,height=4320	这是一个 caps(能力)字符串，定义了视	video/x-raw 表示原始视频数据格式。

	频流的格式和参数。	framerate=30/1 指定帧率为 30 帧每秒。
		width=7680,height=4320 指定视频分辨率为 7680x4320（即 8K 分辨率）。
mpph265enc	mpph265enc 是一个 H.265 编码器，用于将原始视频流编码为 H.265 格式。	
h265parse	h265parse 是一个 H.265 解析器，用于处理 H.265 编码流。	
mp4mux	mp4mux 是一个 MP4 封装器，用于将 H.265 视频流封装成 MP4 文件格式。	
filesink location=test.mp4	filesink 是一个文件输出元素，它将处理过的数据写入指定的文件。	
	location=test.mp4 指定输出文件的路径和名称，生成的 MP4 文件将被命名为 test.mp4。	

3.3.2.3. JPEG 硬编码

```
root@elf2-buildroot:~# gst-launch-1.0 videotestsrc num-buffers=1 !
video/x-raw,framerate=1/1,width=7680,height=4320 ! mppjpegenc ! jpegparse ! queue !
filesink location=test.jpeg
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
Got EOS from element "pipeline0".
Execution ended after 0:00:00.029266878
Setting pipeline to NULL ...
Freeing pipeline ...
//查看/root 目录下生成 test.jpeg 文件
root@elf2-buildroot:~# ls
test.jpeg
```



🕮 说明：运行此命令将生成一个 8K 分辨率的测试视频帧，以 1 帧每秒的速率，编码为 JPEG 格式，并保存为 test.jpeg 文件。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。	
videotestsrc num-buffers=1	videotestsrc 是一个视频测试源元素，它生成一系列的测试视频帧。默认生成的测试视频通常为颜色渐变或其他模式。	
	num-buffers=1 指定只生成 1 帧视频数据。执行完后，视频源将停止输出。	
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。	
video/x-raw,framerate=1/1,width=7680,height=4320	这是一个 caps(能力)字符串，定义了视频流的格式和参数。	video/x-raw 表示原始视频数据格式。
		framerate=1/1 指定帧率为 1 帧每秒。
		width=7680,height=4320 指定视频分辨率为 7680x4320（即 8K 分辨率）。
mppjpegenc	mppjpegenc 是一个 JPEG 编码器，用于将原始视频流编码为 JPEG 格式。	
jpegparse	jpegparse 是一个 JPEG 解析器，用于处理 JPEG 编码流。	
queue	queue 是一个队列元素，用于在数据流中引	

	入缓冲区。
filesink location=test.jpeg	filesink 是一个文件输出元素，它将处理过的数据写入指定的文件。
	location=test.jpeg 指定输出文件的路径和名称，生成的 JPEG 文件将被命名为 test.jpeg。

3.3.3. 视频硬解码

ELF 2 支持 H264，H265、VP8、VP9 视频硬解码，H264 解码器支持 8K@30fps，H265 解码器支持 8K@60fps。

ELF 2 使用 mppvideodec 组件进行视频硬解码，它的输出格式为：NV12，I420，YV12。

3.3.3.1. 解码并播放 H264 格式视频

```
root@elf2-buildroot:~# gst-launch-1.0 filesrc
location=/userdata/media/video/1080p_60fps_h264-30S.mp4 ! qtdemux ! h264parse !
mppvideodec ! waylandsink
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
0:00:05.9 / 0:00:30.0 (19.8 %)
```

🕮 说明：运行此命令将从指定的 MP4 文件中读取 H.264 编码的视频数据，解复用并解析该数据，然后通过硬件加速解码，最后在 Wayland 显示服务器上播放视频。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/video/1080p_60fps_h264-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/video/1080p_60fps_h264-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
qtdemux	qtdemux 是一个 MP4 文件解复用器，它从 MP4 容器中提取出音频和视频流。
h264parse	h264parse 是一个 H.264 视频流解析器，负责解析 H.264 视频数据。
mppvideodec	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 H.264 流进行

	解码。
waylandsink	waylandsink 是一个视频输出元素，用于在 Wayland 显示服务器上显示视频。它将解码后的视频帧呈现到屏幕上。

3.3.3.2. 解码并播放 H264 格式视频带音频

```
root@elf2-buildroot:~# gst-launch-1.0 filesrc
location=/userdata/media/video/1080p_60fps_h264-30S.mp4 ! qtdemux name=demux
demux.video_0 ! queue ! h264parse ! mppvideodec ! waylandsink demux.audio_0 !
queue ! aacparse ! faad ! alsasink device=plughw:1,0
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Redistribute latency...
Redistribute latency...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstAudioSinkClock
0:00:04.5 / 0:00:30.0 (15.0 %)
```

📖 说明：运行此命令将从指定的 MP4 文件中读取 H.264 编码的视频流和 AAC 编码的音频流，解复用并解析这些数据，然后使用硬件加速解码视频并在 Wayland 上播放，同时将音频解码后输出到指定的 ALSA 音频设备。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/video/1080p_60fps_h264-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/video/1080p_60fps_h264-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
qtdemux name=demux	qtdemux 是一个 MP4 文件解复用器，它从 MP4 容器中提取出音频和视频流。
	name=demux 为该解复用器命名为 demux，以便后续引用。
demux.video_0 ! queue	demux.video_0 表示从名为 demux 的解复用器中获取第一个视频流。
	queue 是一个队列元素，用于缓冲数据流，确保后续处理的流畅性。
h264parse	h264parse 是一个 H.264 视频流解析器，负

	责解析 H.264 视频数据。
mppvideodec	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 H.264 流进行解码。
waylandsink	waylandsink 是一个视频输出元素，用于在 Wayland 显示服务器上显示视频。它将解码后的视频帧呈现到屏幕上。
demux.audio_0 ！ queue	demux.audio_0 表示从解复用器中获取第一个音频流。
	queue 再次使用队列元素，用于缓冲音频流，确保流畅播放。
aacparse	aacparse 是 AAC 音频解析器，负责解析 AAC 音频流。
faad	faad 是 AAC 解码器，负责将 AAC 音频流解码为原始音频数据。
alsasink device=plughw:1,0	alsasink 是音频输出元素，使用 ALSA (Advanced Linux Sound Architecture)将音频数据输出到音频设备。
	device=plughw:1,0 指定了要使用的具体音频设备。

3.3.3.3. 解码并播放 H265 格式视频

```
root@elf2-buildroot:~# gst-launch-1.0 filesrc
location=/userdata/media/video/4k_60fps_h265-30S.mp4 ! qtdemux ! h265parse !
mppvideodec ! waylandsink
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
0:00:06.4 / 0:00:30.0 (21.4 %)
```

📖 说明：运行此命令将从指定的 MP4 文件中读取 H.265 编码的视频数据，解复用并解析该数据，然后通过硬件加速解码，最后在 Wayland 显示服务器上播放视频。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/video/4k_60fps_h265-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/video/4k_60fps_h265-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接

	符。它用于将一个元素的输出连接到下一个元素的输入。
qtdemux	qtdemux 是一个 MP4 文件解复用器，它从 MP4 容器中提取出音频和视频流。
h265parse	h265parse 是一个 H.265 视频流解析器，负责解析 H.265 视频数据。
mppvideodec	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 H.265 流进行解码。
waylandsink	waylandsink 是一个视频输出元素，用于在 Wayland 显示服务器上显示视频。它将解码后的视频帧呈现到屏幕上。

3.3.3.4. 解码并播放 H265 格式视频带音频

```
root@elf2-buildroot:~# gst-launch-1.0 filesrc
location=/userdata/media/video/4k_60fps_h265-30S.mp4 ! qtdemux name=demux
demux.video_0 ! queue ! h265parse ! mppvideodec ! waylandsink demux.audio_0 !
queue ! aacparse ! faad ! alsasink device=plughw:1,0
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Redistribute latency...
Redistribute latency...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstAudioSinkClock
0:00:03.4 / 0:00:30.0 (11.3 %)
```

🕮 说明：运行此命令将从指定的 MP4 文件中读取 H.265 编码的视频流和 AAC 编码的音频流，解复用并解析这些数据，然后使用硬件加速解码视频并在 Wayland 上播放，同时将音频解码后输出到指定的 ALSA 音频设备。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/video/4k_60fps_h265-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/video/4k_60fps_h265-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
qtdemux name=demux	qtdemux 是一个 MP4 文件解复用器，它从 MP4 容器中提取出音频和视频流。

	name=demux 为该解复用器命名为 demux，以便后续引用。
demux.video_0 ！ queue	demux.video_0 表示从名为 demux 的解复用器中获取第一个视频流。
	queue 是一个队列元素，用于缓冲数据流，确保后续处理的流畅性。
h265parse	h265parse 是一个 H.265 视频流解析器，负责解析 H.265 视频数据。
mppvideodec	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 H.265 流进行解码。
waylandsink	waylandsink 是一个视频输出元素，用于在 Wayland 显示服务器上显示视频。它将解码后的视频帧呈现到屏幕上。
demux.audio_0 ！ queue	demux.audio_0 表示从解复用器中获取第一个音频流。
	queue 再次使用队列元素，用于缓冲音频流，确保流畅播放。
aacparse	aacparse 是 AAC 音频解析器，负责解析 AAC 音频流。
faad	faad 是 AAC 解码器，负责将 AAC 音频流解码为原始音频数据。
alsasink device=plughw:1,0	alsasink 是音频输出元素，使用 ALSA (Advanced Linux Sound Architecture)将音频数据输出到音频设备。
	device=plughw:1,0 指定了要使用的具体音频设备。

3.3.3.5. 解码并播放 VP9 格式视频

```
root@elf2-buildroot:~# gst-launch-1.0 filesrc
location=/userdata/media/video/1080p_60fps_vp9-30S.mp4 ! qtdemux ! vp9parse !
mppvideodec ! waylandsink
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
0:00:02.5 / 0:00:30.0 (8.7 %)
```

📖 说明：运行此命令将从指定的 MP4 文件中读取 VP9 编码的视频数据，解复用并解析该数据，然后通过硬件加速解码，最后在 Wayland 显示服务器上播放视频。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。

filesrc location=/userdata/media/video/1080p_60fps_vp9-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/video/1080p_60fps_vp9-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
qtdemux	qtdemux 是一个 MP4 文件解复用器，它从 MP4 容器中提取出音频和视频流。
vp9parse	vp9parse 是一个 VP9 视频流解析器，负责解析 VP9 视频数据。
mppvideodec	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 VP9 流进行解码。
waylandsink	waylandsink 是一个视频输出元素，用于在 Wayland 显示服务器上显示视频。它将解码后的视频帧呈现到屏幕上。

3.3.3.6. 解码并播放 VP9 格式视频带音频

```
root@elf2-buildroot:~# gst-launch-1.0 filesrc
location=/userdata/media/video/1080p_60fps_vp9-30S.mp4 ! qtdemux name=demux
demux.video_0 ! queue ! vp9parse ! mppvideodec ! waylandsink demux.audio_0 !
queue ! aacparse ! faad ! alsasink device=plughw:1,0
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstAudioSinkClock
0:00:03.2 / 0:00:30.0 (10.7 %)
```

📖 说明：运行此命令将从指定的 MP4 文件中读取 VP9 编码的视频流和 AAC 编码的音频流，解复用并解析这些数据，然后使用硬件加速解码视频并在 Wayland 上播放，同时将音频解码后输出到指定的 ALSA 音频设备。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/video/1080p_60fps_vp9-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/video/1080p_60fps_vp9-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个

	元素的输入。
qtdemux name=demux	qtdemux 是一个 MP4 文件解复用器，它从 MP4 容器中提取出音频和视频流。
	name=demux 为该解复用器命名为 demux，以便后续引用。
demux.video_0 ! queue	demux.video_0 表示从名为 demux 的解复用器中获取第一个视频流。
	queue 是一个队列元素，用于缓冲数据流，确保后续处理的流畅性。
vp9parse	vp9parse 是一个 VP9 视频流解析器，负责解析 VP9 视频数据。
mppvideodec	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 VP9 流进行解码。
waylandsink	waylandsink 是一个视频输出元素，用于在 Wayland 显示服务器上显示视频。它将解码后的视频帧呈现到屏幕上。
demux.audio_0 ! queue	demux.audio_0 表示从解复用器中获取第一个音频流。
	queue 再次使用队列元素，用于缓冲音频流，确保流畅播放。
aacparse	aacparse 是 AAC 音频解析器，负责解析 AAC 音频流。
faad	faad 是 AAC 解码器，负责将 AAC 音频流解码为原始音频数据。
alsasink device=plughw:1,0	alsasink 是音频输出元素，使用 ALSA (Advanced Linux Sound Architecture)将音频数据输出到音频设备。
	device=plughw:1,0 指定了要使用的具体音频设备。

3.3.3.7. 解码并播放 VP8 格式视频

```
root@elf2-buildroot:~# gst-launch-1.0 filesrc
location=/userdata/media/video/1080p_30fps_vp8.mp4 ! matroskademux ! queue !
mppvideodec ! waylandsink
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
0:00:09.2 / 0:00:30.1 (30.7 %)
```

📖 说明：运行此命令将从指定的 MP4 文件中提取 VP8 编码的视频，进行解复用、解码后，通过 Wayland 显示服务器输出视频。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
location=/userdata/media/video/1080p_30fps_vp8.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/video/1080p_30fps_vp8.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
matroskademux	matroskademux 是一个解复用器，用于从特定格式的多媒体容器中提取音频和视频流。
queue	queue 是一个队列元素，用于缓冲数据流，确保后续处理的流畅性。
mppvideodec	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 VP8 流进行解码。
waylandsink	waylandsink 是一个视频输出元素，用于在 Wayland 显示服务器上显示视频。它将解码后的视频帧呈现到屏幕上。

3.3.3.8. 解码并播放 VP8 格式视频带音频

```
root@elf2-buildroot:~# gst-launch-1.0 filesrc
location=/userdata/media/video/1080p_30fps_vp8.mp4 typefind=true ! video/webm !
matroskademux name=dec dec. ! queue ! mppvideodec ! waylandsink dec. ! queue !
decodebin ! audioconvert ! audioresample ! alsasink device=plughw:1,0
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstAudioSinkClock
0:00:02.4 / 0:00:30.1 (8.3 %)
```

🕮 说明：运行此命令将从指定的 MP4 文件中提取 VP8 编码的视频和音频流，进行解复用、解码后，通过 Wayland 显示服务器输出视频，并通过 ALSA 输出音频。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
location=/userdata/media/video/1080p_30fps_vp8.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/video/1080p_30

	fps_vp8.mp4 播放的视频文件的路径。
typefind=true	启用 typefind 功能，GStreamer 将会自动检测输入文件的媒体类型，以便选择适合的解复用器和解码器。
i	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
video/webm	指定视频流的类型为 WebM 格式。这个元素用于将流标记为 WebM 格式。
matroskademux name=dec	matroskademux 是一个解复用器，用于从特定格式的多媒体容器中提取音频和视频流。
	name=dec 为解复用器命名，以便后续引用。
dec. ! queue ! mppvideodec ! waylandsink	dec.这是对先前命名的 dec 解复用器的引用，用于访问其输出。
	queue 是一个队列元素，用于缓冲数据流，确保后续处理的流畅性。
	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 VP8 流进行解码。
	waylandsink 是一个视频输出元素，用于在 Wayland 显示服务器上显示视频。它将解码后的视频帧呈现到屏幕上。
dec. ! queue ! decodebin ! audioconvert ! audioresample ! alsasink device=plughw:1,0	dec.再次引用 dec 解复用器，用于处理音频流的输出。
	queue 为音频流添加一个队列，以缓冲数据，确保音频处理的稳定性
	decodebin 是一个自动解码器，能够根据输入流类型选择合适的解码器。它会处理来自 dec 解复用器的音频流。
	audioconvert 元素用于音频格式转换，确保音频流的格式兼容后续处理。
	audioresample 用于音频重采样，确保音频以合适的采样率输出，以适应不同的音频设备。
	alsasink device=plughw:1,0 使用 ALSA 音频输出元素将解码后的音频播放出来，device=plughw:1,0 指定了使用的音频设备。

3.3.4. 摄像头测试

ELF 2 支持 OV13855 MIPI 摄像头，同时也支持 UVC 摄像头。首先来测试一下 UVC 摄像头，这里以林柏视进程测试，将 USB 摄像头插入开发板，将自动安装 UVC 驱动。

3.3.4.1. UVC Camera 测试

1、摄像头识别检测和格式支持查询

摄像头识别检测，列出系统中所有视频设备：

```
root@elf2-buildroot:~# v4l2-ctl --list-devices
[   205.931251] m00_b_dw9763 3-000c: dw9763_open: enter, power.usage_count(1)!
rk_hdmirx (fdee0000.hdmirx-controller):
[   205.931362] m00_b_dw9763 3-000c: dw9763_open: exit, power.usage_count(2)!
        /dev/video20
[   205.931404] dw9763 3-000c: cmd 0x80685600 not supported

[   205.931434] dw9763 3-000c: cmd 0xc1007c00 not supported
rkisp-statistics (platform: rkisp):
[   205.931462] m00_b_dw9763 3-000c: dw9763_close: enter, power.usage_count(2)!
        /dev/video18
[   205.931483] m00_b_dw9763 3-000c: dw9763_close: exit, power.usage_count(1)!
        /dev/video19

rkcif-mipi-lvds (platform:rkcif):
        /dev/media0

rkcif (platform:rkcif-mipi-lvds):
        /dev/video0
        /dev/video1
        /dev/video2
        /dev/video3
        /dev/video4
        /dev/video5
        /dev/video6
        /dev/video7
        /dev/video8
        /dev/video9
        /dev/video10

rkisp_mainpath (platform:rkisp0-vir0):
        /dev/video11
        /dev/video12
        /dev/video13
        /dev/video14
        /dev/video15
        /dev/video16
        /dev/video17
        /dev/media1

4K USB Camera: 4K USB Camera (usb-xhci-hcd.4.auto-1):
```

```
        /dev/video21
        /dev/video22
        /dev/media2
```

格式支持查询：

```
root@elf2-buildroot:~# v4l2-ctl --list-formats-ext -d /dev/video21
//查看摄像头支持的格式
ioctl: VIDIOC_ENUM_FMT
        Type: Video Capture

        [0]: 'YUYV' (YUYV 4:2:2)
                Size: Discrete 640x480
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1920x1080
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1280x960
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1280x720
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 640x360
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 640x480
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
        [1]: 'MJPG' (Motion-JPEG, compressed)
                Size: Discrete 640x480
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 3840x2160
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 2560x1440
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
```

```
                Size: Discrete 1920x1080
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1280x960
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1280x720
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 640x360
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 640x480
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
        [2]: 'NV12' (Y/CbCr 4:2:0)
                Size: Discrete 640x480
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1920x1080
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1280x960
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1280x720
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 640x360
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 640x480
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
```

2、摄像头采集格式查询

摄像头采集格式查询：

```
root@elf2-buildroot:~# v4l2-ctl -V -d /dev/video21
Format Video Capture:
```

```
        Width/Height      : 640/480
        Pixel Format      : 'YUYV' (YUYV 4:2:2)
        Field             : None
        Bytes per Line    : 1280
        Size Image        : 0
        Colorspace        : sRGB
        Transfer Function : Rec. 709
        YCbCr/HSV Encoding: ITU-R 601
        Quantization      : Default (maps to Limited Range)
        Flags
```

3、摄像头图像预览

```
root@elf2-buildroot:~# gst-launch-1.0 v4l2src device=/dev/video21 ! videoconvert !
video/x-raw,format=NV12,width=640,height=480 ! waylandsink
Setting pipeline to PAUSED ...
Pipeline is live and does not need PREROLL ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
Redistribute latency...
0:00:19.7 / 99:99:99.
```

📖 说明：运行此命令将从/dev/video21 设备获取视频流，转换视频格式为 NV12，并将其显示在 Waylan d 窗口中。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
v4l2src device=/dev/video21	v4l2src 是 GStreamer 中用于访问 V4L2 设备的源插件。它可以从视频捕捉设备（如摄像头）读取视频流。
	device=/dev/video21 指定了要使用的视频设备，/dev/video21 是设备的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
videoconvert	videoconvert 是一个 GStreamer 插件，用于转换视频格式。
video/x-raw,format=NV12,width=640,height=480	video/x-raw 是指定视频流的能力描述符，表示原始视频数据。
	format=NV12 指定输出的视频格式为 NV12。
	width=640 和 height=480 指定输出视频的分辨率为 640x480 像素。
waylandsink	waylandsink 是一个视频输出元素，用于在 Wayland 显示服务器上显示视频。它将解码后的视频帧呈现到屏幕上。

4、摄像头拍照

```
root@elf2-buildroot:~# gst-launch-1.0 v4l2src device=/dev/video21 num-buffers=1 ! videoconvert ! video/x-raw,format=NV12,width=640,height=480 ! mppjpegenc ! filesink location=pic.jpg
Setting pipeline to PAUSED ...
Pipeline is live and does not need PREROLL ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
Redistribute latency...
Got EOS from element "pipeline0".
Execution ended after 0:00:00.310412782
Setting pipeline to NULL ...
Freeing pipeline ...
//查看/root 目录下生成 pic.jpg 文件
root@elf2-buildroot:~# ls
pic.jpg
```

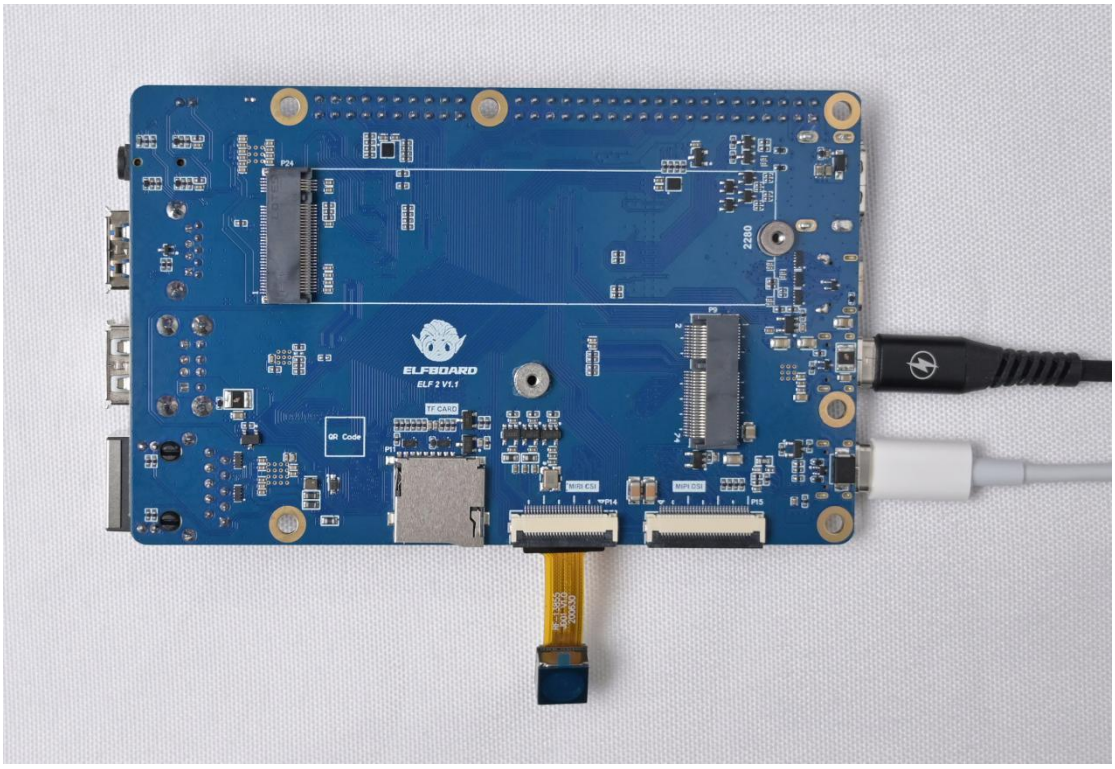
📖 说明：运行此命令将从/dev/video21 设备捕获一帧图像，经过格式转换、JPEG 编码后，将其保存为名为 pic.jpg 的文件。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
v4l2src device=/dev/video21 num-buffers=1	v4l2src 是 GStreamer 中用于访问 V4L2 设备的源插件。它可以从视频捕捉设备（如摄像头）读取视频流。
	device=/dev/video21 指定了要使用的视频设备，/dev/video21 是设备的路径。
	num-buffers=1 从视频源读取的缓冲区数量为 1，只捕获一帧图像。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
videoconvert	videoconvert 是一个 GStreamer 插件，用于转换视频格式。
video/x-raw,format=NV12,width=640,height=480	video/x-raw 是指定视频流的能力描述符，表示原始视频数据。
	format=NV12 指定输出的视频格式为 NV12。
	width=640 和 height=480 指定输出视频的分辨率为 640x480 像素。
mppjpegenc	mppjpegenc 是 GStreamer 中的一个插件，用于将原始视频帧编码为 JPEG 格式。
filesink location=pic.jpg	filesink 是 GStreamer 中的一个插件，用于将数据写入文件。

	location=pic.jpg 指定输出文件的路径和名称，即将生成的 JPEG 文件命名为 pic.jpg。

3.3.4.2. OV13855 测试

OV13855 摄像头模块与 ELF 2 硬件连接如图：



对于 OV13855 等 raw sensor，每一个 sensor 对应 5 个设备节点：

```
COM69 - PuTTY
root@elf2-buildroot:~# grep '' /sys/class/video4linux/video*/name
/sys/class/video4linux/video0/name:stream_cif_mipi_id0
/sys/class/video4linux/video1/name:stream_cif_mipi_id1
/sys/class/video4linux/video10/name:rkcif_tools_id2
/sys/class/video4linux/video11/name:rkisp_mainpath
/sys/class/video4linux/video12/name:rkisp_selfpath
/sys/class/video4linux/video13/name:rkisp_fbcpath
/sys/class/video4linux/video14/name:rkisp_iqtool
/sys/class/video4linux/video15/name:rkisp_rawrd0_m
/sys/class/video4linux/video16/name:rkisp_rawrd2_s
/sys/class/video4linux/video17/name:rkisp_rawrd1_l
/sys/class/video4linux/video18/name:rkisp-statistics
/sys/class/video4linux/video19/name:rkisp-input-params
/sys/class/video4linux/video2/name:stream_cif_mipi_id2
/sys/class/video4linux/video20/name:stream_hdmirx
/sys/class/video4linux/video21/name:4K USB Camera: 4K USB Camera
/sys/class/video4linux/video22/name:4K USB Camera: 4K USB Camera
/sys/class/video4linux/video3/name:stream_cif_mipi_id3
/sys/class/video4linux/video4/name:rkcif_scale_ch0
/sys/class/video4linux/video5/name:rkcif_scale_ch1
/sys/class/video4linux/video6/name:rkcif_scale_ch2
/sys/class/video4linux/video7/name:rkcif_scale_ch3
/sys/class/video4linux/video8/name:rkcif_tools_id0
/sys/class/video4linux/video9/name:rkcif_tools_id1
root@elf2-buildroot:~#
```

Mainpath，指 Rockchip ISP 的一个输出节点，可输出全分辨率图像，一般用来拍照，抓取 Raw 图。
Self Path，指 Rockchip ISP 的一个输出节点，最高只能输出 1080p 分辨率，一般用作预览。
Statistics 3A 统计。

Input-params 3A 参数设置。

OV13855 的测试方法与 UVC Camera 的测试方法基本相同。

1、摄像头识别检测和格式支持查询

```
root@elf2-buildroot:~# v4l2-ctl --list-devices
[   159.770381] m00_b_dw9763 3-000c: dw9763_open: enter, power.usage_count(1)!
rk_hdmirx (fdee0000.hdmirx-controller):
[   159.770422] m00_b_dw9763 3-000c: dw9763_open: exit, power.usage_count(2)!
        /dev/video20
[   159.770435] dw9763 3-000c: cmd 0x80685600 not supported

[   159.770443] dw9763 3-000c: cmd 0xc1007c00 not supported
rkisp-statistics (platform: rkisp):
[   159.770451] m00_b_dw9763 3-000c: dw9763_close: enter, power.usage_count(2)!
        /dev/video18
[   159.770456] m00_b_dw9763 3-000c: dw9763_close: exit, power.usage_count(1)!
        /dev/video19

rkcif-mipi-lvds (platform:rkcif):
        /dev/media0

rkcif (platform:rkcif-mipi-lvds):
        /dev/video0
        /dev/video1
        /dev/video2
        /dev/video3
        /dev/video4
        /dev/video5
        /dev/video6
        /dev/video7
        /dev/video8
        /dev/video9
        /dev/video10

rkisp_mainpath (platform:rkisp0-vir0):
        /dev/video11
        /dev/video12
        /dev/video13
        /dev/video14
        /dev/video15
        /dev/video16
        /dev/video17
        /dev/media1
```

2、摄像头预览

```
root@elf2-buildroot:~# gst-launch-1.0 v4l2src device=/dev/video11 ! video/x-raw,
```

```
format=NV12, width=640, height=480, framerate=30/1 ! waylandsink
Setting pipeline to PAUSED ...
Using mplane plugin for capture
Pipeline is live and does not need PREROLL ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
[  144.625089] rkisp_hw fdcb0000.rkisp: set isp clk = 500000000Hz
New clock: GstSystemClock
[  144.638562] rkcif-mipi-lvds: stream[0] start streaming
[  144.644678] rkcif-mipi-lvds: Allocate dummy buffer, size: 0x01944000
[  144.644733] rockchip-mipi-csi2 mipi0-csi2: stream on, src_sd: 000000005745d45a, sd_name:rockchip-csi2-dphy0
[  144.644740] rockchip-mipi-csi2 mipi0-csi2: stream ON
[  144.644766] rockchip-csi2-dphy0: dphy0, data_rate_mbps 1080
[  144.645125] rockchip-csi2-dphy csi2-dcphy0: csi2_dphy_s_stream stream on:1, dphy0, ret 0
[  144.645137] ov13855 3-0036: ov13855_s_stream: on: 1, 4224x3136@30
Redistribute latency...
0:00:13.6 / 99:99:99.
```

📖 说明：运行此命令将从指定的摄像头设备捕获视频流，转换为 NV12 格式，分辨率 640x480，帧率 30 fps，并通过 Wayland 显示服务器实时显示。

3、摄像头拍照

```
root@elf2-buildroot:~# gst-launch-1.0 v4l2src device=/dev/video11 num-buffers=1 ! video/x-raw,format=NV12,width=640,height=480 ! mppjpegenc ! filesink location=pic.jpg
Setting pipeline to PAUSED ...
Using mplane plugin for capture
Pipeline is live and does not need PREROLL ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
[  185.454999] rkisp_hw fdcb0000.rkisp: set isp clk = 500000000Hz
New clock: GstSystemClock
[  185.466617] rkcif-mipi-lvds: stream[0] start streaming
[  185.470474] rkcif-mipi-lvds: Allocate dummy buffer, size: 0x01944000
[  185.470541] rockchip-mipi-csi2 mipi0-csi2: stream on, src_sd: 000000005745d45a, sd_name:rockchip-csi2-dphy0
[  185.470547] rockchip-mipi-csi2 mipi0-csi2: stream ON
[  185.470573] rockchip-csi2-dphy0: dphy0, data_rate_mbps 1080
[  185.470865] rockchip-csi2-dphy csi2-dcphy0: csi2_dphy_s_stream stream on:1, dphy0, ret 0
[  185.470875] ov13855 3-0036: ov13855_s_stream: on: 1, 4224x3136@30
Redistribute latency...
Got EOS from element "pipeline0".
Execution ended after 0:00:00.240526899
```

```
Setting pipeline to NULL ...
[   185.694528] rkcif-mipi-lvds: stream[0] start stopping, total mode 0x1, cur 0x1
Freeing pipeline ...
//查看是否生成 pic.jpg，可拷贝到 pc 查看
root@elf2-buildroot:~# ls
pic.jpg
```

📖 说明：运行此命令将从指定的摄像头设备/dev/video11 捕获一帧视频，转换为 NV12 格式，并将其编码为 JPEG 格式，然后保存为 pic.jpg 文件。

4、录制 H264 格式视频

```
root@elf2-buildroot:~#  gst-launch-1.0 v4l2src device=/dev/video11 num-buffers=100 ! video/x-raw,format=NV12, width=640,height=480 ! tee name=t ! queue ! mpph264enc ! queue ! h264parse ! qtmux ! filesink location=13855_h264.mp4 t. ! queue ! waylandsink
//摄像头预览时编码 H264
Setting pipeline to PAUSED ...
Using mplane plugin for capture
Pipeline is live and does not need PREROLL ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
[   240.599989] rkisp_hw fdcb0000.rkisp: set isp clk = 500000000Hz
[   240.615863] rkcif-mipi-lvds: stream[0] start streaming
[   240.621480] rkcif-mipi-lvds: Allocate dummy buffer, size: 0x01944000
[   240.621556] rockchip-mipi-csi2 mipi0-csi2: stream on, src_sd: 000000005745d45a, sd_name:rockchip-csi2-dphy0
[   240.621565] rockchip-mipi-csi2 mipi0-csi2: stream ON
[   240.621595] rockchip-csi2-dphy0: dphy0, data_rate_mbps 1080
[   240.621954] rockchip-csi2-dphy csi2-dcphy0: csi2_dphy_s_stream stream on:1, dphy0, ret 0
[   240.621971] ov13855 3-0036: ov13855_s_stream: on: 1, 4224x3136@30
Redistribute latency...
Redistribute latency...
Redistribute latency...
WARNING: from element /GstPipeline:pipeline0/GstWaylandSink:waylandsink0: A lot of buffers are being dropped.
Additional debug info:
../libs/gst/base/gstbasesink.c(3143): gst_base_sink_is_too_late (): /GstPipeline:pipeline0/GstWaylandSink:waylandsink0:
There may be a timestamping problem, or this computer is too slow.
WARNING: from element /GstPipeline:pipeline0/GstWaylandSink:waylandsink0: A lot of buffers are being dropped.
Additional debug info:
../libs/gst/base/gstbasesink.c(3143): gst_base_sink_is_too_late (): /GstPipeline:pipeline0/GstWaylandSink:waylandsink0:
There may be a timestamping problem, or this computer is too slow.
```

```
WARNING: from element /GstPipeline:pipeline0/GstWaylandSink:waylandsink0: A lot of buffers are being dropped.
Additional debug info:
../libs/gst/base/gstbasesink.c(3143): gst_base_sink_is_too_late (): /GstPipeline:pipeline0/GstWaylandSink:waylandsink0:
There may be a timestamping problem, or this computer is too slow.
WARNING: from element /GstPipeline:pipeline0/GstWaylandSink:waylandsink0: A lot of buffers are being dropped.
Additional debug info:
../libs/gst/base/gstbasesink.c(3143): gst_base_sink_is_too_late (): /GstPipeline:pipeline0/GstWaylandSink:waylandsink0:
There may be a timestamping problem, or this computer is too slow.
WARNING: from element /GstPipeline:pipeline0/GstWaylandSink:waylandsink0: A lot of buffers are being dropped.
Additional debug info:
../libs/gst/base/gstbasesink.c(3143): gst_base_sink_is_too_late (): /GstPipeline:pipeline0/GstWaylandSink:waylandsink0:
There may be a timestamping problem, or this computer is too slow.
WARNING: from element /GstPipeline:pipeline0/GstWaylandSink:waylandsink0: A lot of buffers are being dropped.
Additional debug info:
../libs/gst/base/gstbasesink.c(3143): gst_base_sink_is_too_late (): /GstPipeline:pipeline0/GstWaylandSink:waylandsink0:
There may be a timestamping problem, or this computer is too slow.
Got EOS from element "pipeline0".
Execution ended after 0:00:06.858398190
Setting pipeline to NULL ...
Freeing pipeline ...
//查看/root 目录下生成 13855_h264.mp4 文件
root@elf2-buildroot:~# ls
13855_h264.mp4
```

📖 说明：运行此命令将从/dev/video11 设备捕获视频流，编码为 H.264 格式，封装成 MP4 文件保存为 13855_h264.mp4，同时在 Wayland 显示设备上实时播放视频。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
v4l2src device=/dev/video11 num-buffers=100	v4l2src 是用于访问 V4L2（Video for Linux 2）设备的源插件，主要用于从视频捕捉设备（如摄像头）读取视频流。
	device=/dev/video11 指定要使用的视频设备路径。
	num-buffers=100 限制从视频源读取的缓冲区数量为 100，意味着最多捕获 100 帧图像。
!	这个符号在 GStreamer 中表示管道的连接

	符。它用于将一个元素的输出连接到下一个元素的输入。
video/x-raw,framerate=30/1,width=7680,height=4320video/x-raw,format=NV12,width=640,height=480	video/x-raw 是指定视频流的能力描述符，表示原始视频数据。
	format=NV12 指定输出的视频格式为 NV12。
	width=640 和 height=480 指定输出视频的分辨率为 640x480 像素。
tee name=t	tee 是一个 GStreamer 插件，用于将输入流复制到多个输出流。
	name=t 为 tee 元素指定一个名称 t，以便在后续引用。
queue	queue 是 GStreamer 中的一个插件，用于缓冲数据流。
mpph264enc	mpph264enc 是用于将原始视频编码为 H.264 格式的插件。
h264parse	h264parse 是用于解析 H.264 数据流的插件。
qtmux	qtmux 是用于将多媒体流封装成 MP4 格式的插件。
filesink location=13855_h264.mp4	filesink 是 GStreamer 中的一个插件，用于将数据写入文件。
	location=13855_h264.mp4 指定输出文件的路径和名称，即将生成的 MP4 文件命名为 13855_h264.mp4。
t.	表示引用之前定义的 tee 元素
waylandsink	waylandsink 是一个视频输出元素，用于在 Wayland 显示服务器上显示视频。它将解码后的视频帧呈现到屏幕上。

5、播放 H264 格式视频

```
root@elf2-buildroot:~# gst-launch-1.0 filesrc location=13855_h264.mp4 ! qtdemux ! queue ! h264parse ! mppvideodec ! waylandsink
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
Got EOS from element "pipeline0".
Execution ended after 0:00:06.620290207
Setting pipeline to NULL ...
```

```
Freeing pipeline ...
```

🕮 说明：运行此命令将从 13855_h264.mp4 文件中读取视频流，解封装并解析 H.264 数据，然后解码并在 Wayland 显示设备上实时播放视频。

第四章 Desktop 文件系统-板卡接口功能测试

本章测试基于 Desktop 文件系统开展。如需进行本章测试，请参考 "RKDevTool 烧写测试" 章节内容，完成此系统的烧录操作。

镜像文件路径：ELF 2 开发板资料包\03-镜像\00-eMMC 镜像\Elf2 Desktop 镜像。

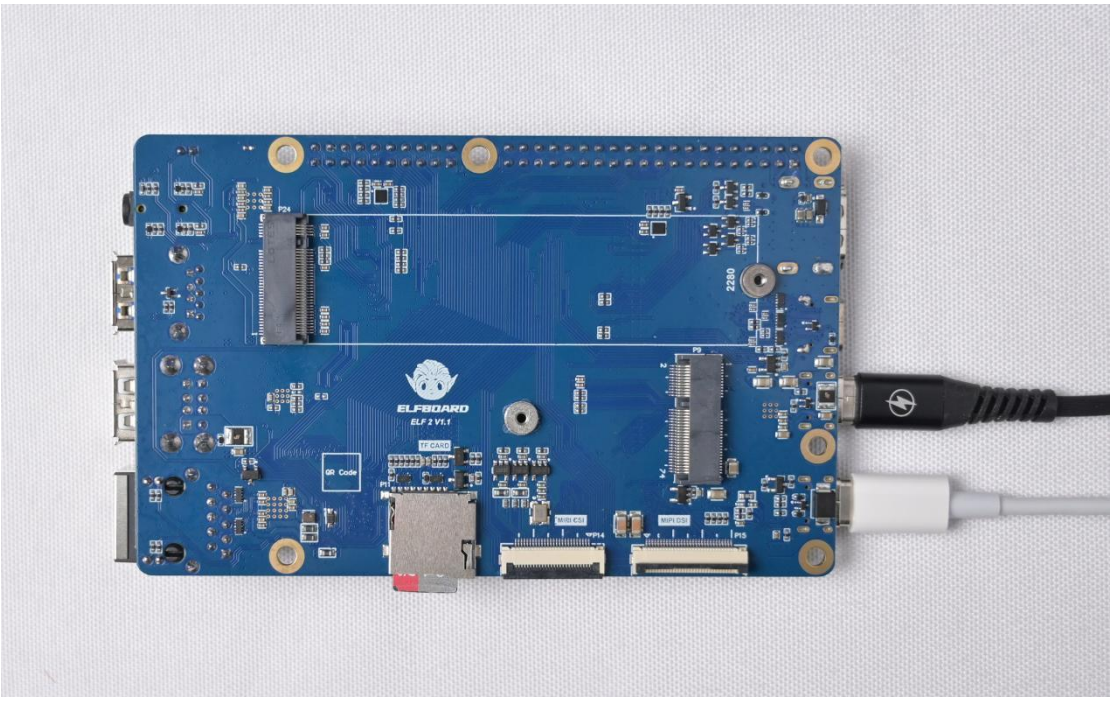
开发板已预装版本为 2.1.0 的 RKNPU2 组件，该组件集成了 0.9.6 版本的 NPU 驱动，采用统一的版本号管理机制，不同版本间可能存在兼容性问题。为确保系统稳定运行，建议用户保持 RKNN-Toolkit2 与 RKNPU2 版本一致，不建议随意更新。若确需升级至最新版本以修复已知问题或提升性能，请参考 Rockchip 官方 GitHub 仓库（https://github.com/rockchip-linux/rknpu）进行操作，并注意在开发板上配套安装 2.1.0 版本的 RKNN Toolkit Lite2 工具，该工具提供 Python 编程接口，支持将通过 RKNN Toolkit2 导出的模型部署至 Rockchip NPU 平台。

4.1. 主板命令行功能测试

ELF 2 内置了丰富的命令行工具可供用户使用。

4.1.1. TF 卡测试

TF 卡与 ELF 2 硬件连接如图：



📖 说明：

- TF 卡挂载目录为/run/media/mmcblk1p1，支持热插拔，终端会打印关于 TF 卡的信息。
- 不同的 TF 卡显示信息可能会有差别，我们测试采用的是闪迪 16G 的 TF 卡。
- 使用 TF 卡测试前需将其格式化成 FAT32 格式。

TF 卡插入 ELF 2 的 TF 卡槽后，系统会自动检查并挂载 TF 卡，挂载成功后，可对 TF 卡进行读写操作。

1.插上 16G TF 卡，没有打印信息。

2.查看挂载目录：

```
root@elf2-desktop:~# mount | grep "mmcblk1p1"
/dev/mmcblk1p1 on /run/media/mmcblk1p1 type vfat
```

```
(rw,relatime,gid=6,fmask=0007,dmask=0007,allow_utime=0020,codepage=936,iochars
et=utf8,shortname=mixed,errors=remount-ro)
```

3.写入测试：

```
root@elf2-desktop:~# dd if=/dev/zero of=/run/media/mmcblk1p1/test bs=1M count=500
conv=fsync
500+0 records in
500+0 records out
524288000 bytes (524 MB, 500 MiB) copied, 21.4785 s, 24.4 MB/s
```

4.读取测试：

⚠**注意：**为确保数据准确，请重启开发板后测试读取速度。

```
root@elf2-desktop:~# dd if=/run/media/mmcblk1p1/test of=/dev/null bs=1M
500+0 records in
500+0 records out
524288000 bytes (524 MB, 500 MiB) copied, 7.78254 s, 67.4 MB/s
```

5.往 TF 卡中写入文件并查看，命令如下：

```
root@elf2-desktop:~# echo 1 > /run/media/mmcblk1p1/test.txt   //写 1 到 test.txt 文件
中
root@elf2-desktop:~# sync                                     //文件同步
root@elf2-desktop:~# cat /run/media/mmcblk1p1/test.txt        //读取 TF 卡中 test.txt 文件
1                                                             //会读到刚才写入的 1
```

6.TF 卡使用完成后，在弹出 TF 卡前，需要使用 umount 卸载 TF 卡：

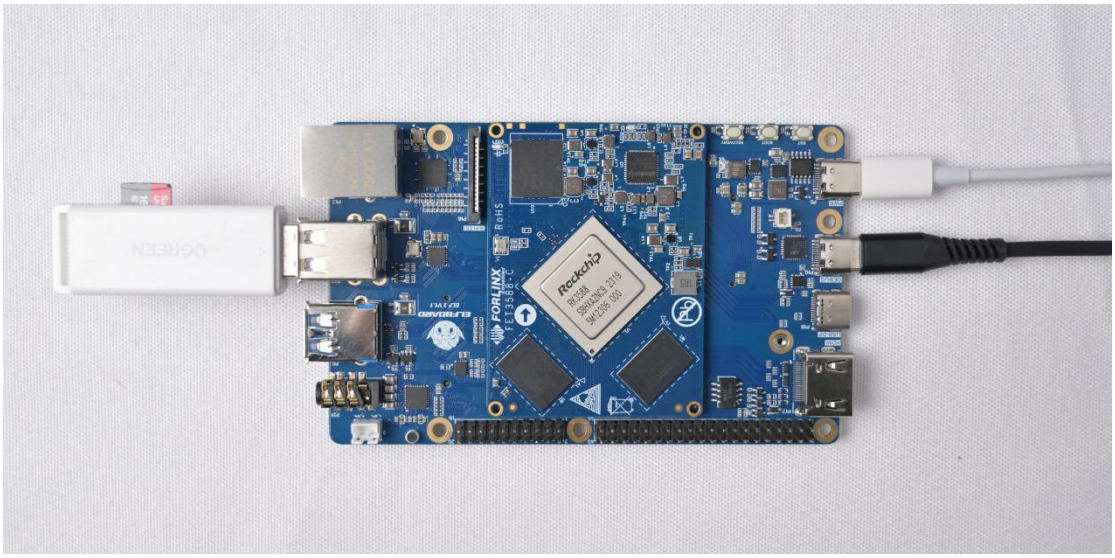
```
root@elf2-desktop:~# umount /run/media/mmcblk1p1
```

⚠**注意：**退出 TF 卡挂载路径后再插拔 TF 卡。

4.1.2. USB 接口测试

4.1.2.1. U 盘测试

U 盘与 ELF 2 硬件连接如图：



🕮 说明：

- 支持 USB 鼠标，USB 键盘，U 盘设备的热插拔。
- 本节测试所用的 U 盘大小为 32G。

- U 盘挂载目录为/run/media/sda1。
- 使用 U 盘测试前需将其格式化成 FAT32 格式。

ELF 2 支持两个 USB2.0 接口和一个 USB3.0 接口。用户可以在任何一个板载 USB HOST 接口上连接 USB 鼠标、 USB 键盘、U 盘等设备，并支持以上设备的热插拔。 这里用挂载 U 盘为例进行演示，目前 U 盘测试支持到 32G，32G 以上并未测试。

1.开发板启动后，插入 U 盘到开发板的 USB HOST 接口，没有打印信息。

2.查看挂载目录，/run/media/sda1 为 USB 存储设备的挂载路径。

```
root@elf2-desktop:~# mount | grep "sda1"
/dev/sda1 on /run/media/sda1 type vfat
(rw,relatime,gid=6,fmask=0007,dmask=0007,allow_utime=0020,codepage=936,
iocharset=utf8,shortname=mixed,errors=remount-ro)
```

3.往 U 盘中写入文件并查看，命令如下：

```
root@elf2-desktop:~# echo 1 > /run/media/sda1/test.txt     //写 1 到 test.txt 文件中
root@elf2-desktop:~# sync                                  //文件同步
root@elf2-desktop:~# cat /run/media/sda1/test.txt          //读取 TF 卡中 test.txt 文件
1                                                          //会读到刚才写入的 1
```

4.U 盘使用完成后，在拔出 U 盘前，需要使用 umount 卸载：

```
root@elf2-desktop:~# umount /run/media/sda1
```

⚠**注意：**退出 U 盘挂载路径后再插拔 U 盘。

4.1.2.2. USB 鼠标测试

鼠标与 ELF 2 硬件连接如图：

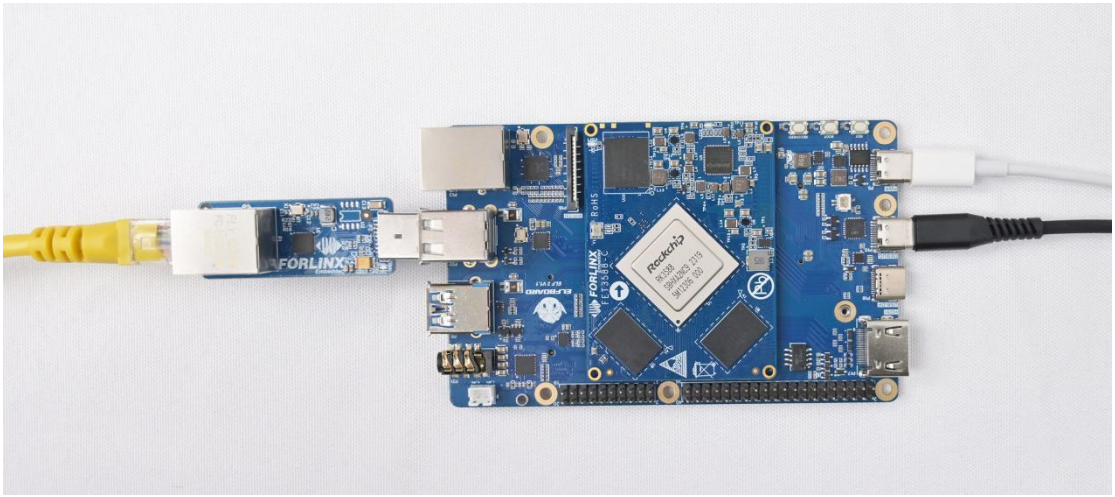


将 USB 鼠标接入 ELF 2 板卡的 USB 接口，没有打印信息。

此时在屏幕上出现箭头光标，鼠标已可正常使用。

4.1.2.3. USB 转网络测试

USB 转网卡模块型号为 RTL8153BI-CG，与 ELF 2 硬件连接如图：



将 USB 转网卡模块插入 ELF 2 开发板的 USB 接口，没有打印信息。

使用如下命令查看是否生成网卡设备：

```
root@elf2-desktop:~# ifconfig -a
eth0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        ether fe:e4:ca:55:40:b4  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 127

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet6 fe80::3687:743:b69e:d5ef  prefixlen 64  scopeid 0x20<link>
        ether 00:0e:c6:87:72:01  txqueuelen 1000  (Ethernet)
        RX packets 15  bytes 5227 (5.2 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 11  bytes 3254 (3.2 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 454  bytes 35390 (35.3 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 454  bytes 35390 (35.3 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

可看到生成网卡 eth1 成功，现在使用 ifconfig 命令设置此网卡 IP：

```
root@elf2-desktop:~# ifconfig eth1 10.10.19.38
root@elf2-desktop:~# ifconfig eth1
eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.10.19.38  netmask 255.0.0.0  broadcast 10.255.255.255
```

```
        inet6 fe80::3687:743:b69e:d5ef  prefixlen 64  scopeid 0x20<link>
        ether 00:0e:c6:87:72:01  txqueuelen 1000  (Ethernet)
        RX packets 492  bytes 409095 (409.0 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 289  bytes 40240 (40.2 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

使用网线连接模块与 PC 机，使用 ping 命令 ping PC 机测试(这里设置 PC 机 IP 为 10.10.19.34)：

```
root@elf2-desktop:~# ping 10.10.19.34 -I eth1 -c 5
PING 10.10.19.34 (10.10.19.34) from 10.10.19.38 eth1: 56(84) bytes of data.
64 bytes from 10.10.19.34: icmp_seq=1 ttl=128 time=0.703 ms
64 bytes from 10.10.19.34: icmp_seq=2 ttl=128 time=0.478 ms
64 bytes from 10.10.19.34: icmp_seq=3 ttl=128 time=0.358 ms
64 bytes from 10.10.19.34: icmp_seq=4 ttl=128 time=0.523 ms
64 bytes from 10.10.19.34: icmp_seq=5 ttl=128 time=0.479 ms

--- 10.10.19.34 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4054ms
rtt min/avg/max/mdev = 0.358/0.508/0.703/0.111 ms
```

4.1.3. 有线网络测试

4.1.3.1. IPV4 基本命令测试

📖 说明：

- 本节示例在本公司的网络环境下进行测试，实际使用中网络环境会有差异，请用户按照自己实际网络环境进行配置。
- ELF 2 有 eth0 一路网卡，出厂默认配置为通过 DHCP 自动获取 IP 地址，用户可以根据下面的指令自行设置 IP。

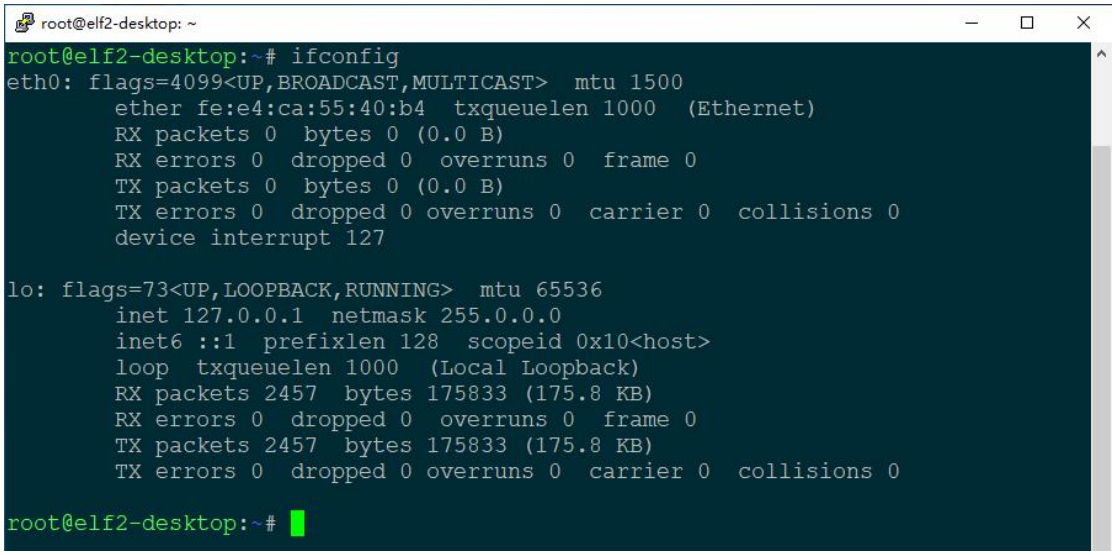
1.在 Linux 系统下，使用 ifconfig 命令可以显示或配置网络设备，使用 ethtool 查询及设置网卡参数。

```
root@elf2-desktop:~# ifconfig
```

📖 说明：

- ‘ifconfig’是 Linux 系统下的一个命令，用于显示和配置网络接口的信息。它可以显示已经配置的网络接口的状态，包括 IP 地址、MAC 地址、网络掩码、广播地址、传输速率等信息。使用‘ifconfig’命令时，可以不带参数，这样会列出系统中所有已经配置的网络接口的状态和信息。也可以使用‘ifconfig’命令加上网络接口的名称，例如‘ifconfig eth0’，这样会显示指定网络接口的详细信息。用户可以使用该命令修改网络接口的参数，例如设置 IP 地址、子网掩码、网关等。

打印信息如下：



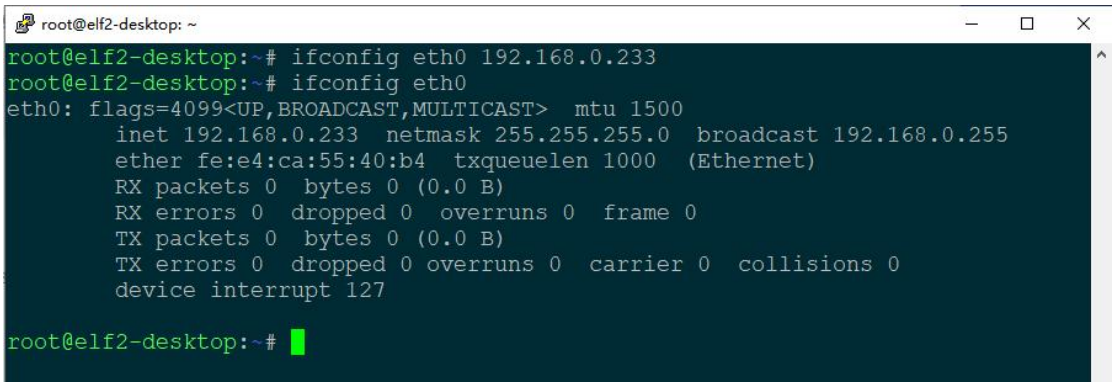
2.设置 IP 地址，将 eth0 IP 设置为 192.168.0.233。

说明：

- 此方法修改 IP 仅在终端中临时生效。

```
root@elf2-desktop:~# ifconfig eth0 192.168.0.233          //设置 IP
root@elf2-desktop:~# ifconfig eth0                        //查看设置后网络状况
```

打印信息如下：



inet 192.168.0.233 可以看出 IP 设置成功。

3.修改 mac 地址。

说明：

- 以下设置方法仅在终端中临时生效。

```
root@elf2-desktop:~# ifconfig eth0 down
root@elf2-desktop:~# ifconfig eth0 hw ether 00:00:00:00:00:01  //eth0 mac 设为 00:
00:00:00:00:01
root@elf2-desktop:~# ifconfig eth0 up
root@elf2-desktop:~# ifconfig eth0
```

打印信息如下：

```
root@elf2-desktop: ~
root@elf2-desktop:~# ifconfig eth0 down
root@elf2-desktop:~# ifconfig eth0 hw ether 00:00:00:00:00:01
root@elf2-desktop:~# ifconfig eth0 up
root@elf2-desktop:~# ifconfig eth0
eth0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 192.168.0.233  netmask 255.255.255.0  broadcast 192.168.0.255
        ether 00:00:00:00:00:01  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 127

root@elf2-desktop:~#
```

HWaddr 00:00:00:00:00:01 可以看出 mac 设置成功。

4.设置子网掩码。

```
root@elf2-desktop:~# ifconfig eth0 netmask 255.255.255.0    //设置 eth0 子网掩码为 255.255.255.0
```

5.设置广播地址。

```
root@elf2-buildroot:~# ifconfig eth0 broadcast 192.168.0.120        //eth0 广播地址设 192.168.0.120
```

6.添加/删除默认网关。

添加默认网关：

```
root@elf2-desktop:~# route add default gw 192.168.0.1      //添加 eth0 默认网关
```

删除默认网关：

```
root@elf2-desktop:~# route del default gw 192.168.0.1          //删除 eth0 默认网关
```

7.关闭/开启网卡。

关闭 eth0 网卡：

```
root@elf2-desktop:~# ifconfig eth0 down                        //关闭 eth0 网卡
```

开启 eth0 网卡：

```
root@elf2-desktop:~# ifconfig eth0 up                          //开启 eth0 网卡
```

4.1.3.2. IPV6 测试

1.设置 IPV6 地址。

```
root@elf2-desktop:~# ip -6 addr add 2001:250:4000:2000::50/64 dev eth0      //设置 IPV6 地址
```

📖 说明：

- 在 Linux 系统中，可以使用‘ip’命令来管理网络接口和配置网络参数。命令中的‘-6’选项用于指定要添加的是 IPv6 地址，‘addr’表示要添加 IP 地址，add 表示添加 IP 地址，后面跟着要添加的 IPv6 地址‘2001:250:4000:2000::50/64’，‘/64’表示网络前缀长度，添加 IP 地址的网络接口名称‘dev eth1’，这里指定的网络接口是‘eth0’。

2.配置电脑 ipv6 地址。

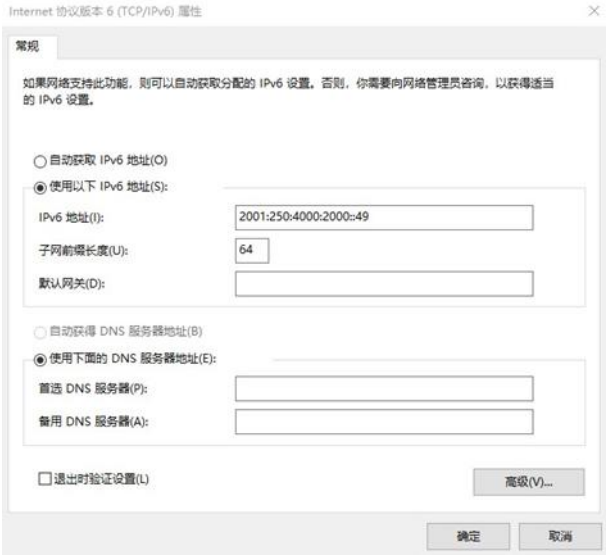
打开控制面板->网络和 Internet->网络和共享中心->更改适配器设置，出现如下界面：



右击以太网，选择属性，取消勾选 ipv4，勾选 ipv6。



双击“Internet 协议版本 6（TCP/IPV6）”，参考下图修改：



将 ELF 2 板卡和电脑用网线直连，使用 ping6 命令测试如下：

```
root@elf2-desktop:~# ping6 2001:250:4000:2000::49
PING 2001:250:4000:2000::49(2001:250:4000:2000::49) 56 data bytes
64 bytes from 2001:250:4000:2000::49: icmp_seq=1 ttl=128 time=1.43 ms
64 bytes from 2001:250:4000:2000::49: icmp_seq=2 ttl=128 time=0.399 ms
64 bytes from 2001:250:4000:2000::49: icmp_seq=3 ttl=128 time=0.501 ms
^C
--- 2001:250:4000:2000::49 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
```

```
rtt min/avg/max/mdev = 0.399/0.640/1.432/0.398 ms
```

4.1.3.3. 以太网配置

🕮 说明：

参考“ IPv4 基本命令测试”章节，可以在命令行中设置板卡的 IP 地址、网关等信息。请注意，这种设置仅为临时生效。如需使其永久生效，需修改相关配置文件。

ELF 2板载一个千兆网卡，插入网线连接网络的情况下，出厂默认配置为通过DHCP自动获取IP地址。如需将eth0网卡设置为静态IP（例如 192.168.0.232），可按以下步骤操作。

查看NetworkManager中当前的网络连接：

```
root@elf2-desktop:~# nmcli con show
NAME                UUID                                  TYPE      DEVICE
Wired connection 1  51065622-78fe-3d8d-b848-d14f8b8d5585  ethernet  eth0
```

这里eth0在NetworkManager中网络连接的名称为“Wired connection 1”。

```
root@elf2-desktop:~# nmcli con mod 'Wired connection 1' ipv4.method manual ipv4.addresses 192.168.0.232/24 ipv4.gateway 192.168.0.1 ipv4.dns 8.8.8.8 connection.autoconnect yes
```

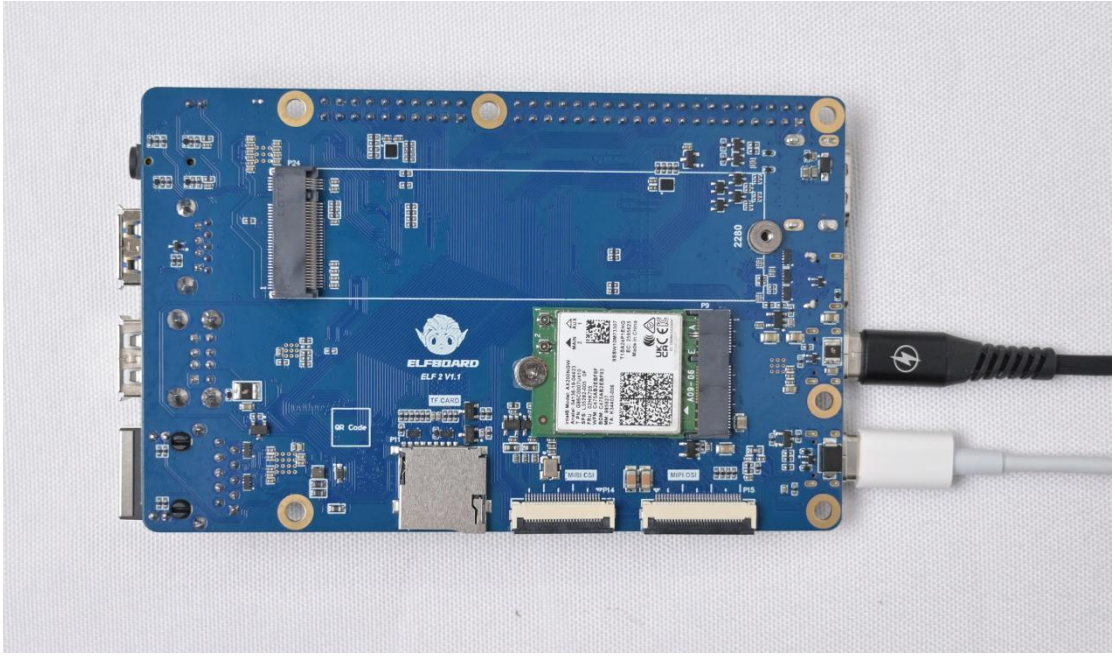
参数	含义
con mod	用于指定需要固定 IP 的网络连接名称
ipv4.address	用于指定需要固定的 Ipv4 地址和子网掩码
Ipv4.gateway	用于指定 Ipv4 网关
ipv4.dns	用于指定 Ipv4 DNS
connection.autoconnect	用于设置自动连接

设置完后激活网络连接，配置生效。

```
root@elf2-desktop:~# nmcli con up 'Wired connection 1'     //激活网络连接
Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/6)
root@elf2-desktop:~# ifconfig eth0                         //查看eth0配置信息已生效
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.0.232  netmask 255.255.255.0  broadcast 192.168.0.255
        inet6 fe80::c5a0:7eb7:3ca2:ee46  prefixlen 64  scopeid 0x20<link>
        ether aa:dc:37:cf:ca:0b  txqueuelen 1000  (Ethernet)
        RX packets 862  bytes 186554 (186.5 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 317  bytes 78463 (78.4 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 127
```

4.1.4. WiFi 测试

wifi&bt 模块与 ELF 2 硬件连接如图：



说明：

- 由于网络环境的不同，所以您在做本实验时，请根据实际情况进行设置。

ELF 2 集成了 wifi&bt 模块 CF-AX200-M，WiFi 模块支持 STA 和 AP 两种模式，以下分别对这两种模式进行操作说明。

ELF 2 上电，启动系统，先使用 lsmod 查看模块加载状态：

```
root@elf2-desktop:~# lsmod
```

```
root@elf2-desktop: ~
root@elf2-desktop:~# lsmod
Module                  Size  Used by
iwlmvm                344064  0
iwlwifi               299008  1 iwlmvm
btusb                  57344  0
btrtl                  24576  1 btusb
btbcm                  24576  1 btusb
btintel                28672  1 btusb
root@elf2-desktop:~#
```

4.1.4.1. STA 模式

该模式即作为一个站点，连接到无线网络中。以下测试中，路由器采用 wpa 加密方式，连接的 WiFi 热点名称为：forlinx，密码为：03123102650。由于网络环境的不同，用户在进行本次测试时，请根据实际情况进行设置：

1.以 CF-AX200-M 模块为例，开发板终端中输入如下命令：

```
root@elf2-desktop:~# cmddemo_wifi.sh -s forlinx -p 03123102650
```

命令中相关参数含义如下：

参数	含义
-s	连接的实际 WiFi 热点名称
-p	后接参数 password 指要连接的实际 WiFi 热点的密码 如果当前热点没有密码，-p 后参数写 NONE

串口打印如下：

```
Device 'wlan0' successfully activated with '01277cc0-1e5d-4473-8710-211674b1978e'.
```

2.检查是否能 ping 外网，在终端中输入如下命令：

```
root@elf2-desktop:~# ping www.elfboard.com -c 3
PING www.elfboard.com (211.149.236.219) 56(84) bytes of data.
64 bytes from 211.149.236.219 (211.149.236.219): icmp_seq=1 ttl=53 time=33.7 ms
64 bytes from 211.149.236.219 (211.149.236.219): icmp_seq=2 ttl=53 time=34.6 ms
64 bytes from 211.149.236.219 (211.149.236.219): icmp_seq=3 ttl=53 time=34.9 ms

--- www.elfboard.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 33.703/34.419/34.905/0.516 ms
```

4.1.4.2. AP 模式

即接入点模式，是一种无线网络工作模式，在这种模式下，设备可以作为一个无线接入点，允许其他无线设备连接到它所提供的网络。

说明：

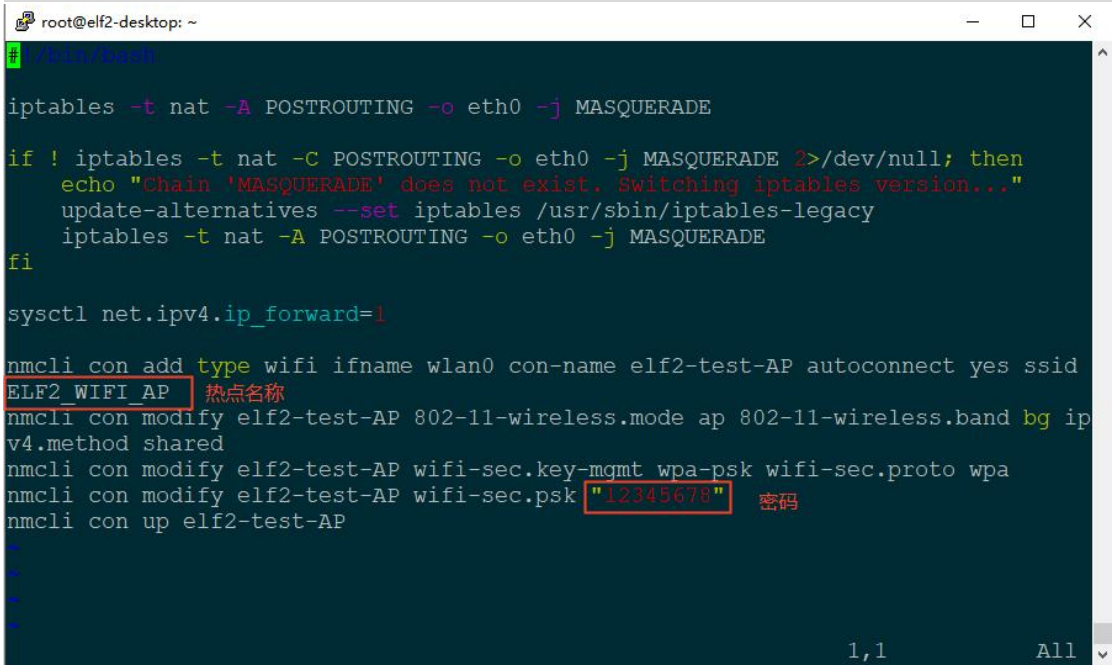
- 进行该项测试前需要保证千兆网卡 eth0 连网，并且网络正常；

WiFi 热点名称：ELF2_WIFI_AP

密码：12345678

热点名称和密码可通过 /usr/bin/cmddemo_hostap.sh 文件查看。

```
root@elf2-desktop:~# vi /usr/bin/cmddemo_hostap.sh
```



执行脚本启动 hostapd 服务，将设备配置为无线接入点：

```
root@elf2-desktop:~# cmddemo_hostap.sh
iptables v1.8.7 (nf_tables): Chain 'MASQUERADE' does not exist
Try `iptables -h' or 'iptables --help' for more information.
Chain 'MASQUERADE' does not exist. Switching iptables version...
update-alternatives: using /usr/sbin/iptables-legacy to provide /usr/sbin/iptables
(iptables) in manual mode
net.ipv4.ip_forward = 1
Connection 'elf2-test-AP' (95c19968-2dc6-459d-83fa-a8feb1d81504) successfully
```

```
added.
Connection successfully activated (D-Bus active path:
/org/freedesktop/NetworkManager/ActiveConnection/6)
```

4.1.5. 蓝牙测试

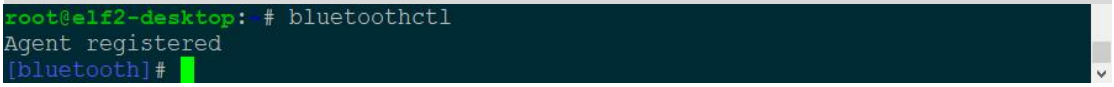
ELF 2 开发板中的 CF-AX200-M 模块，集成了蓝牙功能，本节将演示如何使用蓝牙实现手机与开发板之间的数据传输。

1.ELF 2 上电，启动系统，先使用 lsmod 查看模块加载状态。

```
root@elf2-buildroot:~# lsmod
Module                  Size  Used by
iwlmvm                331776  0
iwlwifi               286720  1 iwlmvm
btusb                  57344  0
btrtl                  24576  1 btusb
btbcm                  24576  1 btusb
btintel                28672  1 btusb
```

2.使用 bluez 命令行工具 bluetoothctl 配置蓝牙。

```
root@elf2-buildroot:~# bluetoothctl
```



查看当前设备信息：

```
[bluetooth]# show
```

```
[bluetooth]# show
Controller EC:8E:77:02:A5:B6 (public)
        Name: elf2-desktop
        Alias: elf2-desktop
        Class: 0x007c0000
        Powered: yes
        Discoverable: no
        DiscoverableTimeout: 0x000000b4
        Pairable: yes
        UUID: Message Notification Se.. (00001133-0000-1000-8000-00805f9b34fb)
        UUID: A/V Remote Control        (0000110e-0000-1000-8000-00805f9b34fb)
        UUID: OBEX Object Push          (00001105-0000-1000-8000-00805f9b34fb)
        UUID: Message Access Server     (00001132-0000-1000-8000-00805f9b34fb)
        UUID: PnP Information           (00001200-0000-1000-8000-00805f9b34fb)
        UUID: IrMC Sync                 (00001104-0000-1000-8000-00805f9b34fb)
        UUID: Vendor specific           (00005005-0000-1000-8000-0002ee000001)
        UUID: Headset                   (00001108-0000-1000-8000-00805f9b34fb)
        UUID: A/V Remote Control Target (0000110c-0000-1000-8000-00805f9b34fb)
        UUID: Generic Attribute Profile (00001801-0000-1000-8000-00805f9b34fb)
        UUID: Phonebook Access Server   (0000112f-0000-1000-8000-00805f9b34fb)
        UUID: Audio Sink                (0000110b-0000-1000-8000-00805f9b34fb)
        UUID: Device Information        (0000180a-0000-1000-8000-00805f9b34fb)
        UUID: Generic Access Profile    (00001800-0000-1000-8000-00805f9b34fb)
        UUID: Handsfree Audio Gateway   (0000111f-0000-1000-8000-00805f9b34fb)
        UUID: Audio Source              (0000110a-0000-1000-8000-00805f9b34fb)
        UUID: OBEX File Transfer        (00001106-0000-1000-8000-00805f9b34fb)
        Modalias: usb:v1D6Bp0246d0540
        Discovering: no
        Roles: central
        Roles: peripheral
Advertising Features:
        ActiveInstances: 0x00 (0)
        SupportedInstances: 0x0c (12)
        SupportedIncludes: tx-power
        SupportedIncludes: appearance
        SupportedIncludes: local-name
        SupportedSecondaryChannels: 1M
        SupportedSecondaryChannels: 2M
        SupportedSecondaryChannels: Coded
[bluetooth]#
```

依次执行以下命令对蓝牙设备控制器进行配置：

```
[ bluetooth ]# power on                                    //启动蓝牙设备
Changing power on succeeded
[ bluetooth ]# pairable on                                 //设置为配对模式
Changing pairable on succeeded
[ bluetooth ]# discoverable on                             //设置为可发现模式
Changing discoverable on succeeded
[CHG] Controller EC:8E:77:02:A5:B6 Discoverable: yes
[ bluetooth ]# agent on                                    //启动代理
Agent is already registered
[ bluetooth ]# default-agent                               //设置当前代理为默认代理
Default agent request successful
```

3.开发板被动配对。

打开手机蓝牙搜索，会出现一个“elf2-desktop”的设备，点击配对，同时开发板上打印信息如下，输入yes。

```
[NEW] Device B4:6E:10:2A:CF:34 vivo X200 Pro
Request confirmation
[agent] Confirm passkey 807221 (yes/no): yes
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001112-0000-1000-8000-00805f9b34fb
```

```
[CHG] Device B4:6E:10:2A:CF:34 Modalias: bluetooth:v00E0p0000d0000
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001105-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000110a-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000110c-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000110e-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001112-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001115-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001116-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000111f-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000112f-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001132-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001200-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001800-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001801-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001855-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 2c042b0a-7f57-4c0a-afcf-1762af70257c
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 8fa9c715-bd1f-596c-a1b0-13162b15c892
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 9fed64fd-e91a-499e-88dd-73dfe023feed
[CHG] Device B4:6E:10:2A:CF:34 ServicesResolved: yes
[CHG] Device B4:6E:10:2A:CF:34 Paired: yes
Authorize service
[agent] Authorize service 00001108-0000-1000-8000-00805f9b34fb (yes/no): yes
Authorize service
[agent] Authorize service 0000110d-0000-1000-8000-00805f9b34fb (yes/no): yes
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001105-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000110a-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000110c-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000110d-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000110e-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001112-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001115-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001116-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000111f-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000112f-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001132-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001200-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001800-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001801-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001855-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 2c042b0a-7f57-4c0a-afcf-1762af70257c
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 8fa9c715-bd1f-596c-a1b0-13162b15c892
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 9fed64fd-e91a-499e-88dd-73dfe023feed
Authorize service
[agent] Authorize service 0000110e-0000-1000-8000-00805f9b34fb (yes/no): yes
```

```
[vivo X200 Pro]#
```

查看和移除连接设备：

```
[vivo X200 Pro]# devices
Device B4:6E:10:2A:CF:34 vivo X200 Pro
[vivo X200 Pro]# remove B4:6E:10:2A:CF:34
```

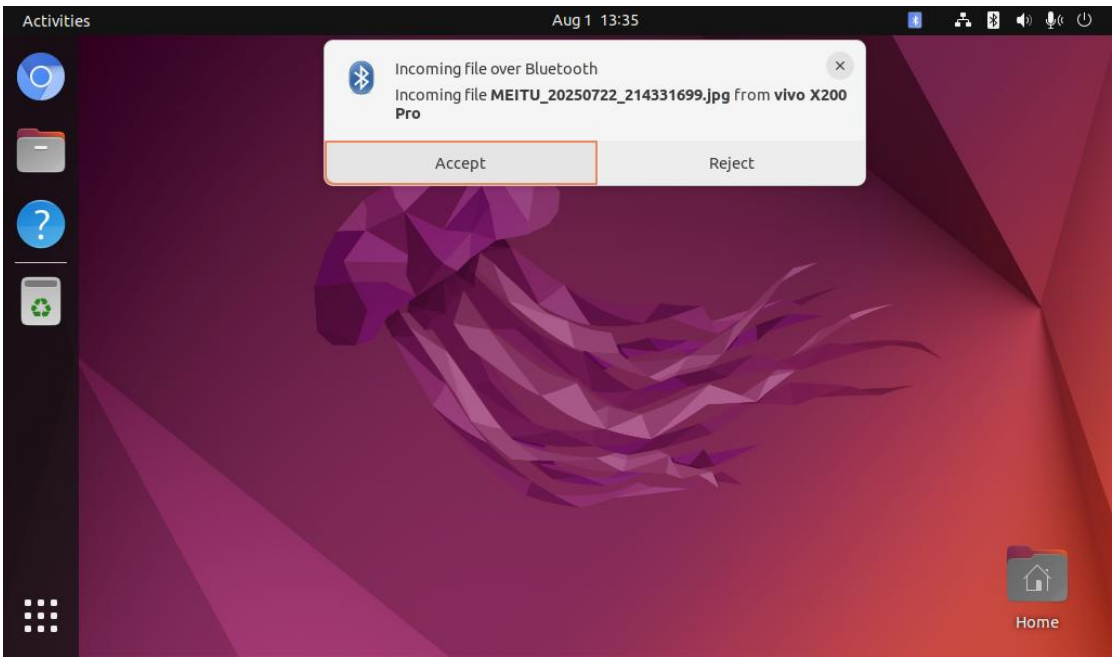
4.开发板主动配对。

除了被动配对，也可以在开发板终端发送主动配对的请求。

```
[bluetooth]# scan on    //搜索可被发现蓝牙
Discovery started
[CHG] Controller EC:8E:77:02:A5:CF Discovering: yes
[NEW] Device B4:6E:10:2A:CF:34 vivo X200 Pro
[NEW] Device 5E:47:90:A6:69:EB 5E-47-90-A6-69-EB
[bluetooth]# scan off    //停止搜索
[bluetooth]# pair B4:6E:10:2A:CF:34    //配对蓝牙
Attempting to pair with B4:6E:10:2A:CF:34
[CHG] Device B4:6E:10:2A:CF:34 Connected: yes
Request confirmation
[agent] Confirm passkey 826399 (yes/no): yes
[CHG] Device B4:6E:10:2A:CF:34 Modalias: bluetooth:v00E0p0000d0000
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001105-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000110a-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000110c-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000110e-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001112-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001115-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001116-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000111f-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 0000112f-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001132-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001200-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001800-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001801-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 00001855-0000-1000-8000-00805f9b34fb
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 2c042b0a-7f57-4c0a-afcf-1762af70257c
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 8fa9c715-bd1f-596c-a1b0-13162b15c892
[CHG] Device B4:6E:10:2A:CF:34 UUIDs: 9fed64fd-e91a-499e-88dd-73dfe023feed
[CHG] Device B4:6E:10:2A:CF:34 ServicesResolved: yes
[CHG] Device B4:6E:10:2A:CF:34 Paired: yes
Pairing successful
[vivo X200 Pro]#
```

5.开发板接收文件。

配对成功后，在手机上可以使用蓝牙发送文件至 ELF 2 开发板上（此操作需要开发板连接屏幕、鼠标），Ubuntu 会提示是否接受， 点击 Accept。接收到的文件默认保存在/home/elf/Downloads 目录下。



6.开发板发送文件。

注意：此操作在 elf 用户下进行。

开发板可以使用蓝牙发送文件至手机上，手机会提示是否接受， 点击接受，进行文件传输。

```
[vivo X200 Pro]# quit
root@elf2-desktop:~# su elf
elf@elf2-desktop:/root$ cd /home/elf/
elf@elf2-desktop:~$ export
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
elf@elf2-desktop:~$ obexctl
[NEW] Client /org/bluez/obex
[obex]# connect B4:6E:10:2A:CF:34
Attempting to connect to B4:6E:10:2A:CF:34
[NEW] Session /org/bluez/obex/client/session0 [default]
[NEW] ObjectPush /org/bluez/obex/client/session0
Connection successful
[B4:6E:10:2A:CF:34]# send /home/elf/Downloads/1.jpg
Attempting to send /home/elf/Downloads/1.jpg to /org/bluez/obex/client/session0
[NEW] Transfer /org/bluez/obex/client/session0/transfer0
Transfer /org/bluez/obex/client/session0/transfer0
        Status: queued
        Name: 1.jpg
        Size: 2306463
        Filename: /home/elf/Downloads/1.jpg
        Session: /org/bluez/obex/client/session0
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Status: active
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 4058 (@4KB/s
09:27)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 110268
```

```
(@106KB/s 00:20)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 224648
(@114KB/s 00:18)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 322688
(@98KB/s 00:20)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 432983
(@110KB/s 00:16)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 531023
(@98KB/s 00:18)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 649488
(@118KB/s 00:13)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 772038
(@122KB/s 00:12)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 870078
(@98KB/s 00:14)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 984458
(@114KB/s 00:11)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 1090668
(@106KB/s 00:11)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 1205048
(@114KB/s 00:09)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 1303088
(@98KB/s 00:10)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 1401128
(@98KB/s 00:09)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 1511423
(@110KB/s 00:07)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 1617633
(@106KB/s 00:06)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 1719758
(@102KB/s 00:05)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 1825968
(@106KB/s 00:04)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 1936263
(@110KB/s 00:03)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 2034303
(@98KB/s 00:02)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 2132343
(@98KB/s 00:01)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Transferred: 2234468
(@102KB/s 00:00)
[CHG] Transfer /org/bluez/obex/client/session0/transfer0 Status: complete
[DEL] Transfer /org/bluez/obex/client/session0/transfer0
[B4:6E:10:2A:CF:34]#
```

每次文件传输都会创建一个新的 OBEX 会话（session0）。传输完成后，会话自动关闭（[DEL] Session），因此不能直接复用。再次发送文件时，需要重新初始化 OBEX 连接。

4.1.6. 看门狗测试

看门狗是嵌入式系统中经常用到的功能，ELF 2中看门狗的设备节点为/dev/watchdog。

参数说明：

参数	含义	说明
settimeout	开启看门狗，设置复位时间，不喂狗	复位时间需大于 2s
keepalive	开启看门狗，设置复位时间，定时 2s 喂狗	复位时间需大于 2s

⚠**注意：**测试时以上两个参数只能使用一个，可通过 ps 查看进程，保证只有一个 cmddemo_wdt 进程。

1.开启看门狗，设置复位时间测试。

执行 60s 复位命令，（可设置最小复位时间间隔为 2s），60s 后 ELF 2 复位：

```
root@elf2-desktop:~# cmddemo_wdt /dev/watchdog settimeout 60 &   //执行 60s 复位命令
[1] 1633
```

🕮 说明：

- 在该命令中，‘cmddemo_wdt’表示要使用 cmddemo_wdt 工具程序；‘/dev/watchdog’表示硬件看门狗的设备文件路径；‘settimeout 60’表示设置超时时间为 60 秒；‘&’表示将 cmddemo_wdt 命令放到后台运行。当用户执行‘cmddemo_wdt /dev/watchdog settimeout 60 &’命令时，它会使用 cmddemo_wdt 工具程序来设置硬件看门狗的超时时间为 60 秒。

等待 60s 后可以看到系统重启。

2.cmddemo_wdt的参数keepalive为设置复位时间，并进行2s定时喂狗，10为设置的复位时间为10s。

```
root@elf2-desktop:~# cmddemo_wdt /dev/watchdog keepalive 10 &
[1] 1571
```

如果 killall 关闭 cmddemo_wdt 应用在后台的运行，则会造成系统在 10s 内不去喂狗而超时重启。

```
root@elf2-desktop:~# killall cmddemo_wdt
[    42.372342] watchdog: watchdog0: watchdog did not stop!
[1]+  Terminated                  cmddemo_wdt /dev/watchdog keepalive 10
```

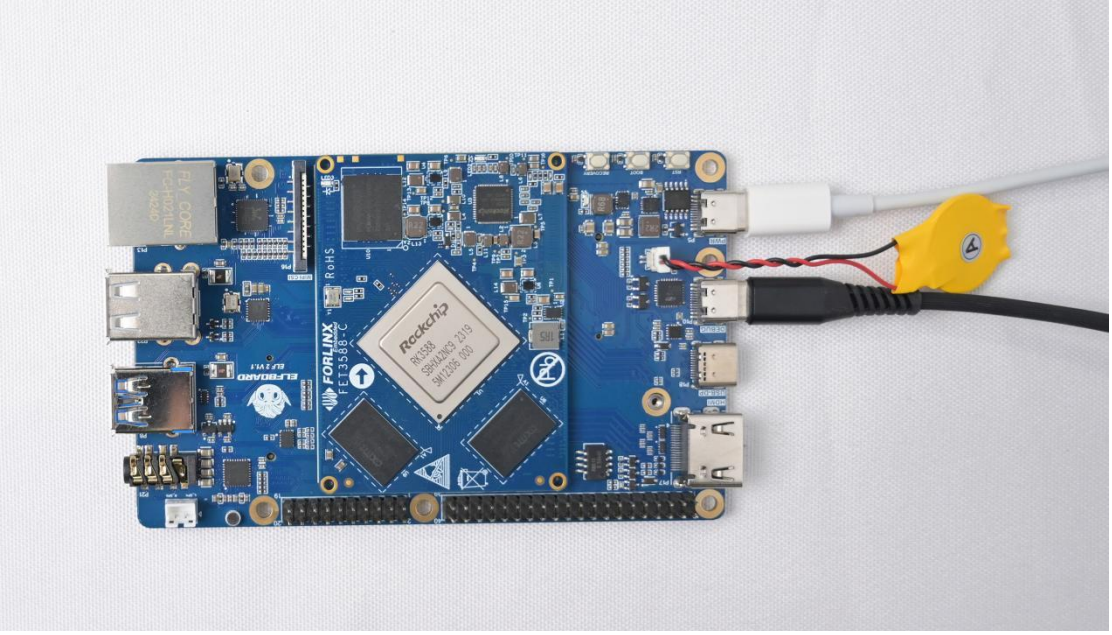
🕮 说明：

- 在该命令中，‘killall’表示终止具有指定名称的进程；‘cmddemo_wdt’表示要终止的进程名称。当用户执行‘killall cmddemo_wdt’命令时，它会终止系统中所有名为 cmddemo_wdt 的进程。

4.1.7. RTC 时钟驱动测试

⚠**注意：**请确保板子上已经安装了电池，并且电池电压正常，请注意，禁止对提供的电池进行充电。

ELF 2 板卡使用的 RTC 芯片为 RX8010，电池与 ELF 2 硬件连接如图：



RTC测试，主要通过使用date和hwclock工具设置软、硬件时间，测试当板子断电再上电的时候，软件时钟读取RTC时钟是否同步。

时间设置。

```
root@elf2-desktop:~# date -s "2024-12-12 17:23:00"   //设置软件时间
Thu Dec 12 05:23:00 PM CST 2024
root@elf2-desktop:~# hwclock -wu                     //将软件时间同步到硬件时间
root@elf2-desktop:~# hwclock -r                      //显示硬件时间
2024-12-12 17:23:08.359834+08:00
```

然后给板子断电再上电，进入系统后读取系统时间，可以看到时间已经同步。

```
root@elf2-desktop:~# date
Thu Dec 12 05:24:01 PM CST 2024
```

4.1.8. 放/录音测试

ELF 2 使用 NAU88C22YG 芯片，提供了 1 路标准 3.5mm 音频插座，1 个 XH2.0-2P 白色插座，可驱动 8Ω 喇叭，最高输出功率为 1W，在进行放音测试前，请将准备好的耳机插入听筒接口，或将喇叭插入底板上的对应插槽上进行测试。

⚠**注意：**在播放音频时，当耳机和喇叭同时连接，默认声音会从耳机输出。而在录音时，如果耳机和板载麦克风同时存在，录音将会通过耳机进行。

4.1.8.1. HDMI 播放声音

用 HDMI 线将 ELF 2 开发板的 HDMI 接口与显示器的 HDMI 接口连接起来，可以正常显示桌面。

1.列出系统中可用的音频播放硬件设备。

```
root@elf2-desktop:~# aplay -l
**** List of PLAYBACK Hardware Devices ****
card 0: rockchipdp0 [rockchip,dp0], device 0: rockchip,dp0 spdif-hifi-0 [rockchip,dp
0 spdif-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 1: rockchipnau8822 [rockchip-nau8822], device 0: dailink-multicodecs nau882
```

```
2-hifi-0 [dailink-multicodecs nau8822-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 2: rockchiphdmi0 [rockchip-hdmi0], device 0: rockchip-hdmi0 i2s-hifi-0 [rockch
ip-hdmi0 i2s-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
```

相关参数含义如下，alpay 更多使用方法可通过 aplay --help 查看。

-l	列出所有声卡和数字音频设备
-L	列出设备名称
-t	文件类型（voc、wav、raw 或 au）
-D	指定声卡设备

使用 aplay -l 列出所有声卡和数字音频设备：

第一个音频卡（card 0）名为 rockchipdp0，具有一个高保真的音频输出设备（device 0），并且该设备有一个可用的子设备。

第二个音频卡（card 1）为 rockchipnau8822，同样具有一个高保真输出设备（device 0），并且也有一个可用的子设备。

第三个音频卡（card 2）为 rockchiphdmi0，同样具有一个高保真输出设备（device 0），并且也有一个可用的子设备。

2.下面使用 rockchiphdmi0 播放指定的 MP3 文件。将耳机插入显示器的音频接口，然后在命令行中执行以下命令，以听到播放的音频。

```
root@elf2-desktop:~# gst-play-1.0 /userdata/media/audio/piano2-CoolEdit.mp3
--audiosink="alsasink device=plughw:2,0"
```

gst-play-1.0	GStreamer 是一个强大的多媒体框架，它提供了一套完整的工具、库和插件，支持音频、视频的处理和播放。gst-play-1.0 是 GStreamer 的一个命令行工具，用于播放多媒体文件。
/userdata/media/audio/piano2-CoolEdit.mp3	播放的音频文件的路径。此处是一个 MP3 格式的音频文件，文件名为 piano2-CoolEdit.mp3
--audiosink="alsasink device=plughw:2,0"	"alsasink device=plughw:2,0" 表示使用 ALSA 作为音频输出，并指定了具体的设备。plughw:2,0 是 ALSA 的设备名称，2 是声卡的索引，0 是设备的索引。它告诉系统使用 card 2 声卡的 device 0 设备进行输出。

4.1.8.2. 耳机播放声音

将耳机插入 ELF 2 开发板的音频插座上，然后在命令行中执行以下命令，以听到播放的音频。

耳机与 ELF 2 硬件连接如图：

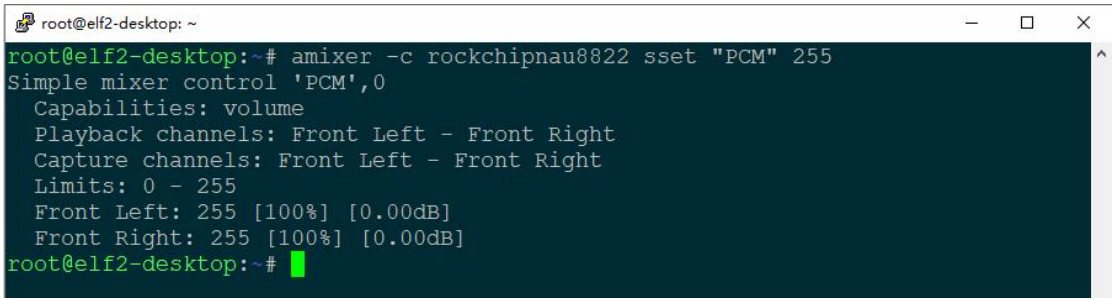


1.查询音频参数

```
root@elf2-desktop:~# amixer -c rockchipnau8822
```

2.设置 PCM 参数

```
root@elf2-desktop:~# amixer -c rockchipnau8822 sset "PCM" 255
```

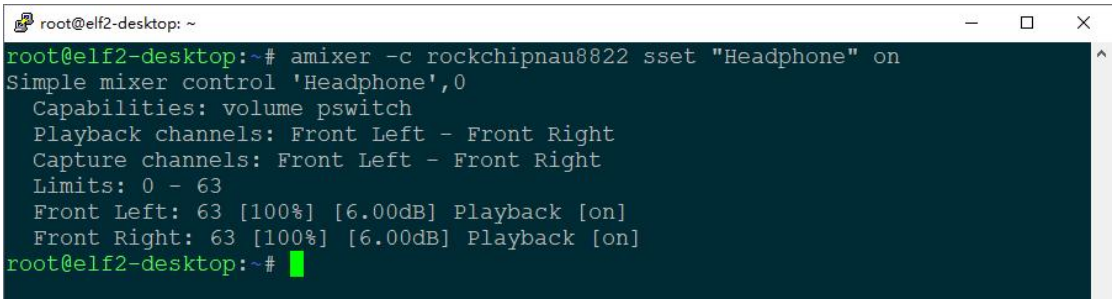


说明：

该行代码是一个命令行命令，用于设置音量控制。amixer 是用于音频混合和控制的命令行工具；-c 后面跟着的是声卡设备 rockchipnau8822 。如果有多个音频设备，使用此选项可以明确指定要控制的设备。sset 是 amixer 的一个参数，用于设置特定控制的音量级别；"PCM"是要设置音量的目标，即 脉冲编码调制（PCM）；255 是要设置的音量级别，可以是 0 到 255 之间的任意整数值，表示音量的百分比。在这里，将 PCM 的音量设置为最大值 255。

3.打开 Speaker

```
root@elf2-desktop:~# amixer -c rockchipnau8822 sset "Headphone" on
```

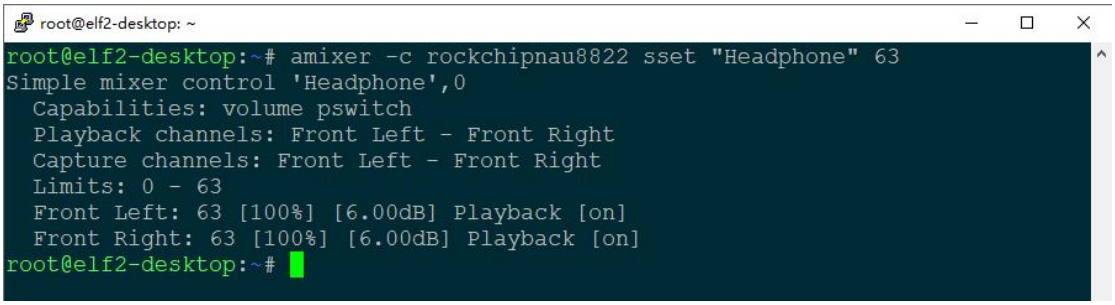


说明：

该行代码是一个命令行命令，用于控制耳机的开关状态。amixer 是用于音频混合和控制的命令行工具； -c 后面跟着的是声卡设备 rockchipnau8822 。如果有多个音频设备，使用此选项可以明确指定要控制的设备。sset 是 amixer 的一个参数，用于设置特定控制的属性；"Headphoner"是要设置的目标，即耳机；on 表示要将耳机打开。这意味着耳机将开始输出音频信号。

4.设置耳机音量

```
root@elf2-desktop:~# amixer -c rockchipnau8822 sset "Headphone" 63
```



说明：

该行代码是一个命令行命令，用于设置耳机的音量级别。amixer 是用于音频混合和控制的命令行工具；sset 是 amixer 的一个参数，用于设置特定控制的音量级别；"Headphone"是要设置的目标，即耳机；63 是要设置的音量级别，表示将耳机的音量设置为 63。该值可以是 0 到 63 之间的任意整数值，表示音量的百分比。在这里，将耳机的音量设置为 100%。

5.使用耳机播放音频

```
root@elf2-desktop:~# gst-play-1.0 /userdata/media/audio/piano2-CoolEdit.mp3
--audiosink="alsasink device=plughw:1,0"
```

4.1.8.3. SPKOUT 播放声音

将喇叭插入 ELF 2 开发板的音频插座上，然后在命令行中执行以下命令，以听到播放的音频。

喇叭与 ELF 2 硬件连接如图：



1.查询音频参数

```
root@elf2-desktop:~# amixer -c rockchipnau8822
```

2.设置 PCM 参数

```
root@elf2-desktop:~# amixer -c rockchipnau8822 sset "PCM" 255
```

```
root@elf2-desktop: ~
root@elf2-desktop:~# amixer -c rockchipnau8822 sset "PCM" 255
Simple mixer control 'PCM',0
  Capabilities: volume
  Playback channels: Front Left - Front Right
  Capture channels: Front Left - Front Right
  Limits: 0 - 255
  Front Left: 255 [100%] [0.00dB]
  Front Right: 255 [100%] [0.00dB]
root@elf2-desktop:~#
```

说明：

该行代码是一个命令行命令，用于设置音量控制。amixer 是用于音频混合和控制的命令行工具；-c 后面跟着的是声卡设备 rockchipnau8822 。如果有多个音频设备，使用此选项可以明确指定要控制的设备。sset 是 amixer 的一个参数，用于设置特定控制的音量级别；"PCM"是要设置音量的目标，即脉冲编码调制（PCM）；255 是要设置的音量级别，可以是 0 到 255 之间的任意整数值，表示音量的百分比。在这里，将 PCM 的音量设置为最大值 255。

3.打开 Speaker

```
root@elf2-desktop:~# amixer -c rockchipnau8822 sset "Speaker" on
```

```
root@elf2-buildroot:~# amixer -c rockchipnau8822 sset "Speaker" on
Simple mixer control 'Speaker',0
  Capabilities: volume pswitch
  Playback channels: Front Left - Front Right
  Capture channels: Front Left - Front Right
  Limits: 0 - 63
  Front Left: 63 [100%] [6.00dB] Playback [on]
  Front Right: 63 [100%] [6.00dB] Playback [on]
root@elf2-buildroot:~#
```

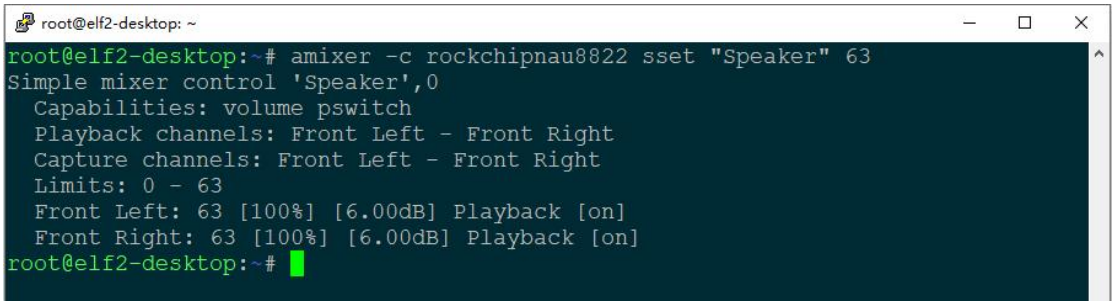
说明：

该行代码是一个命令行命令，用于控制喇叭的开关状态。amixer 是用于音频混合和控制的命令行工具； -c 后面跟着的是声卡设备 rockchipnau8822 。如果有多个音频设备，使用此选项可以明确

指定要控制的设备。sset 是 amixer 的一个参数，用于设置特定控制的属性；"Speaker"是要设置的目标，即喇叭；on 表示要将喇叭打开。这意味着喇叭将开始输出音频信号。

4.设置喇叭音量

```
root@elf2-desktop:~# amixer -c rockchipnau8822 sset "Speaker" 63
```



说明：

该行代码是一个命令行命令，用于设置喇叭的音量级别。amixer 是用于音频混合和控制的命令行工具；sset 是 amixer 的一个参数，用于设置特定控制的音量级别；"Speaker"是要设置的目标，即喇叭；63 是要设置的音量级别，表示将喇叭的音量设置为 63。该值可以是 0 到 63 之间的任意整数值，表示音量的百分比。在这里，将喇叭的音量设置为 100%。

5.使用喇叭播放音频

```
root@elf2-desktop:~# gst-play-1.0 /userdata/media/audio/piano2-CoolEdit.mp3
--audiosink="alsasink device=plughw:1,0"
```

4.1.8.4. MIC 输入

1.查看系统中可用的音频捕获（录音）硬件设备。

```
root@elf2-desktop:~# arecord -l
**** List of CAPTURE Hardware Devices ****
card 1: rockchipnau8822 [rockchip-nau8822], device 0: dailink-multicodecs nau8822-hifi-0 [dailink-multicodecs nau8822-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
```

音频卡（card 0）名为 rockchipnau8822，它的设备编号为 0，支持高保真的音频捕获，并且有一个可用的子设备。

2.下面使用 rockchipnau8822 录制 3 秒的 CD 质量音频，并将其保存为名为 test1.wav 的文件。

```
root@elf2-desktop:~# arecord -D hw:rockchipnau8822,0 -d 3 -f cd -t wav test1.wav
```

说明：

该行代码是一个命令行命令，arecord 是用于录制音频的命令行工具；-D 表示要使用的音频输入设备。hw:rockchipnau8822,0：这里的 hw 表示直接使用硬件设备，rockchipnau8822 是音频设备的名称，0 是设备的索引。这个参数用于选择具体的声卡设备。-d 3 指定录音将持续 3 秒。-f 指定录音的格式。 -t wav test1.wav 指定输出的音频文件格式为 WAV，录制的音频将被保存为 test1.wav。

3.使用 HDMI 播放采集声音

```
root@elf2-desktop:~# aplay -D plughw:2,0 test1.wav
```

4.下面使用 rockchipnau8822 播放 test1.wav 文件。将耳机插入 ELF 2 开发板的音频插座上，然后在命令行中执行以下命令，以听到播放的音频。

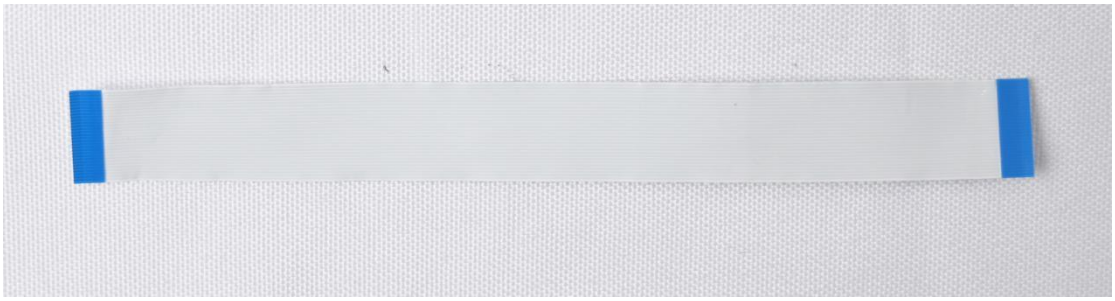
```
root@elf2-desktop:~# aplay -D plughw:1,0 test1.wav
```

4.1.9. LCD 屏测试

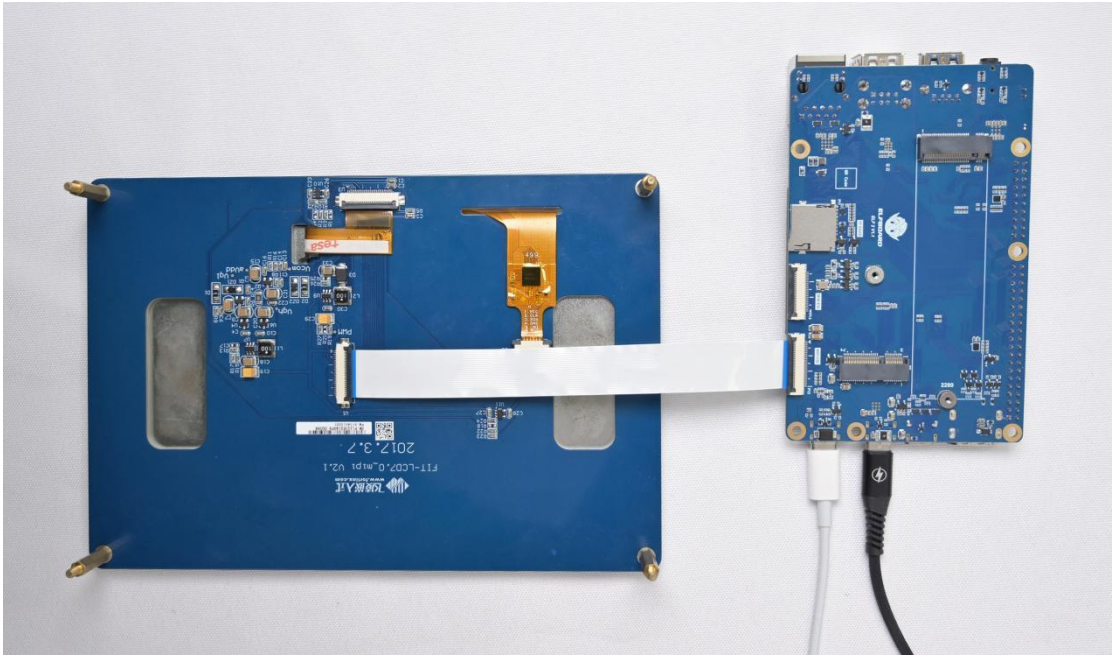
背光的亮度设置范围为（0--255），255表示亮度最高，0表示关闭背光亮度。在MIPI DSI上接上mipi屏幕后，上电启动。进入系统后在终端输入如下命令进行背光测试。

1.屏幕连接

用 FPC 排线进行连接，下图是同向 FPC 排线（蓝色在同一侧）图片，本节测试使用同向 FPC 排线进行连接。



LCD 屏幕与 ELF 2 硬件连接如图：



1.查看支持背光型号。

```
root@elf2-desktop:~# ls /sys/class/backlight/
backlight-dsi0    显示当前支持屏背光型号
```

下面以dsi0为例：

2.查看当前屏幕背光值。

```
root@elf2-desktop:~# cat /sys/class/backlight/backlight-dsi0/brightness
78                                                    //当前背光值为 78
```

3.背光熄灭。

```
root@elf2-desktop:~# echo 0 > /sys/class/backlight/backlight-dsi0/brightness
```

4.LCD 背光亮起。

```
root@elf2-desktop:~# echo 125 > /sys/class/backlight/backlight-dsi0/brightness
```

4.1.10. 调频测试

说明：四核 A55 分别是 cpu0、cpu1、cpu2、cpu3; 四核 A76 分别是 cpu5、cpu6、cpu7、cpu8。此过程以 cpu0 为例操作，实际过程 cpu1、cpu2、cpu3 会同时改变； 单独操作 cpu4、 cpu5、cpu6、cpu7 相互之间不会受到影响。

1.当前内核中支持的所有 cpufreq governor 类型。

```
root@elf2-desktop:~# cat
/sys/devices/system/cpu/cpu0/cpufreq/scaling_available_governors
interactive conservative ondemand userspace powersave performance schedutil
```

其中 userspace 表示用户模式，在此模式下允许其他用户程序调节 CPU 频率。

2.查看当前 CPU 支持的频率档位。

```
root@elf2-desktop:~# cat
/sys/devices/system/cpu/cpu0/cpufreq/scaling_available_frequencies
408000 600000 816000 1008000 1200000 1416000 1608000 1800000
```

3.设置为用户模式，修改频率为 1800000。

```
root@elf2-desktop:~# echo userspace > /sys/devices/system/cpu/cpu0/cpufreq/scali
ng_governor
root@elf2-desktop:~# echo 1800000 > /sys/devices/system/cpu/cpu0/cpufreq/scaling
_setspeed
```

4.查看修改后当前频率。

```
root@elf2-desktop:~# cat /sys/devices/system/cpu/cpu0/cpufreq/cpuinfo_cur_freq
1800000
```

4.1.11. 温度测试

查看 CPU 当前温度值。

```
root@elf2-desktop:~# cat /sys/class/thermal/thermal_zone0/temp
47153                                                    //温度值即为 47.1℃
```

查看内核中 cpu 降频温度值。

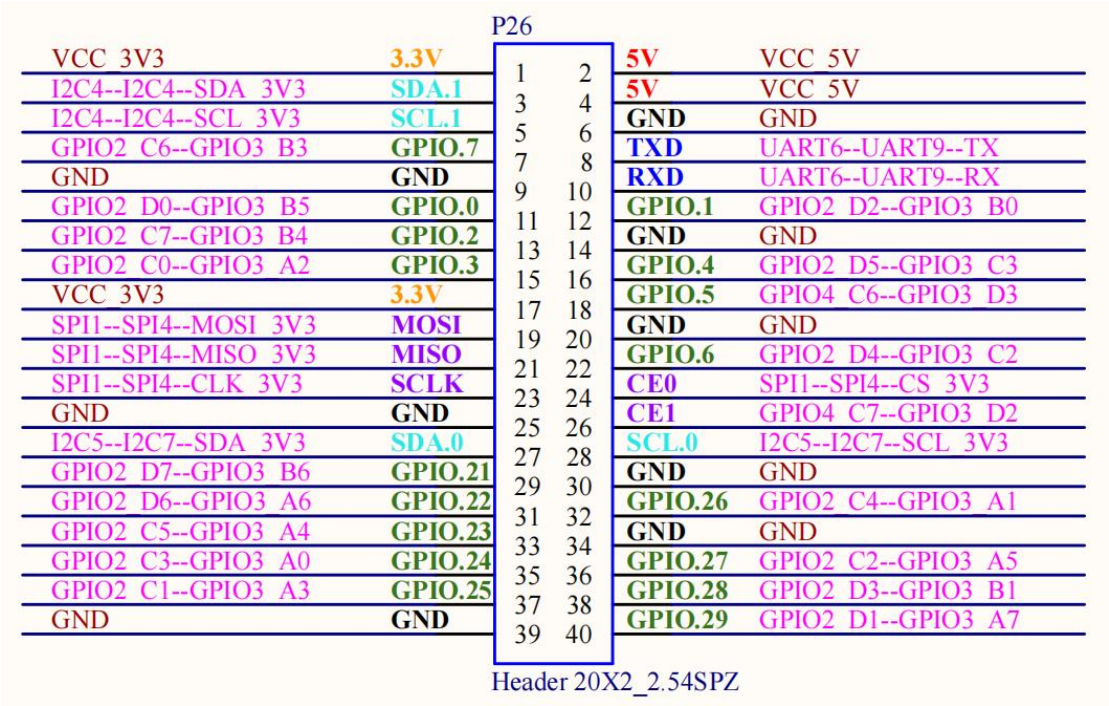
```
root@elf2-desktop:~# cat /sys/devices/virtual/thermal/thermal_zone0/trip_point_0_te
mp
75000                                                    //温度值 75℃
```

查看内核中 cpu 重启温度值。

```
root@elf2-desktop:~# cat /sys/devices/virtual/thermal/thermal_zone0/trip_point_1_te
mp
85000                                                    //温度值 85℃
```

4.1.12. GPIO 命令行测试

⚠注意：GPIO引脚的电平范围为0~3.3V，严禁对此引脚接入超过3.3V的电压。



（原理图左侧是3576核心板引脚，右侧是3588核心板引脚）

GPIO 的命名规则为 GPIOn_xy 其中 x 的表现形式有 ABCD 四种。在 GPIO 计算编号的过程中，A 对应 1，B 对应 2，C 对应 3，D 对应 4。

计算公式为：

$$GPIO_{n_xy} = n \times 32 + （x - 1） \times 8 + y$$

以 GPIO3_B3 为例，计算其 GPIO 编号。

$$\text{GPIO3_B3} = 3 \times 32 + (2 - 1) \times 8 + 3 = 107$$

将 GPIO 引脚配置为输出模式后，在完成相关设置操作后，可使用万用表测量该引脚的电压状态，以验证 GPIO 输出模式是否配置成功。。

```
root@elf2-desktop:~# echo 107 > /sys/class/gpio/export        //系统需要导出控制的
GPIO 引脚编号
root@elf2-desktop:~# echo out > /sys/class/gpio/gpio107/direction //设置为输出
root@elf2-desktop:~# echo 1 > /sys/class/gpio/gpio107/value //设置 IO 输出高电平
root@elf2-desktop:~# echo 0 > /sys/class/gpio/gpio107/value //设置 IO 输出低电平
root@elf2-desktop:~# echo 107 > /sys/class/gpio/unexport       //系统取消导出
```

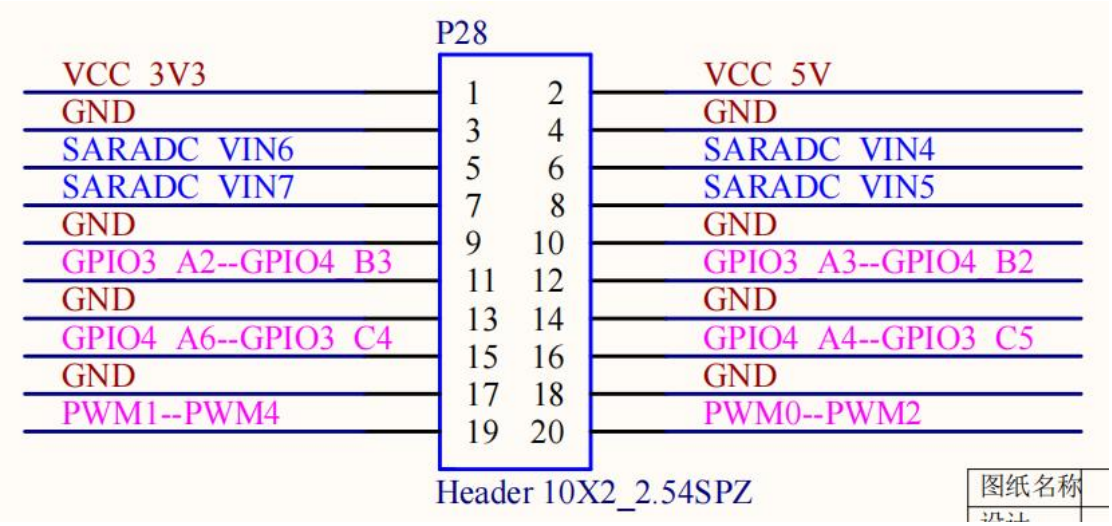
将 GPIO 引脚配置为输入模式后，通过向该引脚施加高电平或低电平信号，随后可以使用 cat 命令读取并查看该引脚的当前电平状态。

```
root@elf2-desktop:~# echo 107 > /sys/class/gpio/export        //系统需要导出控制的
GPIO 引脚编号
root@elf2-desktop:~# echo in > /sys/class/gpio/gpio107/direction //设置为输出
root@elf2-desktop:~# cat /sys/class/gpio/gpio107/value //查看引脚状态
root@elf2-desktop:~# echo 107 > /sys/class/gpio/unexport       //系统取消导出
```

4.1.13. PWM 测试

⚠注意：PWM引脚的电平范围为0~3.3V，严禁对此引脚接入超过3.3V的电压。

ELF 2底板上引出2路PWM接口，位于底板P28。



启动开发板，查看 pwm 节点：

```
root@elf2-desktop:~# ls /sys/class/pwm/
pwmchip0  pwmchip1  pwmchip2
```

由于 pwm 驱动中生成的节点是依次向下枚举的，所以 pwmchip0 节点对应的是设备树中的 pwm2，pwmchip1 节点对应的是设备树中的 pwm4，pwmchip2 节点对应的是设备树中的 pwm5。

下面以测试 pwm2 为例，设置 PWM 为输出：

```
root@elf2-desktop:~# echo 0 > /sys/class/pwm/pwmchip0/export
```

设置 PWM 周期，单位为 ns。

```
root@elf2-desktop:~# echo 1000000 > /sys/class/pwm/pwmchip0/pwm0/period
```

设置 PWM 占空比为 50%。

```
root@elf2-desktop:~# echo 500000 > /sys/class/pwm/pwmchip0/pwm0/duty_cycle
```

使能 PWM 功能。

```
root@elf2-desktop:~# echo 1 > /sys/class/pwm/pwmchip0/pwm0/enable
```

失能 PWM 功能。

```
root@elf2-desktop:~# echo 0 > /sys/class/pwm/pwmchip0/pwm0/enable
```

4.1.14. UART 测试

⚠注意：UART引脚的电平范围为0~3.3V，严禁对此引脚接入超过3.3V的电压。

🗁 路径：ELF 2 开发板资料包\05-常用工具\04-测试工具\SerialTool1.4.0a_Win32_Setup.zip

ELF 2 板卡底板原理图中标示引出的 UART2、UART9 共 2 路串口，其中 UART2 为调试串口。UART9 在开发板中的默认设备名称为 ttyS9。

UART	设备节点	说明
UART2	/dev/ttyFIQ0	调试串口，不能直接用于该测试
UART9	/dev/ttyS9	TTL 电平，可用于测试

本次测试采用 UART9(ttyS9)，分别对应 PIN8，PIN10。通过开发板的 UART 和电脑串口工具软件之间的数据收发，来进行串口测试。

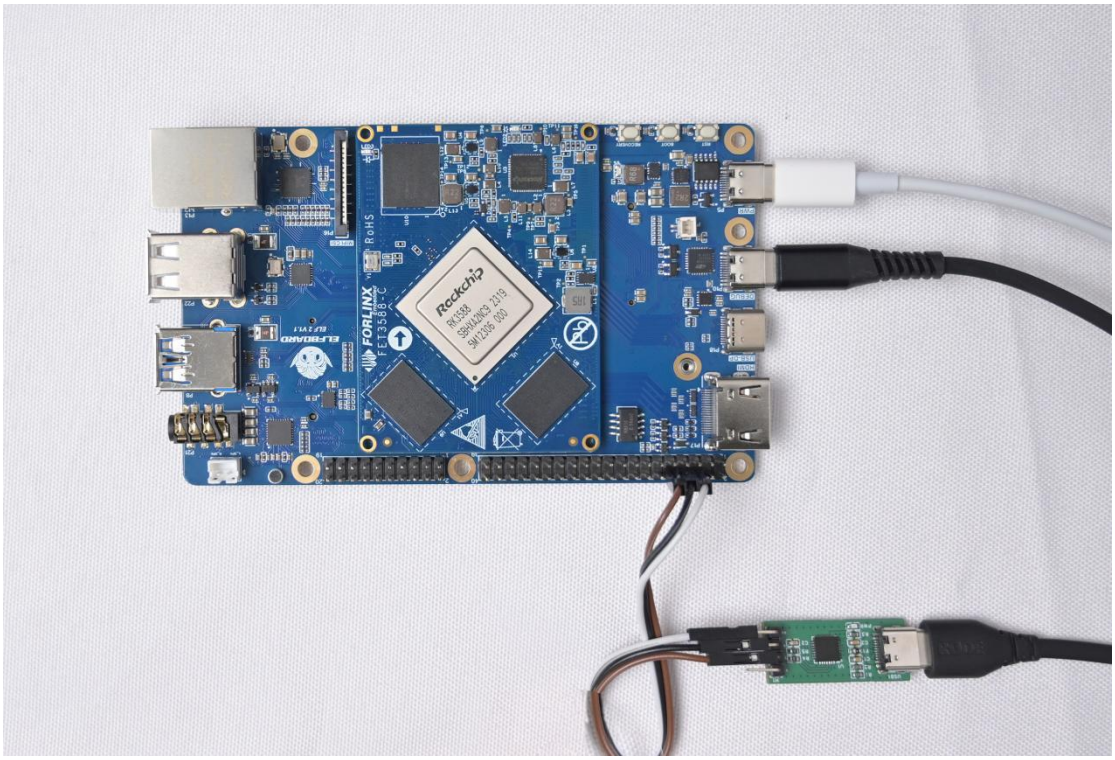
P26

VCC_3V3	3.3V	1	2	5V	VCC_5V
I2C4--I2C4--SDA_3V3	SDA.1	3	4	5V	VCC_5V
I2C4--I2C4--SCL_3V3	SCL.1	5	6	GND	GND
GPIO2_C6--GPIO3_B3	GPIO.7	7	8	TXD	UART6--UART9--TX
GND	GND	9	10	RXD	UART6--UART9--RX
GPIO2_D0--GPIO3_B5	GPIO.0	11	12	GPIO.1	GPIO2_D2--GPIO3_B0
GPIO2_C7--GPIO3_B4	GPIO.2	13	14	GND	GND
GPIO2_C0--GPIO3_A2	GPIO.3	15	16	GPIO.4	GPIO2_D5--GPIO3_C3
VCC_3V3	3.3V	17	18	GPIO.5	GPIO4_C6--GPIO3_D3
SPI1--SPI4--MOSI_3V3	MOSI	19	20	GND	GND
SPI1--SPI4--MISO_3V3	MISO	21	22	GPIO.6	GPIO2_D4--GPIO3_C2
SPI1--SPI4--CLK_3V3	SCLK	23	24	CE0	SPI1--SPI4--CS_3V3
GND	GND	25	26	CE1	GPIO4_C7--GPIO3_D2
I2C5--I2C7--SDA_3V3	SDA.0	27	28	SCL.0	I2C5--I2C7--SCL_3V3
GPIO2_D7--GPIO3_B6	GPIO.21	29	30	GND	GND
GPIO2_D6--GPIO3_A6	GPIO.22	31	32	GPIO.26	GPIO2_C4--GPIO3_A1
GPIO2_C5--GPIO3_A4	GPIO.23	33	34	GND	GND
GPIO2_C3--GPIO3_A0	GPIO.24	35	36	GPIO.27	GPIO2_C2--GPIO3_A5
GPIO2_C1--GPIO3_A3	GPIO.25	37	38	GPIO.28	GPIO2_D3--GPIO3_B1
GND	GND	39	40	GPIO.29	GPIO2_D1--GPIO3_A7

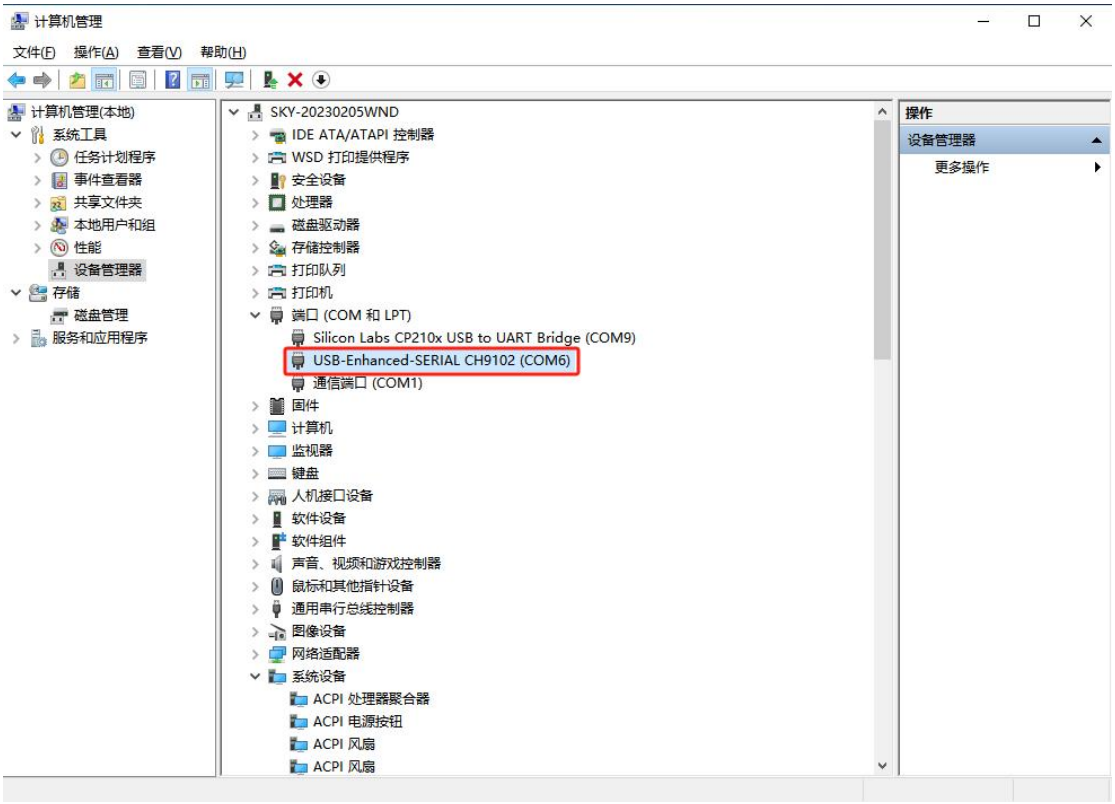
Header 20X2_2.54SPZ

（原理图左侧是3576核心板引脚，右侧是3588核心板引脚）

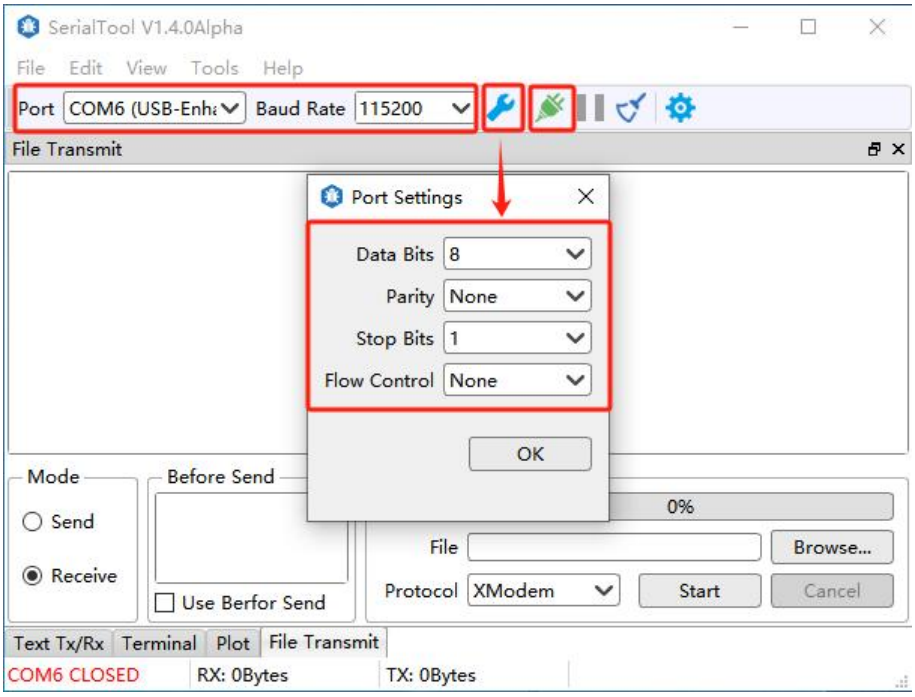
TTL 转 USB 模块与 ELF 2 硬件连接如图：



1.开发板和电脑通过 TTL 转 USB 模块连接好后，开发板上电，在电脑设备管理器查看识别为 COM6（用户以自己实际识别的 COM 口设置参数）。



2.打开电脑串口工具，选择端口 COM6、设置波特率 115200，点击蓝色端口设置图标进行设置，8 位数据位、1 位停止位、无校验、无流控制，点击 OK，点击绿色图标打开串口。



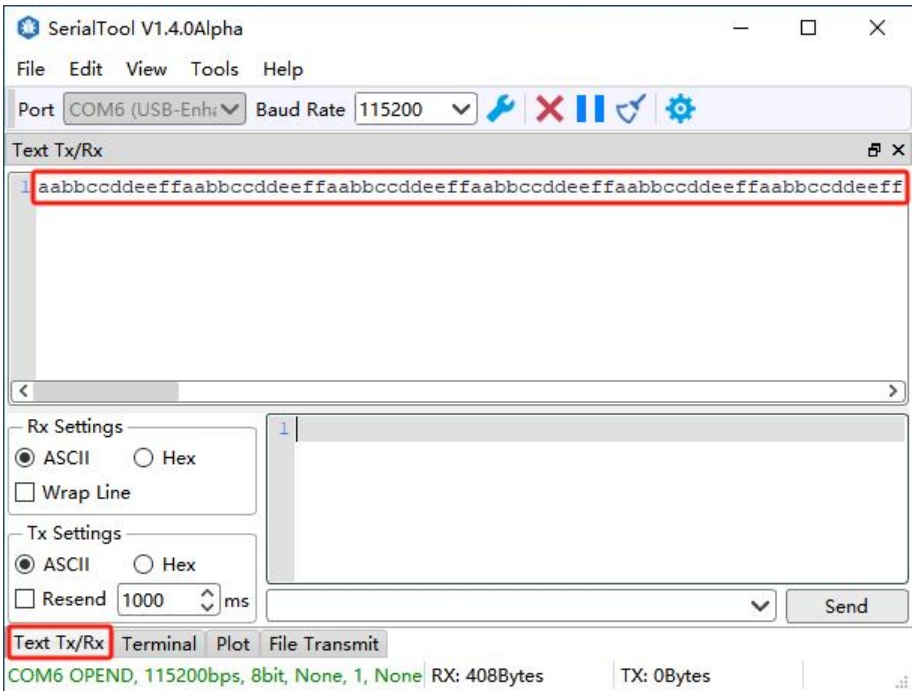
在开发板终端输入如下命令，测试程序将自动发送“aabbccddeeff”。

```
root@elf2-desktop:~# cmddemo_serialport ttyS9 -o -b 115200 -t aabbccddeeff 1
baudrate=115200,data_bit=8,stop_bit=1,check='O'
set done!
[nwrite=12] aabbccddeeff          //发送的数据
```

说明：

‘cmddemo_serialport ttyS9 -o -b 115200 -t aabbccddeeff 1’这段代码是一个命令行 程序的命令行参数。cmddemo_serialport 程序的名称；ttyS9 表示要连接到 ttyS9 这个设备；-o 表示设置为奇校验；-b 115200 表示设置波特率为 115200；-t aabbccddeeff 表示要发送的数据为 aabbccddeeff；1 表示间隔 1 秒发送一次数据。

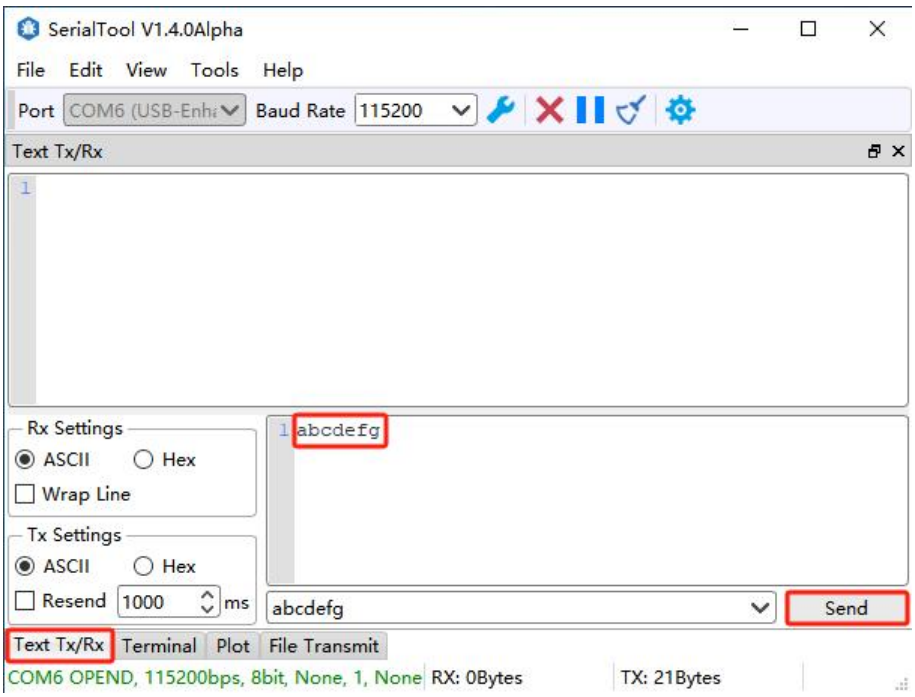
此时查看串口助手，接收到数据如下。



在开发板终端输入如下命令，开发板接收串口工具发来的数据。

```
root@elf2-desktop:~# cmddemo_serialport ttyS9 -b 115200
```

电脑串口工具发送“abcdefg”。



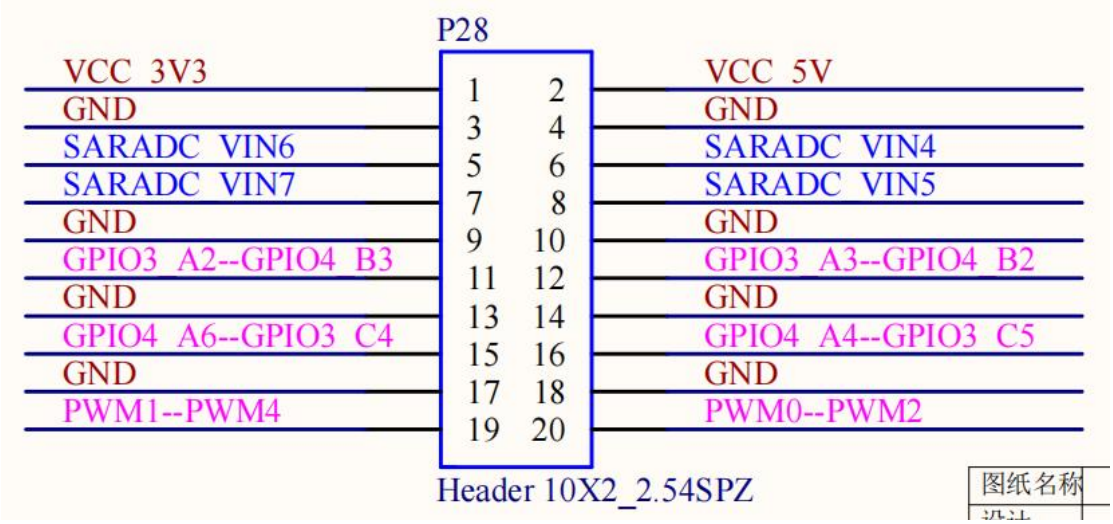
此时开发板接收到数据。

```
baudrate=115200,data_bit=8,stop_bit=1,check='N'
set done!
[nread=7] abcdefg                    //接收到的数据
```

4.1.15. ADC 测试

⚠注意：ADC引脚的电平范围为0~1.8V，严禁对此引脚接入超过1.8V的电压。

ELF 2开发板内部提供了8路ADC，ADC引脚硬件原理图如下。



（原理图左侧是3576核心板引脚，右侧是3588核心板引脚）

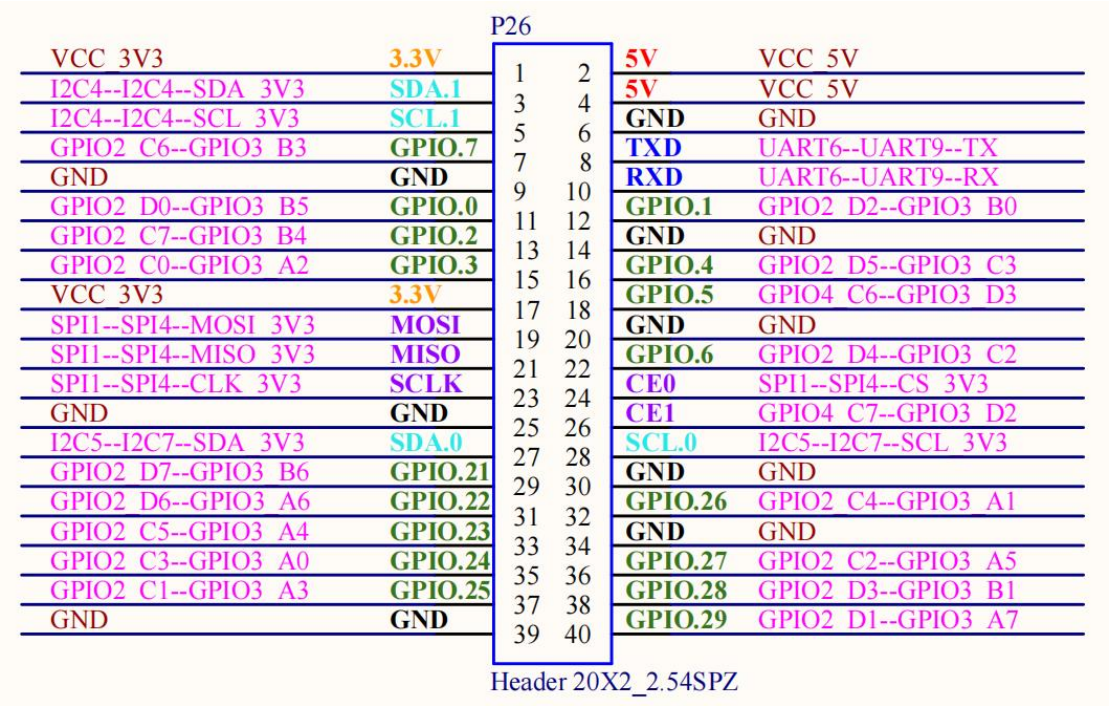
在P28的5引脚输入电压，当前芯片使用1.8V参考电压对应12位ADC最大值4096。

```
root@elf2-desktop:~# cd /sys/bus/iio/devices/iio:device0
root@elf2-desktop:/sys/bus/iio/devices/iio:device0# cat in_voltage6_raw
3638
```

4.1.16. SPI 测试

⚠注意：SPI引脚的电平范围为0~3.3V，严禁对此引脚接入超过3.3V的电压。

ELF 2 底板上引出 1 路 SPI 接口，默认软件上将其配置为 spidev 用于回环测试。测试时，请参考原理图短接 MOSI (PIN19)与 MISO (PIN21)，然后分别使用下面的命令进行测试。



（原理图左侧是3576核心板引脚，右侧是3588核心板引脚）

未短接 MOSI 与 MISO 时，执行测试指令：

```
root@elf2-desktop:~# cmddemo_spi spidev4.0
speed=1000000hz,delay=0,data_bit=8,mode=0,lsb=0,no_cs='0,
set done!
[/dev/spidev4.0 nwrite=38] 0xff  0xff  0xff  0xff  0xff  0xff  0x40  0x00  0x00
0x00  0x00  0x95  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff
0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xde  0xad  0xbe  0xef  0xba
0xad  0xf0  0x0d
[/dev/spidev4.0 nread=38 ] 0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff
0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff
0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff
0xff
```

短接 MOSI 与 MISO 时，执行测试指令：

```
root@elf2-desktop:~# cmddemo_spi spidev4.0
speed=1000000hz,delay=0,data_bit=8,mode=0,lsb=0,no_cs='0,
set done!
[/dev/spidev4.0 nwrite=38] 0xff  0xff  0xff  0xff  0xff  0xff  0x40  0x00  0x00
0x00  0x00  0x95  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff
0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xde  0xad  0xbe  0xef  0xba
0xad  0xf0  0x0d
[/dev/spidev4.0 nread=38 ] 0xff  0xff  0xff  0xff  0xff  0xff  0x40  0x00  0x00
0x00  0x00  0x95  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff
0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xff  0xde  0xad  0xbe  0xef  0xba
0xad  0xf0  0x0d
```

4.1.17. TYPE-C 测试

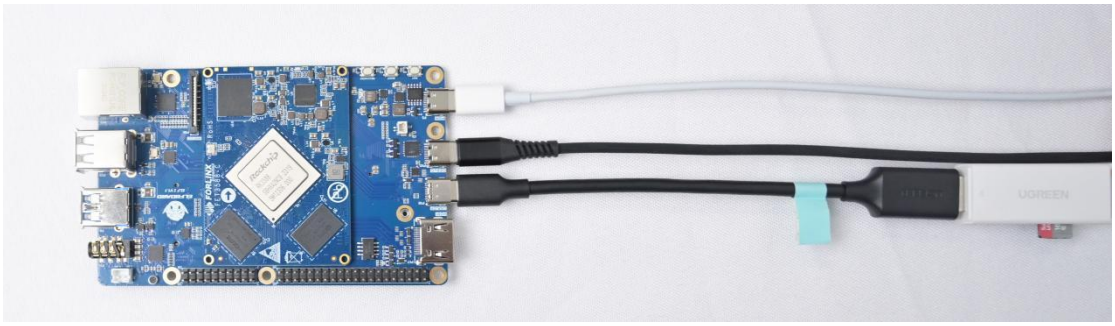
ELF 2 包含 1 个 TYPE-C 接口，USB-DP 的 HOST/DEVICE 模式自动识别。Device 模式可以用它来进行刷机、ADB 文件传输、调试，Host 模式可以插入普通的 USB 设备。

Device 模式：



Host 模式：

OTG 转 USB 模块与 ELF 2 硬件连接如图：



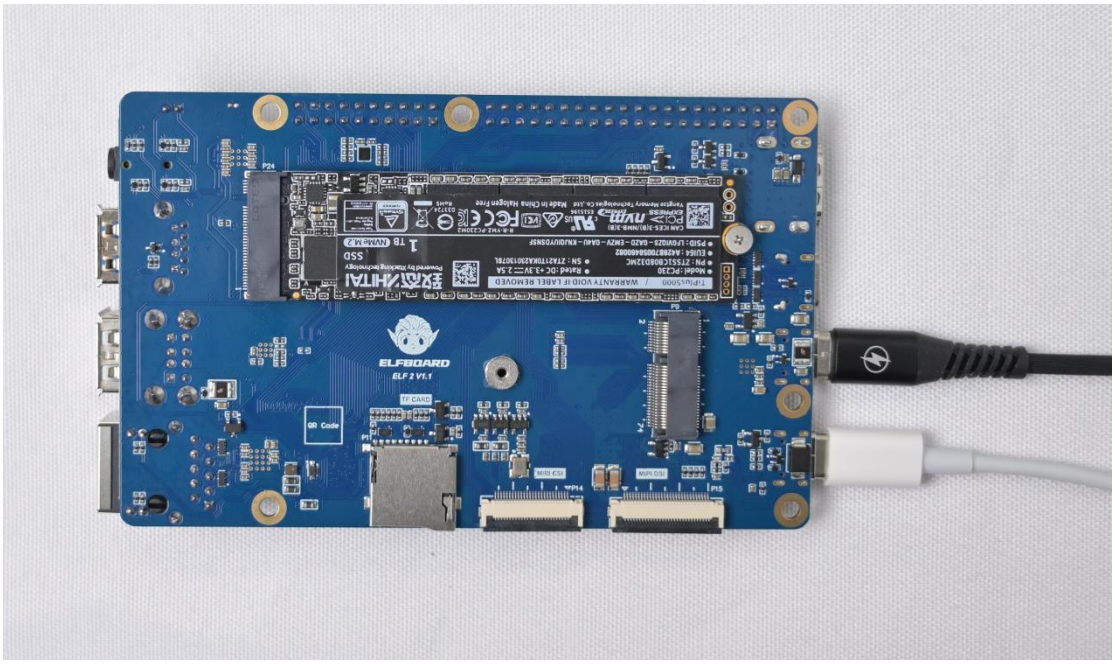
模块与 ELF 2 连接后可以看到 U 盘已经成功挂载。

```
root@elf2-desktop: ~
root@elf2-desktop:~# [   23.815314] dwc3 fc000000.usb: request ffffff8002f1b800
was not queued to ep0out

root@elf2-desktop:~# mount | grep "sda1"
/dev/sda1 on /run/media/sda1 type vfat (rw,relatime,gid=6,fmask=0007,dmask=0007,
allow_utime=0020,codepage=936,iocharset=utf8,shortname=mixed,errors=remount-ro)
root@elf2-desktop:~#
```

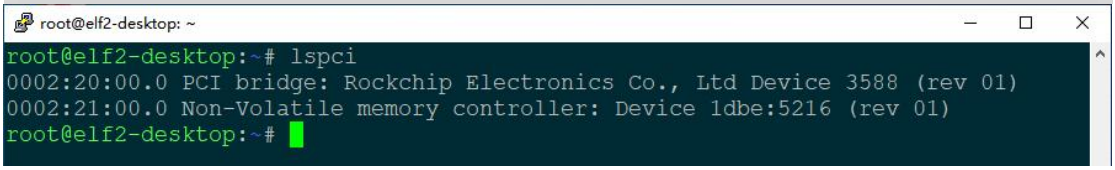
4.1.18. PCIE 测试

ELF 2 板卡有 1 个 PCIE2.0 接口，PCIE 模块与 ELF 2 硬件连接如图：



系统上电前将 PCIE 模块插入底板 PCIE 卡槽。上电后启动后，通过 lspci 可以看到对应设备枚举成功。

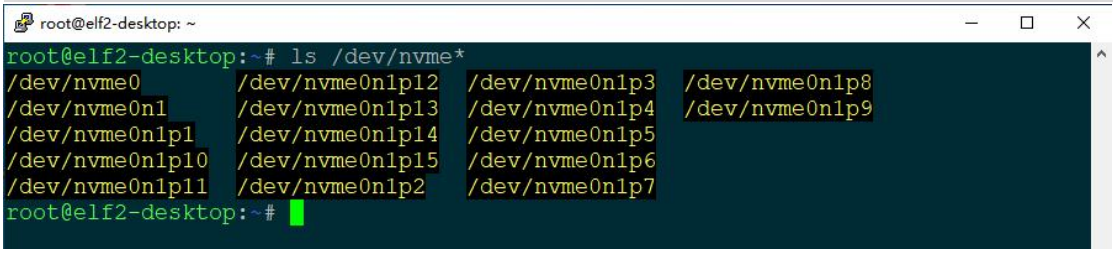
```
root@elf2-desktop:~# lspci
```



由于 pcie 设备类型较多，有可能默认不被内核支持需自行添加编译设备对应的驱动程序。

以 PCIE 硬盘举例，linux 内核默认已经包含该驱动。插入 PCIE 硬盘，此时/dev 目录下有 nvme 设备。

```
root@elf2-desktop:~# ls /dev/nvme*
```



4.1.19. eMMC 测试

下面简单测试 eMMC 的读写速度，以读写 ext4 文件系统为例。

1.写入测试：

```
root@elf2-desktop:~# dd if=/dev/zero of=/test bs=1M count=500 conv=fsync
500+0 records in
500+0 records out
524288000 bytes (524 MB, 500 MiB) copied, 2.39099 s, 219 MB/s
```

说明：

dd：这是一个用于转换和复制文件的命令。

if=/dev/zero：指定输入文件（if）。/dev/zero 是一个特殊的设备文件，读取时会不断输出零字节。

of=/test：指定输出文件（of）。这里 /test 是将数据写入的目标文件。

bs=1M：设置块大小为 1MB，意味着每次读写 1MB 的数据。

count=500：指定读取的块数为 500。因此总共要写入的数据量为 500MB（500 x 1MB）。

conv=fsync：在完成写入后，强制刷新输出文件的缓存，以确保所有数据都写入存储设备。

2.读取测试：

⚠**注意：**为确保数据准确，请重启开发板后测试读取速度。

```
root@elf2-desktop:~# dd if=/test of=/dev/null bs=1M
500+0 records in
500+0 records out
524288000 bytes (524 MB, 500 MiB) copied, 1.6154 s, 325 MB/s
```

说明：

if=/test：指定输入文件，这里 /test 是之前写入的文件，包含了 500MB 的零数据。

of=/dev/null：指定输出文件，/dev/null 是一个特殊的设备文件，写入到它的数据会被丢弃，因此可以用来测试读取速度而不实际保存数据。

bs=1M：设置块大小为 1MB，意味着每次读取 1MB 的数据。

4.2. 多媒体测试

在进行 Desktop 文件系统测试时，需要以 elf 用户身份进行操作，并且需要配置相应的环境变量，以确保图形界面程序能够正确地将图形输出定向到本地的 X 服务器上。

4.2.1. 音频和视频播放体验

4.2.1.1. 用 gst-play 播放器播放视频和音频

Gplay 是基于 GStreamer 实现的音视频播放器，运行也十分简单。

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
elf@elf2-desktop:~$ gst-play-1.0 /userdata/media/video/1080p_60fps_h265-30S.mp4
 --audiosink="alsasink device=plughw:1,0"
//播放带声音视频文件，由耳机放音测试
Press 'k' to see a list of keyboard shortcuts.
Now playing /userdata/media/video/1080p_60fps_h265-30S.mp4
Redistribute latency...
Redistribute latency...
Redistribute latency...
0:00:30.0 / 0:00:30.0
Reached end of play list.
```

📖 说明：运行此命令将使用 GStreamer 播放一个 30 秒长的 H.265 编码的 1080p 60fps 视频文件。

gst-play-1.0	这是 GStreamer 的一个工具，主要用于播放音视频文件，1.0 表示这是 GStreamer 的 1.0 版本。
/userdata/media/video/1080p_60fps_h265-30S.mp4	播放的视频文件的路径。

gst-play-1.0 是一个功能强大的命令行工具，允许用户播放多媒体文件，并通过多种选项来控制播放行为和输出设备，下面是 gst-play-1.0 命令选项说明。

类别	选项/命令	描述
使用说明	gst-play-1.0 [OPTION...] FILE1|URI1 [FILE2|URI2] [FILE3|URI3] ...	gst-play-1.0：命令名称，表示调用 GStreamer 媒体播放器。
		[OPTION...]：可选参数，用于指定播放器的不同配置和行为（例如音量、输出设备等）。
		FILE1|URI1：第一个必需的参数，表示要播放的音频或视频文件的路径或 URI。
		[FILE2|URI2、[FILE3|URI3]：可选参数，可以指定一个或多个额外的文件或 URI，以便连续播放多个媒体项。
帮助选项	-h, --help	显示帮助信息，提供基本使用说明。

	--help-all	显示所有可用的帮助选项。
	--help-gst	显示 GStreamer 的选项和设置。
应用程序选项	-v, --verbose	启用详细模式，输出状态信息和属性通知，便于调试和监控播放进程。
	--flags	控制播放行为，设置 playbin 的 flags 属性。
	--version	打印 GStreamer 的版本信息并退出。
	--videosink	指定视频输出设备，默认使用 autovideosink。
	--audiosink	指定音频输出设备，默认使用 autoaudiosink。
	--gapless	启用无缝播放功能，避免播放列表中曲目之间出现间隙。
	--shuffle	随机播放播放列表中的曲目。
	--no-interactive	禁用键盘的交互控制，播放过程中将不会响应用户的键盘输入。
	--volume	设置播放音量。
	--playlist	指定一个播放列表文件，方便批量播放。
	-q, --quiet	不打印任何输出（除了错误信息），适合减少输出信息的场景。

4.2.1.2. 使用 gst-launch 播放视频

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
elf@elf2-desktop:~$ gst-launch-1.0 filesrc location=
/userdata/media/video/1080p_60fps_h265-30S.mp4 ! qtdemux ! queue ! h265parse !
mppvideodec ! autovideosink
//仅播放视频
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
0:00:01.0 / 0:00:30.0 (3.6 %)
```

📖 说明：运行此命令将播放一个名为 1080p_60fps_h265-30S.mp4 的视频文件，使用 H.265 编码和 1080p 分辨率，帧率为 60fps，持续 30 秒。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建

	和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/video/1080p_60fps_h265-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location= /userdata/media/video/1080p_60fps_h265-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
qtdemux	qtdemux 是一个 MP4 文件解复用器，它从 MP4 容器中提取出音频和视频流。
h265parse	h265parse 是一个 H.265 视频流解析器，负责解析 H.265 视频数据。
mppvideodec	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 H.265 流进行解码。
autovideosink	autovideosink 是一个视频输出元素，用于显示视频流。autovideosink 会自动选择最合适的视频输出设备（如窗口、屏幕等）来显示视频 。

gst-launch-1.0 是一个命令行工具，用于构建和运行媒体处理管道，下面是 gst-launch-1.0 命令选项说明。

类别	选项/命令	描述
使用说明	gst-launch-1.0 [OPTION...] PIPELINE-DESCRIPTION	gst-launch-1.0：命令的名称，用于启动 GStreamer 的管道处理工具。
		[OPTION...]：可选参数部分，允许用户在运行命令时指定特定的选项。
		PIPELINE-DESCRIPTION：管道描述部分，定义了数据流的元素、如何连接这些元素以及它们的属性
帮助选项	-h, --help	显示帮助信息，提供基本使用说明。
	--help-all	显示所有可用的帮助选项。
	--help-gst	显示 GStreamer 的选项和设置。
应用程序选项	-t, --tags	输出标签（也称为元数据）。
	-c, --toc	输出 TOC（章节和版本信息）。
	-v, --verbose	启用详细模式，输出状态信息和属性通知，便于调试和监

		控。
	-q, --quiet	禁用任何进度信息的打印。
	-m, --messages	输出消息。
	-X, --exclude=PROPERTY-NAME	如果启用了详细输出，指定不输出某个属性的状态信息（可多次使用）。
	-f, --no-fault	不安装故障处理程序。
	-e, --eos-on-shutdown	在关闭管道之前，强制对源发送 EOS（结束信号）。
	--version	打印版本信息并退出。

4.2.1.3. 使用 gst-launch 播放音频

```
elf@elf2-desktop:~$ aplay -l
**** List of PLAYBACK Hardware Devices ****
card 0: rockchipdp0 [rockchip,dp0], device 0: rockchip,dp0 spdif-hifi-0 [rockchip,dp0 spdif-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 1: rockchipnau8822 [rockchip-nau8822], device 0: dailink-multicodecs nau8822-hifi-0 [dailink-multicodecs nau8822-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 2: rockchiphdmi0 [rockchip-hdmi0], device 0: rockchip-hdmi0 i2s-hifi-0 [rockchip-hdmi0 i2s-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
elf@elf2-desktop:~$ gst-launch-1.0 filesrc location=/userdata/media/audio/piano2-CoolEdit.mp3 ! id3demux ! mpegaudioparse ! mpg123audiodec ! alsasink device=plughw:1,0
//仅播放音频，device=plughw:1,0 由耳机放音测试。
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
New clock: GstAudioSinkClock
handling interrupt.
Interrupt: Stopping pipeline ...
Execution ended after 0:00:02.665159268
Setting pipeline to PAUSED ...
Setting pipeline to READY ...
Setting pipeline to NULL ...
Freeing pipeline ...
```

🕮 说明：运行此命令将从指定的 MP3 文件中读取音频数据，解析并解码音频流，最终通过指定的 ALSA 音频设备播放出来。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/audio/piano2-CoolEdit.mp3	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/audio/piano2-CoolEdit.mp3 播放的音频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
id3demux	这是一个解复用器，用于解析 MP3 文件中的 ID3 标签（如标题、艺术家、专辑等）和音频数据。它将 MP3 文件中的音频流和元数据分开。
mpegaudioparse	这是一个解析器，用于解析 MPEG 音频流。它处理从 id3demux 传递过来的音频数据，为后续解码做准备。
mpg123audiodec	这是一个音频解码器，专门用于解码 MP3 音频流。它将经过解析的音频数据转换为原始 PCM 格式，以便播放。
alsasink device=plughw:1,0	alsasink 是一个音频输出元素，使用 ALSA（高级 Linux 声音架构）播放音频。
	device=plughw:1,0 指定了使用的音频设备，plughw:1,0 表示使用第一个硬件设备。可以根据系统的实际音频设备调整这个参数。

4.2.1.4. 使用 gst-launch 播放视频和音频

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
elf@elf2-desktop:~$ gst-launch-1.0 filesrc location=
/userdata/media/video/1080p_60fps_h265-30S.mp4 ! qtdemux name=dec dec. !
queue ! h265parse ! mppvideodec ! autovideosink dec. ! queue ! decodebin ! alsasink
device=plughw:1,0
//播放带声音视频文件，device=plughw:1,0 由耳机放音测试
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Redistribute latency...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstAudioSinkClock
^Chandling interrupt. (2.6 %)
```

📖 说明：运行此命令将从指定的 MP4 文件中提取 H.265 编码的视频和音频流，进行解复用、解码后，通过 X11 显示服务器输出视频，并通过 ALSA 输出音频。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/video/1080p_60fps_h265-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/video/1080p_60fps_h265-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
qtdemux name=dec	qtdemux 是一个解复用器，用于解析 MP4 文件格式（QuickTime 格式）。
	name=dec 给解复用器命名为 dec，这样可以在后面引用它。
dec. ! queue ! h265parse ! mppvideodec ! autovideosink	dec.引用名为 dec 的解复用器的输出，处理视频流。
	queue 是一个队列元素，用于缓冲数据流，确保后续处理的流畅性。
	h265parse 是一个 H.265 视频流解析器，负责解析 H.265 视频数据。
	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 H.265 流进行解码。
	autovideosink 是一个视频输出元素，用于显示视频流。autovideosink 会自动选择最合适的视频输出设备（如窗口、屏幕等）来显示视频 。
dec. ! queue ! decodebin ! alsasink device=plughw:1,0	dec.再次引用 dec 解复用器的输出，这次处理音频流。
	queue 添加一个队列，用于音频流的处理。
	decodebin 自动选择合适的解码器来解码音频流。decodebin 可以处理多种音频格式，因此会选择适当的解码器。
	alsasink device=plughw:1,0 使用 ALSA 音频输出元素将解码后的音频播放出来，device=plughw:1,0 指定了使用的音频设备。

4.2.2. 视频硬编码

ELF 2 最大支持 8K@30fps/H.265 和 8K@30fps/H.264 视频编码。

4.2.2.1. 视频硬编码 H.264

```
elf@elf2-desktop:~$ gst-launch-1.0 videotestsrc num-buffers=600 !
video/x-raw,framerate=30/1,width=7680,height=4320 ! mpph264enc ! h264parse !
mp4mux ! filesink location=test.mp4
```

```
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
Got EOS from element "pipeline0".
Execution ended after 0:00:53.676009268
Setting pipeline to NULL ...
Freeing pipeline ...
//查看/home/elf 目录下生成 test.mp4 文件
elf@elf2-desktop:~$ ls
test.mp4
```

🕮 说明：运行此命令将生成一个 8K 分辨率、30 帧每秒的测试视频，编码为 H.264 格式，并将其封装为 MP4 文件，最后保存为 test.mp4。

<table>
<tr><td>gst-launch-1.0</td><td colspan="2">这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。</td></tr>
<tr><td rowspan="2">videotestsrc num-buffers=600</td><td colspan="2">videotestsrc 是一个视频测试源元素，它生成一系列的测试视频帧。默认生成的测试视频通常为颜色渐变或其他模式。</td></tr>
<tr><td colspan="2">num-buffers=600 指定生成 600 帧视频数据，达到这个数量后，视频源将停止输出。</td></tr>
<tr><td>!</td><td colspan="2">这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。</td></tr>
<tr><td rowspan="3">video/x-raw,framerate=30/1,width=7680,height=4320</td><td rowspan="3">这是一个caps(能力)字符串，定义了视频流的格式和参数。</td><td>video/x-raw 表示原始视频数据格式。</td></tr>
<tr><td>framerate=30/1 指定帧率为30帧每秒。</td></tr>
<tr><td>width=7680,height=4320 指定视频分辨率为 7680x4320（即 8K 分辨率）。</td></tr>
<tr><td>mpph264enc</td><td colspan="2">mpph264enc 是一个 H.264 编码器，用于将原始视频流编码为 H.264 格式。</td></tr>
<tr><td>h264parse</td><td colspan="2">h264parse 是一个 H.264 解析器，用于处理 H.264 编码流。</td></tr>
<tr><td>mp4mux</td><td colspan="2">mp4mux 是一个 MP4 封装器，用于将 H.264 视频流封装成 MP4 文件格式。</td></tr>
<tr><td>filesink location=test.mp4</td><td colspan="2">filesink 是一个文件输出元素，它将处理过的数据写入指定的文件。</td></tr>
</table>

	location=test.mp4 指定输出文件的路径和名称，生成的 MP4 文件将被命名为 test.mp4。

4.2.2.2. 视频硬编码 H.265

```
elf@elf2-desktop:~$ gst-launch-1.0 videotestsrc num-buffers=600 ! video/x-raw,framerate=30/1,width=7680,height=4320 ! mpph265enc ! h265parse ! mp4mux ! filesink location=test.mp4
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
Got EOS from element "pipeline0".
Execution ended after 0:00:56.942676054
Setting pipeline to NULL ...
Freeing pipeline ...
//查看/home/elf 目录下生成 test.mp4 文件
elf@elf2-desktop:~$ ls
test.mp4
```

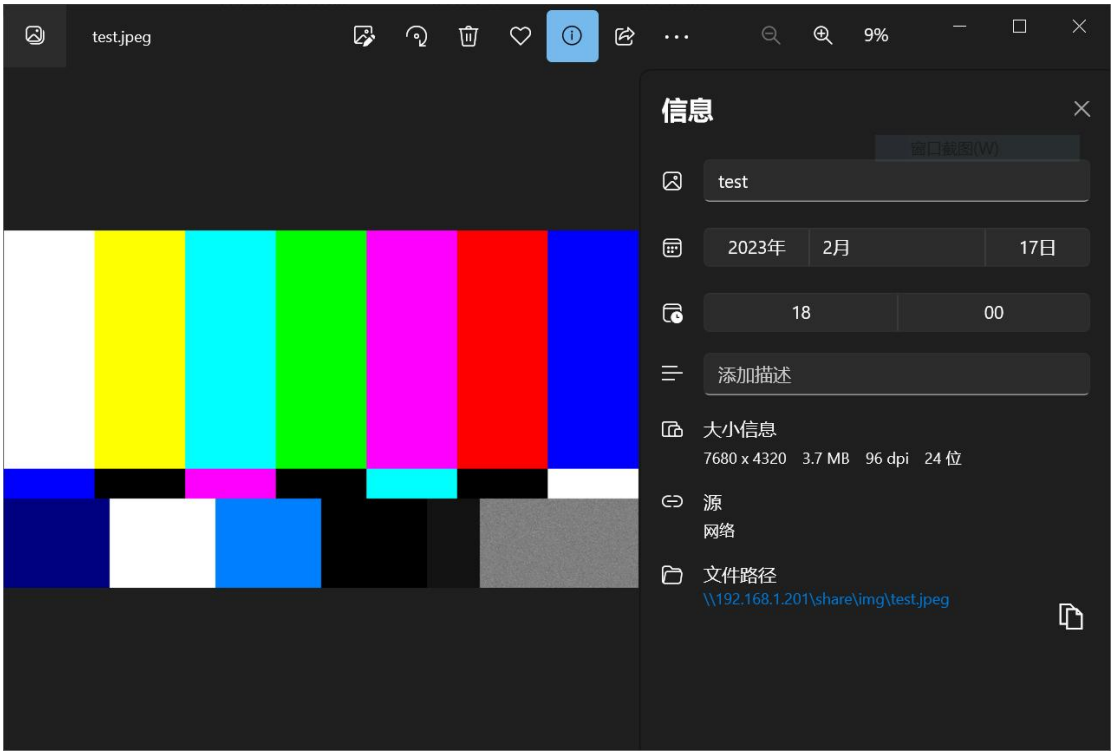
🕮 说明：运行此命令将生成一个 8K 分辨率、30 帧每秒的测试视频，编码为 H.265 格式，并将其封装为 MP4 文件，最后保存为 test.mp4。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。	
videotestsrc num-buffers=600	videotestsrc 是一个视频测试源元素，它生成一系列的测试视频帧。默认生成的测试视频通常为颜色渐变或其他模式。	
	num-buffers=600 指定生成 600 帧视频数据，达到这个数量后，视频源将停止输出。	
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。	
video/x-raw,framerate=30/1,width=7680,height=4320	这是一个 caps(能力) 字符串，定义了视频流的格式和参数。	video/x-raw 表示原始视频数据格式。
		framerate=30/1 指定帧率为 30 帧每秒。
		width=7680,height=4320 指定视频分辨率为 7680x4320（即 8K 分辨率）。
mpph265enc	mpph265enc 是一个 H.265 编码器，用于将	

	原始视频流编码为 H.265 格式。
h265parse	h265parse 是一个 H.265 解析器，用于处理 H.265 编码流。
mp4mux	mp4mux 是一个 MP4 封装器，用于将 H.265 视频流封装成 MP4 文件格式。
filesink location=test.mp4	filesink 是一个文件输出元素，它将处理过的数据写入指定的文件。
	location=test.mp4 指定输出文件的路径和名称，生成的 MP4 文件将被命名为 test.mp4。

4.2.2.3. JPEG 硬编码

```
elf@elf2-desktop:~$ gst-launch-1.0 videotestsrc num-buffers=1 !
video/x-raw,framerate=1/1,width=7680,height=4320 ! mppjpegenc ! jpegparse ! queue !
filesink location=test.jpeg
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
Got EOS from element "pipeline0".
Execution ended after 0:00:00.029266878
Setting pipeline to NULL ...
Freeing pipeline ...
//查看/home/elf 目录下生成 test.jpeg 文件
elf@elf2-desktop:~$ ls
test.jpeg
```



🕮 说明：运行此命令将生成一个 8K 分辨率的测试视频帧，以 1 帧每秒的速率，编码为 JPEG 格式，并保存为 test.jpeg 文件。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。	
videotestsrc num-buffers=1	videotestsrc 是一个视频测试源元素，它生成一系列的测试视频帧。默认生成的测试视频通常为颜色渐变或其他模式。	
	num-buffers=1 指定只生成 1 帧视频数据。执行完后，视频源将停止输出。	
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。	
video/x-raw,framerate=1/1,width=7680,height=4320	这是一个 caps(能力)字符串，定义了视频流的格式和参数。	video/x-raw 表示原始视频数据格式。
		framerate=1/1 指定帧率为 1 帧每秒。
		width=7680,height=4320 指定视频分辨率为 7680x4320（即 8K 分辨率）。
mppjpegenc	mppjpegenc 是一个 JPEG 编码器，用于将原始视频流编码为 JPEG 格式。	
jpegparse	jpegparse 是一个 JPEG 解析器，用于处理 JPEG 编码流。	
queue	queue 是一个队列元素，用于在数据流中引	

	入缓冲区。
filesink location=test.jpeg	filesink 是一个文件输出元素，它将处理过的数据写入指定的文件。
	location=test.jpeg 指定输出文件的路径和名称，生成的 JPEG 文件将被命名为 test.jpeg。

4.2.3. 视频硬解码

ELF 2 支持 H264，H265、VP8、VP9 视频硬解码，H264 解码器支持 8K@30fps，H265 解码器支持 8K@60fps。

ELF 2 使用 mppvideodec 组件进行视频硬解码，它的输出格式为：NV12，I420，YV12。

4.2.3.1. 解码并播放 H264 格式视频

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
elf@elf2-desktop:~$ gst-launch-1.0 filesrc
location=/userdata/media/video/1080p_60fps_h264-30S.mp4 ! qtdemux ! h264parse !
mppvideodec ! autovideosink
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
0:00:05.9 / 0:00:30.0 (19.8 %)
```

📖 说明：运行此命令将从指定的 MP4 文件中读取 H.264 编码的视频数据，解复用并解析该数据，然后通过硬件加速解码，最后在 X11 显示服务器上播放视频。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/video/1080p_60fps_h264-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/video/1080p_60fps_h264-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
qtdemux	qtdemux 是一个 MP4 文件解复用器，它从 MP4 容器中提取出音频和视频流。
h264parse	h264parse 是一个 H.264 视频流解析器，负责解析 H.264 视频数据。
mppvideodec	mppvideodec 使用 MPP（Media Processi

	ng Platform）视频解码器对 H.264 流进行解码。
	autovideosink 是一个视频输出元素，用于显示视频流。autovideosink 会自动选择最合适的视频输出设备（如窗口、屏幕等）来显示视频。

4.2.3.2. 解码并播放 H264 格式视频带音频

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
elf@elf2-desktop:~$ gst-launch-1.0 filesrc
location=/userdata/media/video/1080p_60fps_h264-30S.mp4 ! qtdemux name=demux
demux.video_0 ! queue ! h264parse ! mppvideodec ! autovideosink demux.audio_0 !
queue ! aacparse ! faad ! alsasink device=plughw:1,0
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Redistribute latency...
Redistribute latency...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstAudioSinkClock
0:00:04.5 / 0:00:30.0 (15.0 %)
```

🕮 说明：运行此命令将从指定的 MP4 文件中读取 H.264 编码的视频流和 AAC 编码的音频流，解复用并解析这些数据，然后使用硬件加速解码视频并在 X11 上播放，同时将音频解码后输出到指定的 ALSA 音频设备。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/video/1080p_60fps_h264-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/video/1080p_60fps_h264-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
qtdemux name=demux	qtdemux 是一个 MP4 文件解复用器，它从 MP4 容器中提取出音频和视频流。
	name=demux 为该解复用器命名为 demux，以便后续引用。
demux.video_0 ! queue	demux.video_0 表示从名为 demux 的解复用器中获取第一个视频流。

	queue 是一个队列元素，用于缓冲数据流，确保后续处理的流畅性。
h264parse	h264parse 是一个 H.264 视频流解析器，负责解析 H.264 视频数据。
mppvideodec	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 H.264 流进行解码。
autovideosink	autovideosink 是一个视频输出元素，用于显示视频流。autovideosink 会自动选择最合适的视频输出设备（如窗口、屏幕等）来显示视频 。
demux.audio_0 ！ queue	demux.audio_0 表示从解复用器中获取第一个音频流。
	queue 再次使用队列元素，用于缓冲音频流，确保流畅播放。
aacparse	aacparse 是 AAC 音频解析器，负责解析 AAC 音频流。
faad	faad 是 AAC 解码器，负责将 AAC 音频流解码为原始音频数据。
alsasink device=plughw:1,0	alsasink 是音频输出元素，使用 ALSA (Advanced Linux Sound Architecture)将音频数据输出到音频设备。
	device=plughw:1,0 指定了要使用的具体音频设备。

4.2.3.3. 解码并播放 H265 格式视频

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
elf@elf2-desktop:~$ gst-launch-1.0 filesrc
location=/userdata/media/video/4k_60fps_h265-30S.mp4 ! qtdemux ! h265parse !
mppvideodec ! autovideosink
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
0:00:06.4 / 0:00:30.0 (21.4 %)
```

🕮 说明：运行此命令将从指定的 MP4 文件中读取 H.265 编码的视频数据，解复用并解析该数据，然后通过硬件加速解码，最后在 X11 显示服务器上播放视频。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。

filesrc location=/userdata/media/video/4k_60fps_h265-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。 location=/userdata/media/video/4k_60fps_h265-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
qtdemux	qtdemux 是一个 MP4 文件解复用器，它从 MP4 容器中提取出音频和视频流。
h265parse	h265parse 是一个 H.265 视频流解析器，负责解析 H.265 视频数据。
mppvideodec	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 H.265 流进行解码。
autovideosink	autovideosink 是一个视频输出元素，用于显示视频流。autovideosink 会自动选择最合适的视频输出设备（如窗口、屏幕等）来显示视频 。

4.2.3.4. 解码并播放 H265 格式视频带音频

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
elf@elf2-desktop:~$ gst-launch-1.0 filesrc
location=/userdata/media/video/4k_60fps_h265-30S.mp4 ! qtdemux name=demux
demux.video_0 ! queue ! h265parse ! mppvideodec ! autovideosink demux.audio_0 !
queue ! aacparse ! faad ! alsasink device=plughw:1,0
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Redistribute latency...
Redistribute latency...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstAudioSinkClock
0:00:03.4 / 0:00:30.0 (11.3 %)
```

🕮 说明：运行此命令将从指定的 MP4 文件中读取 H.265 编码的视频流和 AAC 编码的音频流，解复用并解析这些数据，然后使用硬件加速解码视频并在 X11 上播放，同时将音频解码后输出到指定的 ALSA 音频设备。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/video/4k_60fps_h265-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。

	location=/userdata/media/video/4k_60fps_h265-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
qtdemux name=demux	qtdemux 是一个 MP4 文件解复用器，它从 MP4 容器中提取出音频和视频流。
	name=demux 为该解复用器命名为 demux，以便后续引用。
demux.video_0 ! queue	demux.video_0 表示从名为 demux 的解复用器中获取第一个视频流。
	queue 是一个队列元素，用于缓冲数据流，确保后续处理的流畅性。
h265parse	h265parse 是一个 H.265 视频流解析器，负责解析 H.265 视频数据。
mppvideodec	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 H.265 流进行解码。
autovideosink	autovideosink 是一个视频输出元素，用于显示视频流。autovideosink 会自动选择最合适的视频输出设备（如窗口、屏幕等）来显示视频 。
demux.audio_0 ! queue	demux.audio_0 表示从解复用器中获取第一个音频流。
	queue 再次使用队列元素，用于缓冲音频流，确保流畅播放。
aacparse	aacparse 是 AAC 音频解析器，负责解析 AAC 音频流。
faad	faad 是 AAC 解码器，负责将 AAC 音频流解码为原始音频数据。
alsasink device=plughw:1,0	alsasink 是音频输出元素，使用 ALSA (Advanced Linux Sound Architecture)将音频数据输出到音频设备。
	device=plughw:1,0 指定了要使用的具体音频设备。

4.2.3.5. 解码并播放 VP9 格式视频

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
elf@elf2-desktop:~$ gst-launch-1.0 filesrc
location=/userdata/media/video/1080p_60fps_vp9-30S.mp4 ! qtdemux ! vp9parse !
mppvideodec ! autovideosink
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
```

```
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
0:00:02.5 / 0:00:30.0 (8.7 %)
```

🕮 说明：运行此命令将从指定的 MP4 文件中读取 VP9 编码的视频数据，解复用并解析该数据，然后通过硬件加速解码，最后在 X 显示服务器上播放视频。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/video/1080p_60fps_vp9-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/video/1080p_60fps_vp9-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
qtdemux	qtdemux 是一个 MP4 文件解复用器，它从 MP4 容器中提取出音频和视频流。
vp9parse	vp9parse 是一个 VP9 视频流解析器，负责解析 VP9 视频数据。
mppvideodec	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 VP9 流进行解码。
autovideosink	autovideosink 是一个视频输出元素，用于显示视频流。autovideosink 会自动选择最合适的视频输出设备（如窗口、屏幕等）来显示视频 。

4.2.3.6. 解码并播放 VP9 格式视频带音频

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
elf@elf2-desktop:~$ gst-launch-1.0 filesrc
location=/userdata/media/video/1080p_60fps_vp9-30S.mp4 ! qtdemux name=demux
demux.video_0 ! queue ! vp9parse ! mppvideodec ! autovideosink demux.audio_0 !
queue ! aacparse ! faad ! alsasink device=plughw:1,0
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstAudioSinkClock
0:00:03.2 / 0:00:30.0 (10.7 %)
```

🕮 说明：运行此命令将从指定的 MP4 文件中读取 VP9 编码的视频流和 AAC 编码的音频流，解复用并解

析这些数据，然后使用硬件加速解码视频并在 X11 上播放，同时将音频解码后输出到指定的 ALSA 音频设备。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
filesrc location=/userdata/media/video/1080p_60fps_vp9-30S.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/video/1080p_60fps_vp9-30S.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
qtdemux name=demux	qtdemux 是一个 MP4 文件解复用器，它从 MP4 容器中提取出音频和视频流。
	name=demux 为该解复用器命名为 demux，以便后续引用。
demux.video_0 ! queue	demux.video_0 表示从名为 demux 的解复用器中获取第一个视频流。
	queue 是一个队列元素，用于缓冲数据流，确保后续处理的流畅性。
vp9parse	vp9parse 是一个 VP9 视频流解析器，负责解析 VP9 视频数据。
mppvideodec	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 VP9 流进行解码。
autovideosink	autovideosink 是一个视频输出元素，用于显示视频流。autovideosink 会自动选择最合适的视频输出设备（如窗口、屏幕等）来显示视频 。
demux.audio_0 ! queue	demux.audio_0 表示从解复用器中获取第一个音频流。
	queue 再次使用队列元素，用于缓冲音频流，确保流畅播放。
aacparse	aacparse 是 AAC 音频解析器，负责解析 AAC 音频流。
faad	faad 是 AAC 解码器，负责将 AAC 音频流解码为原始音频数据。
alsasink device=plughw:1,0	alsasink 是音频输出元素，使用 ALSA (Advanced Linux Sound Architecture)将音频数据输出到音频设备。
	device=plughw:1,0 指定了要使用的具体音频设备。

4.2.3.7. 解码并播放 VP8 格式视频

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
```

```
elf@elf2-desktop:~$ gst-launch-1.0 filesrc
location=/userdata/media/video/1080p_30fps_vp8.mp4 ! matroskademux ! queue !
mppvideodec ! autovideosink
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
0:00:09.2 / 0:00:30.1 (30.7 %)
```

📖 说明：运行此命令将从指定的 MP4 文件中提取 VP8 编码的视频，进行解复用、解码后，通过 X11 显示服务器输出视频。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
location=/userdata/media/video/1080p_30fps_vp8.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。 location=/userdata/media/video/1080p_30fps_vp8.mp4 播放的视频文件的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
matroskademux	matroskademux 是一个解复用器，用于从特定格式的多媒体容器中提取音频和视频流。
queue	queue 是一个队列元素，用于缓冲数据流，确保后续处理的流畅性。
mppvideodec	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 VP8 流进行解码。
autovideosink	autovideosink 是一个视频输出元素，用于显示视频流。autovideosink 会自动选择最合适的视频输出设备（如窗口、屏幕等）来显示视频 。

4.2.3.8. 解码并播放 VP8 格式视频带音频

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
elf@elf2-desktop:~$ gst-launch-1.0 filesrc
location=/userdata/media/video/1080p_30fps_vp8.mp4 typefind=true ! video/webm !
matroskademux name=dec dec. ! queue ! mppvideodec ! autovideosink dec. ! queue !
decodebin ! audioconvert ! audioresample ! alsasink device=plughw:1,0
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Redistribute latency...
```

```
Redistribute latency...
Pipeline is PREROLLED ...
Prerolled, waiting for async message to finish...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstAudioSinkClock
0:00:02.4 / 0:00:30.1 (8.3 %)
```

🕮 说明：运行此命令将从指定的 MP4 文件中提取 VP8 编码的视频和音频流，进行解复用、解码后，通过 X11 显示服务器输出视频，并通过 ALSA 输出音频。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
location=/userdata/media/video/1080p_30fps_vp8.mp4	filesrc 是一个文件源元素，用于读取指定路径的文件。
	location=/userdata/media/video/1080p_30fps_vp8.mp4 播放的视频文件的路径。
typefind=true	启用 typefind 功能，GStreamer 将会自动检测输入文件的媒体类型，以便选择适合的解复用器和解码器。
i	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
video/webm	指定视频流的类型为 WebM 格式。这个元素用于将流标记为 WebM 格式。
matroskademux name=dec	matroskademux 是一个解复用器，用于从特定格式的多媒体容器中提取音频和视频流。
	name=dec 为解复用器命名，以便后续引用。
dec. ! queue ! mppvideodec ! autovideosink	dec.这是对先前命名的 dec 解复用器的引用，用于访问其输出。
	queue 是一个队列元素，用于缓冲数据流，确保后续处理的流畅性。
	mppvideodec 使用 MPP（Media Processing Platform）视频解码器对 VP8 流进行解码。
	autovideosink 是一个视频输出元素，用于显示视频流。autovideosink 会自动选择最合适的视频输出设备（如窗口、屏幕等）来显示视频 。
dec. ! queue ! decodebin ! audioconvert ! audioresample ! alsasink device=plughw:1,0	dec.再次引用 dec 解复用器，用于处理音频流的输出。
	queue 为音频流添加一个队列，以缓冲数据，确保音频处理的稳定性
	decodebin 是一个自动解码器，能够根据输

	入流类型选择合适的解码器。它会处理来自 dec 解复用器的音频流。
	audioconvert 元素用于音频格式转换，确保音频流的格式兼容后续处理。
	audioresample 用于音频重采样，确保音频以合适的采样率输出，以适应不同的音频设备。
	alsasink device=plughw:1,0 使用 ALSA 音频输出元素将解码后的音频播放出来，device=plughw:1,0 指定了使用的音频设备。

4.2.4. 摄像头测试

ELF 2 支持 OV13855 MIPI 摄像头，同时也支持 UVC 摄像头。首先来测试一下 UVC 摄像头，这里以林柏视进程测试，将 USB 摄像头插入开发板，将自动安装 UVC 驱动。

4.2.4.1. UVC Camera 测试

1、摄像头识别检测和格式支持查询

摄像头识别检测，列出系统中所有视频设备：

```
elf@elf2-desktop:~$ v4l2-ctl --list-devices
[   205.931251] m00_b_dw9763 3-000c: dw9763_open: enter, power.usage_count(1)!
rk_hdmirx (fdee0000.hdmirx-controller):
[   205.931362] m00_b_dw9763 3-000c: dw9763_open: exit, power.usage_count(2)!
        /dev/video20
[   205.931404] dw9763 3-000c: cmd 0x80685600 not supported

[   205.931434] dw9763 3-000c: cmd 0xc1007c00 not supported
rkisp-statistics (platform: rkisp):
[   205.931462] m00_b_dw9763 3-000c: dw9763_close: enter, power.usage_count(2)!
        /dev/video18
[   205.931483] m00_b_dw9763 3-000c: dw9763_close: exit, power.usage_count(1)!
        /dev/video19

rkcif-mipi-lvds (platform:rkcif):
        /dev/media0

rkcif (platform:rkcif-mipi-lvds):
        /dev/video0
        /dev/video1
        /dev/video2
        /dev/video3
        /dev/video4
        /dev/video5
        /dev/video6
        /dev/video7
        /dev/video8
```

```
        /dev/video9
        /dev/video10

rkisp_mainpath (platform:rkisp0-vir0):
        /dev/video11
        /dev/video12
        /dev/video13
        /dev/video14
        /dev/video15
        /dev/video16
        /dev/video17
        /dev/media1

4K USB Camera: 4K USB Camera (usb-xhci-hcd.4.auto-1):
        /dev/video21
        /dev/video22
        /dev/media2
```

格式支持查询：

```
elf@elf2-desktop:~$ v4l2-ctl --list-formats-ext -d /dev/video21
//查看摄像头支持的格式
ioctl: VIDIOC_ENUM_FMT
        Type: Video Capture

        [0]: 'YUYV' (YUYV 4:2:2)
                Size: Discrete 640x480
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1920x1080
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1280x960
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1280x720
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 640x360
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 640x480
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
```

```
                Interval: Discrete 0.033s (30.000 fps)
                Interval: Discrete 0.040s (25.000 fps)
        [1]: 'MJPG' (Motion-JPEG, compressed)
                Size: Discrete 640x480
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 3840x2160
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 2560x1440
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1920x1080
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1280x960
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1280x720
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 640x360
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 640x480
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
        [2]: 'NV12' (Y/CbCr 4:2:0)
                Size: Discrete 640x480
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1920x1080
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1280x960
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.040s (25.000 fps)
                Size: Discrete 1280x720
```

```
                Interval: Discrete 0.033s (30.000 fps)
                Interval: Discrete 0.040s (25.000 fps)
        Size: Discrete 640x360
                Interval: Discrete 0.033s (30.000 fps)
                Interval: Discrete 0.040s (25.000 fps)
        Size: Discrete 640x480
                Interval: Discrete 0.033s (30.000 fps)
                Interval: Discrete 0.040s (25.000 fps)
                Interval: Discrete 0.033s (30.000 fps)
                Interval: Discrete 0.040s (25.000 fps)
```

2、摄像头采集格式查询

摄像头采集格式查询：

```
elf@elf2-desktop:~$ v4l2-ctl -V -d /dev/video21
Format Video Capture:
        Width/Height      : 640/480
        Pixel Format      : 'YUYV' (YUYV 4:2:2)
        Field             : None
        Bytes per Line    : 1280
        Size Image        : 0
        Colorspace        : sRGB
        Transfer Function : Rec. 709
        YCbCr/HSV Encoding: ITU-R 601
        Quantization      : Default (maps to Limited Range)
        Flags
```

3、摄像头图像预览

```
elf@elf2-desktop:~$  export  DISPLAY=:0.0   //执行环境变量
elf@elf2-desktop:~$ gst-launch-1.0 v4l2src device=/dev/video21 ! videoconvert !
video/x-raw,format=NV12,width=640,height=480 ! autovideosink
Setting pipeline to PAUSED ...
Pipeline is live and does not need PREROLL ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
Redistribute latency...
0:00:19.7 / 99:99:99.
```

📖 说明：运行此命令将从/dev/video21 设备获取视频流，转换视频格式为 NV12，并将其显示在 X11 窗口中。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
v4l2src device=/dev/video21	v4l2src 是 GStreamer 中用于访问 V4L2 设备的源插件。它可以从视频捕捉设备（如摄像头）读取视频流。
	device=/dev/video21 指定了要使用的视频

	设备，/dev/video21 是设备的路径。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
videoconvert	videoconvert 是一个 GStreamer 插件，用于转换视频格式。
video/x-raw,format=NV12,width=640,height=480	video/x-raw 是指定视频流的能力描述符，表示原始视频数据。
	format=NV12 指定输出的视频格式为 NV12。
	width=640 和 height=480 指定输出视频的分辨率为 640x480 像素。
autovideosink	autovideosink 是一个视频输出元素，用于显示视频流。autovideosink 会自动选择最合适的视频输出设备（如窗口、屏幕等）来显示视频 。

4、摄像头拍照

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
elf@elf2-desktop:~$ gst-launch-1.0 v4l2src device=/dev/video21 num-buffers=1 ! videoconvert ! video/x-raw,format=NV12,width=640,height=480 ! mppjpegenc ! filesink location=pic.jpg
Setting pipeline to PAUSED ...
Pipeline is live and does not need PREROLL ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
Redistribute latency...
Got EOS from element "pipeline0".
Execution ended after 0:00:00.310412782
Setting pipeline to NULL ...
Freeing pipeline ...
//查看/root 目录下生成 pic.jpg 文件
elf@elf2-desktop:~$ ls
pic.jpg
```

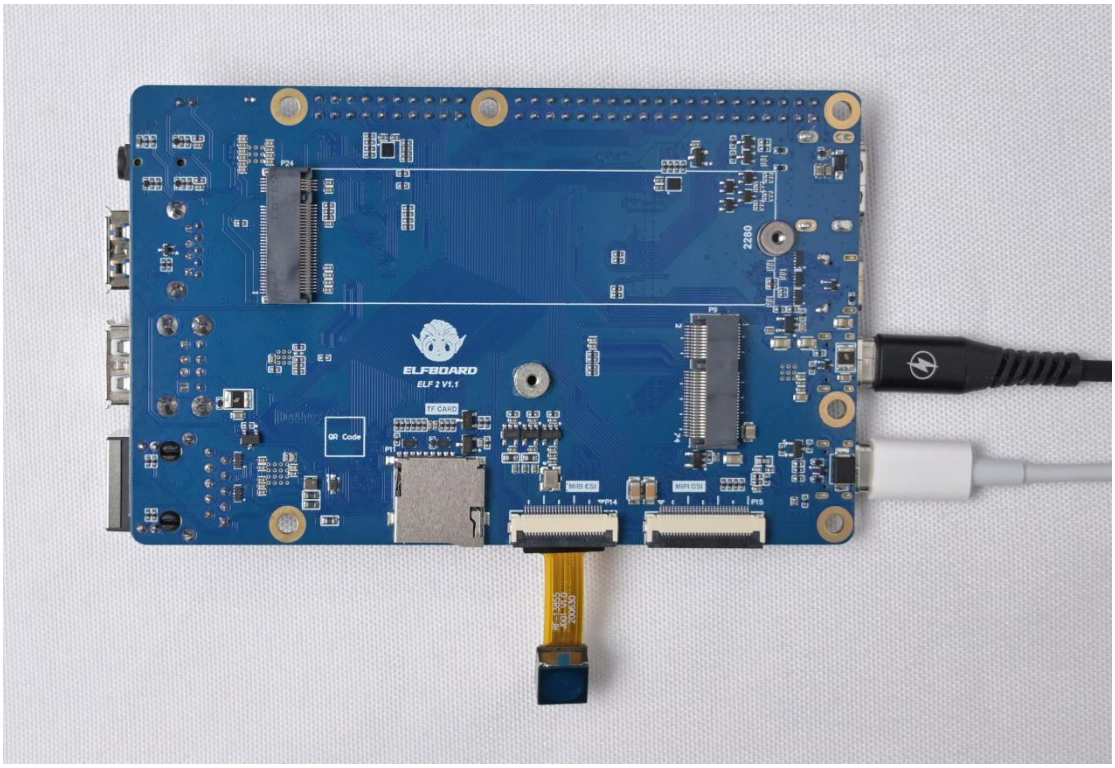
📖 说明：运行此命令将从/dev/video21 设备捕获一帧图像，经过格式转换、JPEG 编码后，将其保存为名为 pic.jpg 的文件。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
v4l2src device=/dev/video21 num-buffers=1	v4l2src 是 GStreamer 中用于访问 V4L2 设备的源插件。它可以从视频捕捉设备（如摄像头）读取视频流。
	device=/dev/video21 指定了要使用的视频设备，/dev/video21 是设备的路径。

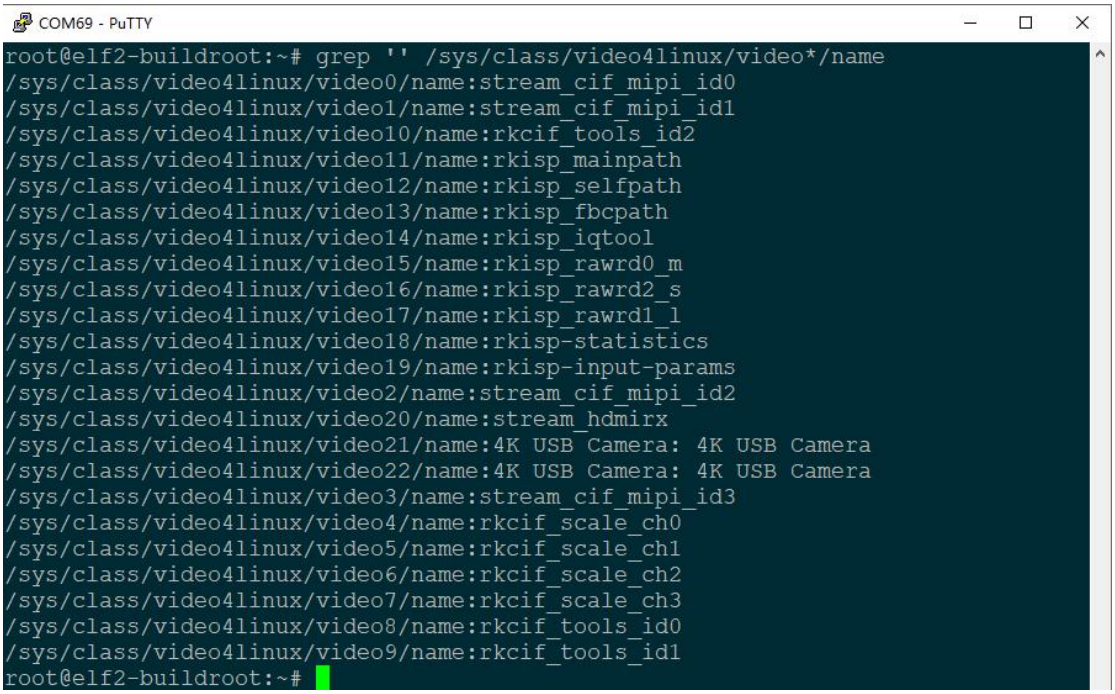
	num-buffers=1 从视频源读取的缓冲区数量为 1，只捕获一帧图像。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
videoconvert	videoconvert 是一个 GStreamer 插件，用于转换视频格式。
video/x-raw,format=NV12,width=640,height=480	video/x-raw 是指定视频流的能力描述符，表示原始视频数据。
	format=NV12 指定输出的视频格式为 NV12。
	width=640 和 height=480 指定输出视频的分辨率为 640x480 像素。
mppjpegenc	mppjpegenc 是 GStreamer 中的一个插件，用于将原始视频帧编码为 JPEG 格式。
filesink location=pic.jpg	filesink 是 GStreamer 中的一个插件，用于将数据写入文件。
	location=pic.jpg 指定输出文件的路径和名称，即将生成的 JPEG 文件命名为 pic.jpg。

4.2.4.2. OV13855 测试

OV13855 摄像头模块与 ELF 2 硬件连接如图：



对于 OV13855 等 raw sensor，每一个 sensor 对应 5 个设备节点：



Mainpath，指 Rockchip ISP 的一个输出节点，可输出全分辨率图像，一般用来拍照，抓取 Raw 图。

Self Path，指 Rockchip ISP 的一个输出节点，最高只能输出 1080p 分辨率，一般用作预览。

Statistics 3A 统计。

Input-params 3A 参数设置。

OV13855 的测试方法与 UVC Camera 的测试方法基本相同。

1、摄像头识别检测和格式支持查询

```
elf@elf2-desktop:~$ v4l2-ctl --list-devices
[   159.770381] m00_b_dw9763 3-000c: dw9763_open: enter, power.usage_count(1)!
rk_hdmirx (fdee0000.hdmirx-controller):
[   159.770422] m00_b_dw9763 3-000c: dw9763_open: exit, power.usage_count(2)!
        /dev/video20
[   159.770435] dw9763 3-000c: cmd 0x80685600 not supported

[   159.770443] dw9763 3-000c: cmd 0xc1007c00 not supported
rkisp-statistics (platform: rkisp):
[   159.770451] m00_b_dw9763 3-000c: dw9763_close: enter, power.usage_count(2)!
        /dev/video18
[   159.770456] m00_b_dw9763 3-000c: dw9763_close: exit, power.usage_count(1)!
        /dev/video19

rkcif-mipi-lvds (platform:rkcif):
        /dev/media0

rkcif (platform:rkcif-mipi-lvds):
        /dev/video0
        /dev/video1
        /dev/video2
```

```
            /dev/video3
            /dev/video4
            /dev/video5
            /dev/video6
            /dev/video7
            /dev/video8
            /dev/video9
            /dev/video10

rkisp_mainpath (platform:rkisp0-vir0):
            /dev/video11
            /dev/video12
            /dev/video13
            /dev/video14
            /dev/video15
            /dev/video16
            /dev/video17
            /dev/media1
```

2、摄像头预览

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
elf@elf2-desktop:~$ gst-launch-1.0 v4l2src device=/dev/video11 ! video/x-raw, format=NV12, width=640, height=480, framerate=30/1 ! autovideosink
Setting pipeline to PAUSED ...
Using mplane plugin for capture
Pipeline is live and does not need PREROLL ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
[   144.625089] rkisp_hw fdcb0000.rkisp: set isp clk = 500000000Hz
New clock: GstSystemClock
[   144.638562] rkcif-mipi-lvds: stream[0] start streaming
[   144.644678] rkcif-mipi-lvds: Allocate dummy buffer, size: 0x01944000
[   144.644733] rockchip-mipi-csi2 mipi0-csi2: stream on, src_sd: 000000005745d45a, sd_name:rockchip-csi2-dphy0
[   144.644740] rockchip-mipi-csi2 mipi0-csi2: stream ON
[   144.644766] rockchip-csi2-dphy0: dphy0, data_rate_mbps 1080
[   144.645125] rockchip-csi2-dphy csi2-dcphy0: csi2_dphy_s_stream stream on:1, dphy0, ret 0
[   144.645137] ov13855 3-0036: ov13855_s_stream: on: 1, 4224x3136@30
Redistribute latency...
0:00:13.6 / 99:99:99.
```

🕮 说明：运行此命令将从指定的摄像头设备捕获视频流，转换为 NV12 格式，分辨率 640x480，帧率 30 fps，并通过 X11 显示服务器实时显示。

3、摄像头拍照

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
```

```
elf@elf2-desktop:~$ gst-launch-1.0 v4l2src device=/dev/video11 num-buffers=1 ! video/x-raw,format=NV12,width=640,height=480 ! mppjpegenc ! filesink location=pic.jpg
Setting pipeline to PAUSED ...
Using mplane plugin for capture
Pipeline is live and does not need PREROLL ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
[   185.454999] rkisp_hw fdcb0000.rkisp: set isp clk = 500000000Hz
New clock: GstSystemClock
[   185.466617] rkcif-mipi-lvds: stream[0] start streaming
[   185.470474] rkcif-mipi-lvds: Allocate dummy buffer, size: 0x01944000
[   185.470541] rockchip-mipi-csi2 mipi0-csi2: stream on, src_sd: 000000005745d45a, sd_name:rockchip-csi2-dphy0
[   185.470547] rockchip-mipi-csi2 mipi0-csi2: stream ON
[   185.470573] rockchip-csi2-dphy0: dphy0, data_rate_mbps 1080
[   185.470865] rockchip-csi2-dphy csi2-dcphy0: csi2_dphy_s_stream stream on:1, dphy0, ret 0
[   185.470875] ov13855 3-0036: ov13855_s_stream: on: 1, 4224x3136@30
Redistribute latency...
Got EOS from element "pipeline0".
Execution ended after 0:00:00.240526899
Setting pipeline to NULL ...
[   185.694528] rkcif-mipi-lvds: stream[0] start stopping, total mode 0x1, cur 0x1
Freeing pipeline ...
//查看是否生成 pic.jpg，可拷贝到 pc 查看
elf@elf2-desktop:~$ ls
pic.jpg
```

📖 说明：运行此命令将从指定的摄像头设备/dev/video11 捕获一帧视频，转换为 NV12 格式，并将其编码为 JPEG 格式，然后保存为 pic.jpg 文件。

4、录制 H264 格式视频

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
elf@elf2-desktop:~$ gst-launch-1.0 v4l2src device=/dev/video11 num-buffers=100 ! video/x-raw,format=NV12, width=640,height=480 ! tee name=t ! queue ! mpph264enc ! queue ! h264parse ! qtmux ! filesink location=13855_h264.mp4 t. ! queue ! autovideosink
//摄像头预览时编码 H264
Setting pipeline to PAUSED ...
Pipeline is live and does not need PREROLL ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
New clock: GstSystemClock
[   625.709873] rockchip-mipi-csi2 mipi0-csi2: stream on, src_sd: 0000000092b9b984, sd_name:rockchip-csi2-dphy0
```

```
[   625.709883] rockchip-mipi-csi2 mipi0-csi2: stream ON
rga_api version 1.10.0_[8]
Redistribute latency...
Redistribute latency...
WARNING: from element
/GstPipeline:pipeline0/GstAutoVideoSink:autovideosink0/GstXvImageSink:autovideosi
nk0-actual-sink-xvimage: A lot of buffers are being dropped.
Additional debug info:
../libs/gst/base/gstbasesink.c(3143): gst_base_sink_is_too_late ():
/GstPipeline:pipeline0/GstAutoVideoSink:autovideosink0/GstXvImageSink:autovideosi
nk0-actual-sink-xvimage:
There may be a timestamping problem, or this computer is too slow.
WARNING: from element
/GstPipeline:pipeline0/GstAutoVideoSink:autovideosink0/GstXvImageSink:autovideosi
nk0-actual-sink-xvimage: A lot of buffers are being dropped.
Additional debug info:
../libs/gst/base/gstbasesink.c(3143): gst_base_sink_is_too_late ():
/GstPipeline:pipeline0/GstAutoVideoSink:autovideosink0/GstXvImageSink:autovideosi
nk0-actual-sink-xvimage:
There may be a timestamping problem, or this computer is too slow.
WARNING: from element
/GstPipeline:pipeline0/GstAutoVideoSink:autovideosink0/GstXvImageSink:autovideosi
nk0-actual-sink-xvimage: A lot of buffers are being dropped.
Additional debug info:
../libs/gst/base/gstbasesink.c(3143): gst_base_sink_is_too_late ():
/GstPipeline:pipeline0/GstAutoVideoSink:autovideosink0/GstXvImageSink:autovideosi
nk0-actual-sink-xvimage:
There may be a timestamping problem, or this computer is too slow.
WARNING: from element
/GstPipeline:pipeline0/GstAutoVideoSink:autovideosink0/GstXvImageSink:autovideosi
nk0-actual-sink-xvimage: A lot of buffers are being dropped.
Additional debug info:
../libs/gst/base/gstbasesink.c(3143): gst_base_sink_is_too_late ():
/GstPipeline:pipeline0/GstAutoVideoSink:autovideosink0/GstXvImageSink:autovideosi
nk0-actual-sink-xvimage:
There may be a timestamping problem, or this computer is too slow.
WARNING: from element
/GstPipeline:pipeline0/GstAutoVideoSink:autovideosink0/GstXvImageSink:autovideosi
nk0-actual-sink-xvimage: A lot of buffers are being dropped.
Additional debug info:
../libs/gst/base/gstbasesink.c(3143): gst_base_sink_is_too_late ():
/GstPipeline:pipeline0/GstAutoVideoSink:autovideosink0/GstXvImageSink:autovideosi
nk0-actual-sink-xvimage:
There may be a timestamping problem, or this computer is too slow.
```

```
WARNING: from element /GstPipeline:pipeline0/GstAutoVideoSink:autovideosink0/GstXvImageSink:autovideosink0-actual-sink-xvimage: A lot of buffers are being dropped.
Additional debug info:
../libs/gst/base/gstbasesink.c(3143): gst_base_sink_is_too_late (): /GstPipeline:pipeline0/GstAutoVideoSink:autovideosink0/GstXvImageSink:autovideosink0-actual-sink-xvimage:
There may be a timestamping problem, or this computer is too slow.
Got EOS from element "pipeline0".
Execution ended after 0:00:06.852649813
Setting pipeline to NULL ...
[  632.569164] rockchip-mipi-csi2 mipi0-csi2: stream off, src_sd: 0000000092b9b984, sd_name:rockchip-csi2-dphy0
[  632.569186] rockchip-mipi-csi2 mipi0-csi2: stream OFF
Freeing pipeline ...
//查看/root 目录下生成 13855_h264.mp4 文件
elf@elf2-desktop:~$ ls
13855_h264.mp4
```

📖 说明：运行此命令将从/dev/video11 设备捕获视频流，编码为 H.264 格式，封装成 MP4 文件保存为 13855_h264.mp4，同时在 X11 显示设备上实时播放视频。

gst-launch-1.0	这是 GStreamer 的命令行工具，用于构建和运行数据流管道，允许用户以命令行的方式播放、处理或转换音视频。
v4l2src device=/dev/video11 num-buffers=100	v4l2src 是用于访问 V4L2（Video for Linux 2）设备的源插件，主要用于从视频捕捉设备（如摄像头）读取视频流。
	device=/dev/video11 指定要使用的视频设备路径。
	num-buffers=100 限制从视频源读取的缓冲区数量为 100，意味着最多捕获 100 帧图像。
!	这个符号在 GStreamer 中表示管道的连接符。它用于将一个元素的输出连接到下一个元素的输入。
video/x-raw,framerate=30/1,width=7680,height=4320video/x-raw,format=NV12,width=640,height=480	video/x-raw 是指定视频流的能力描述符，表示原始视频数据。
	format=NV12 指定输出的视频格式为 NV12。
	width=640 和 height=480 指定输出视频的分辨率为 640x480 像素。
tee name=t	tee 是一个 GStreamer 插件，用于将输入流复制到多个输出流。
	name=t 为 tee 元素指定一个名称 t，以便在后续引用。
queue	queue 是 GStreamer 中的一个插件，用于

	缓冲数据流。
mpph264enc	mpph264enc 是用于将原始视频编码为 H.264 格式的插件。
h264parse	h264parse 是用于解析 H.264 数据流的插件。
qtmux	qtmux 是用于将多媒体流封装成 MP4 格式的插件。
filesink location=13855_h264.mp4	filesink 是 GStreamer 中的一个插件，用于将数据写入文件。
	location=13855_h264.mp4 指定输出文件的路径和名称，即将生成的 MP4 文件命名为 13855_h264.mp4。
t.	表示引用之前定义的 tee 元素
autovideosink	autovideosink 是一个视频输出元素，用于显示视频流。autovideosink 会自动选择最合适的视频输出设备（如窗口、屏幕等）来显示视频 。

5、播放 H264 格式视频

```
elf@elf2-desktop:~$ export DISPLAY=:0.0  //执行环境变量
elf@elf2-desktop:~$ gst-launch-1.0 filesrc location=13855_h264.mp4 ! qtdemux ! queue ! h264parse ! mppvideodec ! autovideosink
Setting pipeline to PAUSED ...
Pipeline is PREROLLING ...
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
Redistribute latency...
New clock: GstSystemClock
Got EOS from element "pipeline0".
Execution ended after 0:00:06.621154540
Setting pipeline to NULL ...
Freeing pipeline ...
```

📖 说明：运行此命令将从 13855_h264.mp4 文件中读取视频流，解封装并解析 H.264 数据，然后解码并在 X11 显示设备上实时播放视频。

第五章 烧写系统

ELF 2 开发板目前支持 OTG 和 TF 卡两种烧写方式。在用户资料中提供了相应的烧写工具，用户可选择任意一种方式进行镜像烧写。

5.1. 烧写所需镜像

路径：ELF 2_Lite 开发板资料包\03-镜像\00-eMMC 镜像\buildroot 镜像\update.img

烧写所需的镜像文件为 update.img，该文件是由 uboot.img、boot.img、rootfs.img 等多个镜像和文件组合打包而成的集成固件。通过烧录工具，可将 update.img 一次性完整烧录至设备中，实现系统更新或设备的首次系统写入。该镜像为设备固件的完整打包形式，适用于系统整体升级或出厂烧录。

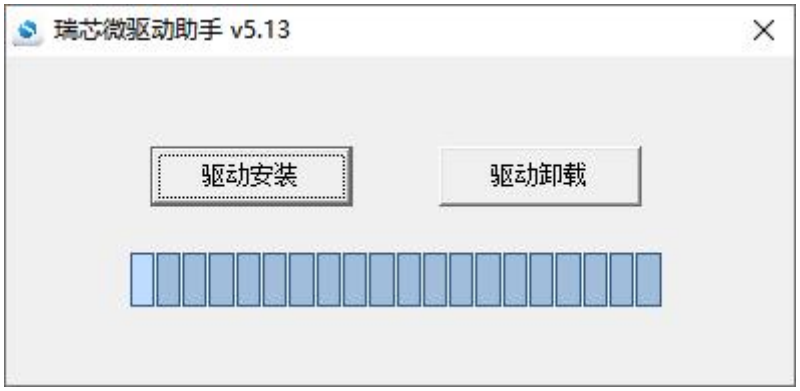
5.2. OTG 烧写系统

5.2.1. OTG 驱动安装

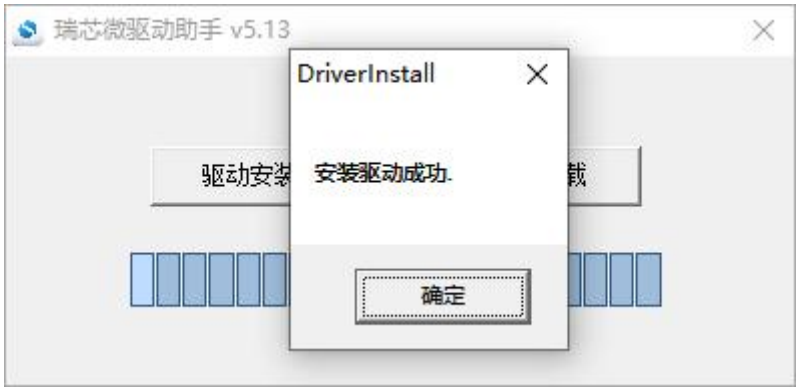
路径：ELF 2 开发板资料包\05-常用工具\03-烧写工具\OTG 烧写\DriverAssitant_v5.13.zip

将上述路径文件解压到任意目录，以管理员权限运行。

打开 DriverInstall.exe 程序。



点击“驱动安装”。

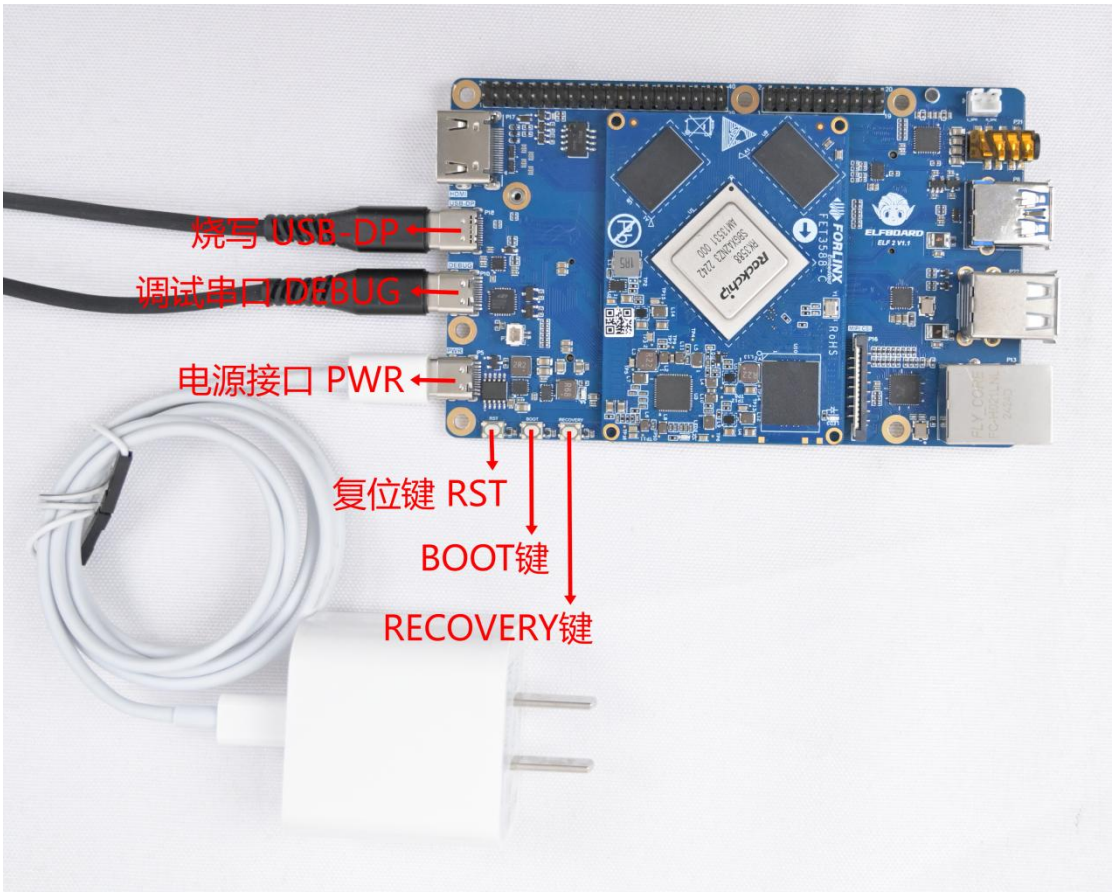


5.2.2. OTG 烧写测试

5.2.2.1. RKDevTool 烧写测试

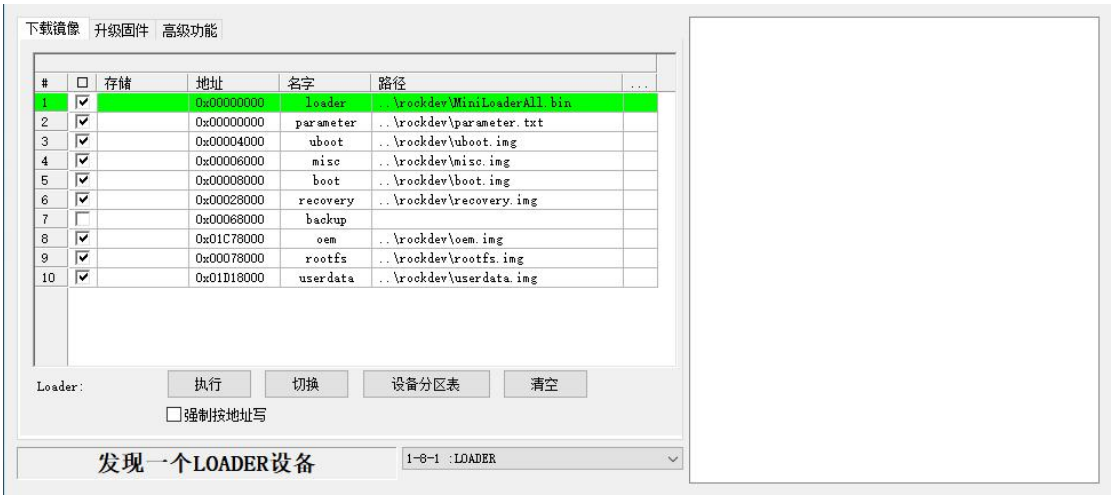
🗁 路径：ELF 2 开发板资料包\05-常用工具\03-烧写工具\OTG 烧写\RKDevTool_Release_v3.31.zip

这是瑞芯微提供的一款开发工具，使用前将其解压到全英文路径下，用 Type-C 线连接开发板 USB-DP 口和主机。打开瑞芯微开发工具，开发板上电，按住开发板的 RECOVERY 键不要松开，按一下 RST 键进行系统复位，抬起 RST 键，直至瑞芯微开发工具提示“发现一个 LOADER 设备”，再抬起 RECOVERY 键。



⚠**注意：**识别设备的操作是开发板上电时 RECOVERY 按键是按下的状态。

⚠**注意：**理论上瑞芯微开发工具解压目录随意，但有用户反馈瑞芯微开发工具解压目录需为全英文，若打开开发工具后与下图不一致，请考虑解压其在全英文目录下。



点击“高级功能”选项，点击“擦除所有”，等待擦除完成。点击“升级固件”选项，点击“固件”按钮选择完整的升级镜像 update.img。程序将对固件进行解析，因此需要等待一会。

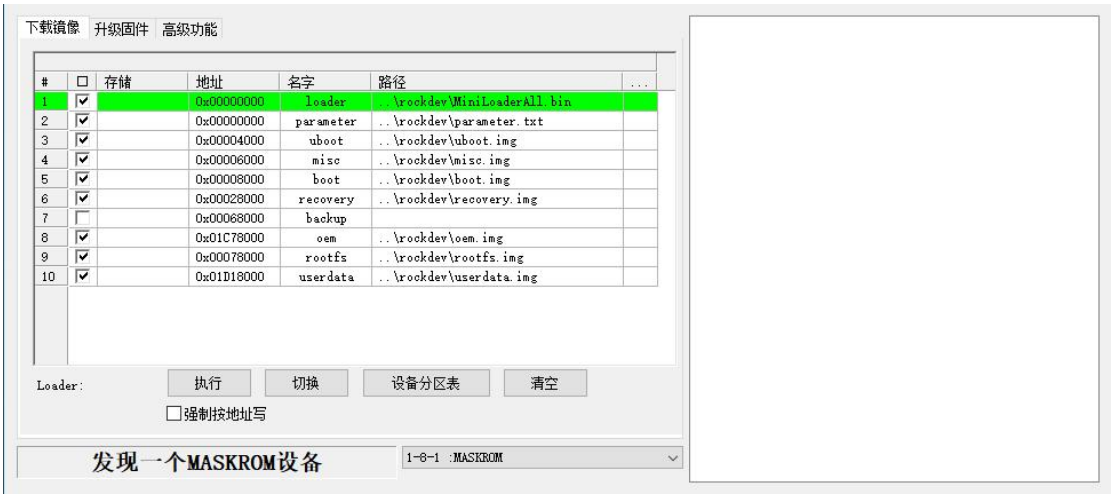


点击“升级”按钮进行固件更新，等待更新完成。



MASKROM 模式介绍

如果 loader 损坏无法进入 LOADER 模式时，可以按住 BOOT 键然后按 RST 键进入 MASKROM 模式进行烧写。



此时系统将提示发现一个 MASKROM 设备，点击“升级固件”选项，点击“固件”按钮选择完整的升级镜像 update.img，点击“升级”按钮进行固件更新，等待更新完成。

⚠**注意：**MASKROM 模式下不要点击“设备分区表”，为无效操作。

⚠**注意：**MASKROM 模式下单独烧写不会清除 UBOOT 环境变量。

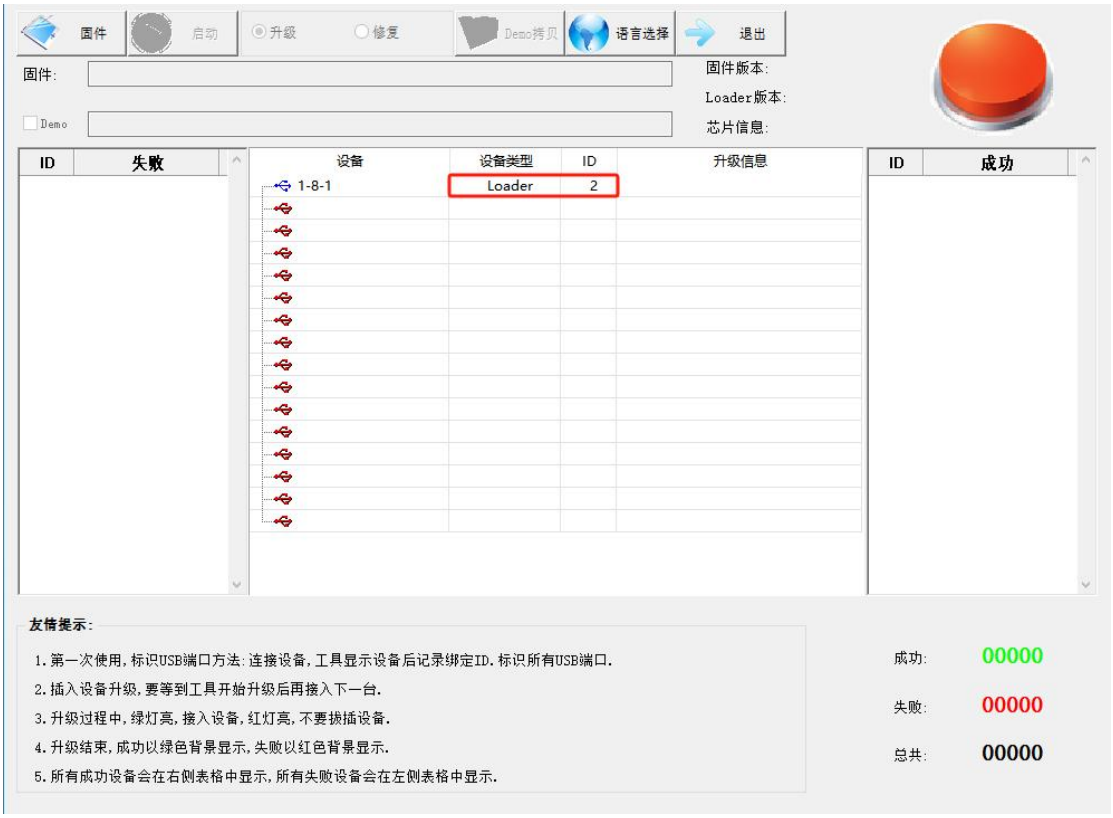
5.2.2.2. FactoryTool 烧写测试

🗁 路径：ELF 2 开发板资料包\05-常用工具\03-烧写工具\OTG 烧写\FactoryTool_v1.91.zip

FactoryTool 是工厂批量 OTG 烧写时使用到的工具，其不需要读取镜像，可以批量烧写，另外其可烧写某些较大的镜像文件，若 RKDevTool 兼容性不满足时，亦可尝试该方式，使用前将其解压到全英文路径下，用 Type-C 线连接开发板 USB-DP 接口和主机，按住开发板的 RECOVERY 键不要松开，然后按一下 RST 键系统复位，大约两秒后松开 RECOVERY 键。瑞芯微开发工具上将提示发现 Loader 设备。

⚠**注意：**识别设备的操作是开发板上电时 RECOVERY 按键是按下的状态。

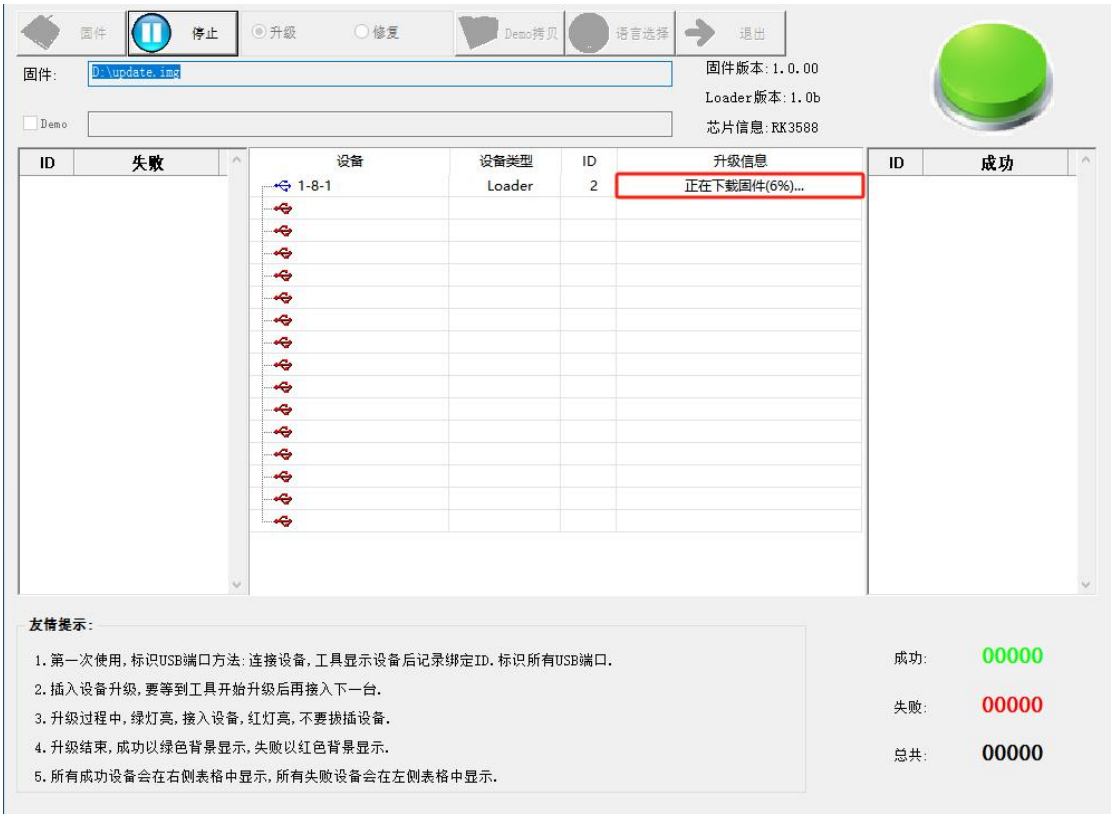
⚠**注意：**理论上解压目录随意，但有用户反馈瑞芯微开发工具解压目录需为全英文，若打开开发工具后与下图不一致，请考虑解压其在全英文目录下。

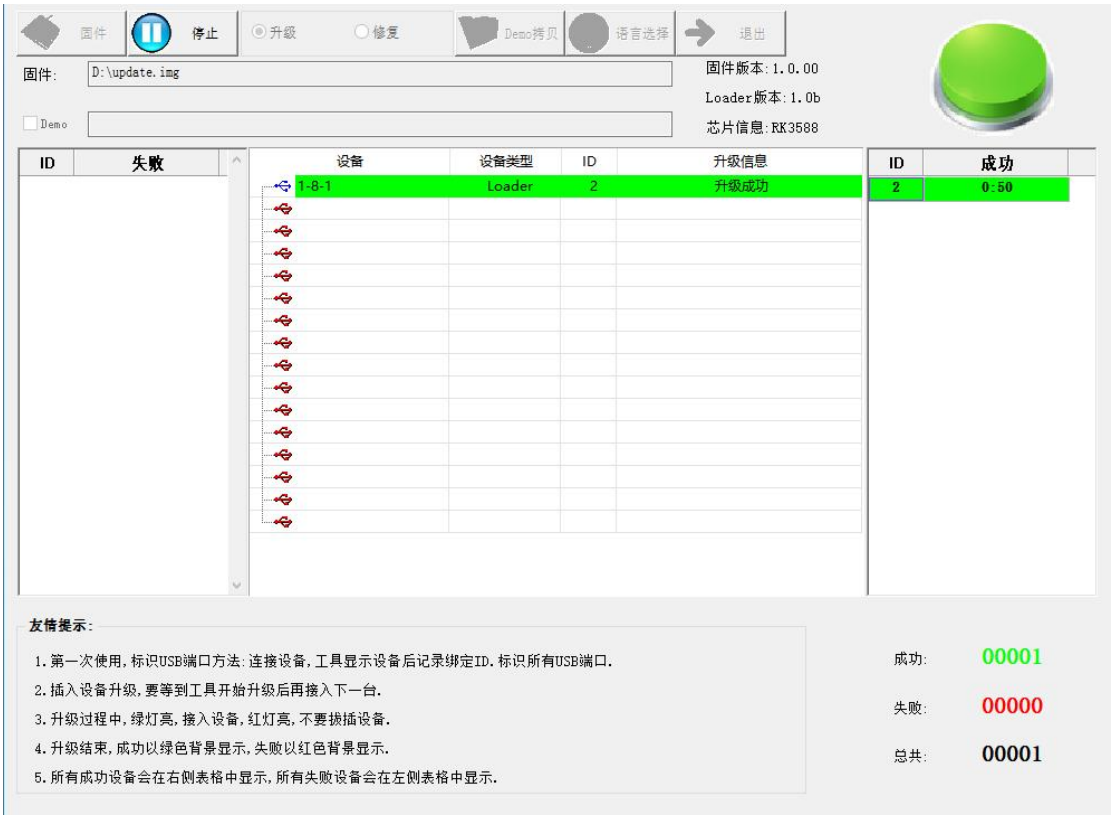


点击固件按钮，选择要更新的固件，固件的路径请根据实际存放位置进行调整。



点击启动按钮将自动开始烧写。





等待烧写完成后，点击停止按钮，再点击退出按钮，最后重启系统。

5.3. TF 卡烧写系统

烧写 TF 卡制作与烧写测试。

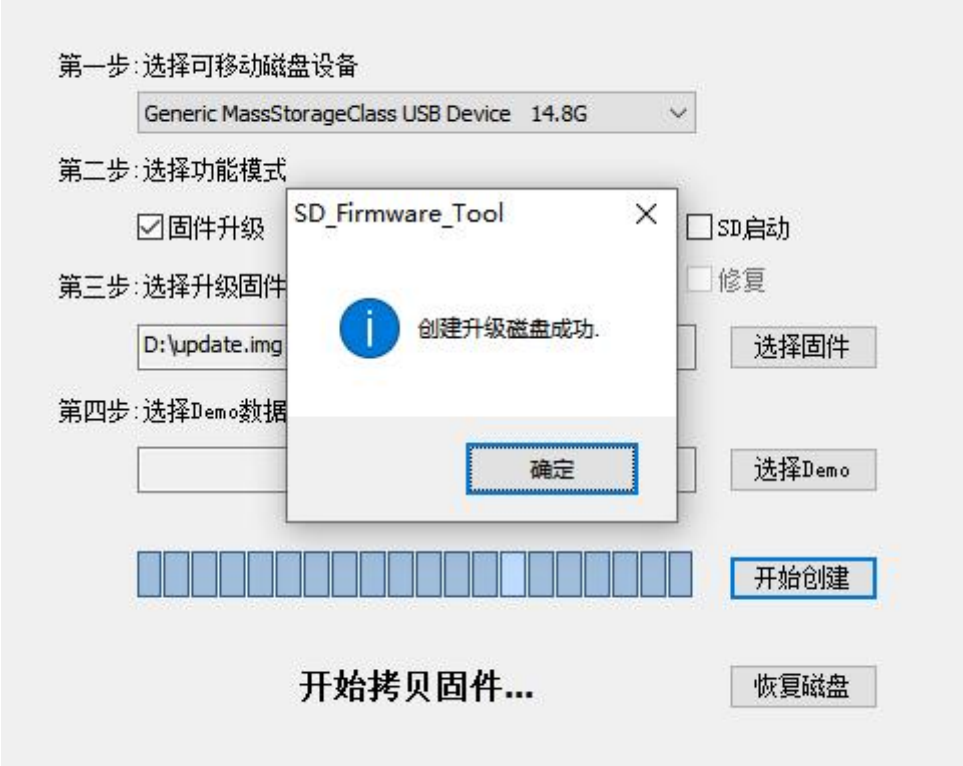
⚠**注意：**测试 TF 卡容量最大为 32G，使用 32G 以上 TF 卡可能会烧写失败。

将 ELF 2 开发板资料包\05-常用工具\03-烧写工具\TF 卡烧写\SDDiskTool_v1.78.zip 拷贝到 windows 任意目录。以管理员权限运行 SD_Firmware_Tool.exe。



TF 卡通过读卡器插入电脑 USB 接口，选择磁盘设备，勾选固件升级，选择升级固件 update.img，点击开始创建。





将 TF 卡插入开发板并上电启动，系统将自动进入烧写流程。烧写成功有如下两种判断方法：

1、批量生产时，可以根据核心板的蓝色指示灯来判断烧写是否完成，烧写过程中的指示灯变化如下：

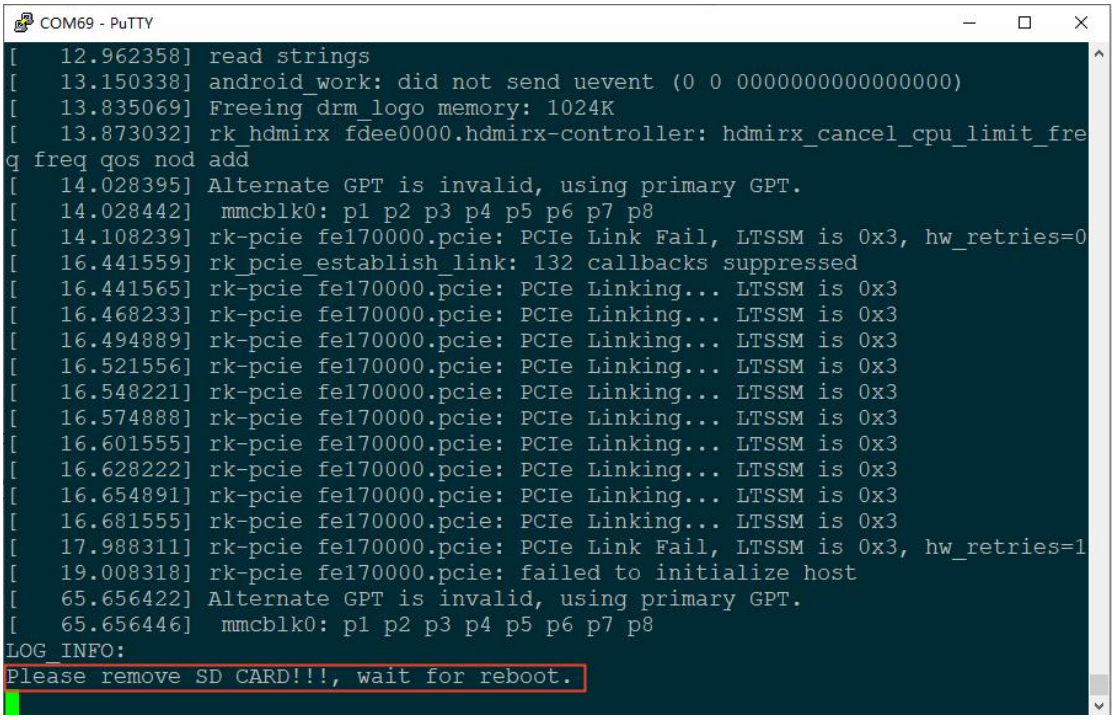
1）内核启动阶段：心跳灯模式，规律的间歇性闪烁。

2）烧写准备阶段：EMMC 指示灯，熄灭。

3）烧写进行阶段：EMMC 指示灯，常亮。

4）烧写完成阶段：心跳灯模式，规律的间歇性闪烁。

2、屏幕和调试串口都将提示：Please remove SD CARD!!!, wait for reboot.



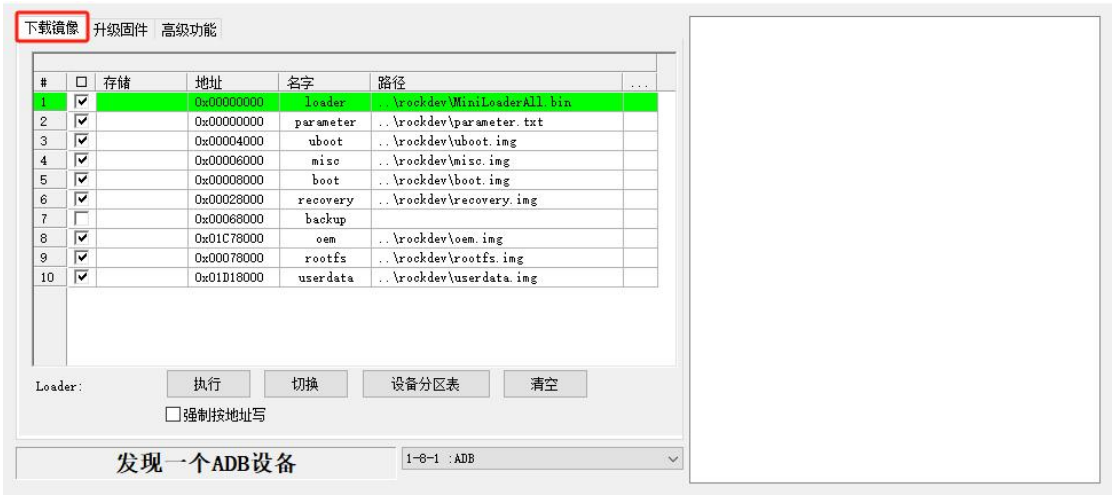
烧写完成后，**请勿断电，直接**拔出 TF 卡，系统将自动重启。若未拔出 TF 卡，重启后系统将再次进入烧写模式。若拔出了 TF 卡，系统未自动重启，重新上电或者按 RST 键手动重启也可完成烧写。烧写过程中请耐心等待。

5.4. 单独更新镜像

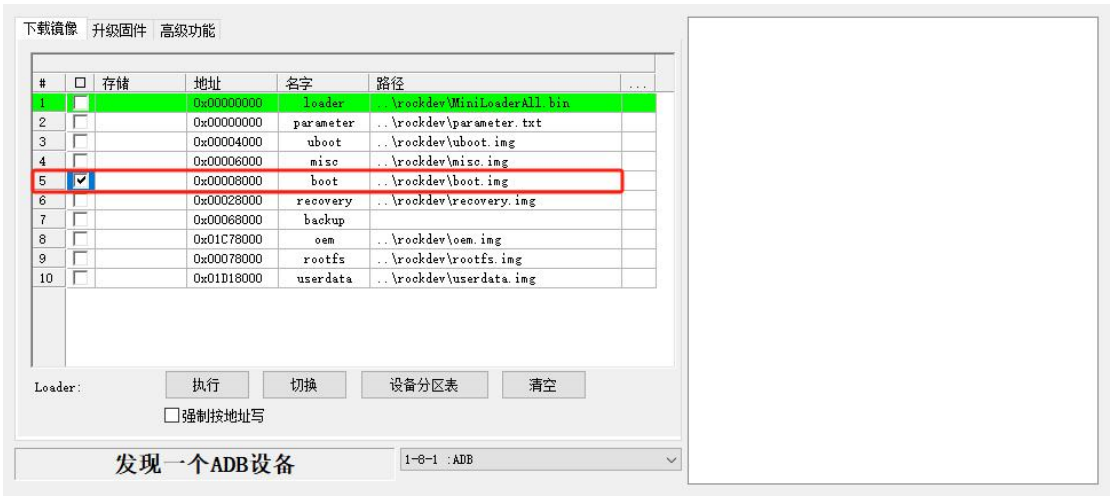
5.4.1. 单独更新内核

除了整体更新之外，RKDevTool 还支持单步烧录，例如只更改了内核部分，只需要单独编译内核之后，将生成的 kernel/boot.img 更新到开发板中。

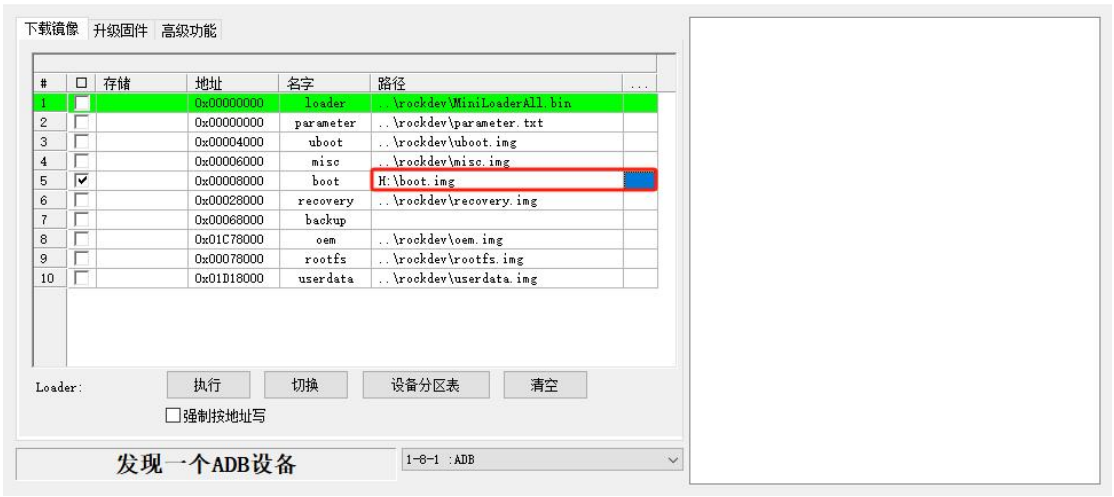
点击“下载镜像”按钮。



勾选烧录 boot 分区。

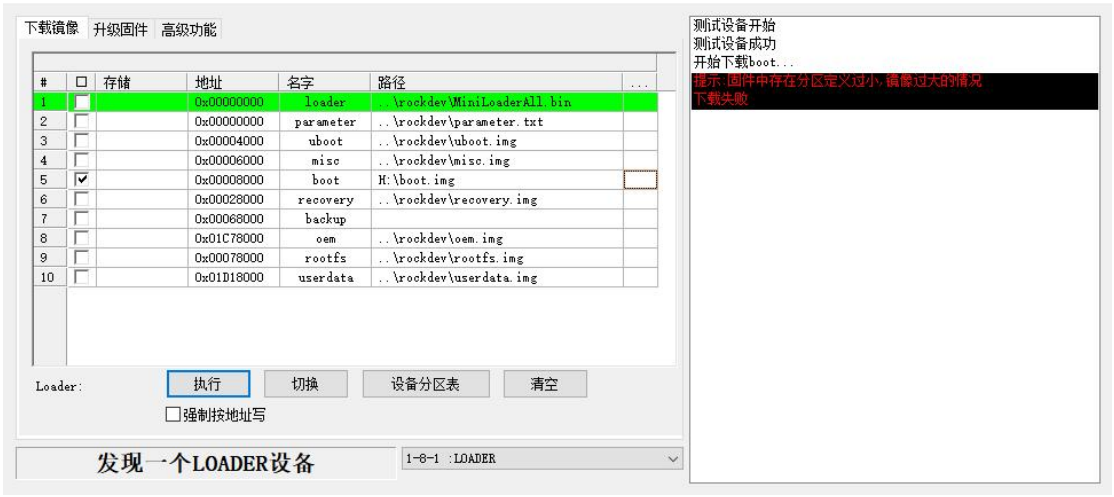


选择 boot.img 镜像。

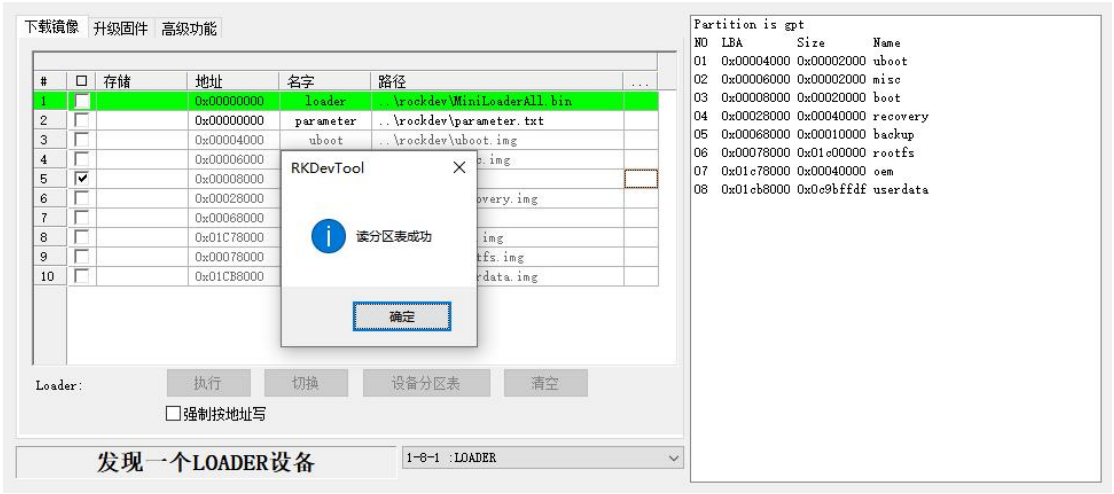


同样，用 Type-C 线连接开发板 USB-DP 口和主机，按住开发板的 RECOVERY 键不要松开，然后按一下 RST 键系统复位，大约两秒后松开 RECOVERY，烧录工具会显示“发现一个 LOADER 设备”，最后点击“执行”。

如果显示“boot 超过 Flash 大小，下载失败”。



需要点击“设备分区表”，重新识别设备分区表。



再次点击“执行”进行烧录。



5.4.2. 单独更新 uboot

烧写 u-boot/uboot.img 文件，具体操作可参考“单独更新内核”章节。

5.4.3. 单独更新文件系统

5.4.3.1. 更新 buildroot 系统

烧写 buildroot/output/elf2_fs/images/rootfs.ext2 文件，具体操作可参考“单独更新内核”章节。

5.4.3.2. 更新 Desktop 系统

烧写 ubuntu/jammy-rootfs.img 文件，具体操作可参考“单独更新内核”章节。

5.4.4. 修改 buildroot 文件系统的方法

1.使用 mount 命令，将 ELF2-linux-source/prebuilts/elfboard/buildroot 路径下的 rootfs.ext4 这个文件挂载到指定文件夹。

```
elf@ubuntu:~/work/ELF2-linux-source$ cd prebuilts/elfboard/buildroot
elf@ubuntu:~/work/ELF2-linux-source/prebuilts/elfboard/buildroot$ mkdir rootfs
elf@ubuntu:~/work/ELF2-linux-source/prebuilts/elfboard/buildroot$ sudo mount
rootfs.ext4 rootfs/
[sudo] password for elf:
```

2.进入挂载文件夹，可以看到整体的根文件目录。

```
elf@ubuntu:~/work/ELF2-linux-source/prebuilts/elfboard/buildroot$ cd rootfs/
elf@ubuntu:~/work/ELF2-linux-source/prebuilts/elfboard/buildroot/rootfs$ ls
bin                etc       linuxrc       opt              sbin       udisk
black.jpg          home      lost+found    proc             sdcard     userdata
busybox.fragment   info      media         rockchip-test    sys        usr
data               lib       mnt           root             system     var
dev                lib64     oem           run              tmp        vendor
```

可以通过 cp 命令将“编译并运行命令行应用”章节编译的 cmddemo_wdt 应用拷贝到文件系统的 root 目录下。

```
elf@ubuntu:~/work/ELF2-linux-source/prebuilts/elfboard/buildroot/rootfs$ sudo
cp ../../../../app/elf/elf_cmd/cmddemo_wdt/cmddemo_wdt root/
[sudo] password for elf:
```

退出 rootfs 目录，解除挂载。

```
elf@ubuntu:~/work/ELF2-linux-source/prebuilts/elfboard/buildroot/rootfs$ cd ..
elf@ubuntu:~/work/ELF2-linux-source/prebuilts/elfboard/buildroot$ sudo umount rootfs
[sudo] password for elf:
```

5.单独烧录 rootfs.ext4，可参考“单独更新内核”章节。

5.4.5. 修改 Desktop 文件系统的方法

1.使用 mount 命令，将 jammy-rootfs.img 挂载到指定文件夹。

```
elf@ubuntu:~/work/ELF2-linux-source$ cd ubuntu/
elf@ubuntu:~/work/ELF2-linux-source/ubuntu$ mkdir rootfs
elf@ubuntu:~/work/ELF2-linux-source/ubuntu$ sudo mount jammy-rootfs.img rootfs/
[sudo] password for elf:
```

2.进入挂载文件夹，可以看到整体的根文件目录。

```
elf@ubuntu:~/work/ELF2-linux-source/ubuntu$ cd rootfs/
elf@ubuntu:~/work/ELF2-linux-source/ubuntu/rootfs$ ls
bin   dev  home  lost+found  mnt  proc  run   snap  swapfile  tmp  var
boot  etc  lib   media       opt  root  sbin  srv   sys       usr
```

3.可以通过 cp 命令将“4.2.5 编译并运行命令行应用”章节编译的 cmddemo_wdt 应用拷贝到文件系统的 root 目录下。

```
elf@ubuntu:~/work/ELF2-linux-source/ubuntu/rootfs$ sudo cp ../../app/elf/elf_cmd/cmddemo_wdt/cmddemo_wdt root/
[sudo] password for elf:
```

4.安装软件包 chroot path(挂载路径) /bin/bash -c "apt-get install xxx“,例如安装 net-tools 工具。

```
elf@ubuntu:~/work/ELF2-linux-source/ubuntu/rootfs$ sudo chroot . /bin/bash -c "apt-get install net-tools"
```

5.退出 rootfs 目录，解除挂载。

```
elf@ubuntu:~/work/ELF2-linux-source/ubuntu/rootfs$ cd ..
elf@ubuntu:~/work/ELF2-linux-source/ubuntu$ sudo umount rootfs
[sudo] password for elf:
```

5.单独烧录 jammy-rootfs.img，可参考“单独更新内核”章节。

5.4.6. 单独更新 logo

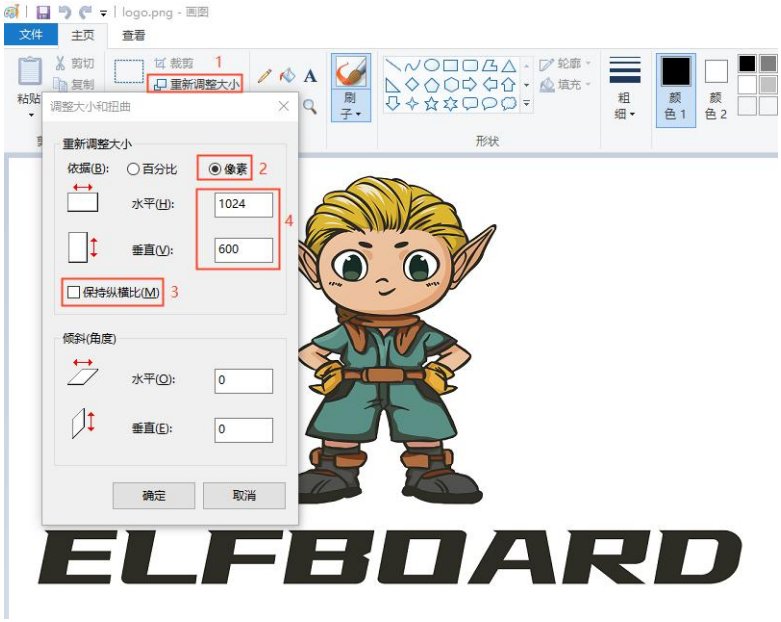
1.以下图为例，制作一张 24 位 bmp 格式的图片。



①将图片保存之后，右击图片，打开方式选择画图。



②将图片分辨率修改为 1024x600。



③分辨率修改完成后，点击文件，选择另存为，文件名为 logo，保存类型选择 24 位位图（*.bmp;*.dib），点击保存。



出现如下提示信息，点击确定即可。



至此图片制作完成。

2.若要更换 U-Boot 阶段的 logo，只需将制作好的 BMP 格式图片命名为 logo.bmp，并替换掉 kernel 目录下的原 logo.bmp 文件。若要更换内核阶段的 logo，则需将 BMP 格式图片命名为 logo_kernel.bmp，并替换 kernel 目录中的 logo_kernel.bmp 文件。

3.更换之后重新编译内核，并进行烧写内核文件，可参考“单独更新内核”章节。

4.观察屏幕，已显示我们更换的图片。

第六章 开发环境搭建

本章节将从环境搭建入手，说明相关步骤。由于环境搭建过程中可能会遇到一些不可预见的问题，建议初学者直接使用我们已搭建好的开发环境，以便快速上手并缩短开发时间。

在 Linux 系统中，通常有三种安装方式：真机双系统、真机单系统和虚拟机。每种安装方式都有其优缺点，本文仅介绍如何在虚拟机中搭建 Ubuntu 系统。关于计算机硬件的要求，我们建议内存至少为 16GB，这样在为虚拟机分配一定内存（建议 10GB 及以上）的同时，仍能在 Windows 上进行其他操作，否则会导致 Windows 的操作变得非常缓慢。

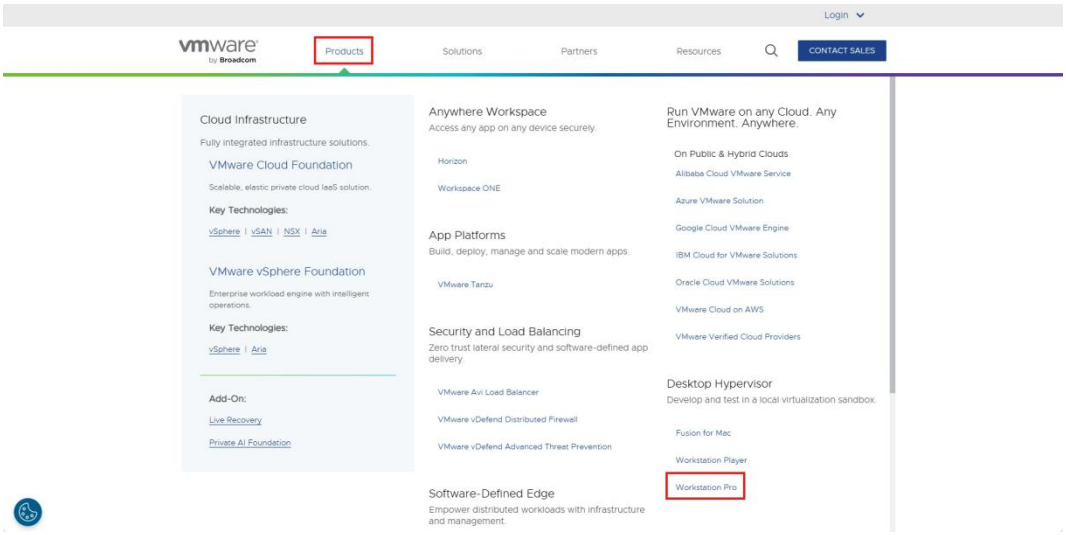
6.1. VMware 虚拟机软件安装

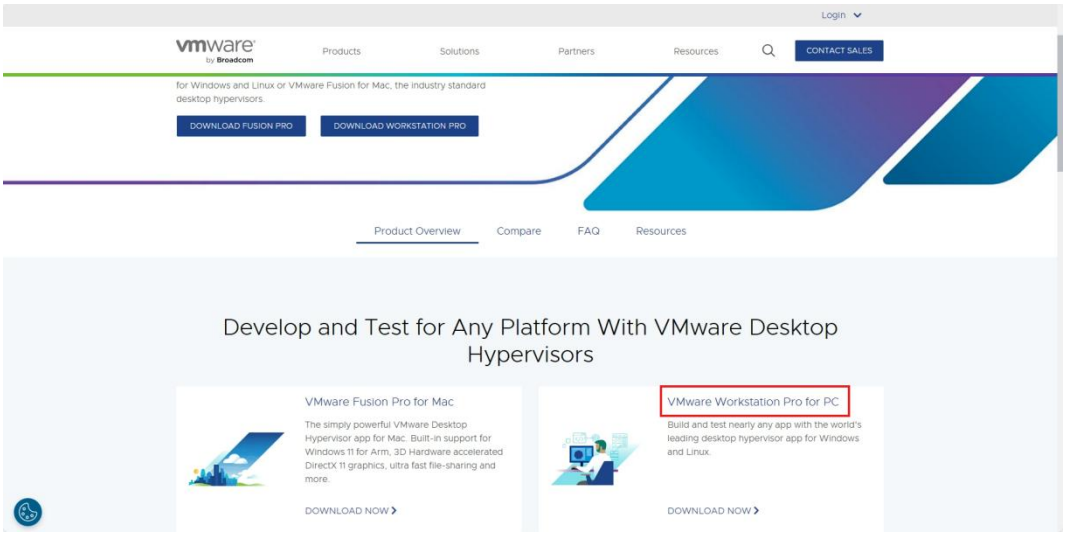
本章主要介绍 VMware 虚拟机的安装，以 VMware workstation 17 Pro v17 为例展示操作系统的安装配置过程。

6.1.1. VMware 软件的下载与购买

⚠**注意**：由于 VMware 官网内容可能会调整，链接和信息可能有变化。建议查阅时随时留意最新更新，并参考官方文档获取准确信息。

登陆 VMware 官网 https://www.vmware.com/下载 Workstation Pro 并获取产品密匙。VMware 是付费软件，需要自行购买，或者可以选择使用试用。





目前官网最新的 Workstation Pro 已经更新到 Pro17，本文选择了 Pro17 版本进行使用和讲解。

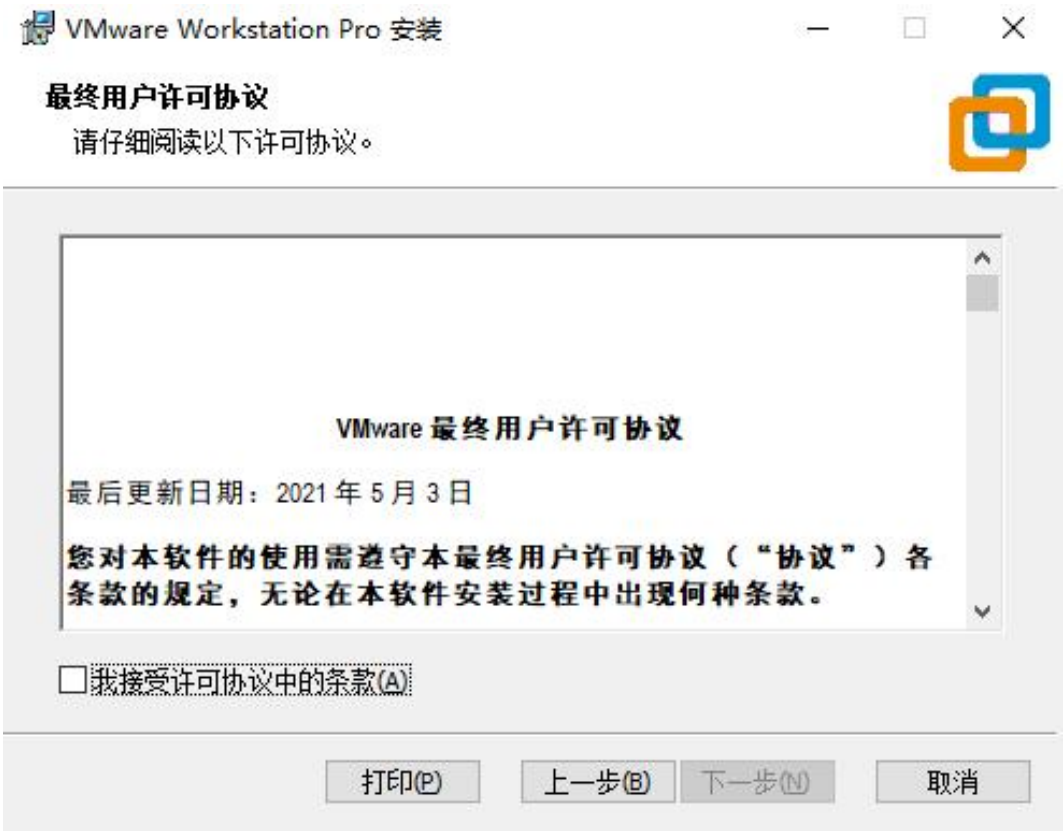
我们已经提供了 Vmware Workstation 软件，大家可以直接使用，路径：ELF 2 开发板资料包\05-常用工具\02-环境搭建工具\VMware-workstation-full-17.0.0-20800274.exe

6.1.2. VMware 软件的安装

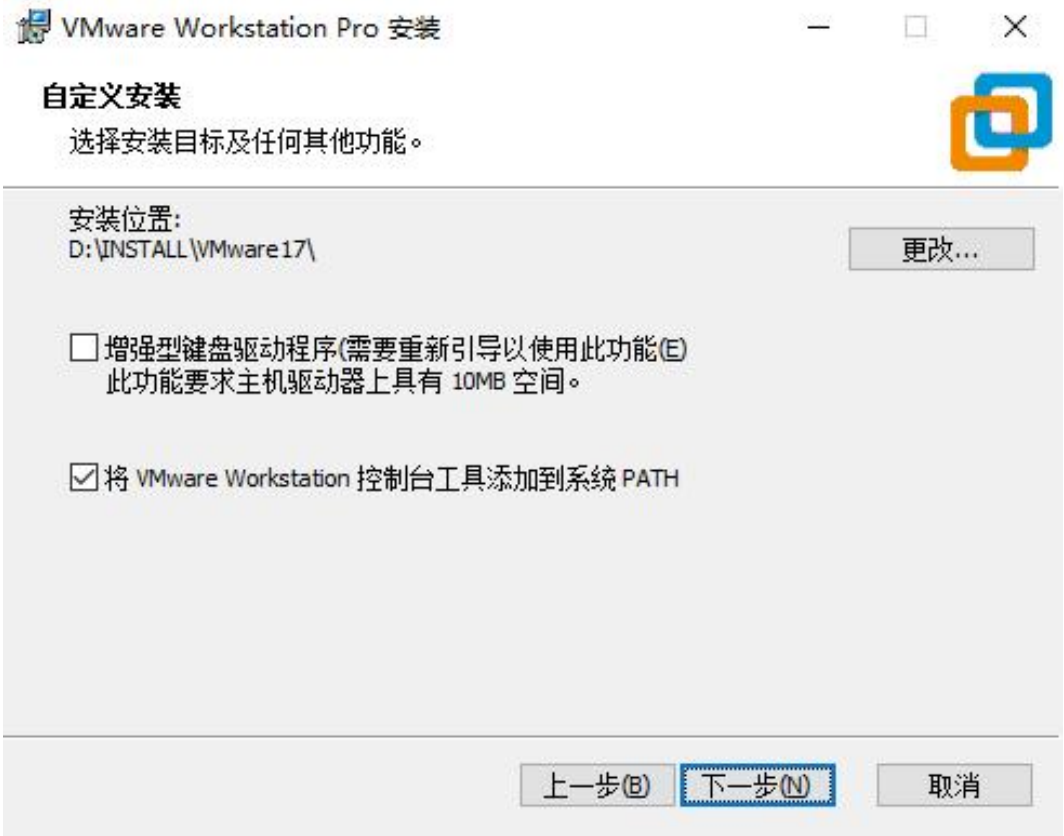
双击 VMware-workstation-full-17.0.0-20800274.exe 进入安装向导。



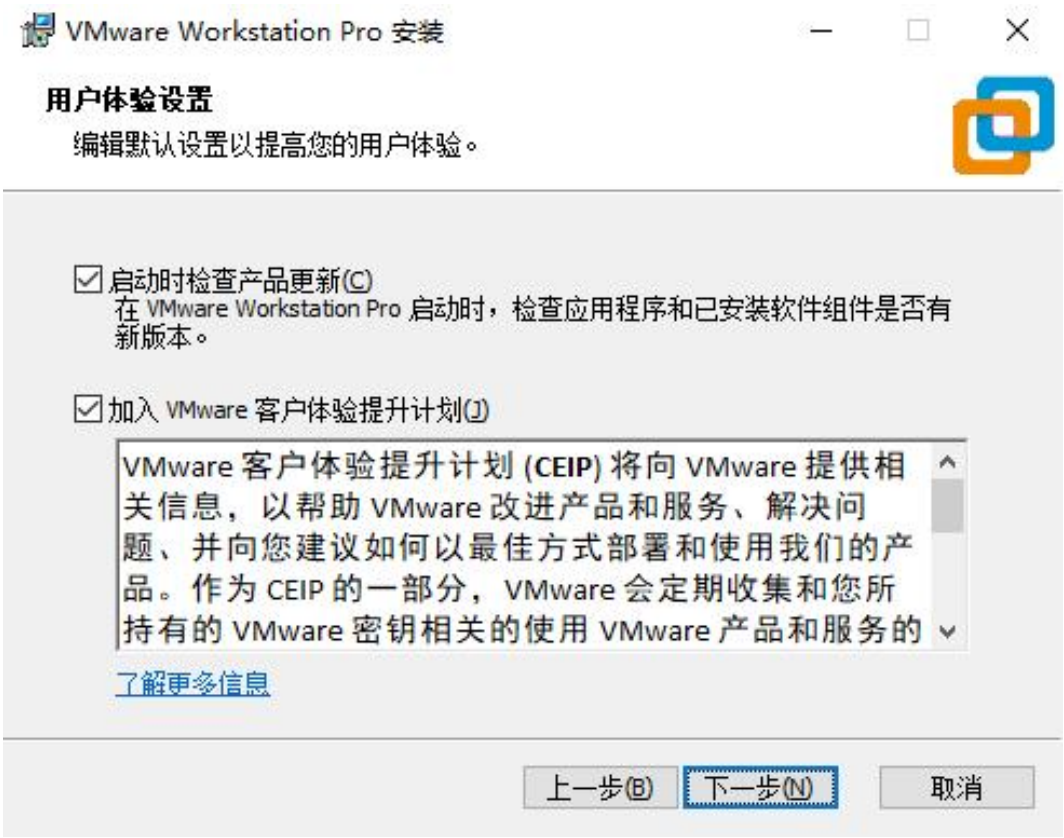
点击“下一步”。



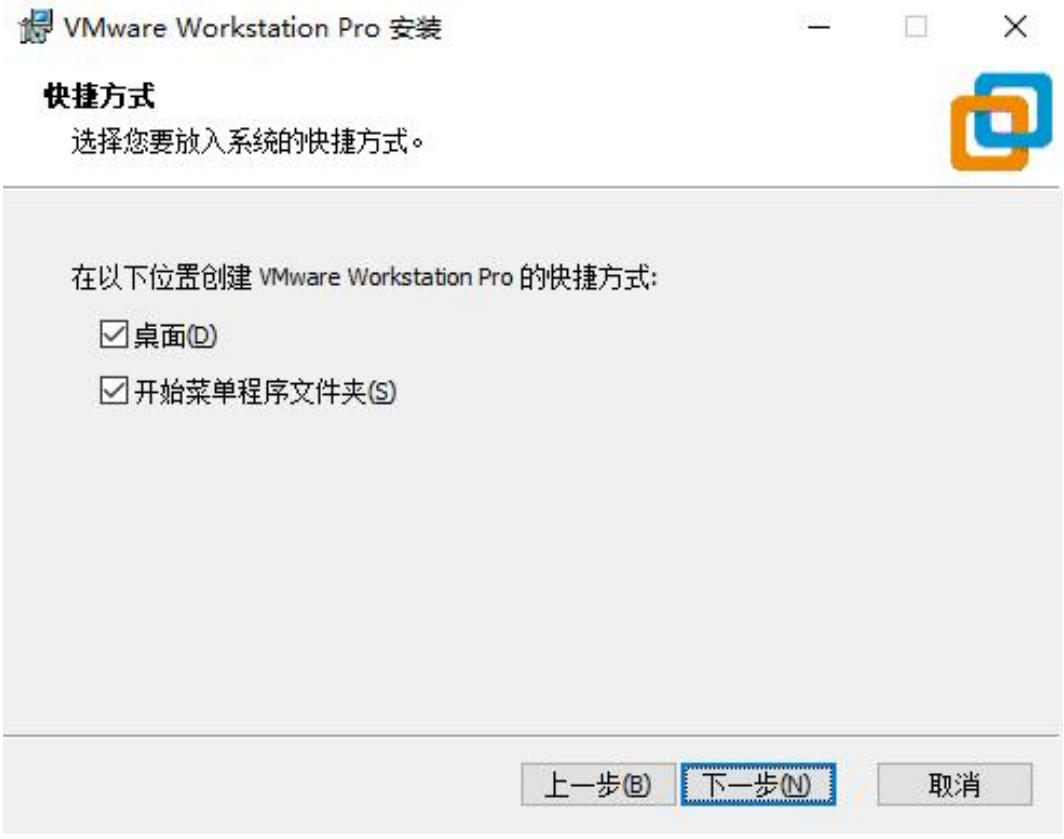
勾选我接受许可协议中的条款，点击“下一步”。



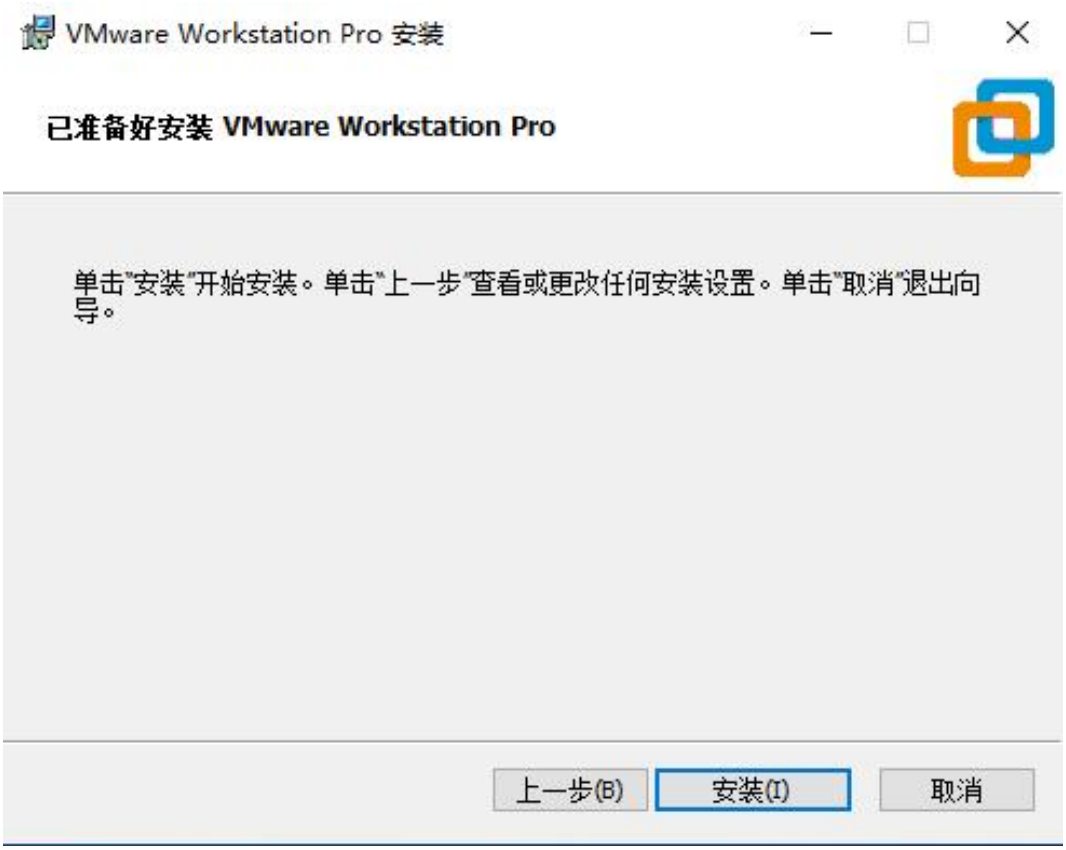
修改安装位置，装到自己电脑安装软件的分区，点击“下一步”。



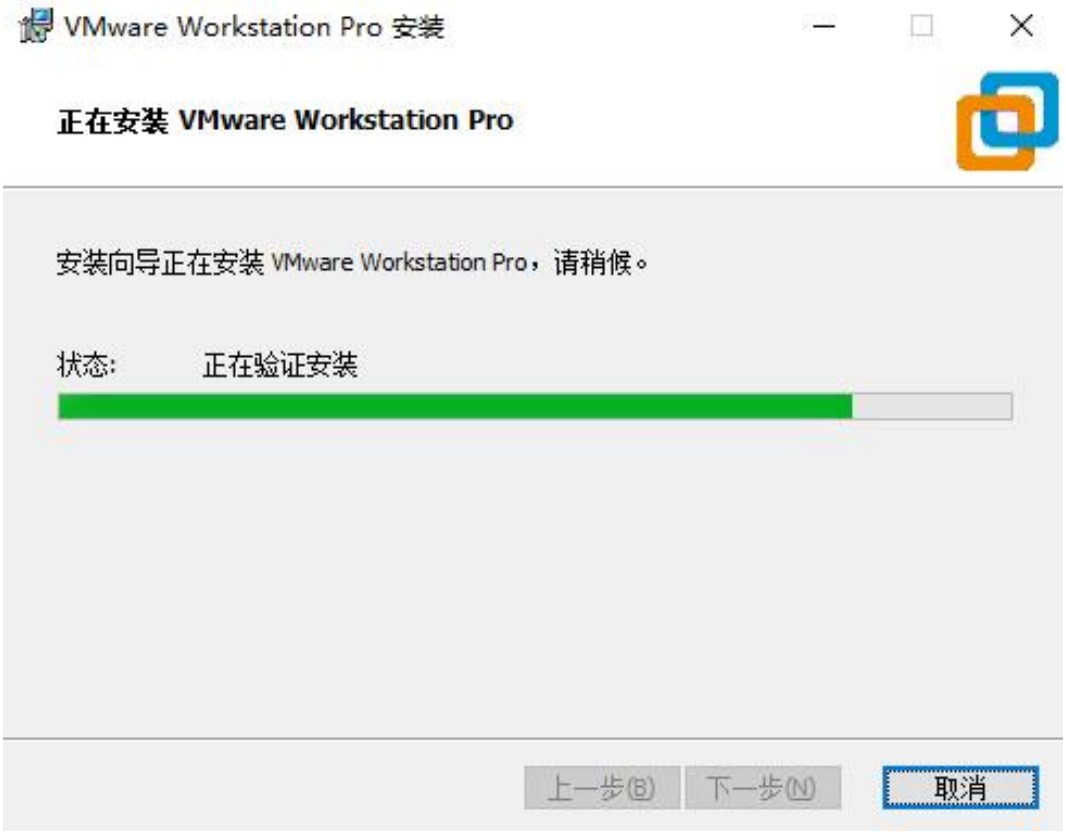
勾选，点击“下一步”。



勾选添加快捷方式，点击“下一步”。

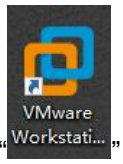

VMware Workstation Pro 安装
已准备好安装 VMware Workstation Pro
单击"安装"开始安装。单击"上一步"查看或更改任何安装设置。单击"取消"退出向导。
上一步(B)
安装(I)
取消

点击“安装”。


VMware Workstation Pro 安装
正在安装 VMware Workstation Pro
安装向导正在安装 VMware Workstation Pro，请稍候。
状态:
正在验证安装
上一步(B)
下一步(N)
取消

等待安装完成。



点击完成安装。双击桌面上的 VMware 图标“ 

”



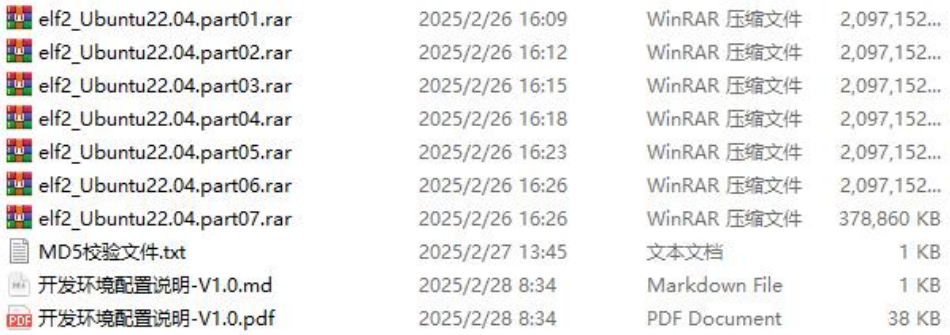
学习可以选择试用，若用户需要长期使用，需要到官方购买，填写许可证。

6.2. 加载已有 ubuntu 开发环境

对于零基础的初学者，建议先使用资料包中提供的开发环境来进行 ubuntu 系统的基本操作，在对 Li

nux 系统有了基本了解之后再搭建属于自己的 ubuntu 系统。提供的开发环境已经做了一些基本配置和网络服务的搭建（详细内容可见开发环境中的说明文档），开发者可以直接进行开发操作。

在 ELF 2 开发板资料包\07-开发环境中有以下文件：

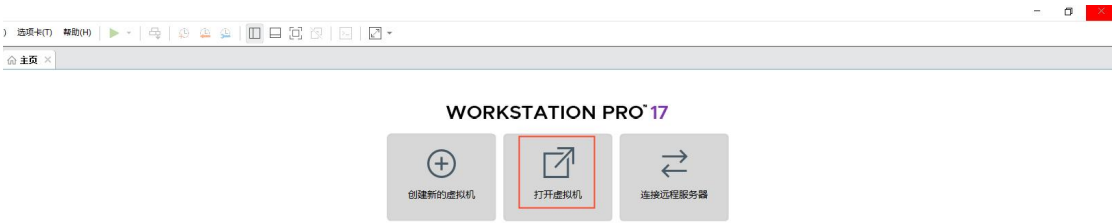


开发环境资料中有 MD5 校验文件，用户下载完开发环境资料，先对开发环境压缩包进行 MD5 校验（MD5 校验可以在网络上选择 MD5 在线工具校验，也可以下载 MD5 校验工具进行校验，可根据实际情况选择），查看校验码和校验文件中校验码是否一致，若一致则下载文件正常；若不一致，则文件可能有破损，需要重新下载。

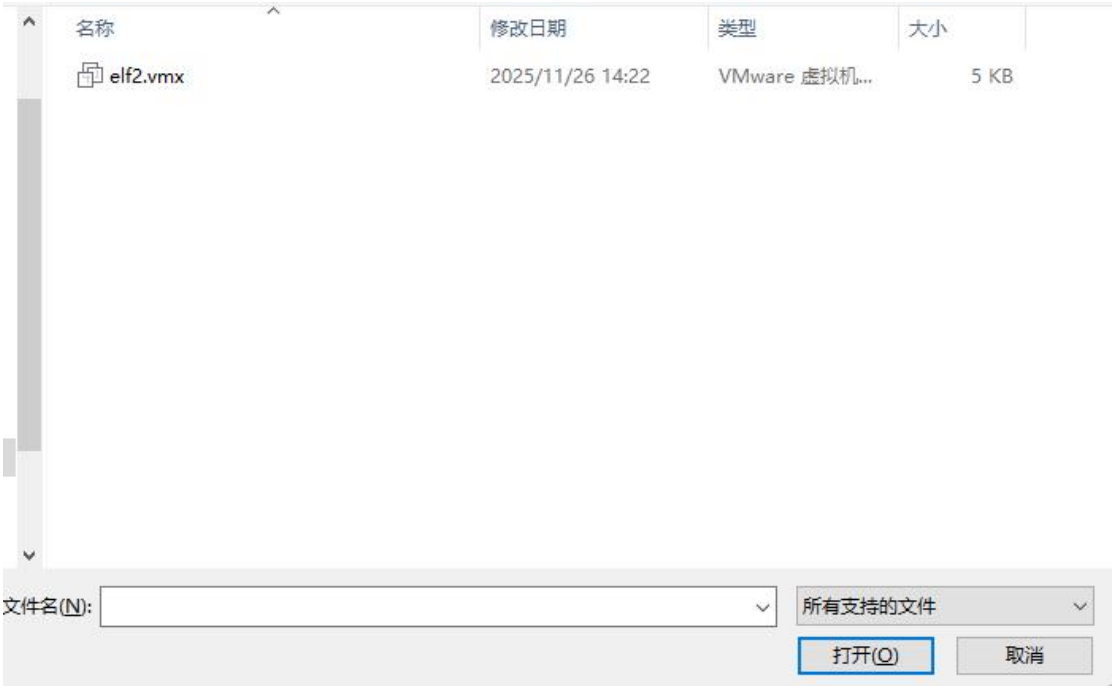
选中所有压缩包，右键解压到当前文件夹或自己的目录。解压完成后得到开发环境 elf2。elf2 开发环境文件夹中的 elf2.vmx 为虚拟机要打开的文件。



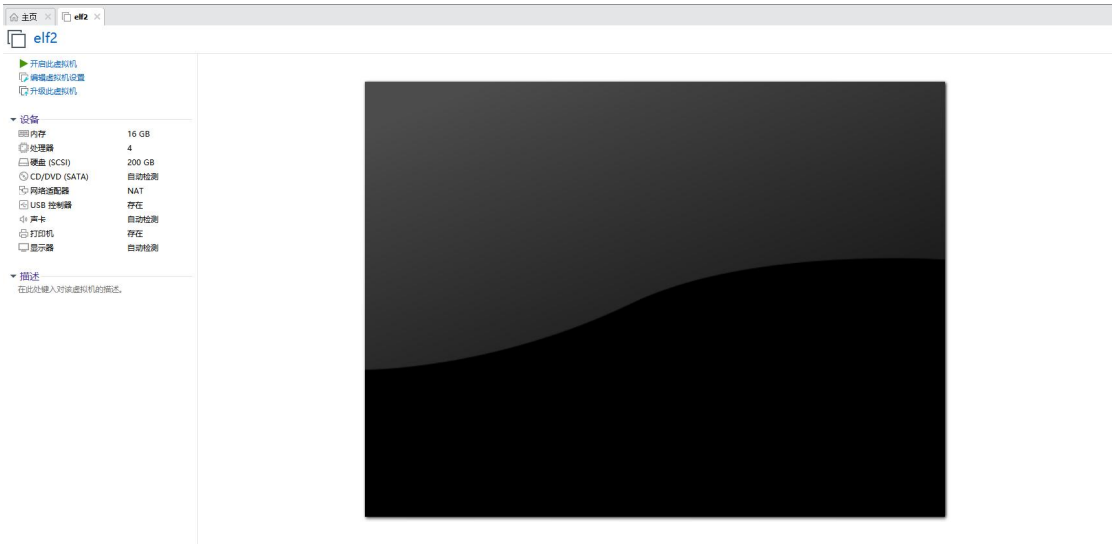
打开安装好的虚拟机。



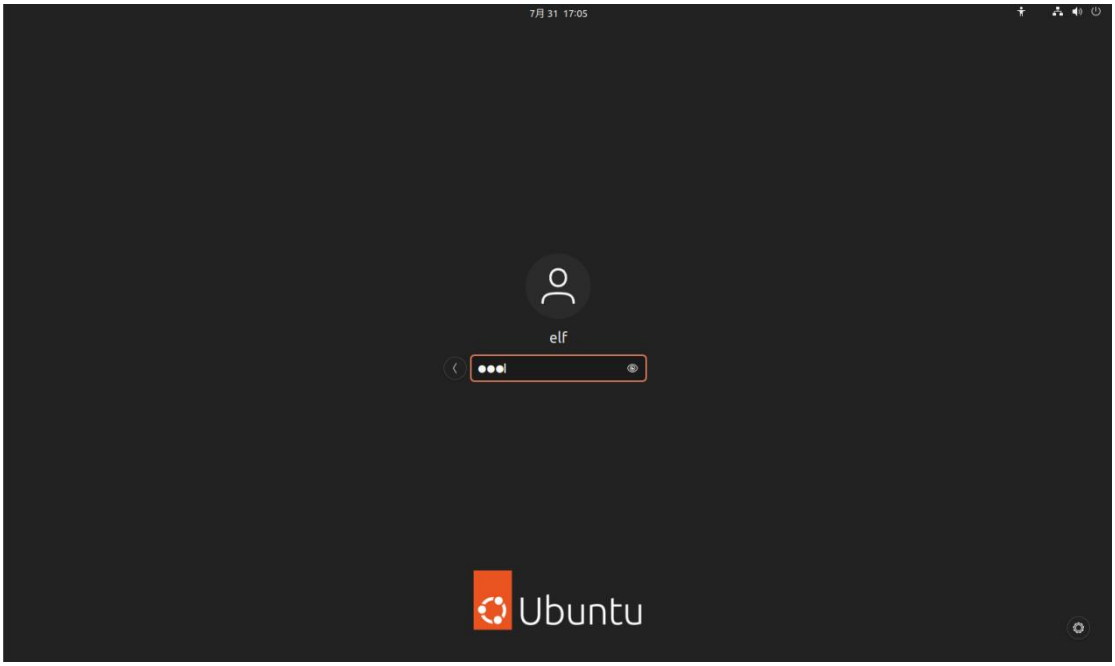
选择刚解压生成的 elf2 虚拟机文件所在的目录，双击打开启动文件

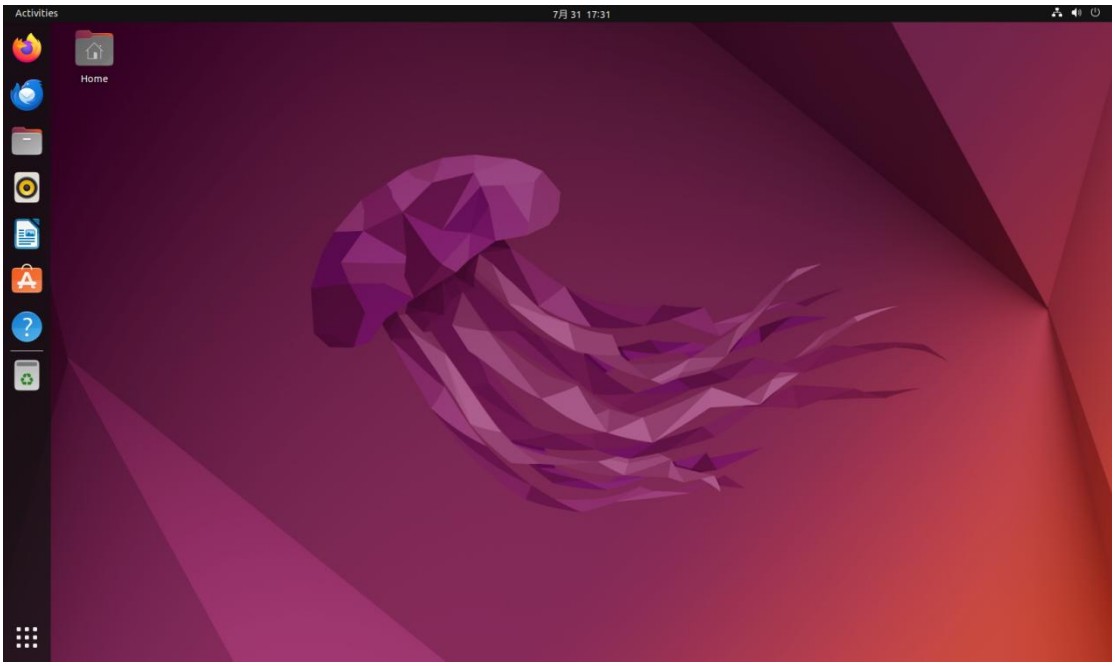


加载完成后点击开启此虚拟机，即可运行，进入系统的界面。



提供的开发环境的账户为 elf，密码为 elf，输入完成后按 Enter 键登录。





6.3. 搭建新的 ubuntu 开发环境

⚠ **注意：初学者不建议自己搭建系统，建议使用已有虚拟机环境，不需要搭建环境的此节可以跳过。**

本章节主要讲解了 ubuntu 系统的搭建过程。

6.3.1. ubuntu 系统搭建

6.3.1.1. 创建 Ubuntu 虚拟机

打开 VMware 软件，点击创建新的虚拟机。进入以下界面，选择自定义，点击“下一步”。



选择对应 VMware 版本的兼容性，版本可在帮助->关于 VMware Workstation 中查看，点击“下一步”。



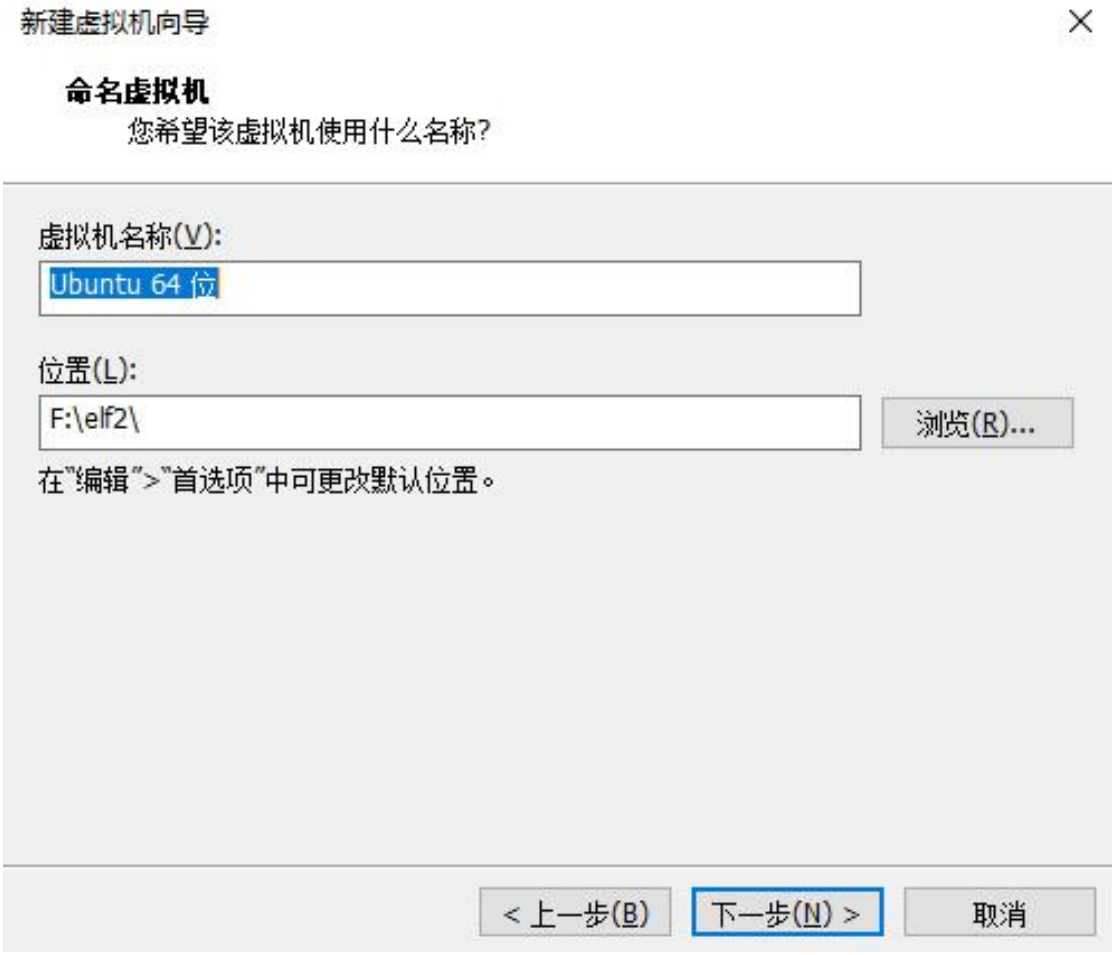
选择稍后安装操作系统，点击“下一步”。


新建虚拟机向导
安装客户机操作系统
虚拟机如同物理机，需要操作系统。您将如何安装客户机操作系统?
安装来源:
安装程序光盘(D):
无可用驱动器
安装程序光盘映像文件(iso)(M):
ubuntu-22.04.4-desktop-amd64.iso
浏览(R)...
稍后安装操作系统(S)。
创建的虚拟机将包含一个空白硬盘。
帮助
< 上一步(B)
下一步(N) >
取消

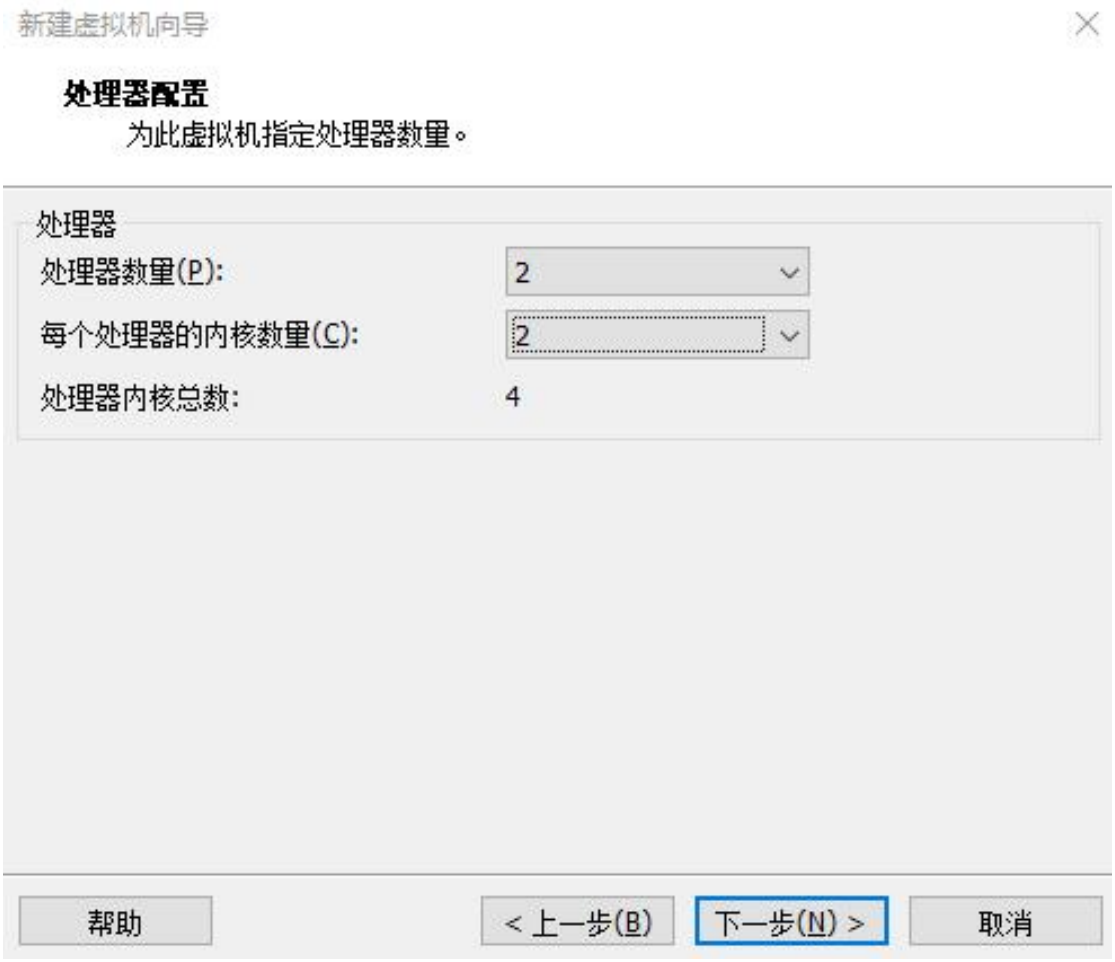
保持默认，点击“下一步”。

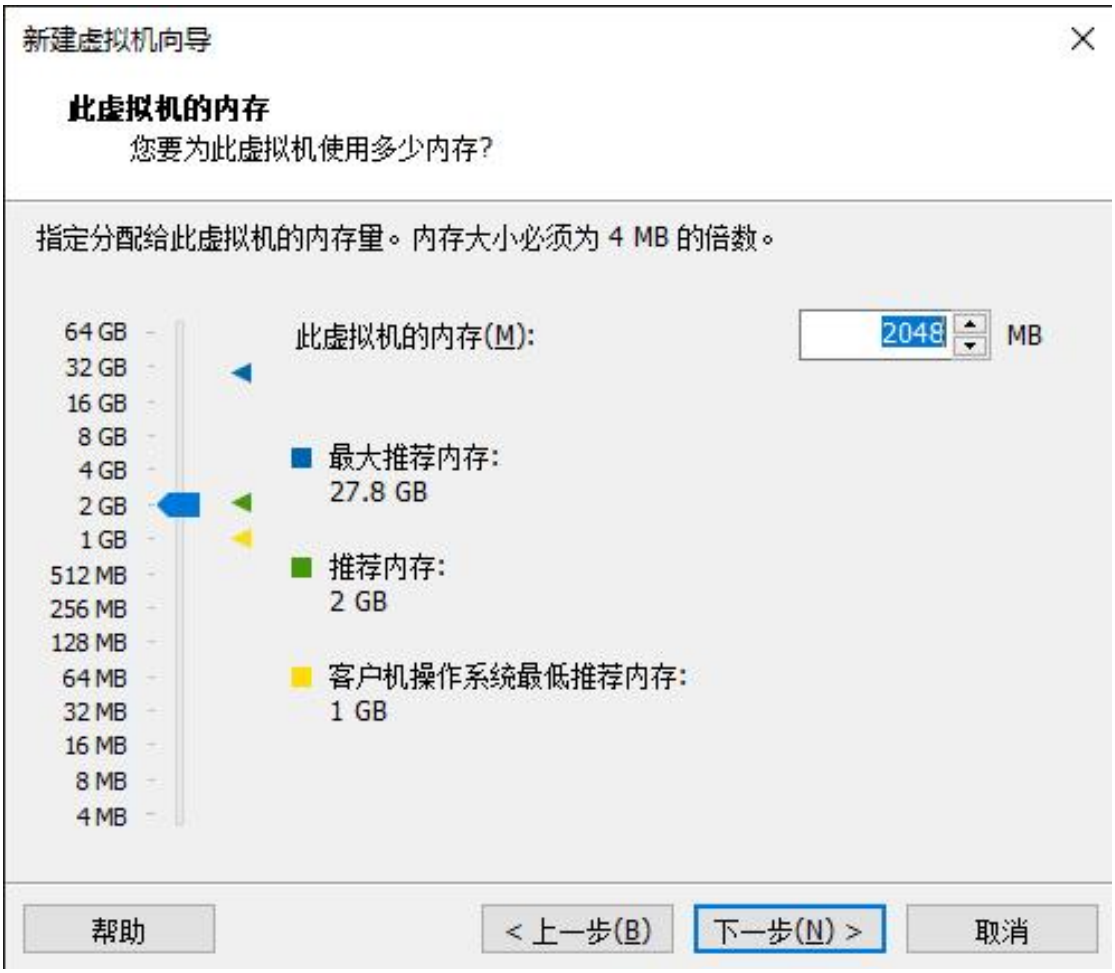


修改虚拟机名称及安装位置，点击“下一步”。

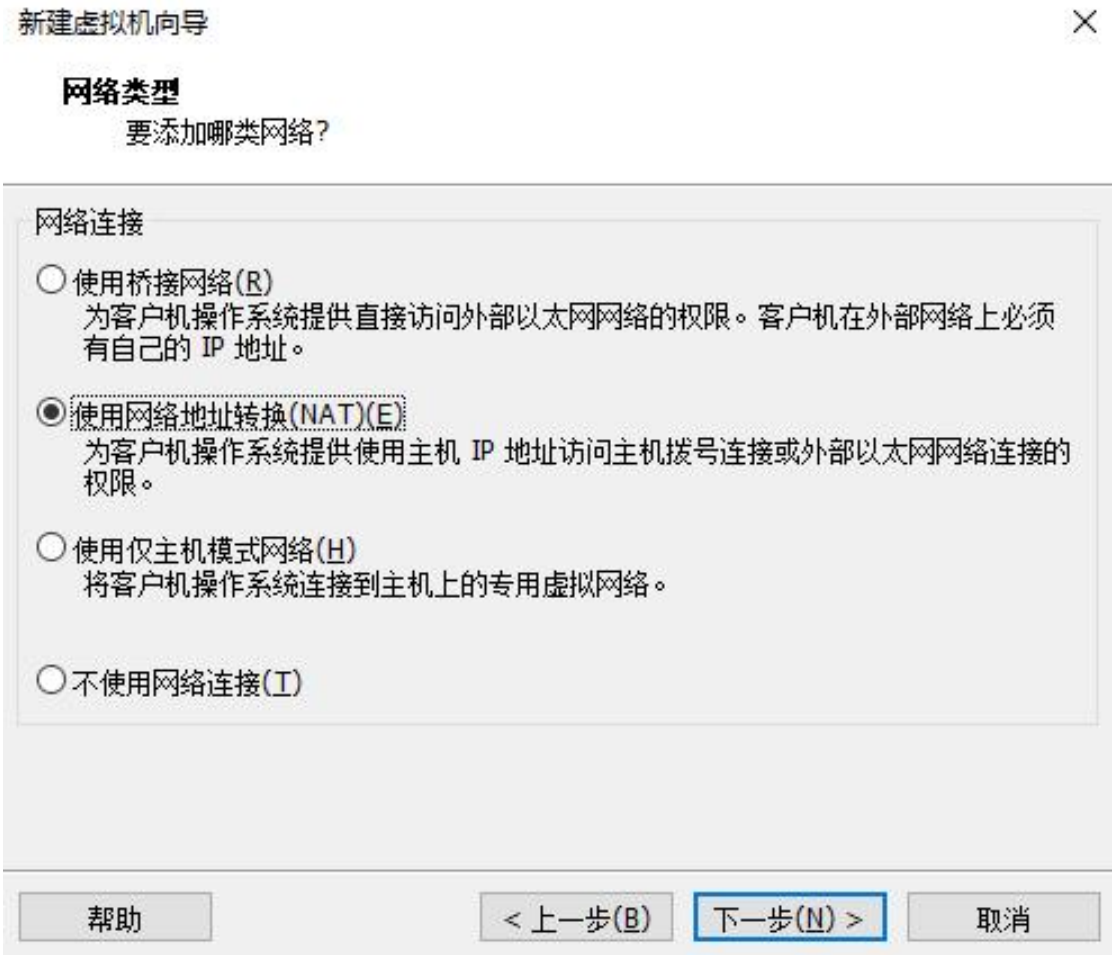


同样按照实际情况设置内存大小。


新建虚拟机向导
处理器配置
为此虚拟机指定处理器数量。
处理器
处理器数量(P):
2
每个处理器的内核数量(C):
2
处理器内核总数:
4
帮助
< 上一步(B)
下一步(N) >
取消



设置网络类型，默认为 NAT 模式，点击“下一步”。后面的步骤保持默认值，直到指定磁盘容量步骤。



IO 控制器类型这里默认选择 LSI 就可以。



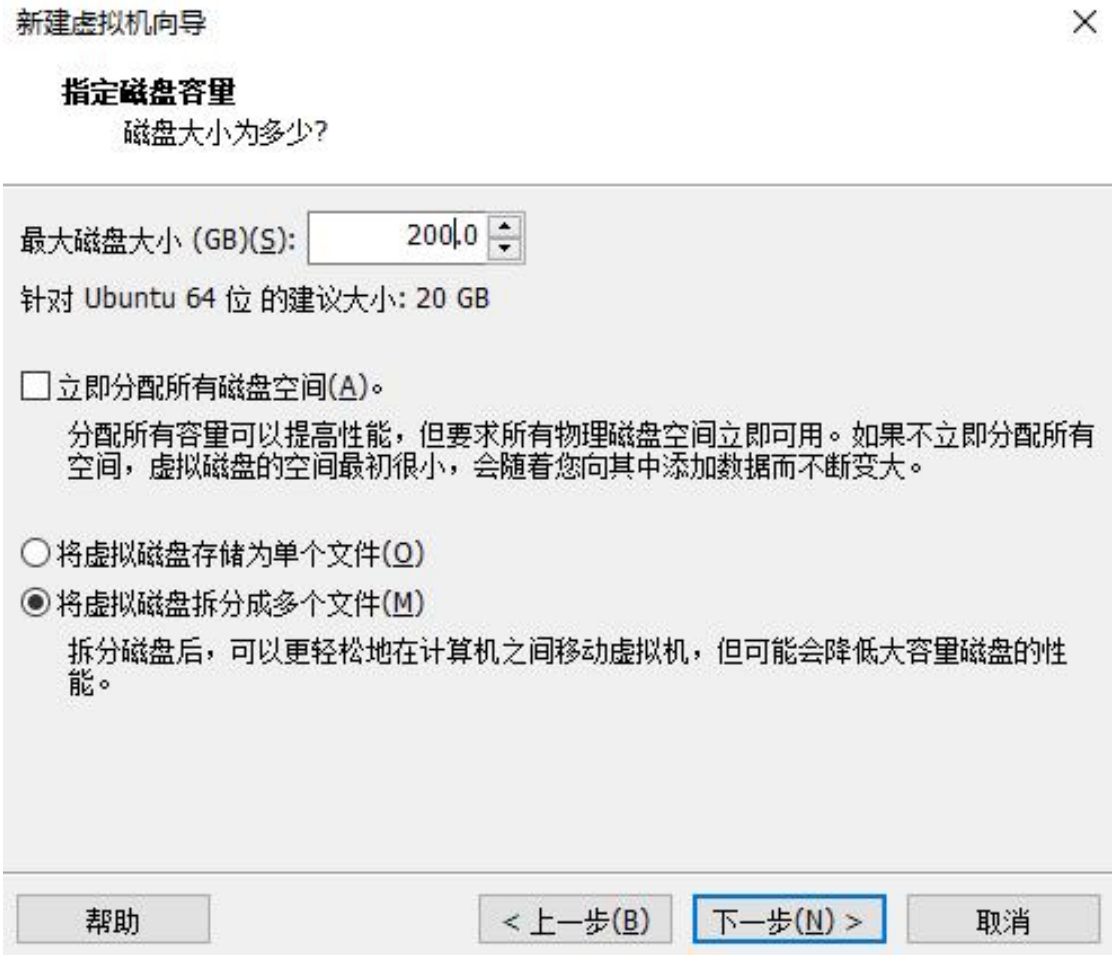
这里同样是默认选择 SCSI。



这里选择创建新的虚拟磁盘。



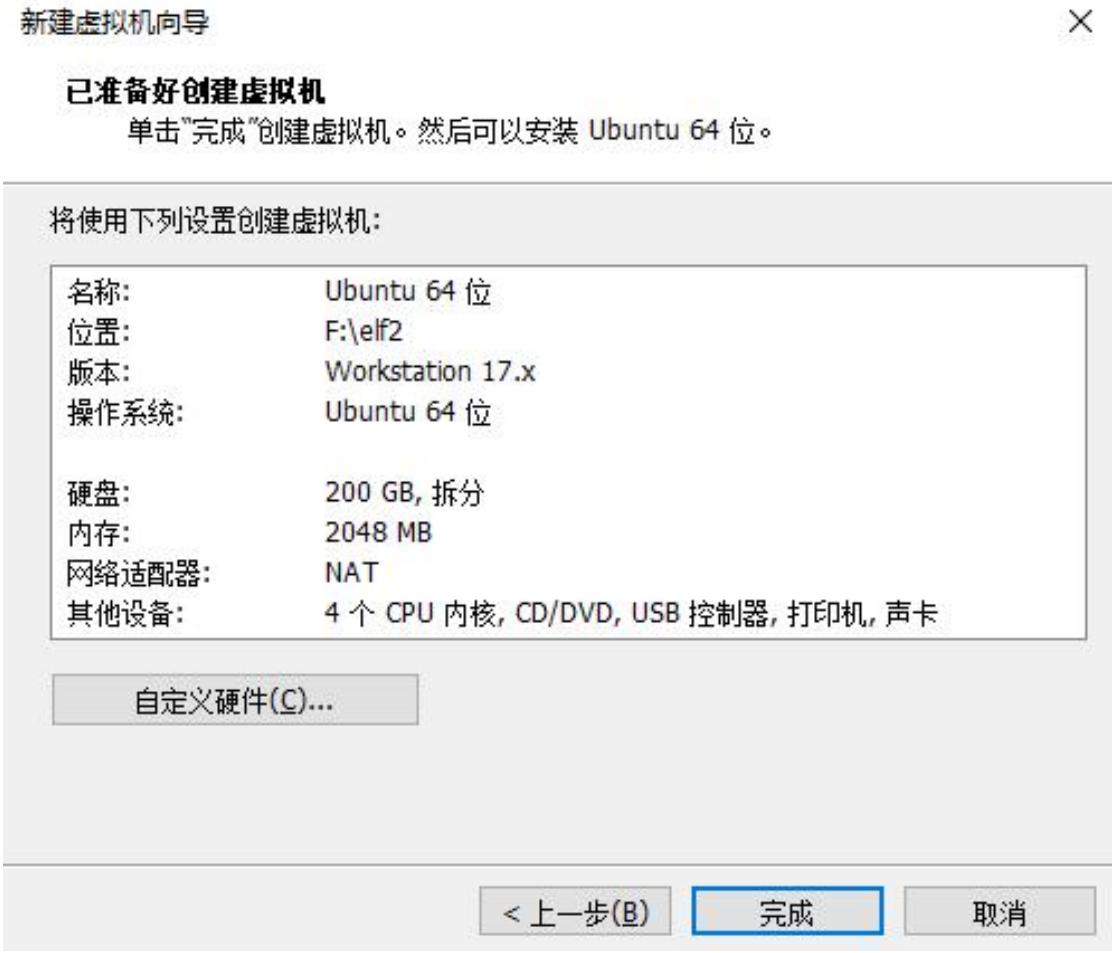
设置磁盘大小为 200G，并选择磁盘的存在形式，然后点击“下一步”完成。



指定磁盘文件，这里默认即可。


新建虚拟机向导
指定磁盘文件
您要在何处存储磁盘文件?
磁盘文件(F)
将使用多个磁盘文件创建一个 200 GB 虚拟磁盘。将根据此文件名自动命名这些磁盘文件。
Ubuntu 64 位.vmdk
浏览(R)...
帮助
< 上一步(B)
下一步(N) >
取消

默认点击“完成”即可。


新建虚拟机向导
已准备好创建虚拟机
单击"完成"创建虚拟机。然后可以安装 Ubuntu 64 位。
将使用下列设置创建虚拟机:
名称: Ubuntu 64 位
位置: F:\elf2
版本: Workstation 17.x
操作系统: Ubuntu 64 位
硬盘: 200 GB, 拆分
内存: 2048 MB
网络适配器: NAT
其他设备: 4 个 CPU 内核, CD/DVD, USB 控制器, 打印机, 声卡
自定义硬件(C)...
< 上一步(B)
完成
取消

至此，虚拟机创建完成。



下一小节中我们介绍 Ubuntu 系统在虚拟机中的安装，其在真机中的安装方法与虚拟机类似。这里我们介绍在虚拟机中安装 Ubuntu 系统的方法。

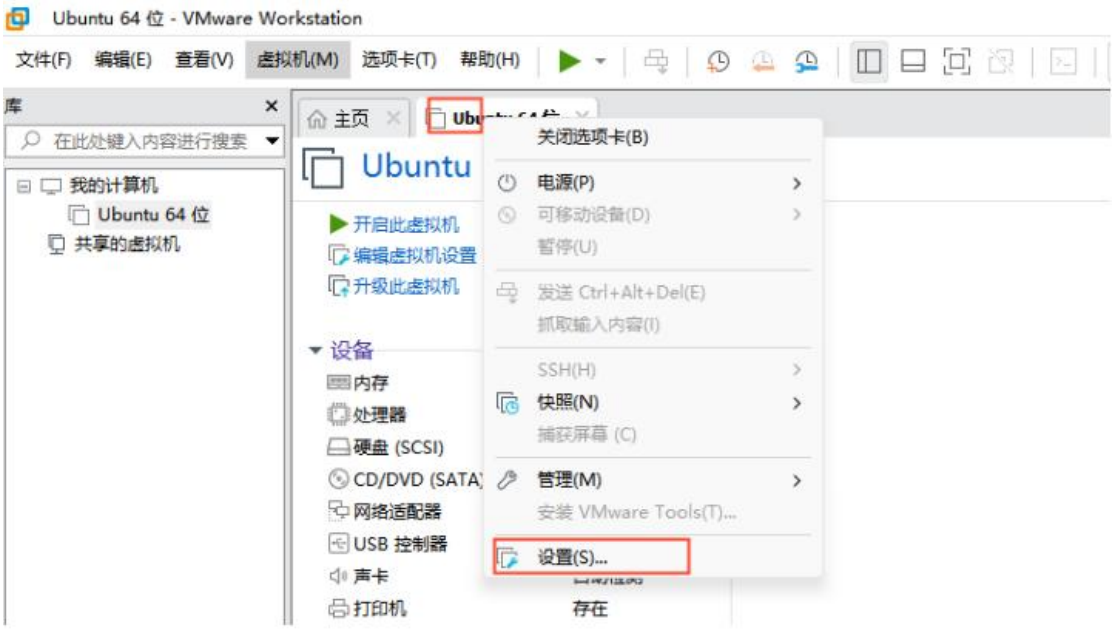
6.3.1.2. 系统安装

我们选择安装的 Ubuntu 版本是 22.04，首先去 Ubuntu 官网获取 Ubuntu22.04 64 位镜像，下载地址为：https://releases.ubuntu.com/jammy/下载“ubuntu-22.04.4-desktop-amd64.iso”这个版本。

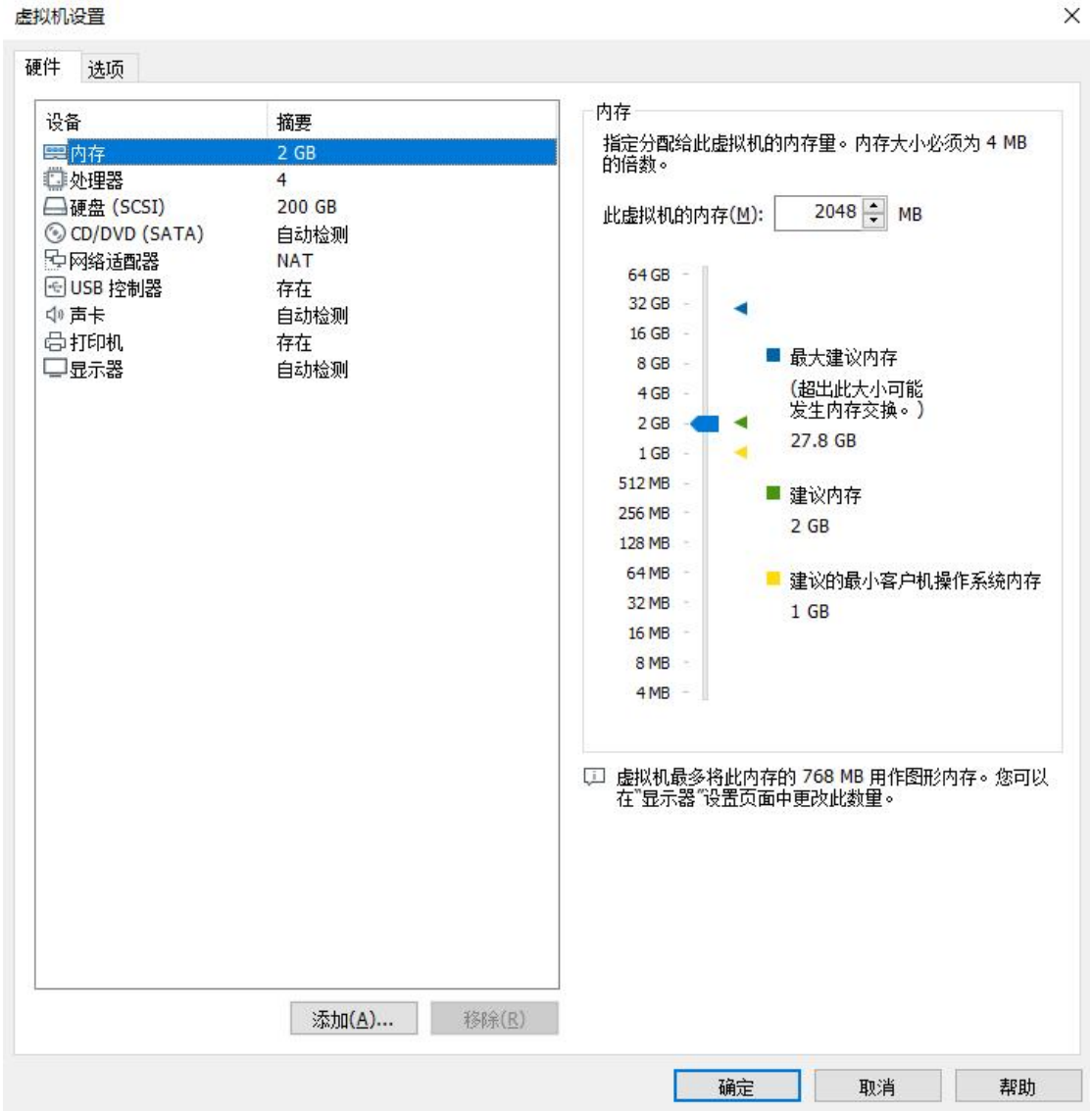
我们已经提供了下载好的镜像，路径：ELF 2 开发板资料包\05-常用工具\02-环境搭建工具\ubuntu22.04.4 镜像



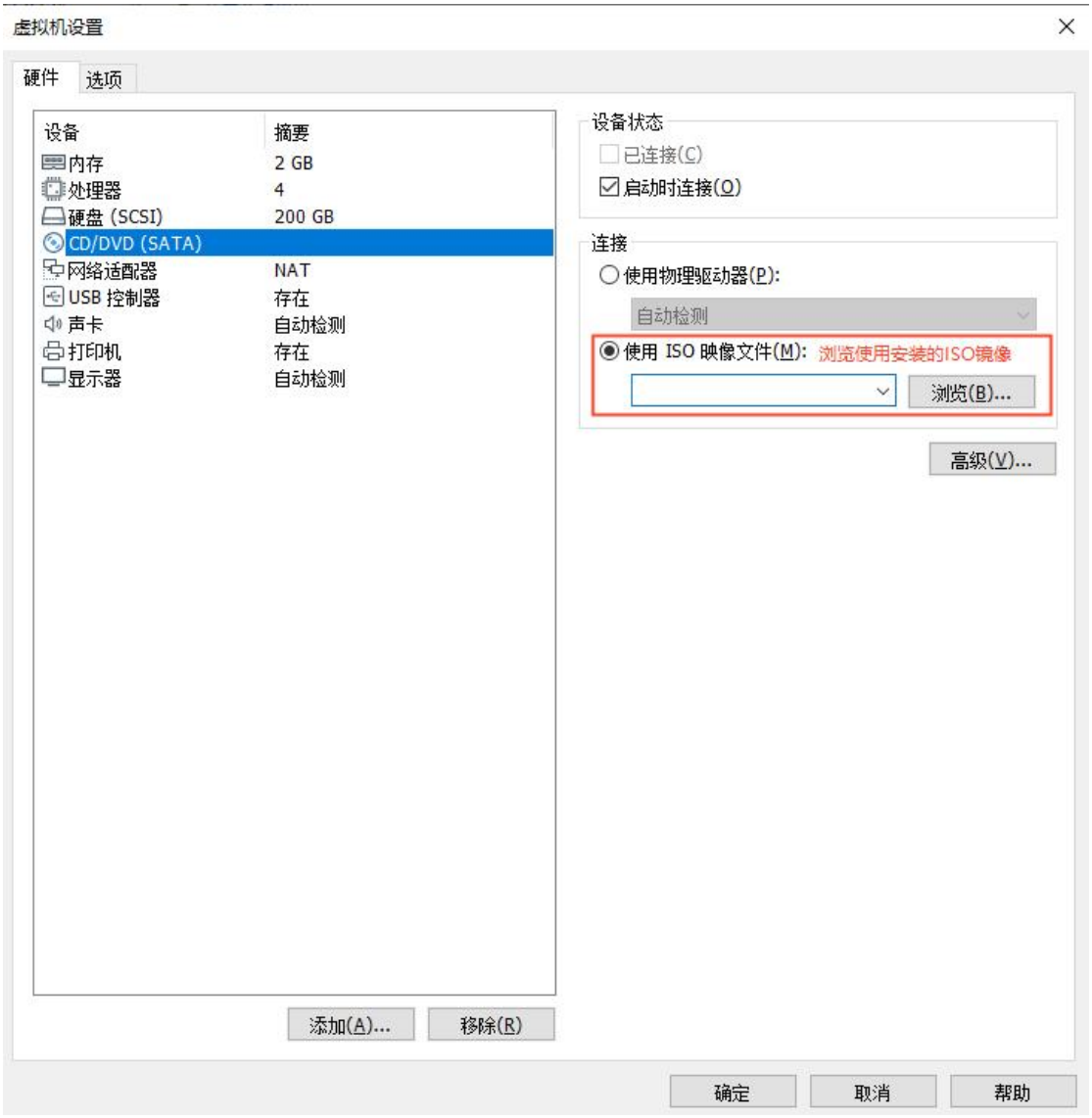
右击刚创建完成的 Ubuntu64 位，选择设置。


Ubuntu 64 位 - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
库
在此处键入内容进行搜索
我的计算机
Ubuntu 64 位
共享的虚拟机
主页
Ubuntu
开启此虚拟机
编辑虚拟机设置
升级此虚拟机
设备
内存
处理器
硬盘 (SCSI)
CD/DVD (SATA)
网络适配器
USB 控制器
声卡
打印机
关闭选项卡(B)
电源(P)
可移动设备(D)
暂停(U)
发送 Ctrl+Alt+Del(E)
抓取输入内容(I)
SSH(H)
快照(N)
捕获屏幕 (C)
管理(M)
安装 VMware Tools(T)...
设置(S)...
存在

弹出“虚拟机设置菜单”。


虚拟机设置
硬件
选项
设备
摘要
内存 2 GB
处理器 4
硬盘 (SCSI) 200 GB
CD/DVD (SATA) 自动检测
网络适配器 NAT
USB 控制器 存在
声卡 自动检测
打印机 存在
显示器 自动检测
添加(A)...
移除(R)
内存
指定分配给此虚拟机的内存里。内存大小必须为 4 MB 的倍数。
此虚拟机的内存(M): 2048 MB
64 GB
32 GB
16 GB
8 GB
4 GB
2 GB
1 GB
512 MB
256 MB
128 MB
64 MB
32 MB
16 MB
8 MB
4 MB
最大建议内存
(超出此大小可能发生内存交换。)
27.8 GB
建议内存
2 GB
建议的最小客户机操作系统内存
1 GB
虚拟机最多将此内存的 768 MB 用作图形内存。您可以在“显示器”设置页面中更改此数量。
确定
取消
帮助

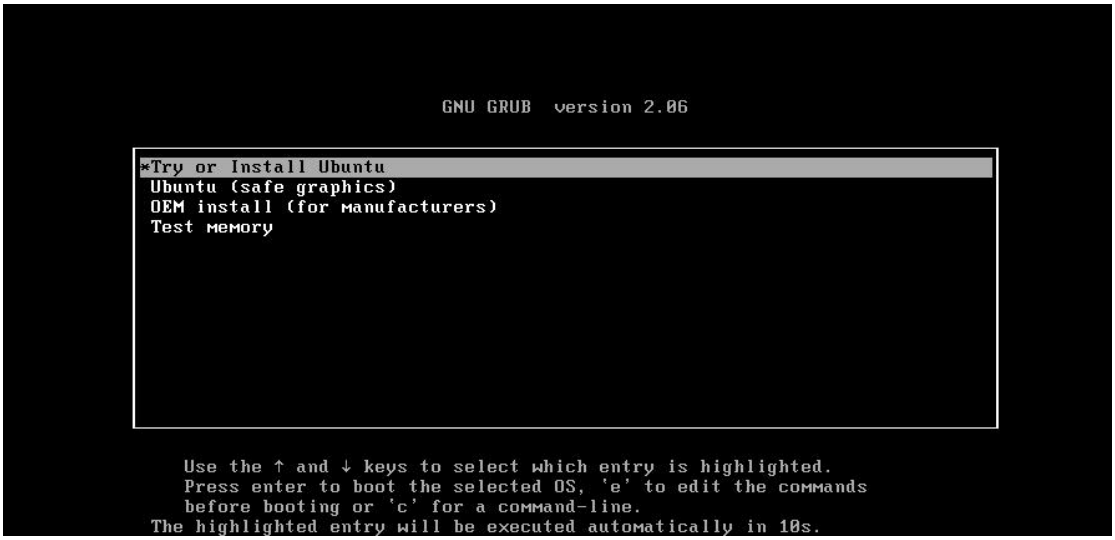
点击 CD/DVD（SATA），选择使用 ISO 映像文件，浏览选择前面下载的 Ubuntu 镜像，然后确定。



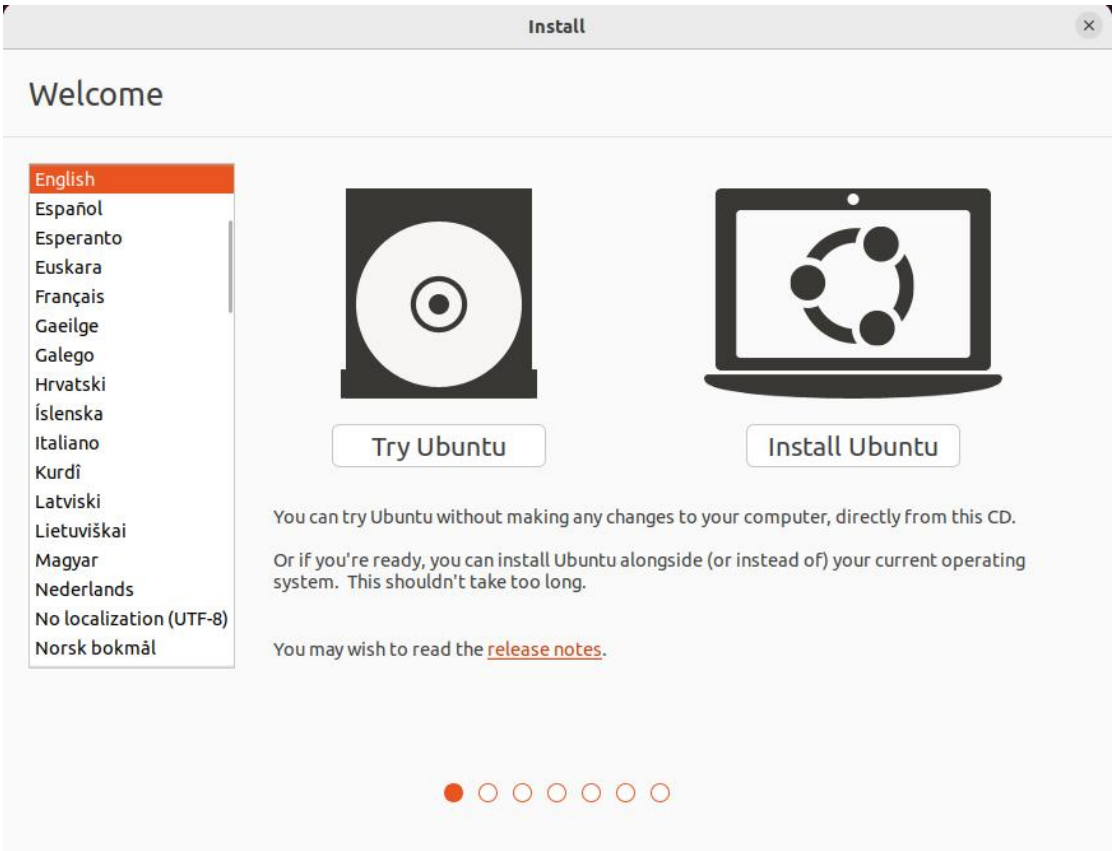
设置好镜像后，保证网络可用，开启虚拟机，进行 Ubuntu 镜像的安装。

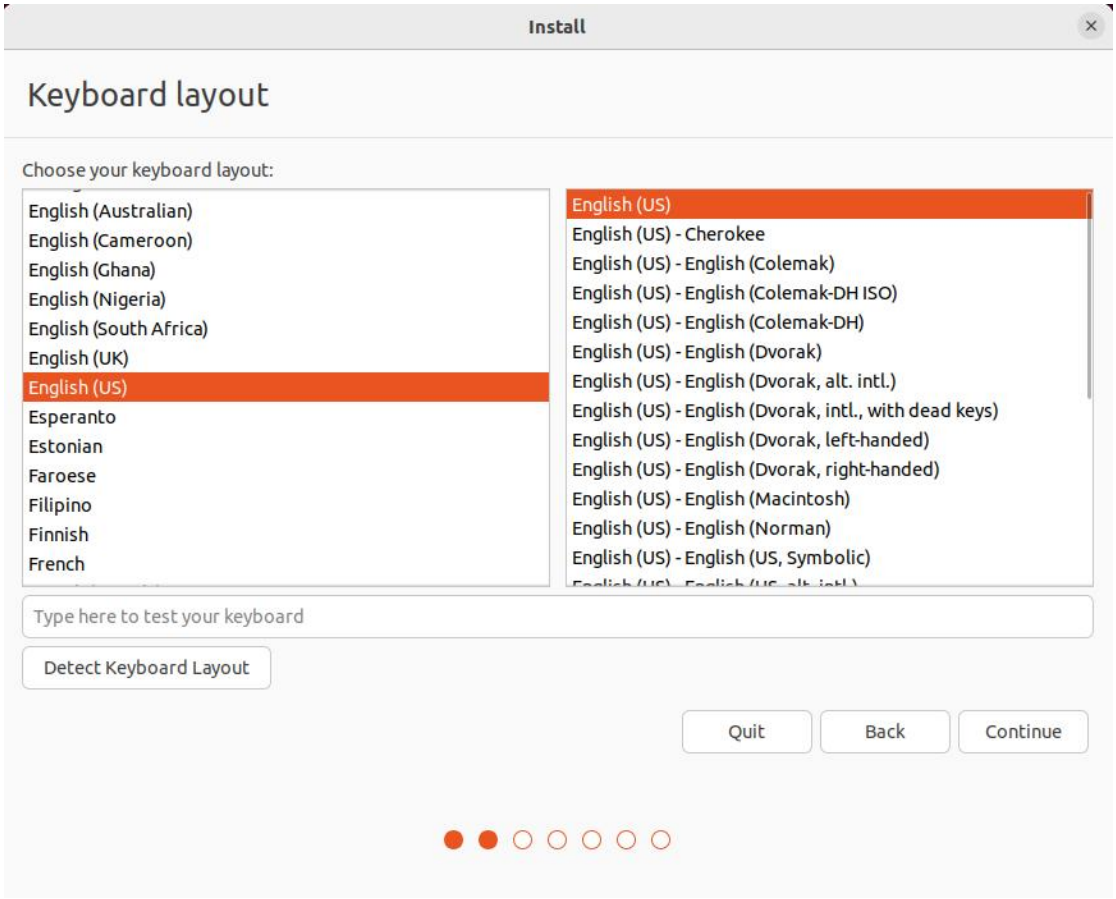


开启虚拟机后，等待出现安装界面如下，选择第一项，回车。

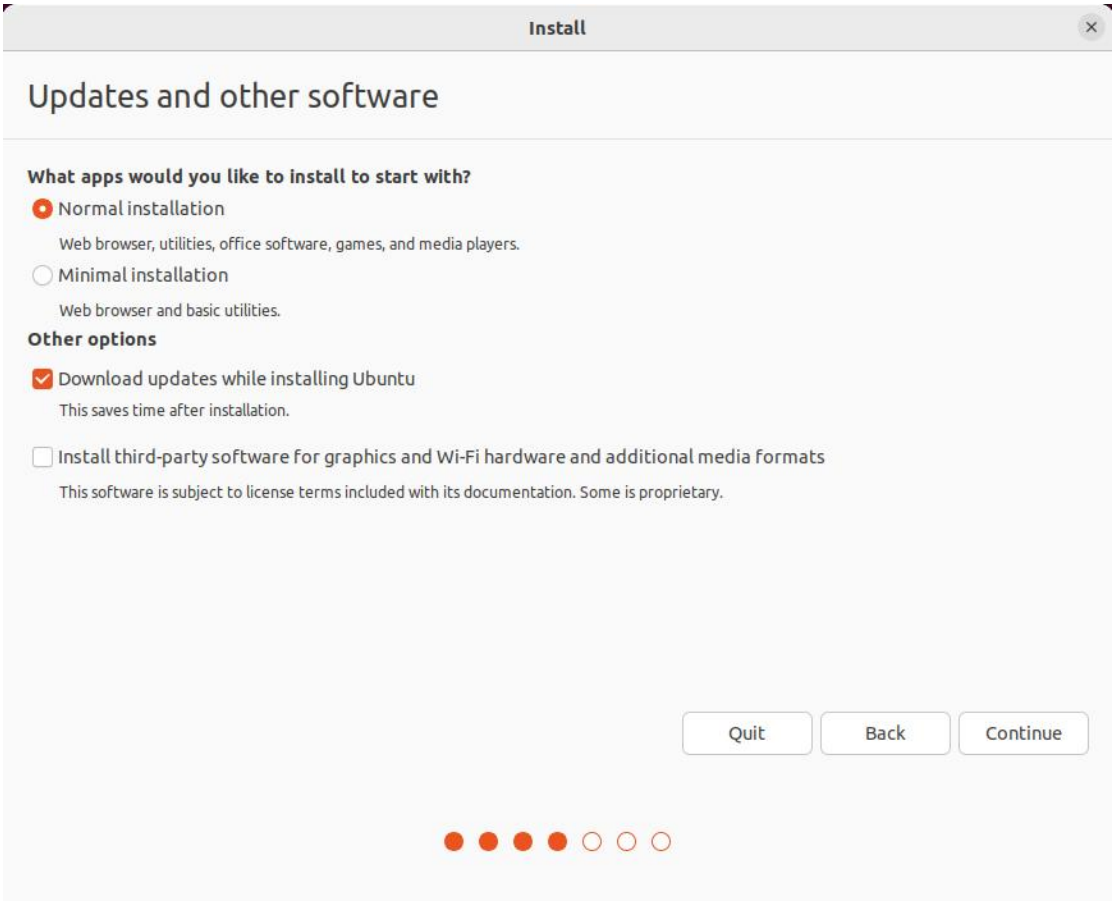


如图左侧选择语言后，点击“Install Ubuntu”后弹出选择语言界面。Ubuntu 默认语言是英文的，当然，也可以选择中文，默认选择的语言在后期也是可以重新设置的，选择完成后“Continue”。

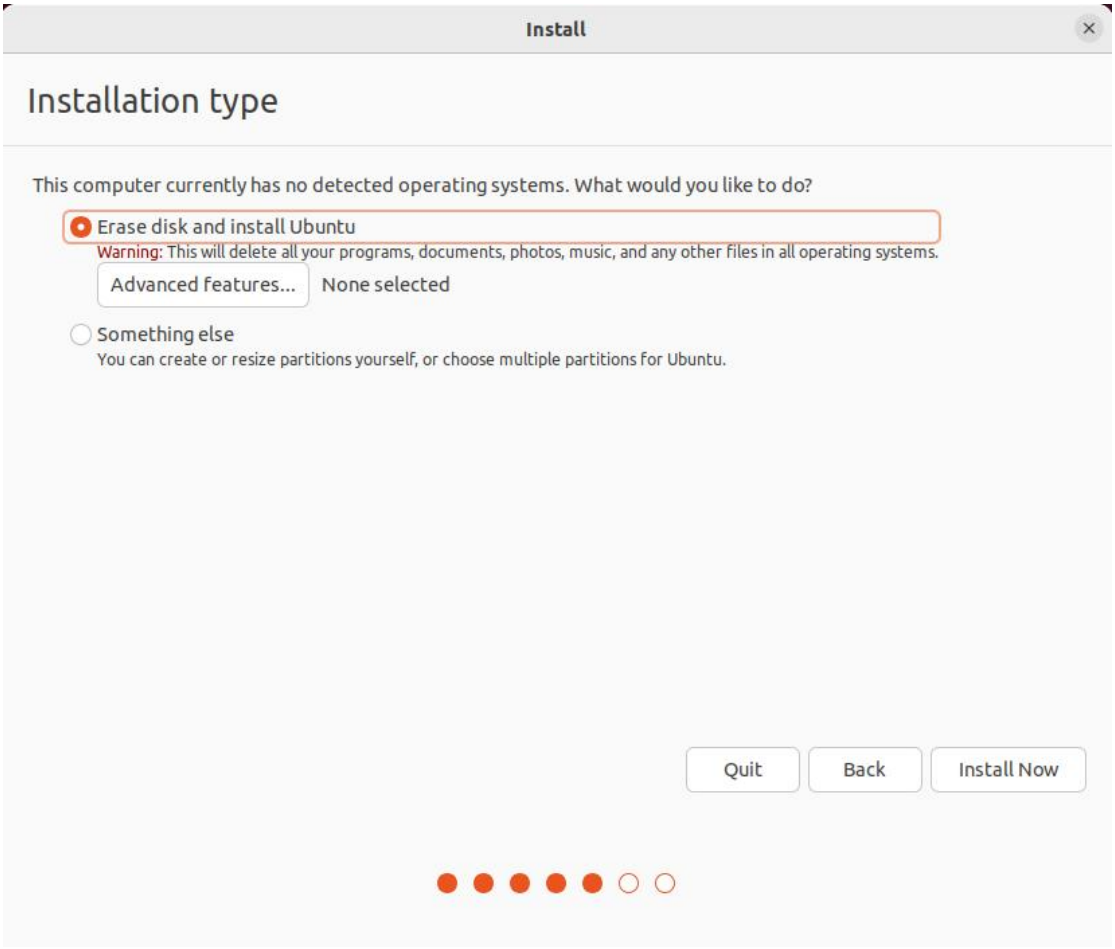




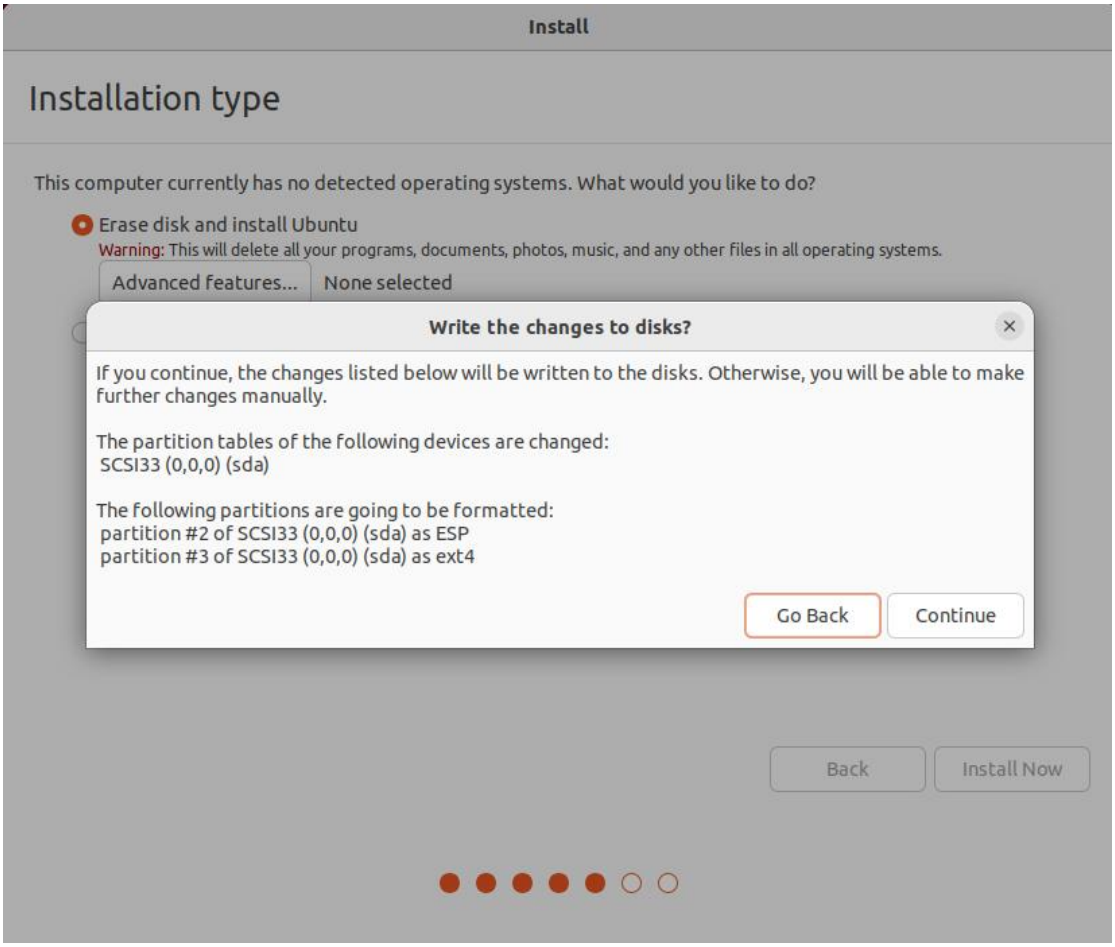
接下来，默认选择“Continue”继续安装，安装过程会很慢，然后点击“Continue”。



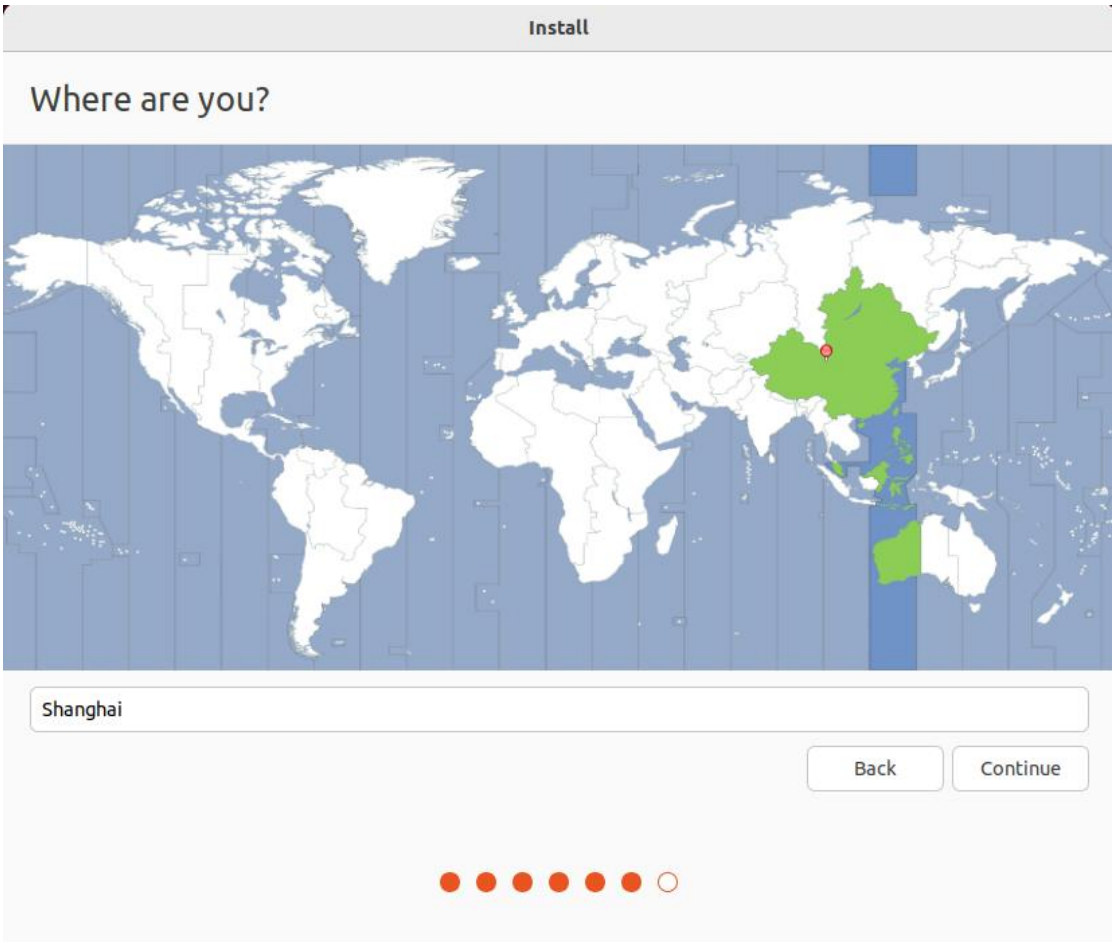
默认，点击 Install Now。



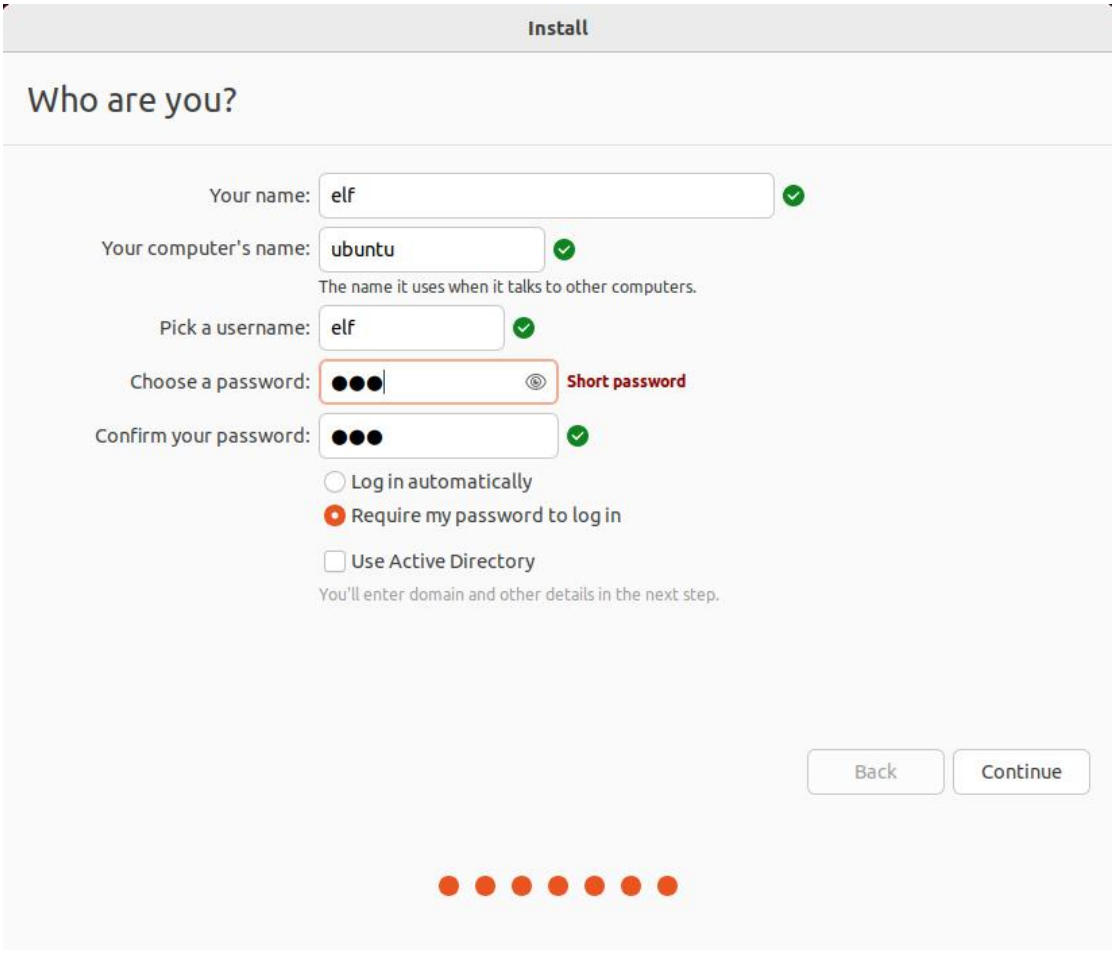
会弹出下图，点击“continue”即可。



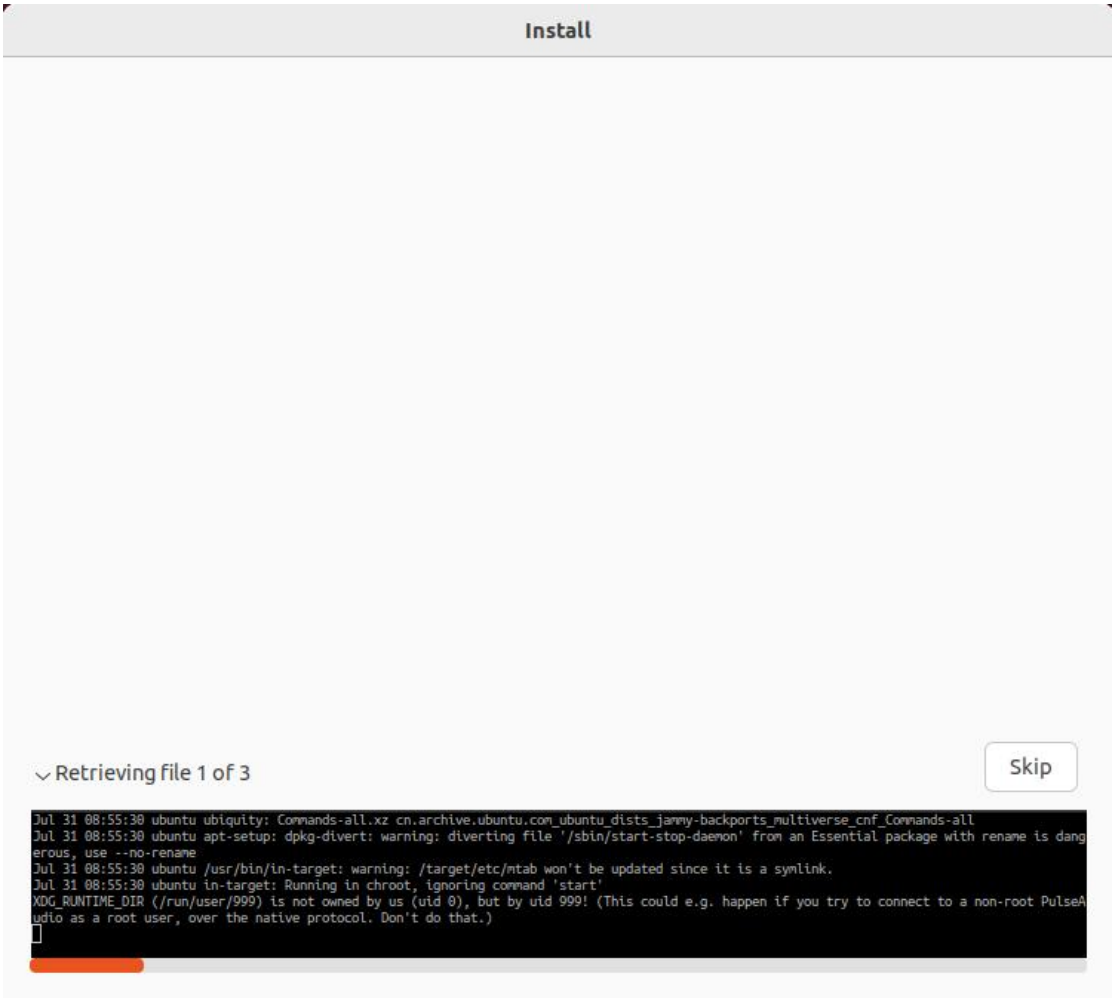
接下来选择时区，这里点击上海时区或输入 Shanghai 即可（不同时区根据实际情况选择即可），点击“继续”。



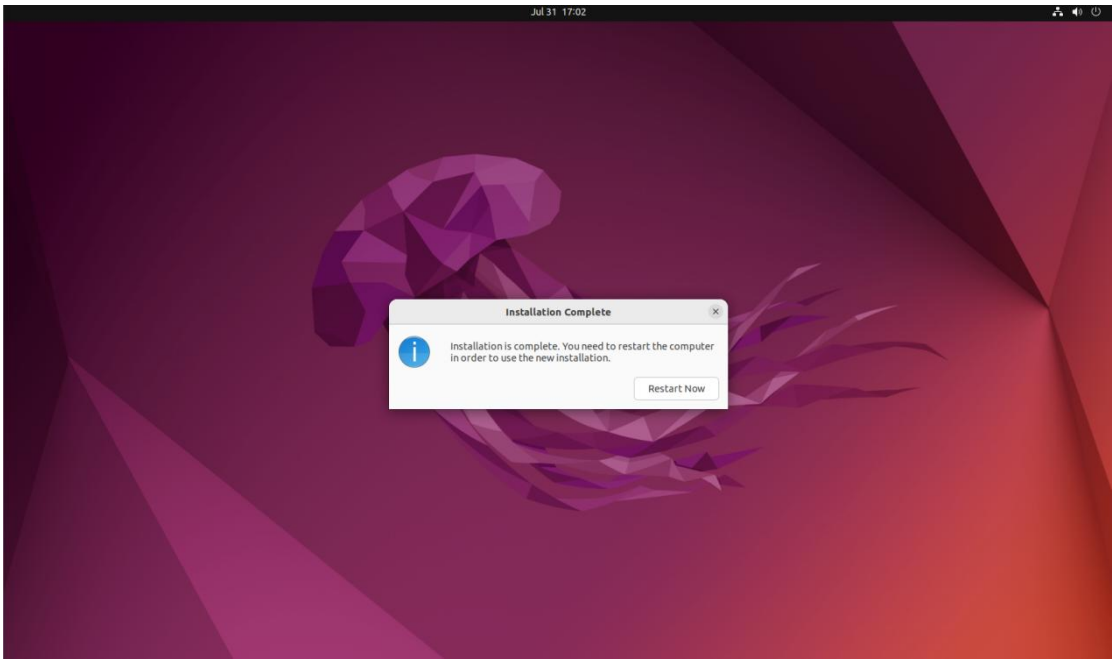
最后设置用户名和密码，点击“continue”就会自动安装。

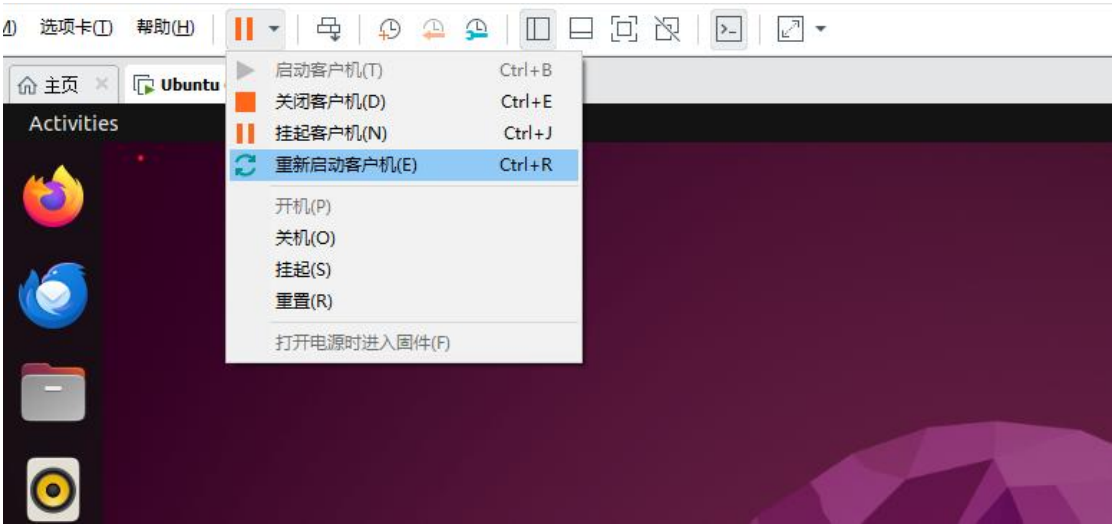


安装过程如下图，网络不好可以 Skip 跳过，不影响安装。

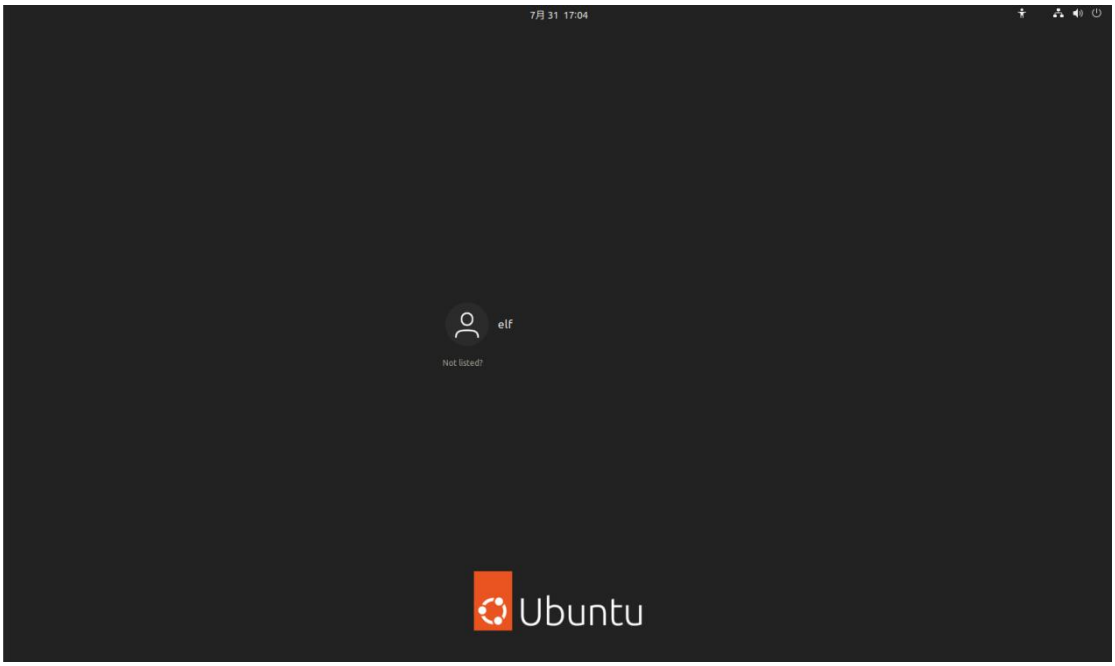


安装完成后显示如下图，点击“Restart Now”重启（或者点击“重新启动客户机”）：

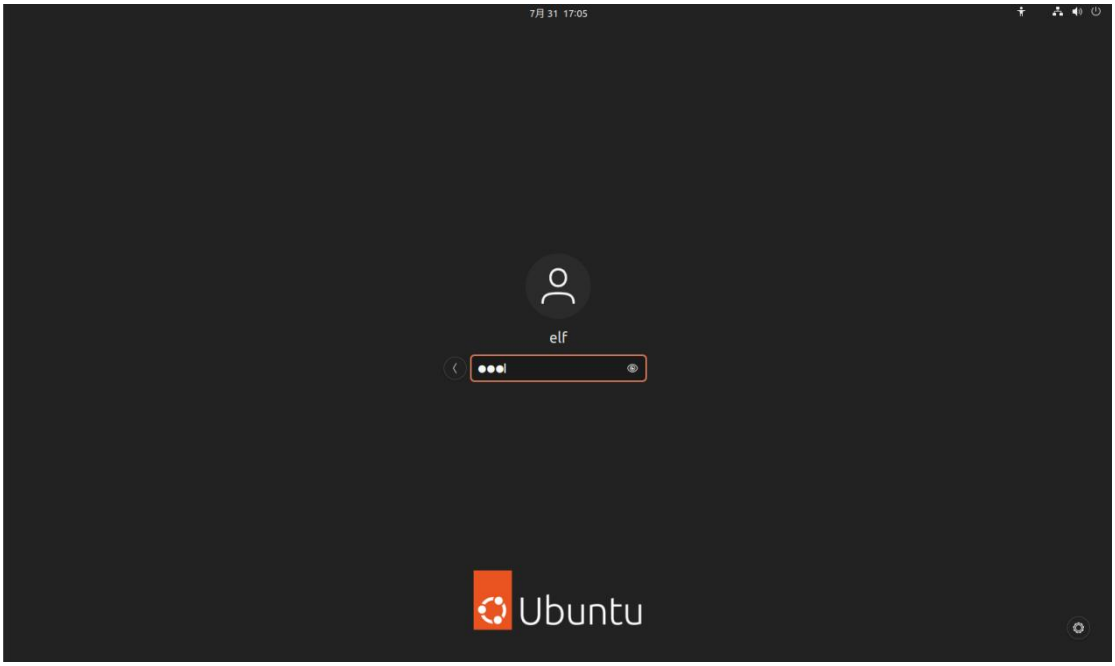




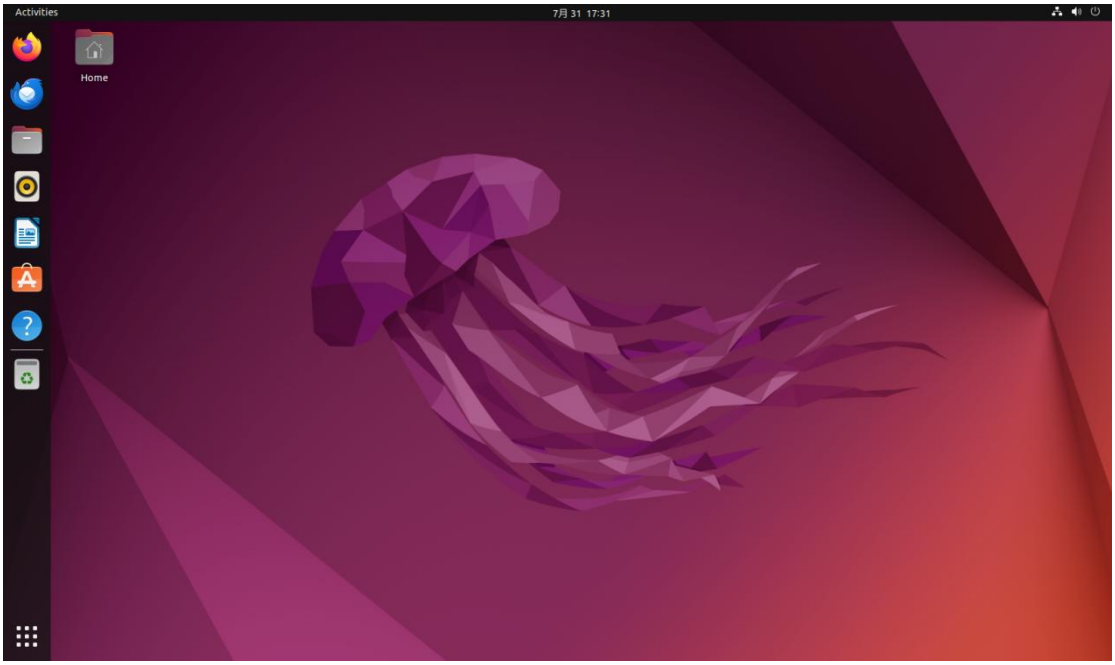
重启完成需要使用用户名和密码登录，登录后系统界面如下图：



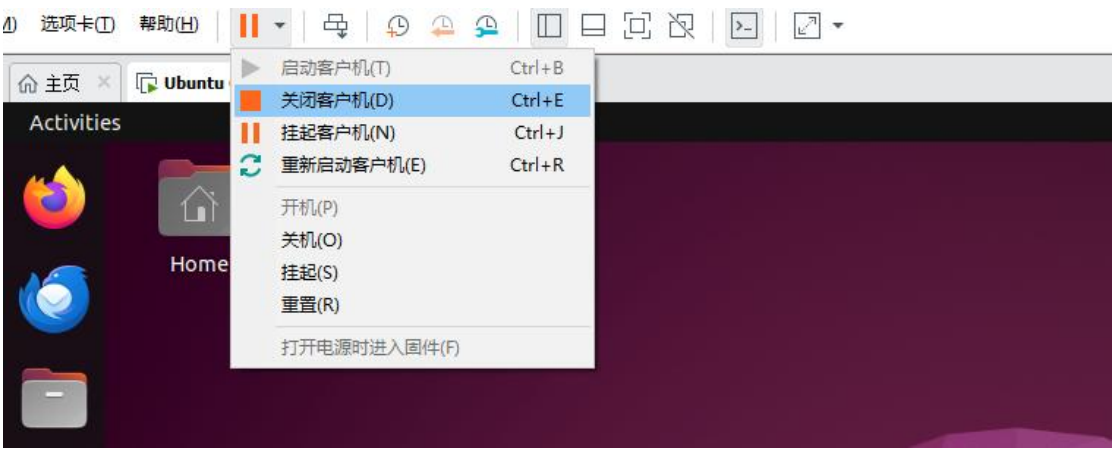
点击账户“elf”，输入密码，回车，完成登录。



进入 ubuntu 系统桌面。

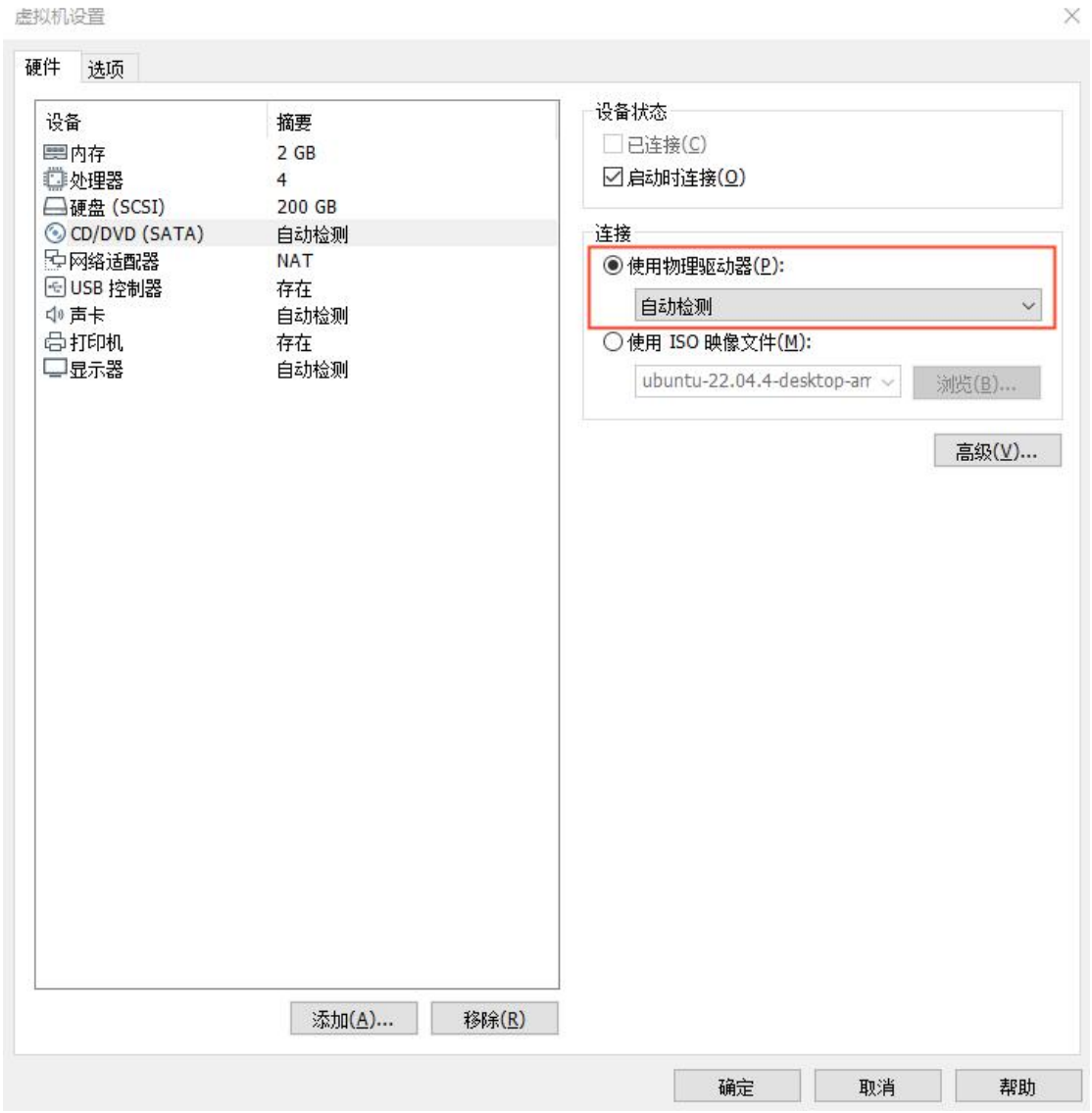


以上，ubuntu 系统安装完成。按照下图配置，点击确定，关闭客户机。



点击“编辑虚拟机设置”，如下图重新设置“使用物理驱动器”。





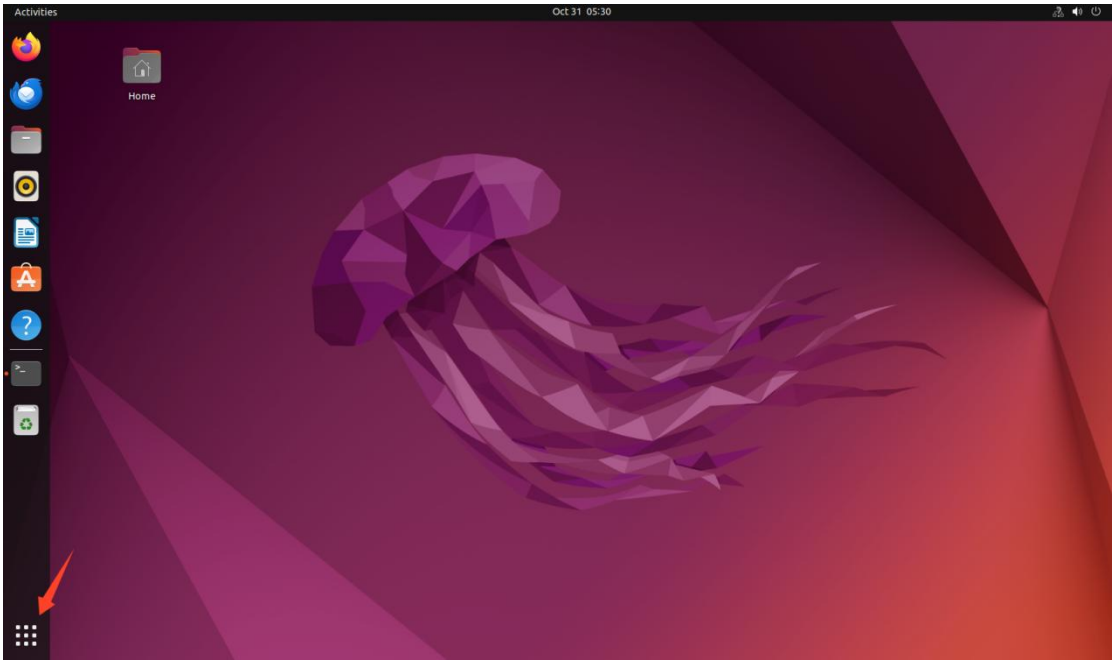
然后重新打开虚拟机，看能否正常启动 Ubuntu。

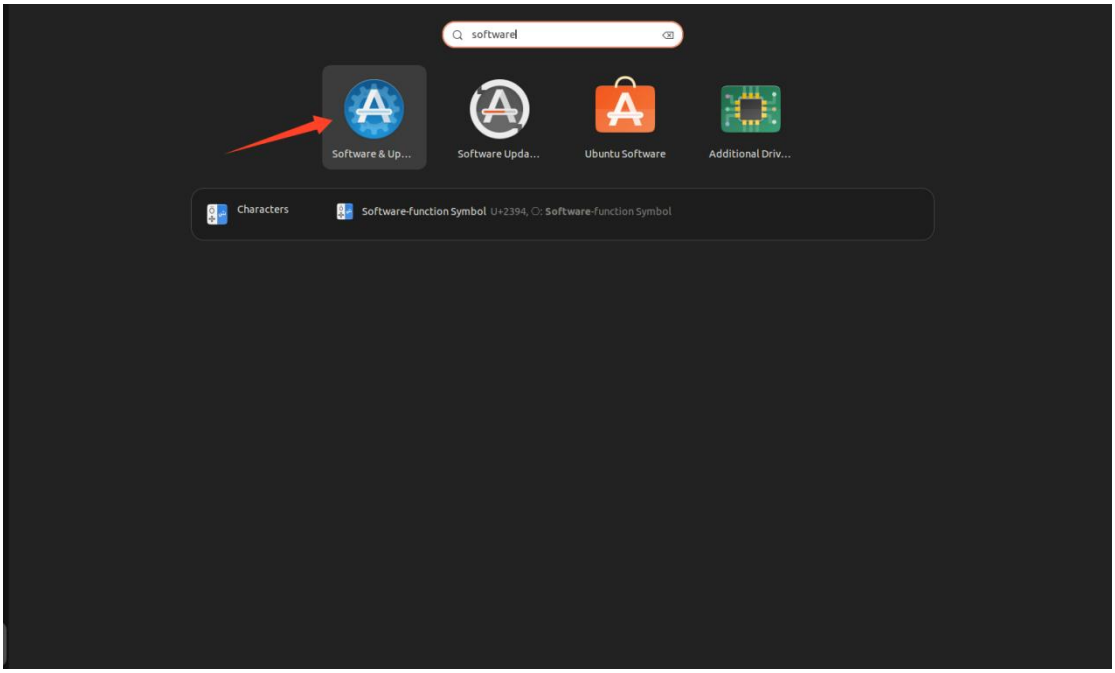
6.3.1.3. VMware Tools 安装

接下来安装 VMware Tools，如果不安装该工具，在 Windows 主机和虚拟机之间无法使用复制粘贴、文件拖拽。可以从命令行手动安装 VMware Tools，对于 Linux 的较高发行版，使用集成的 open-vm-tools 版本。由于目前 Ubuntu22.04 是比较高的发行版本，因此通过集成的 open-vm-tools 版本进行安装 VMware tools。

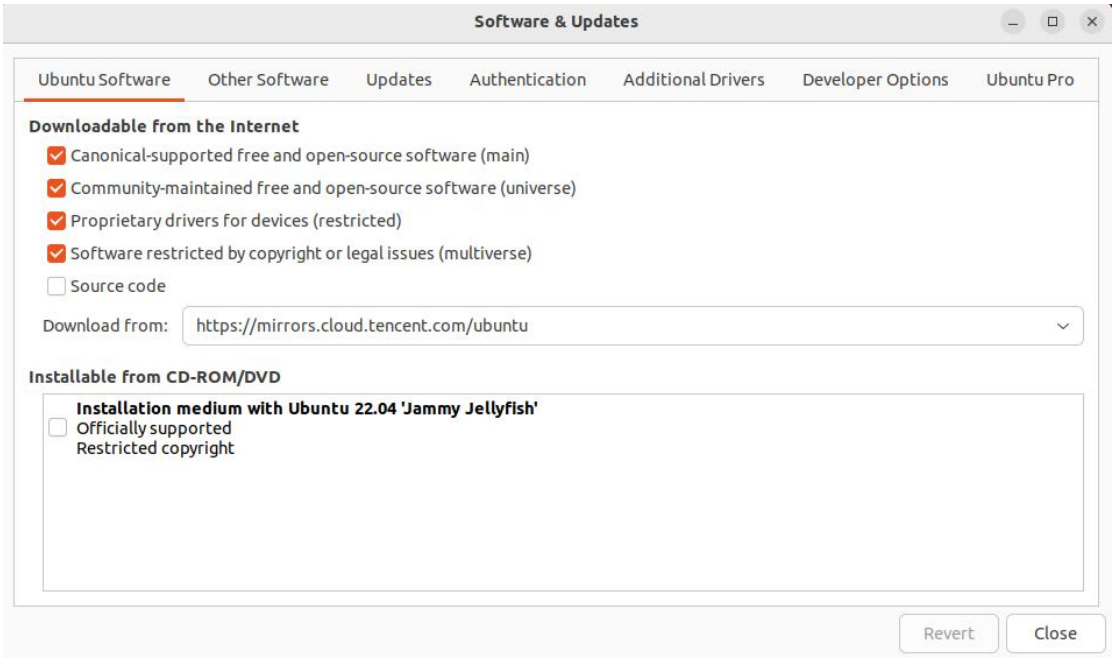
为了成功安装 VMware Tools，首先需要进行换源操作。这样可以确保软件包的下载速度更快，并且能够获取到最新的依赖包。

1.点击左下角的图标按钮，弹出新界面上方的文本框中搜索“software”，即可找到“软件和更新”应用程序，单击图标打开。

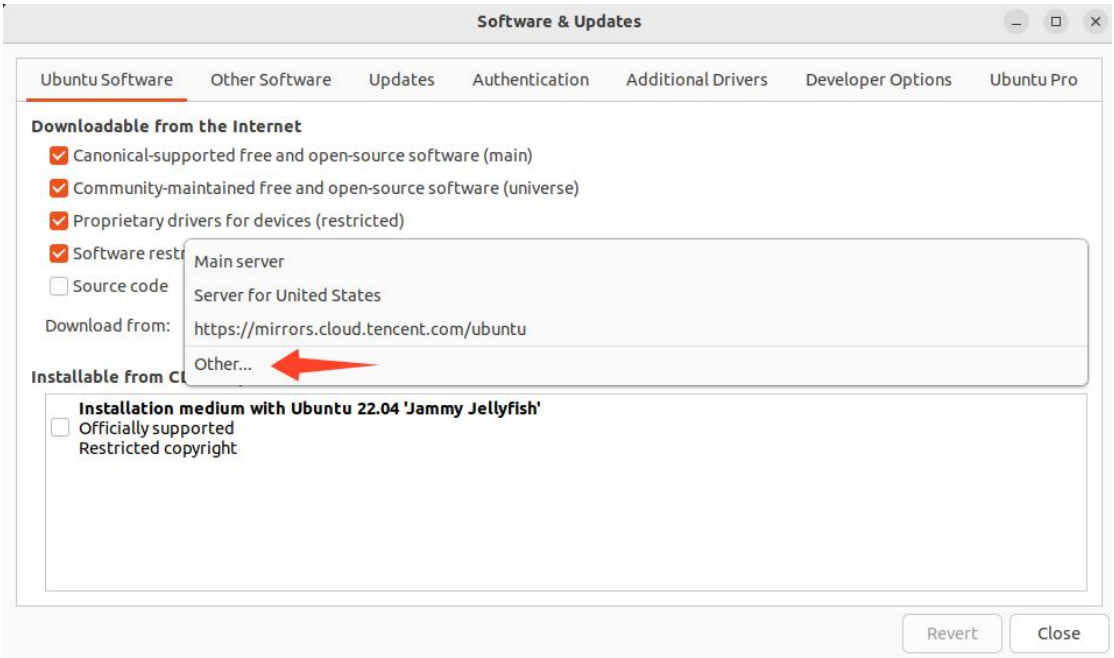




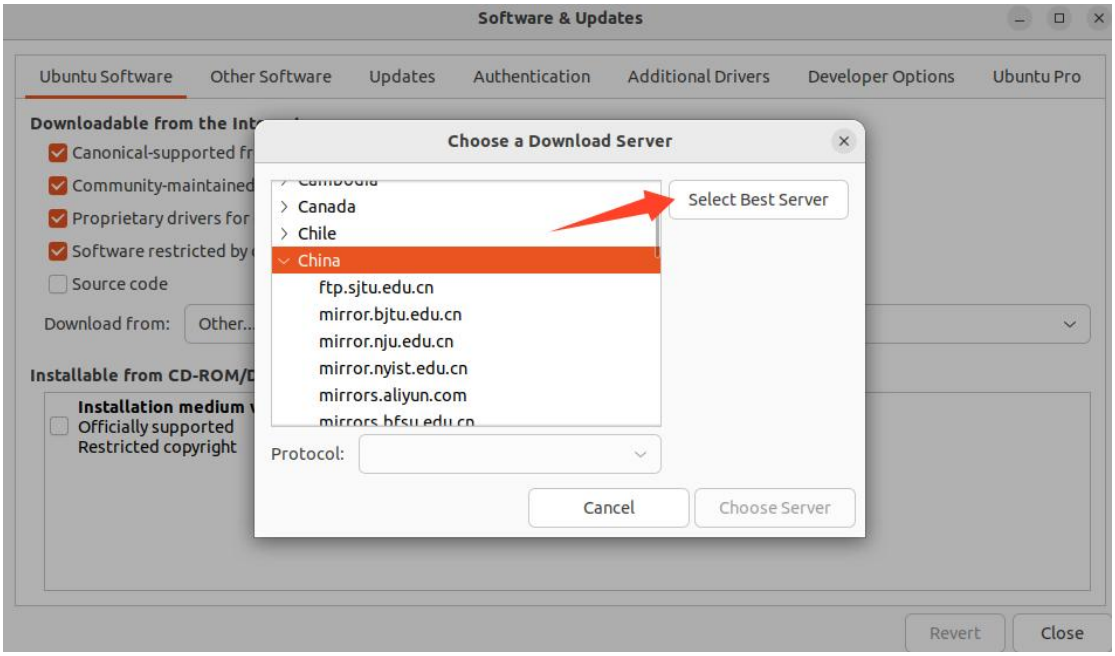
2.选中第一个选项卡“Ubuntu Software”，可以看到下方的服务器。



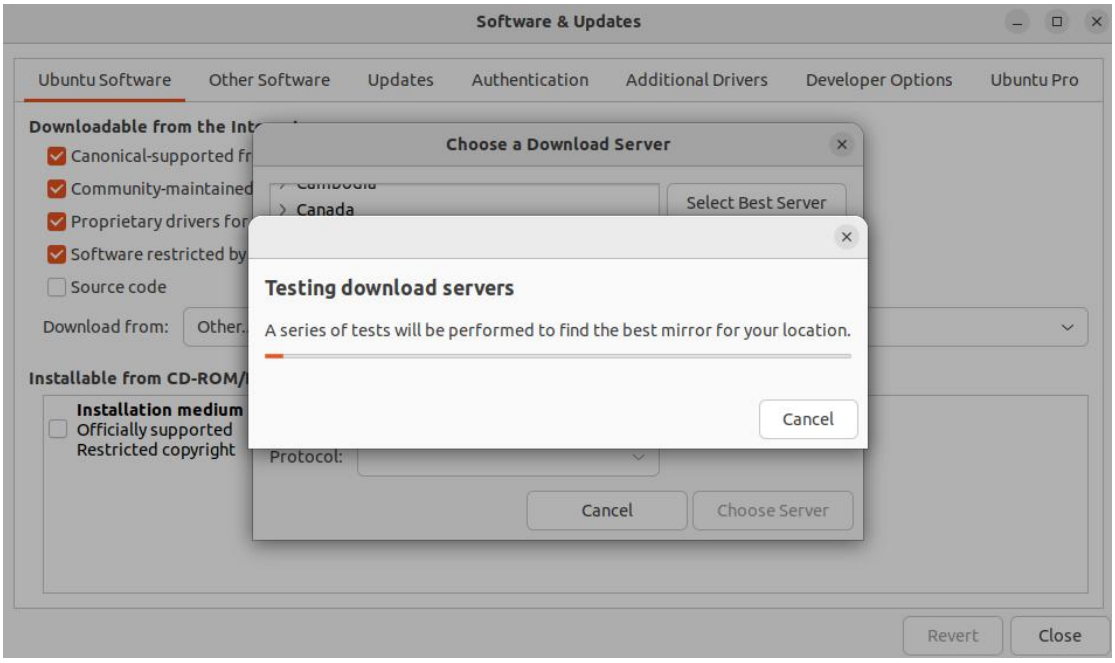
3.点击右侧的小黑倒三角，选中“other...”，将会弹出“选择下载服务器”对话框。



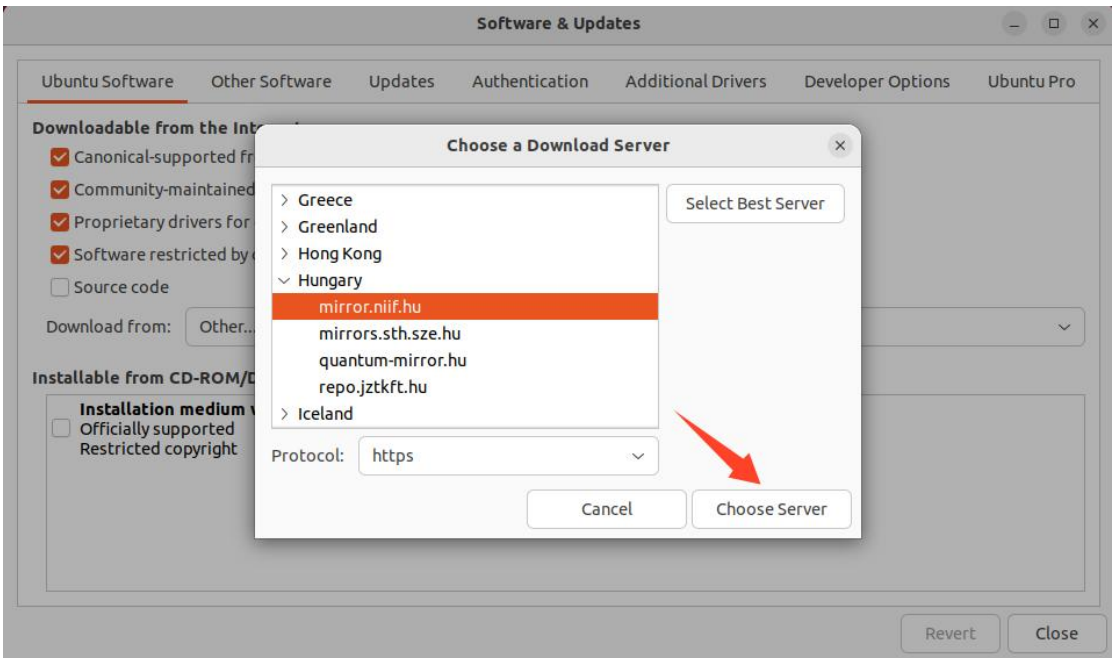
4.选择 China，然后点击右侧的“选择最佳服务器”按钮，系统会自动找到一个最好的服务器。



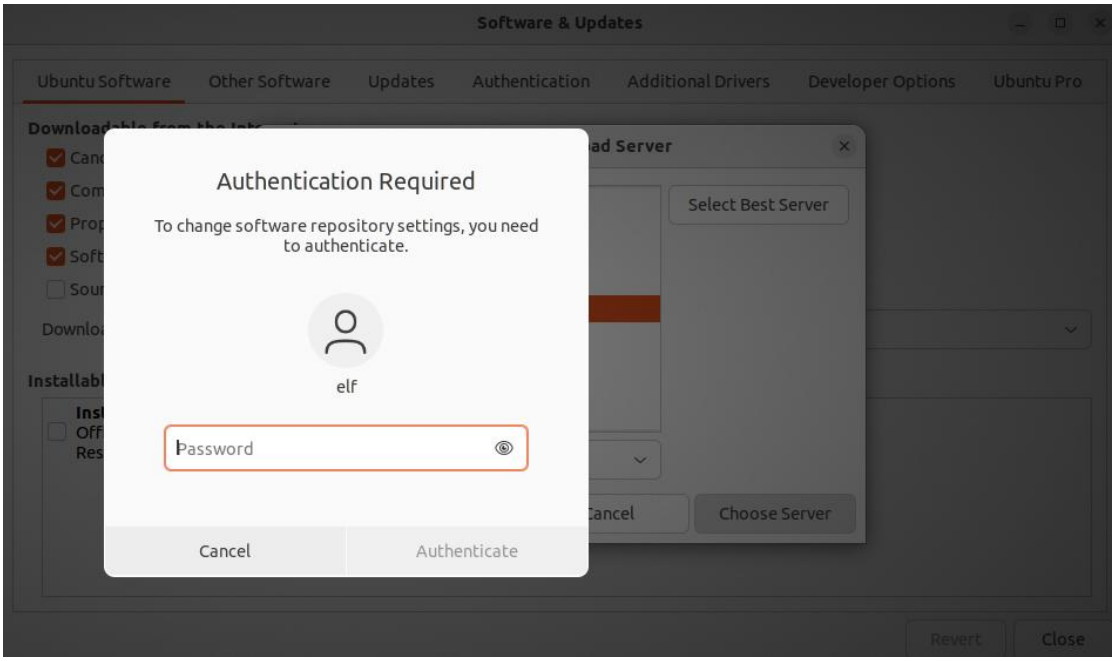
5.系统会自动做一系列测试，测试各个服务器在当前网络状况下的性能表现，从而选出最佳的服务器。



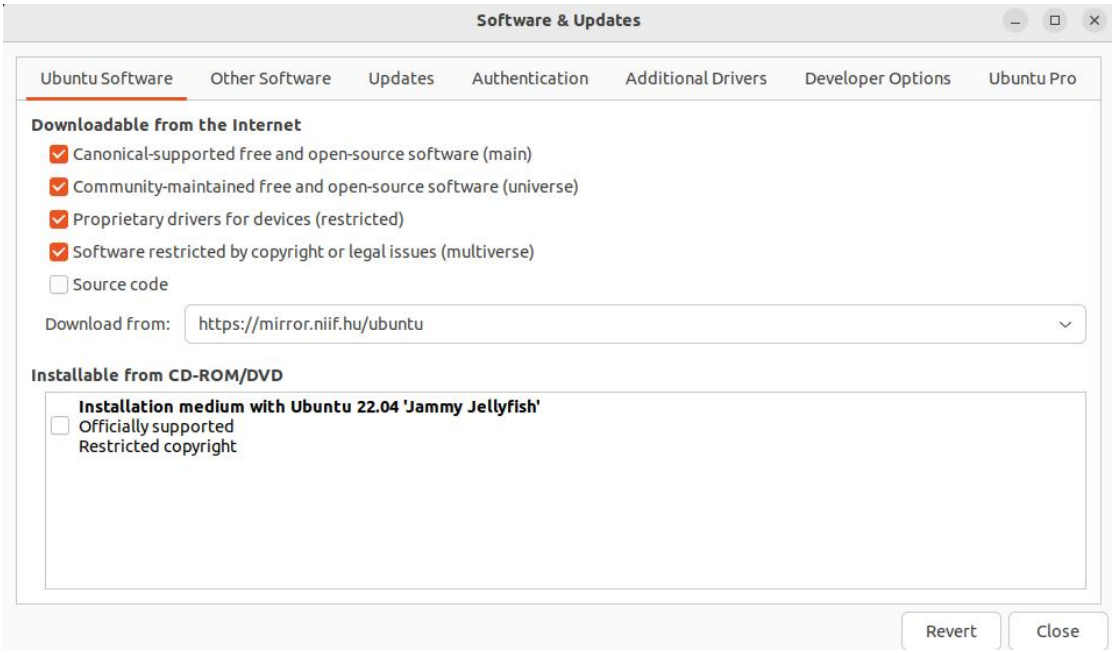
6.系统最终自动重新选择了北京外国语大学镜像站点，这里我们也可以手动选择自己喜欢的站点，确认无误后，点击“选择服务器”按钮。



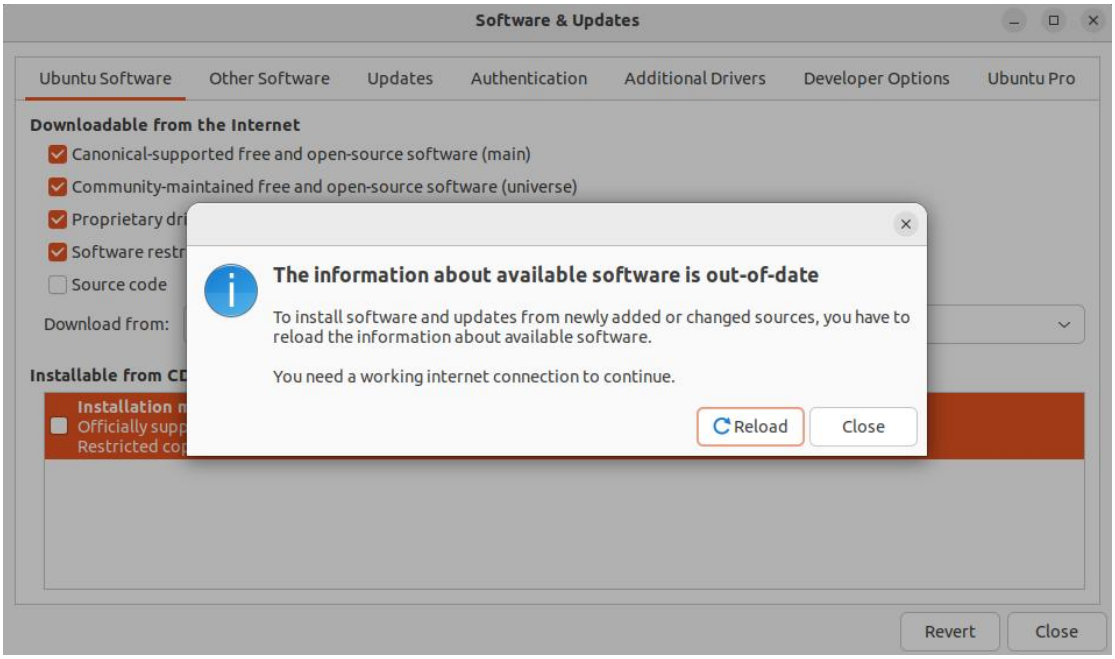
7.由于更换软件源属于敏感操作，会弹出“Authentication Required”对话框，输入密码，点击“Authentication”按钮方可进行下一步。

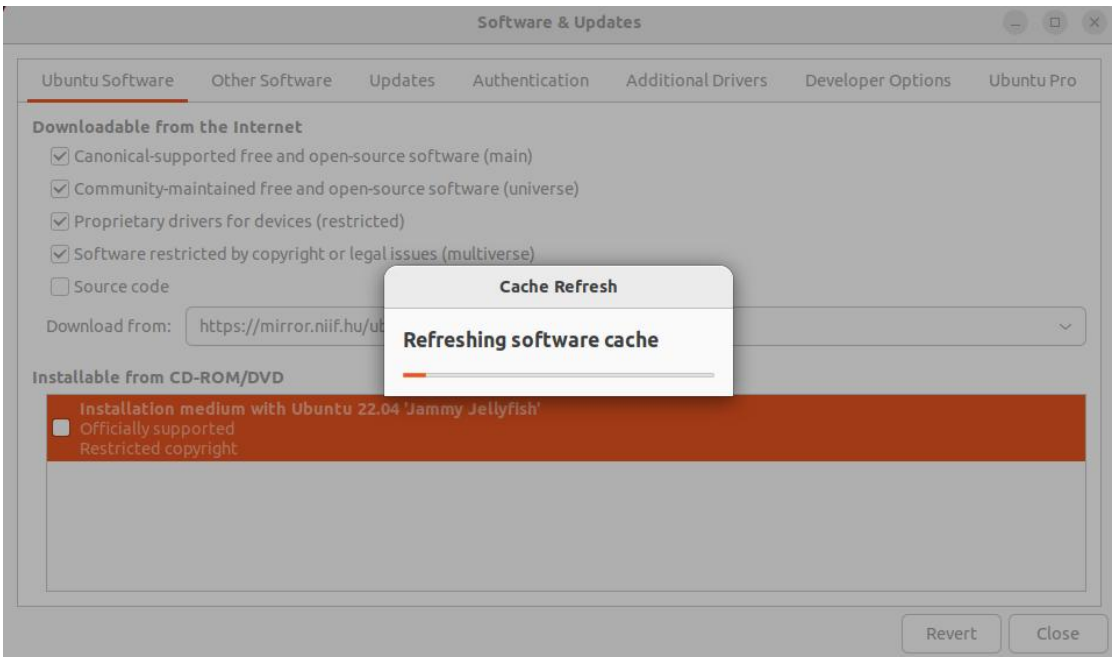


8.下图为更换成功后的状态，可以看到“下载自:”后面已经更新为“http://mirrors.niif.hu/ubuntu”，点击“colse”按钮，完成更新。

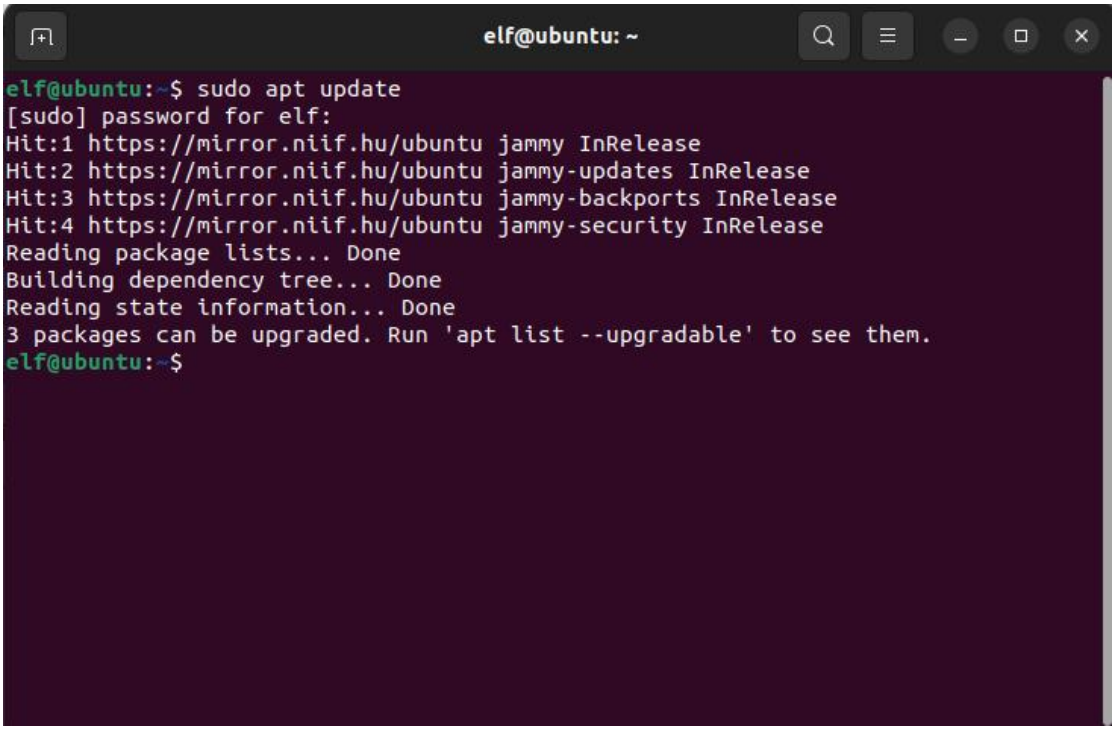


9.会弹出“The information about available software is out-of-date”对话框，意思是更换软件源之后，需要重新向新的软件源更新一下所有软件的版本缓存信息，这里我们点击一下“Reload”按钮立即更新一下即可。





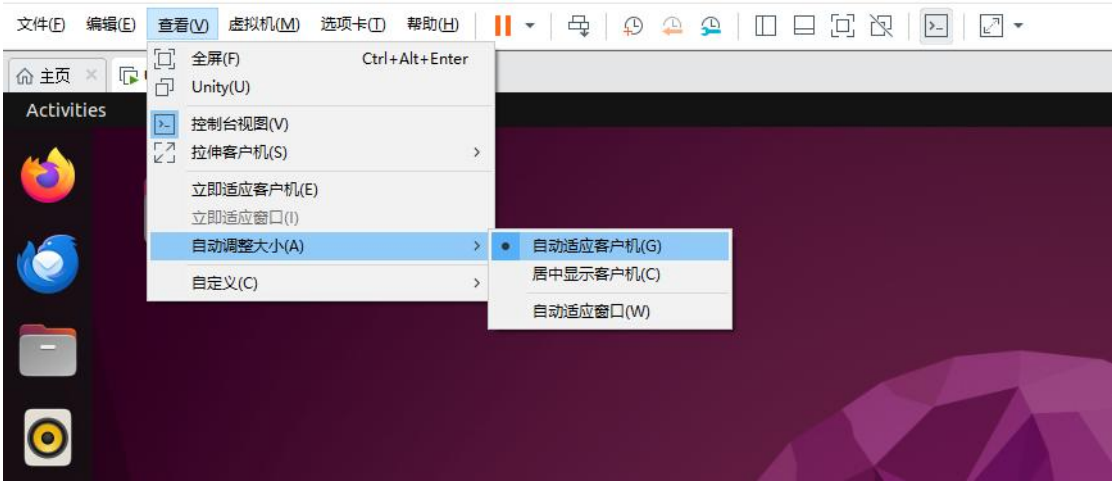
10.软件源更新后，Ubuntu 的源换成了“https://mirror.niif.hu/ubuntu”。



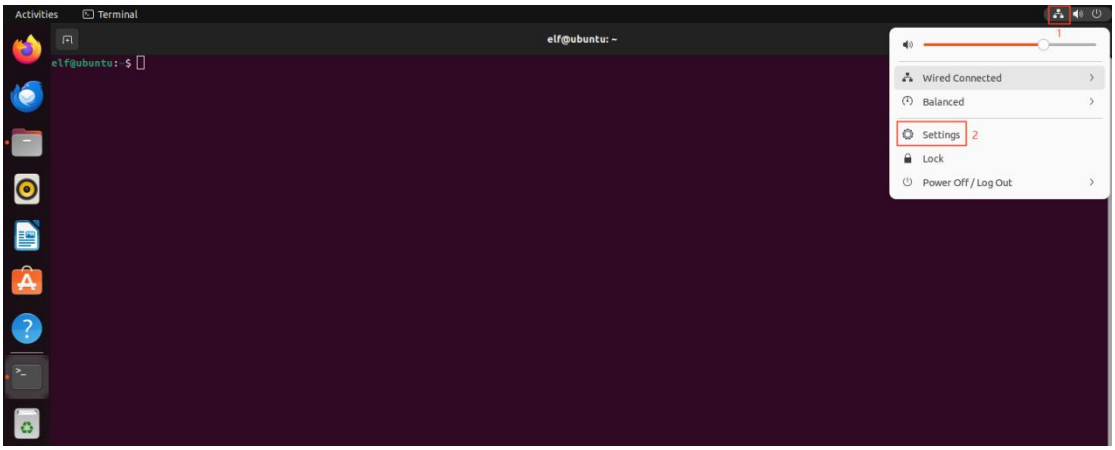
11 安装 VMware tools 工具

```
elf@ubuntu:~$ sudo apt update
elf@ubuntu:~$ sudo apt install open-vm-tools-desktop -y
elf@ubuntu:~$ sudo reboot
```

VMware tools 工具完成后，可以实现 Windows 和 Ubuntu 之间的文件复制粘贴，虚拟机自适应全显等功能。如果虚拟机不能够全屏显示，可以通过点击查看，选择自动调整大小，点击自动适应客户机，即可实现虚拟机的全屏显示，VMware tools 安装成功。



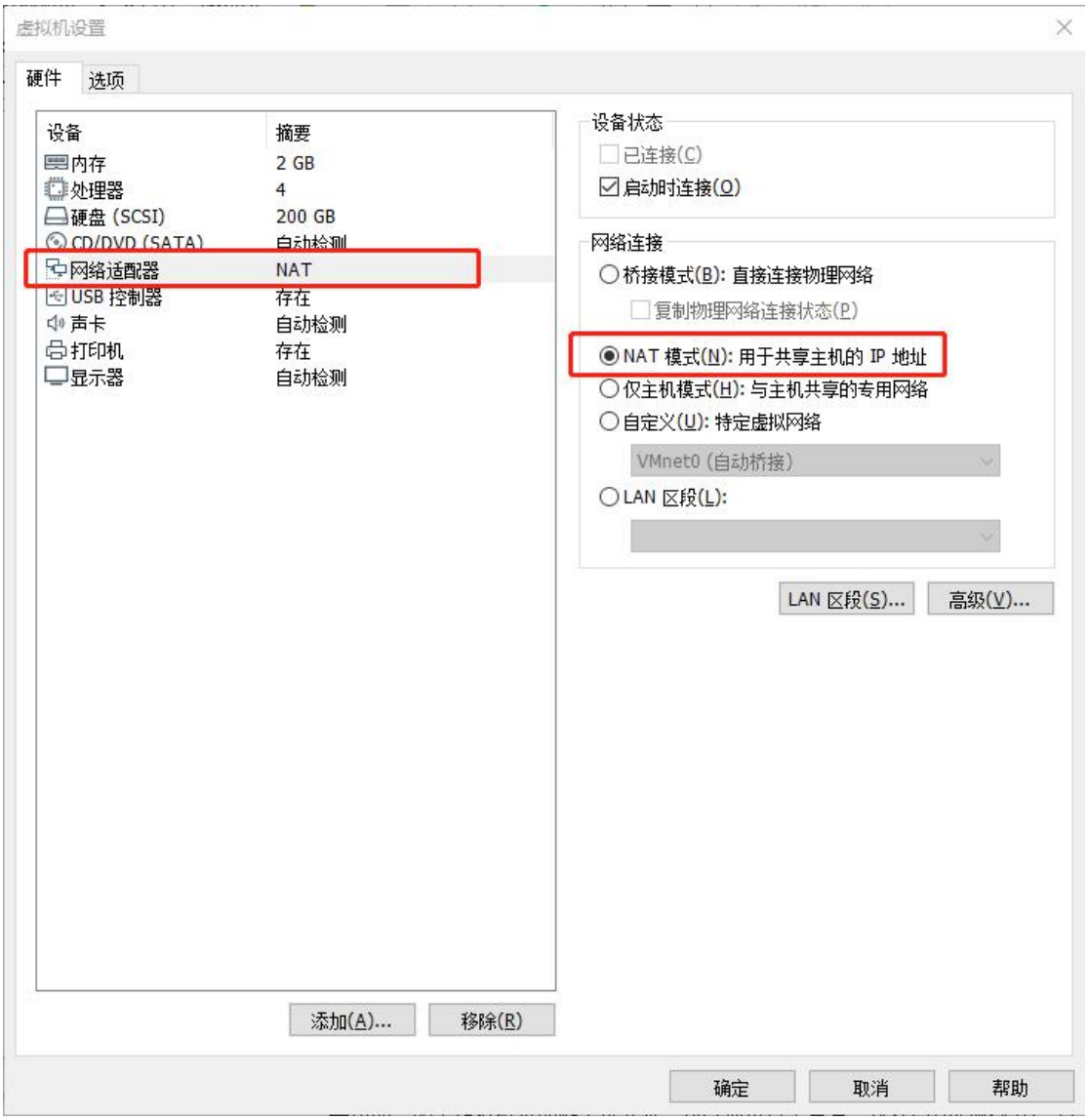
在下图位置进行大部分的系统设置。Ubuntu 上很多设置的需求都可以在这里完成。



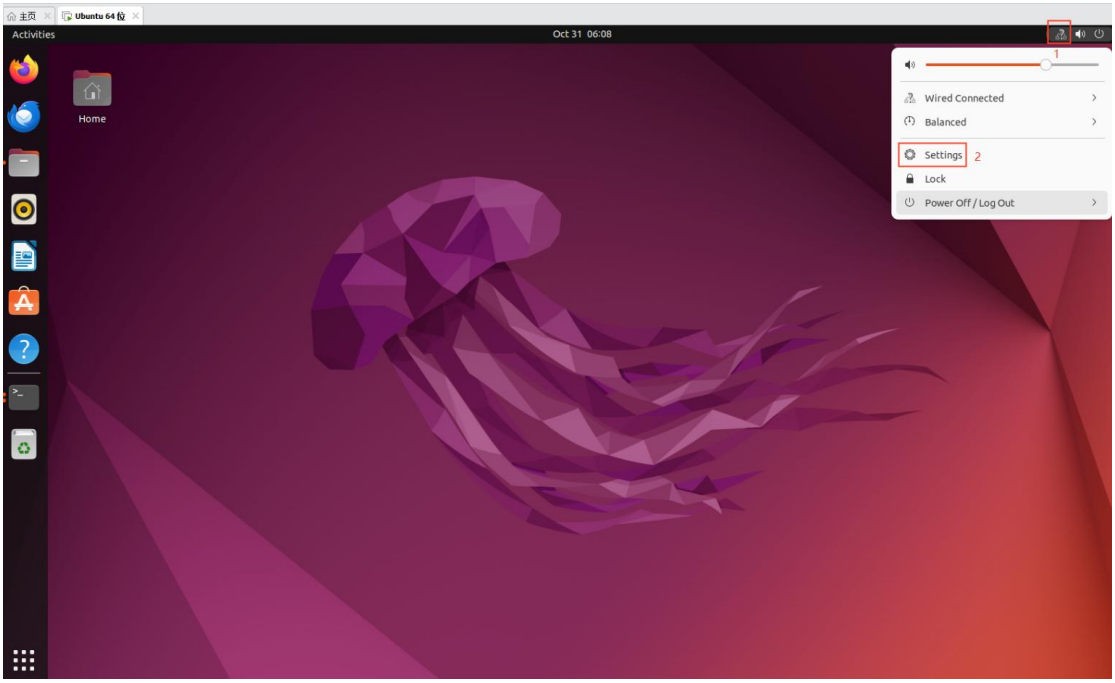
6.3.1.4. Ubuntu 网络设置

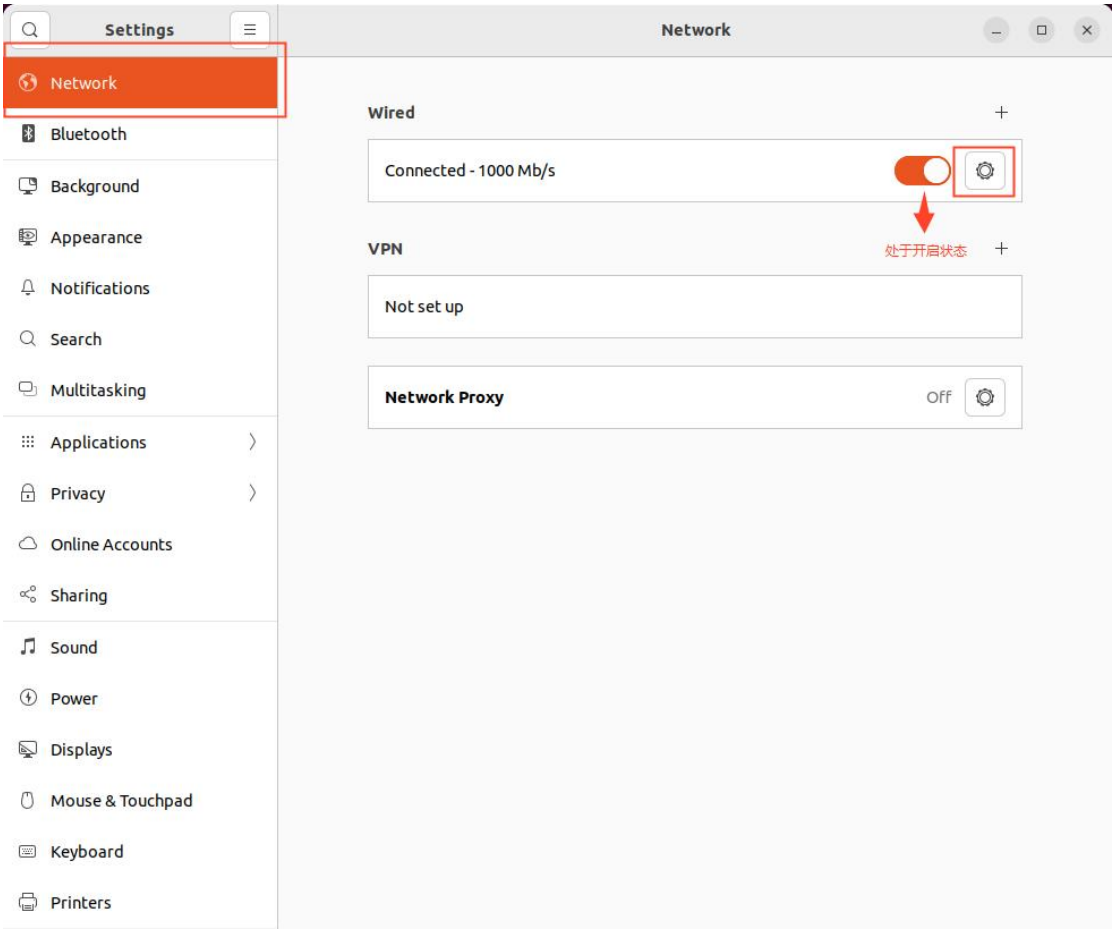
1.NAT 模式

在使用网络前，先确保我们的虚拟机能连接互联网，打开虚拟机设置，网络适配器中的网络连接模式改为“NAT 模式”：

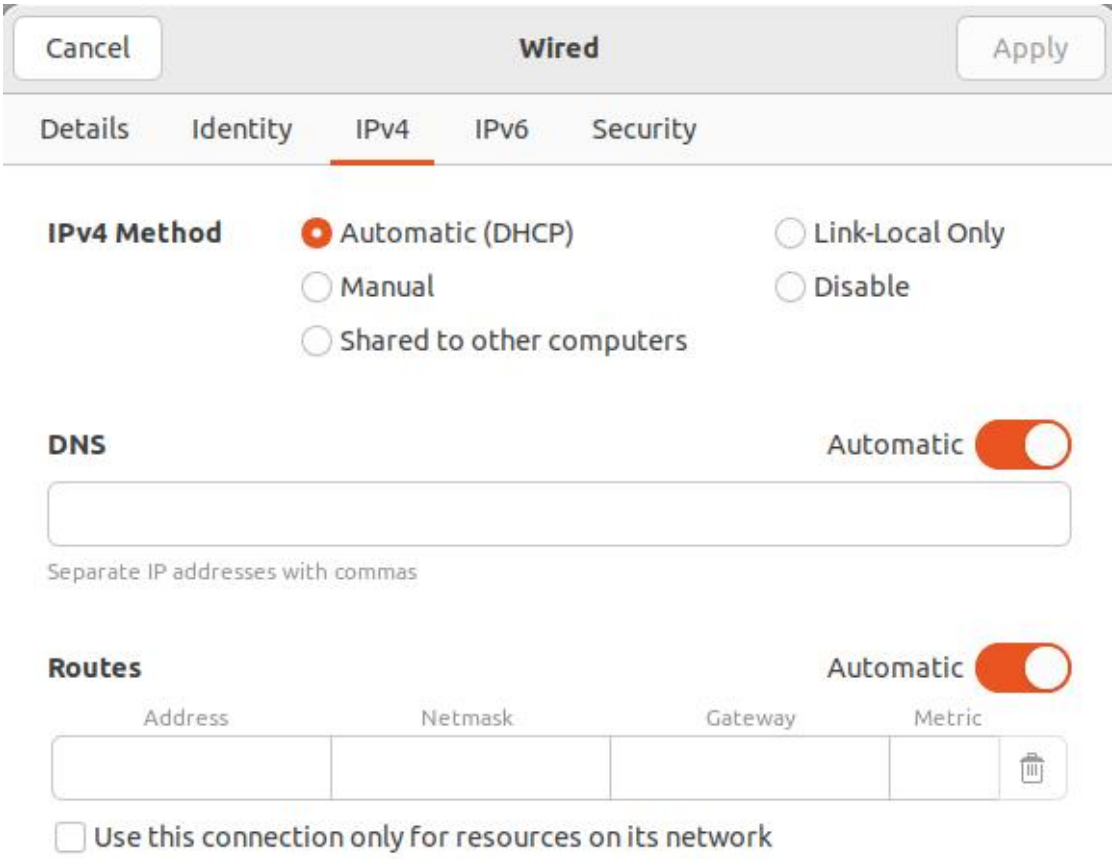


在虚拟机中，VMware 虚拟网卡设置为 NAT 模式时，Ubuntu 环境中网络设置为动态 IP 即可。在这种模式下虚拟 NAT 设备和主机网卡相连通。这是我们虚拟机上外网最常用的方式。


Activities
Oct 31 06:08
Home
Wired Connected
Balanced
Settings
2
Lock
Power Off / Log Out

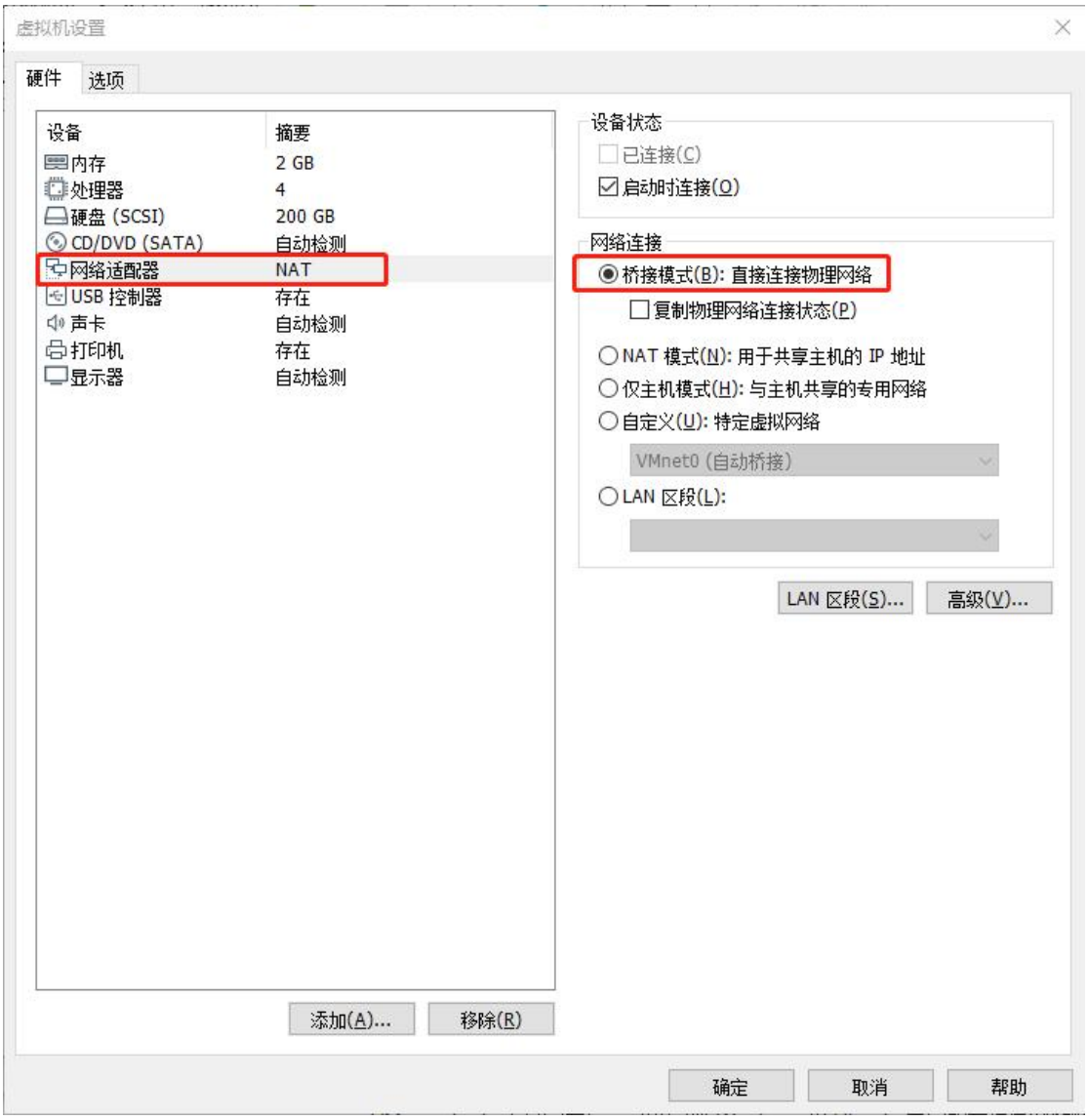

Settings
Network
Network
Bluetooth
Background
Appearance
Notifications
Search
Multitasking
Applications
Privacy
Online Accounts
Sharing
Sound
Power
Displays
Mouse & Touchpad
Keyboard
Printers
Wired
Connected - 1000 Mb/s
处于开启状态
VPN
Not set up
Network Proxy
Off

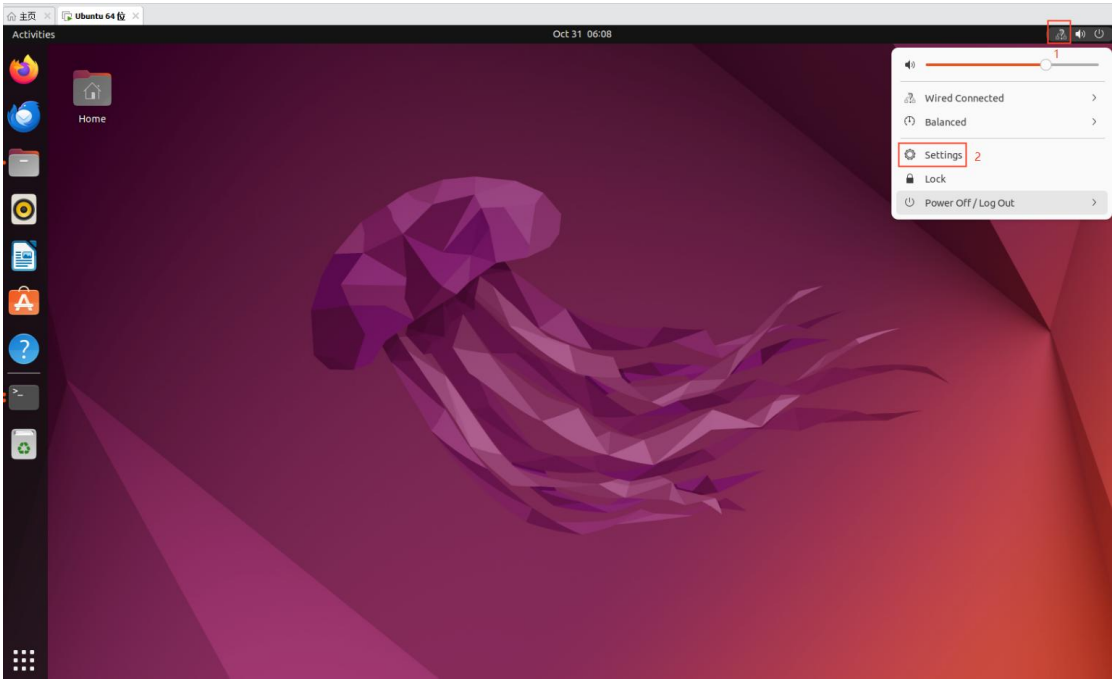
网络设置为动态 ip。

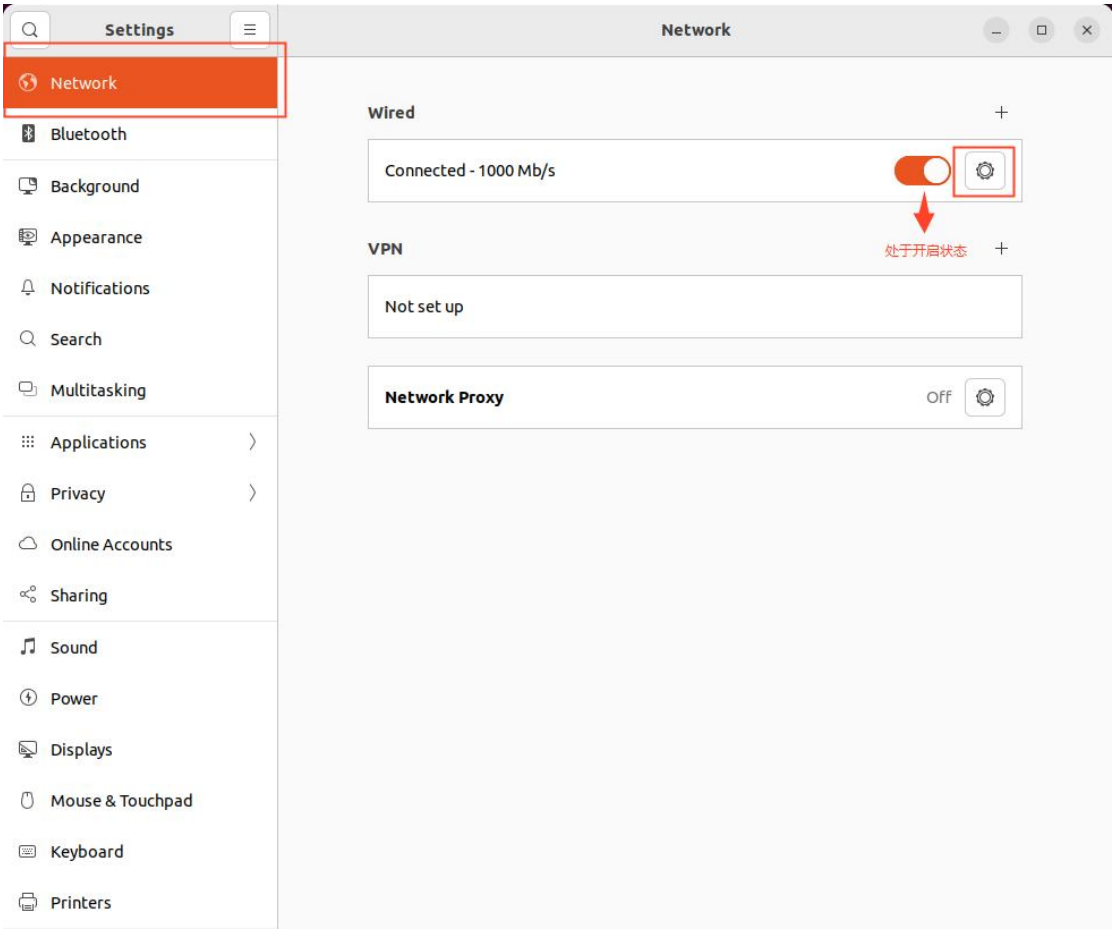


2.桥接模式

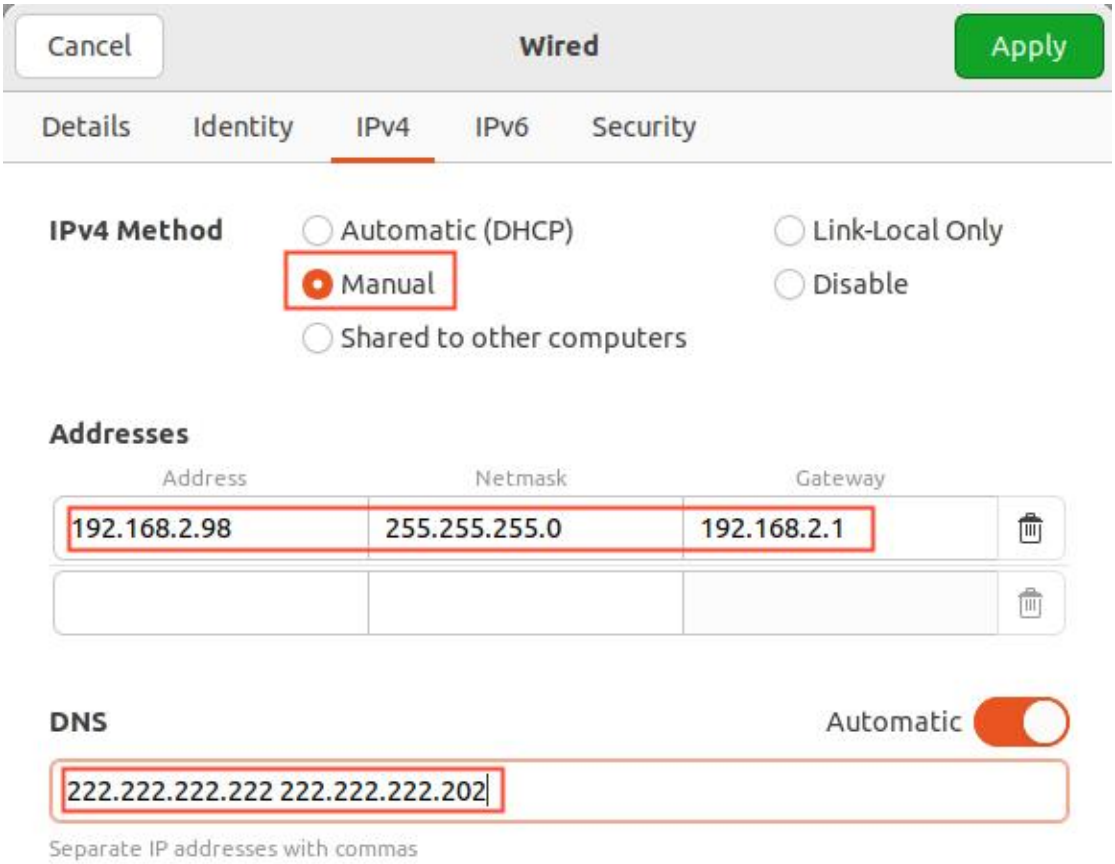
如果在使用 TFTP，SFTP 等服务器时则需要设置虚拟机的网络连接方式为桥接方式。VMware 虚拟网卡设置为桥接模式时，主机网卡和虚拟机网卡通过虚拟网桥进行通信，需要将 ubuntu 的 IP 与主机 IP 设置在同一个网段。


虚拟机设置
硬件
选项
设备
摘要
内存
2 GB
处理器
4
硬盘 (SCSI)
200 GB
CD/DVD (SATA)
自动检测
网络适配器
NAT
USB 控制器
存在
声卡
自动检测
打印机
存在
显示器
自动检测
添加(A)...
移除(R)
设备状态
已连接(C)
启动时连接(O)
网络连接
桥接模式(B): 直接连接物理网络
复制物理网络连接状态(P)
NAT 模式(N): 用于共享主机的 IP 地址
仅主机模式(H): 与主机共享的专用网络
自定义(U): 特定虚拟网络
VMnet0 (自动桥接)
LAN 区段(L):
LAN 区段(S)...
高级(V)...
确定
取消
帮助


Activities
Oct 31 06:08
Home
Wired Connected
Balanced
Settings
Lock
Power Off / Log Out



设置静态 ip，此时 Ubuntu 的 IP 与主机 IP 需设置在同一个网段。



⚠ **注意：网络设置部分涉及到的 IP 以及 DNS 请按照用户自身的实际环境来设置，文档为举例说明。**

6.3.2. 必要库安装

⚠ 注：若您使用的是我们提供的开发环境，本小节可以直接跳过。

在进行 Linux 系统编译之前，需要安装一些工具包。在操作之前，请确保您的计算机或虚拟机能够正常连接互联网。如果在安装过程中遇到网络断开的问题，请按照以下步骤重新进行安装。

安装编译 Linux 必要的包。

```
elf@ubuntu:~$ sudo apt-get update
elf@ubuntu:~$ sudo apt-get install openssh-server vim git fakeroot      //必备工具包的安装
elf@ubuntu:~$ sudo apt-get install ssh make gcc libssl-dev liblz4-tool expect expect-dev g++ patchelf chrpath gawk texinfo chrpath diffstat binfmt-support qemu-user-static live-build bison flex cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev libgucharmap-2-90-dev bzip2 expat gpgv2 cpp-aarch64-linux-gnu libgmp-dev libmpc-dev bc python-is-python3 python2
```

使用网络配置工具和 menuconfig 时还需要安装以下库。

```
elf@ubuntu:~$ sudo apt-get update                    //更新下载源信息
elf@ubuntu:~$ sudo apt-get install libncurses*       //用于生成基于文本的用户界面
elf@ubuntu:~$ sudo apt-get install net-tools         //网络配置工具
```

6.3.3. 交叉编译链安装

🗁 ELF 2 开发板资料包\05-常用工具\01-编译工具安装脚本\aarch64-buildroot-linux-gnu_sdk-buildroot.tar.gz

将上述压缩包拷贝到开发环境/home/elf/目录下，并在该目录下解压。

```
elf@ubuntu:~$ tar -zvxf aarch64-buildroot-linux-gnu_sdk-buildroot.tar.gz
```

进入 aarch64-buildroot-linux-gnu_sdk-buildroot 目录执行 relocate-sdk.sh 脚本。

```
elf@ubuntu:~$ cd aarch64-buildroot-linux-gnu_sdk-buildroot/
elf@ubuntu:~/aarch64-buildroot-linux-gnu_sdk-buildroot$ ./relocate-sdk.sh
```

6.3.4. Qt Creator 安装及配置

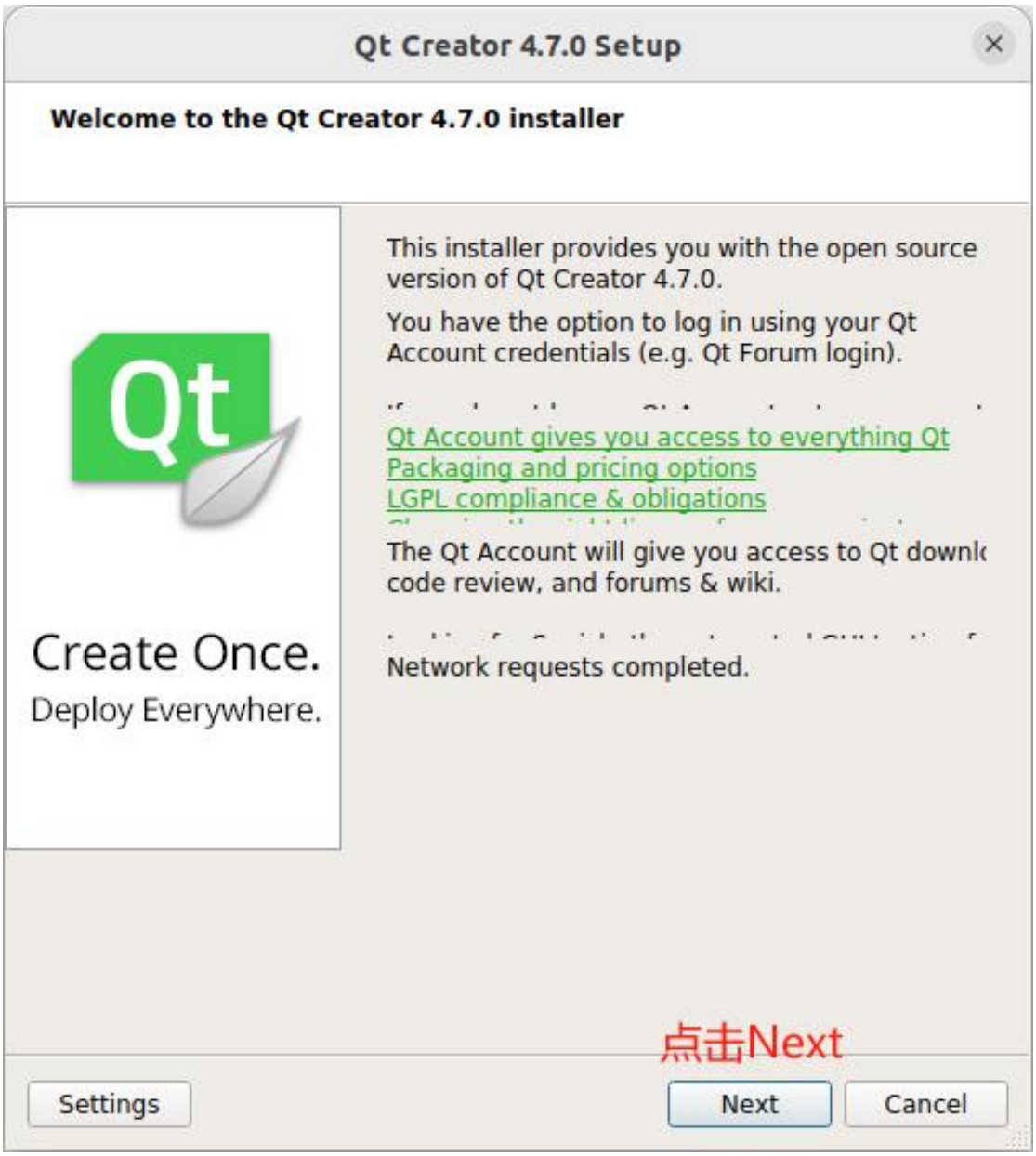
6.3.4.1. Qt Creator 安装

🗁 ELF 2 开发板资料包\05-常用工具\02-环境搭建工具\Qt 安装脚本\qt-creator-opensource-linux-x86_64-4.7.0.run

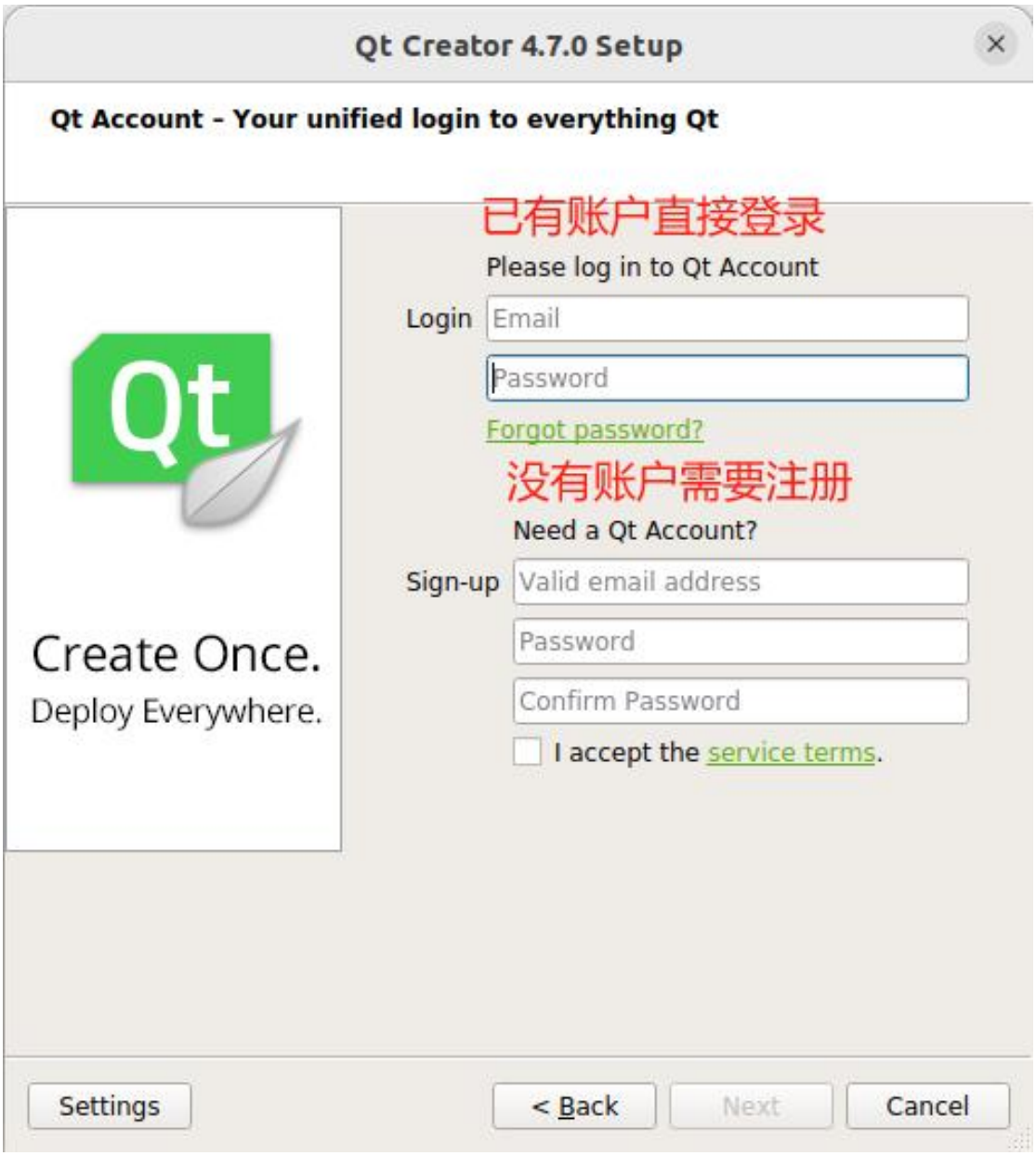
将 qt-creator-opensource-linux-x86_64-4.7.0.run 拷贝到开发环境/home/elf/目录下，执行下面命令。

```
elf@ubuntu:~$ ./qt-creator-opensource-linux-x86_64-4.7.0.run
```

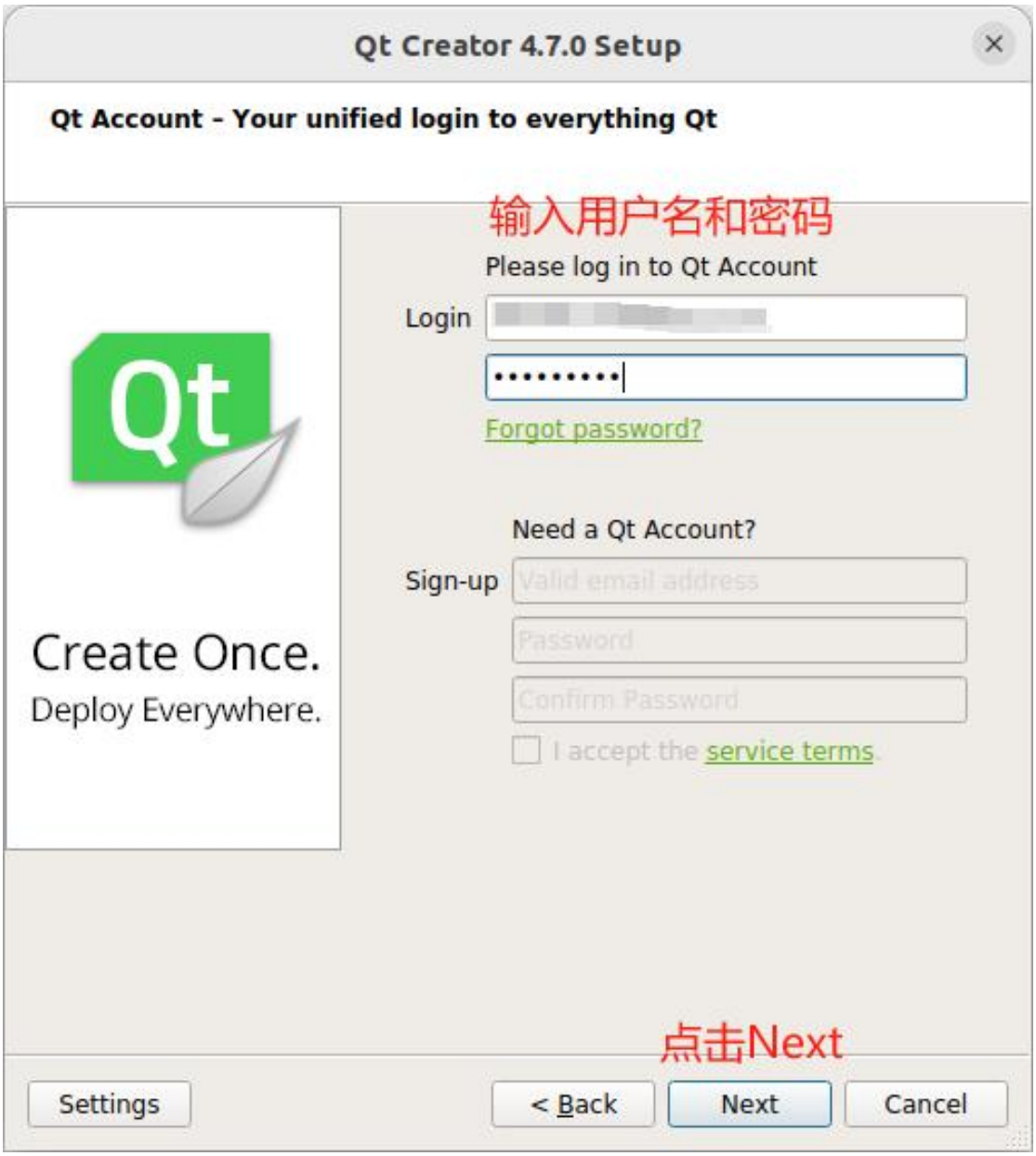
然后会弹出图形界面的安装窗口，按照提示进行安装：



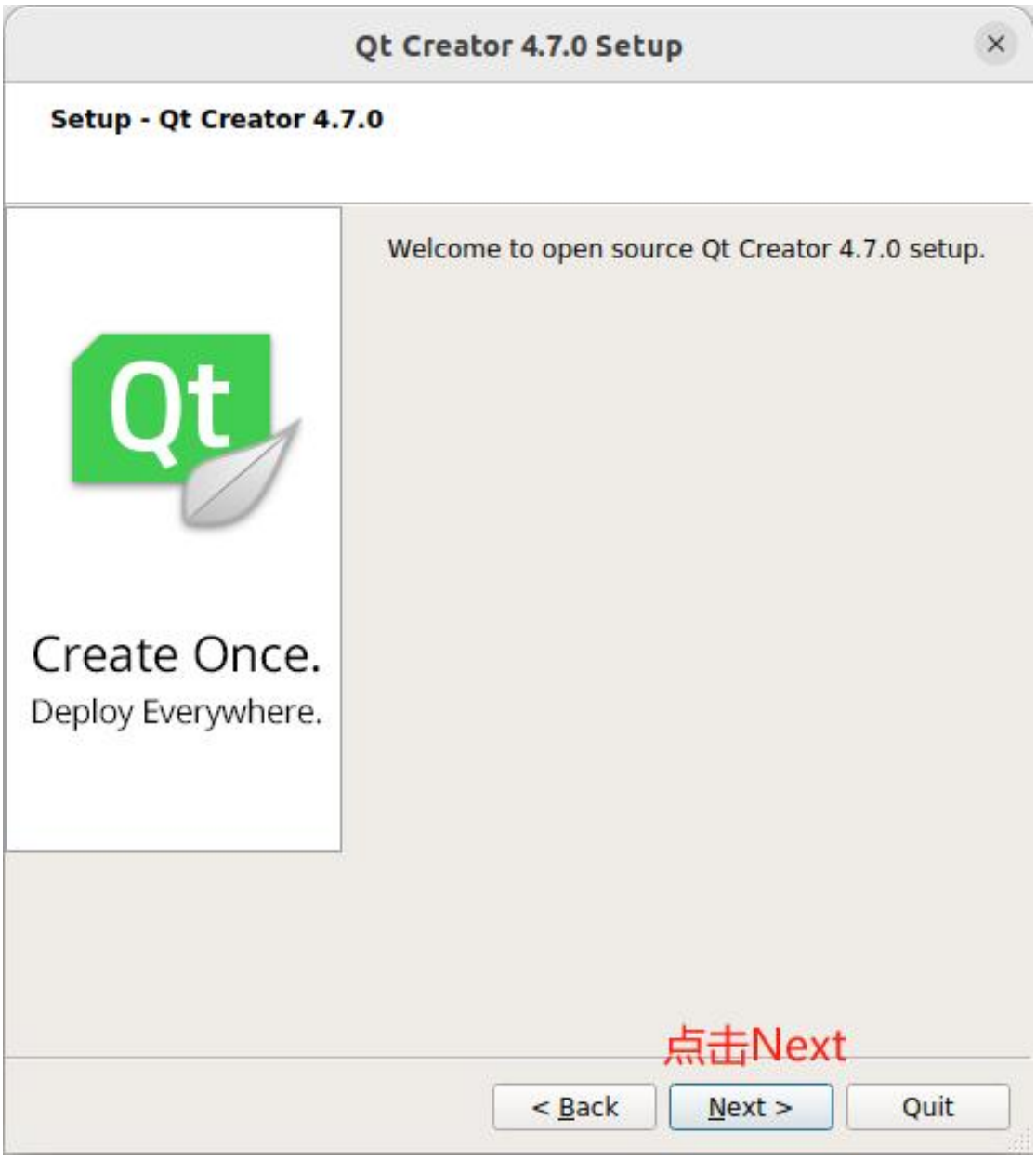
没有 Qt 账户的用户需要注册，已有 Qt 账户的用户直接登录即可。Qt 密码要求为：包含大写字母、小写字母、数字。



输入用户名和密码，点击 Next。


Qt Creator 4.7.0 Setup
Qt Account - Your unified login to everything Qt
输入用户名和密码
Please log in to Qt Account
Login
Forgot password?
Need a Qt Account?
Sign-up
Valid email address
Password
Confirm Password
I accept the service terms
Create Once.
Deploy Everywhere.
点击Next
Settings
< Back
Next
Cancel

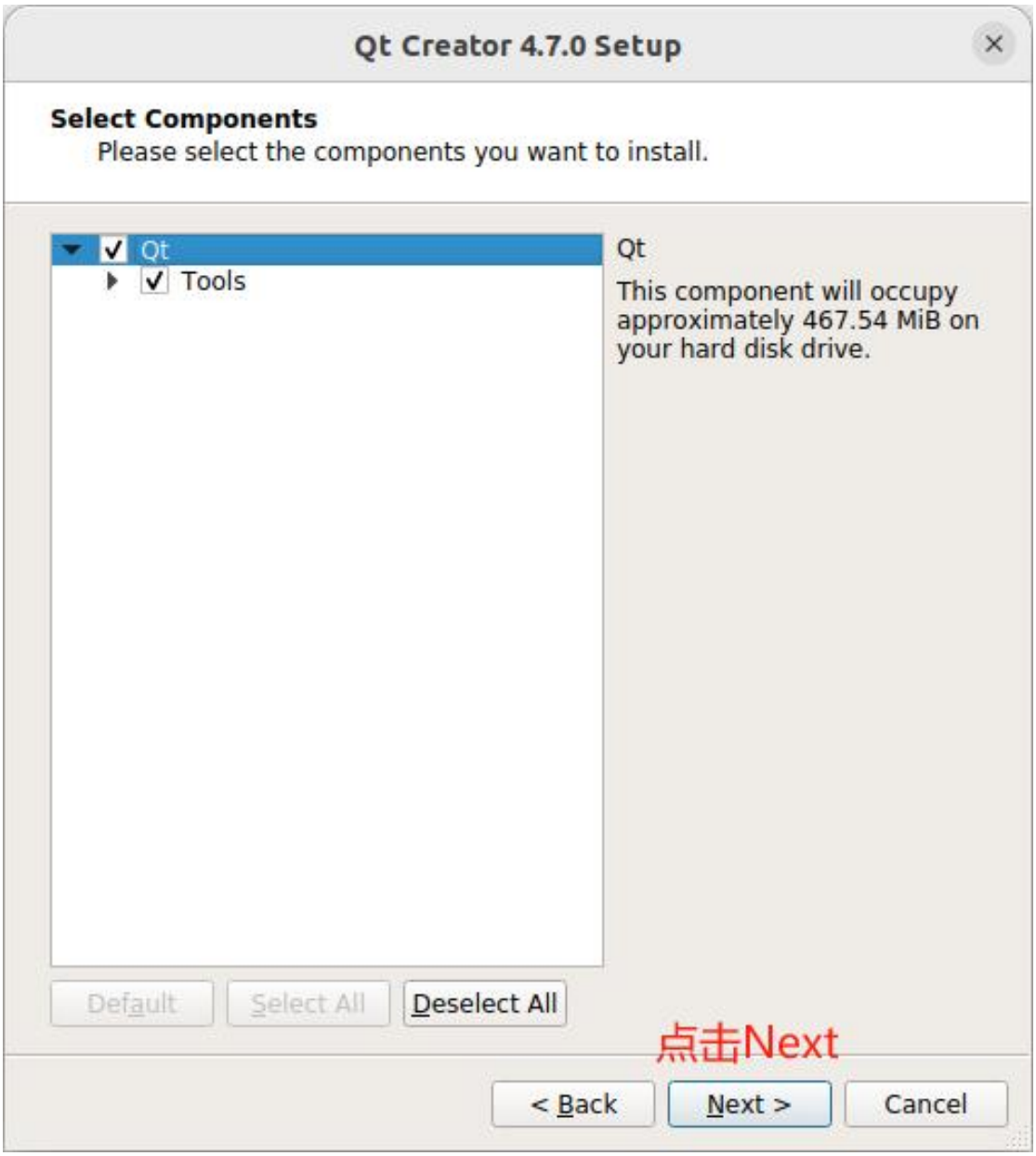
点击 Next。

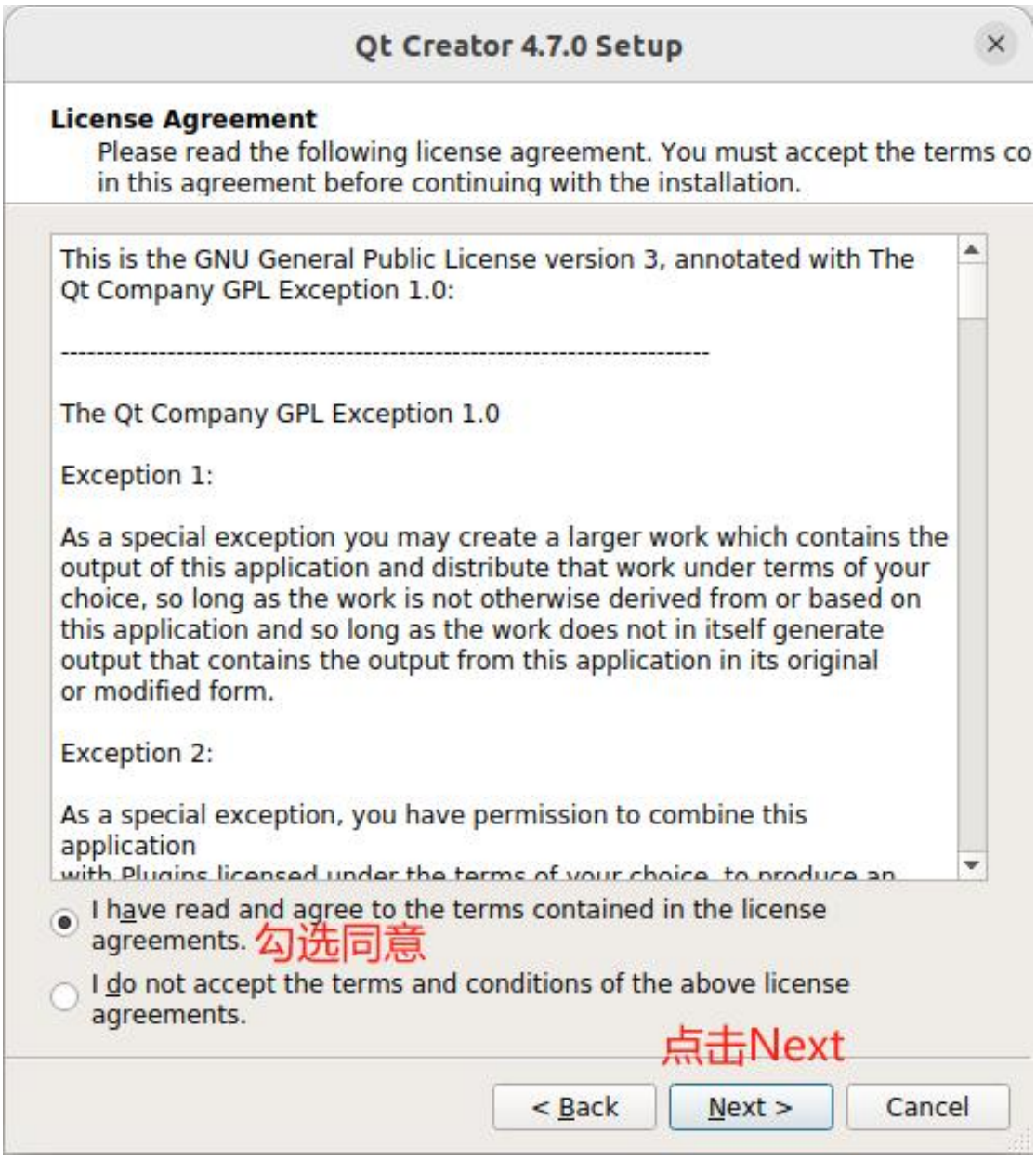


用户可根据自己习惯设置安装路径，这里选择默认路径，点击 Next。

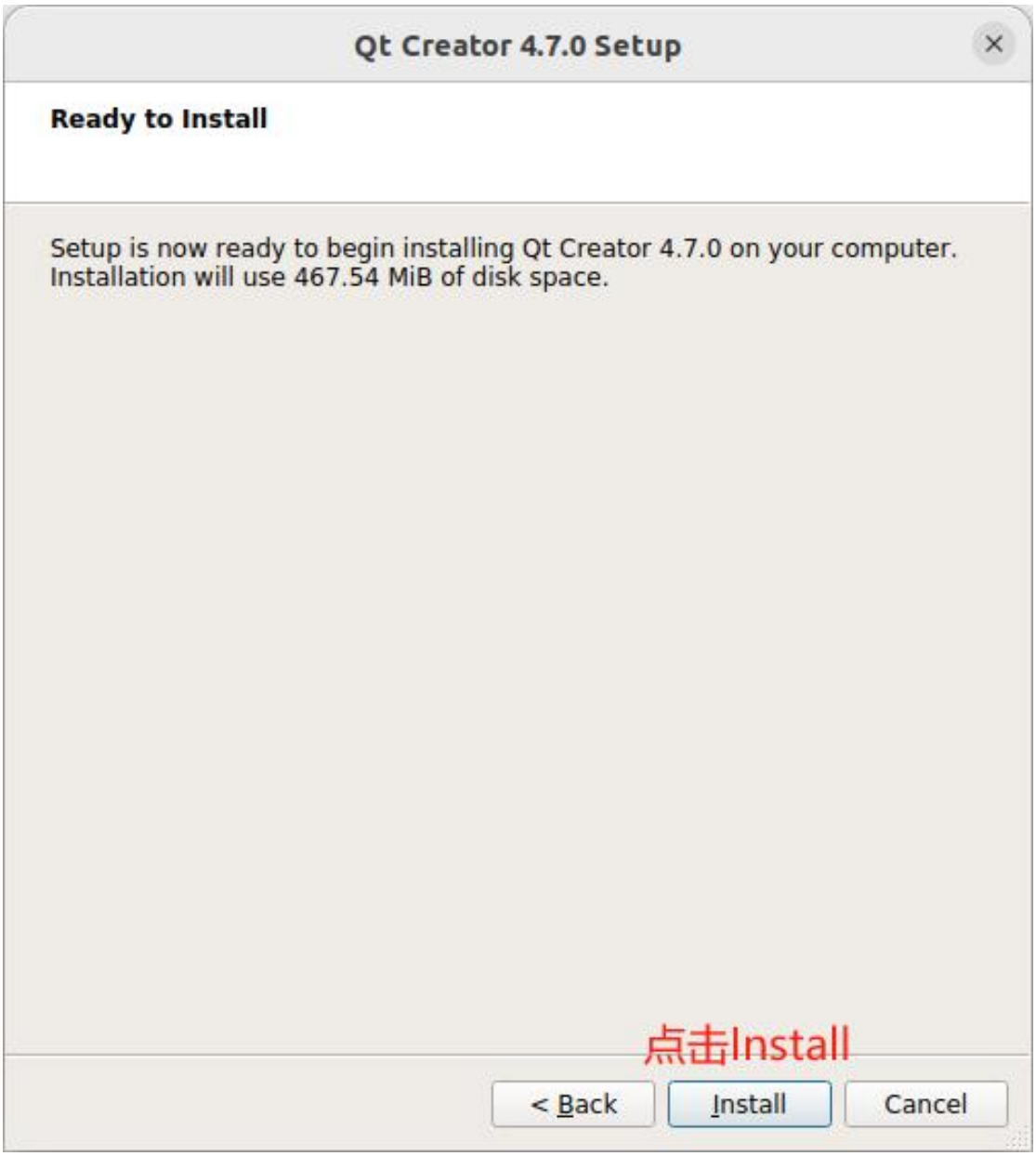

Qt Creator 4.7.0 Setup
Installation Folder
Please specify the directory where Qt Creator 4.7.0 will be installed.
/home/elf/qtcreator-4.7.0
Browse...
选择安装路径
点击Next
< Back
Next >
Cancel

完全安装，点击 Next。

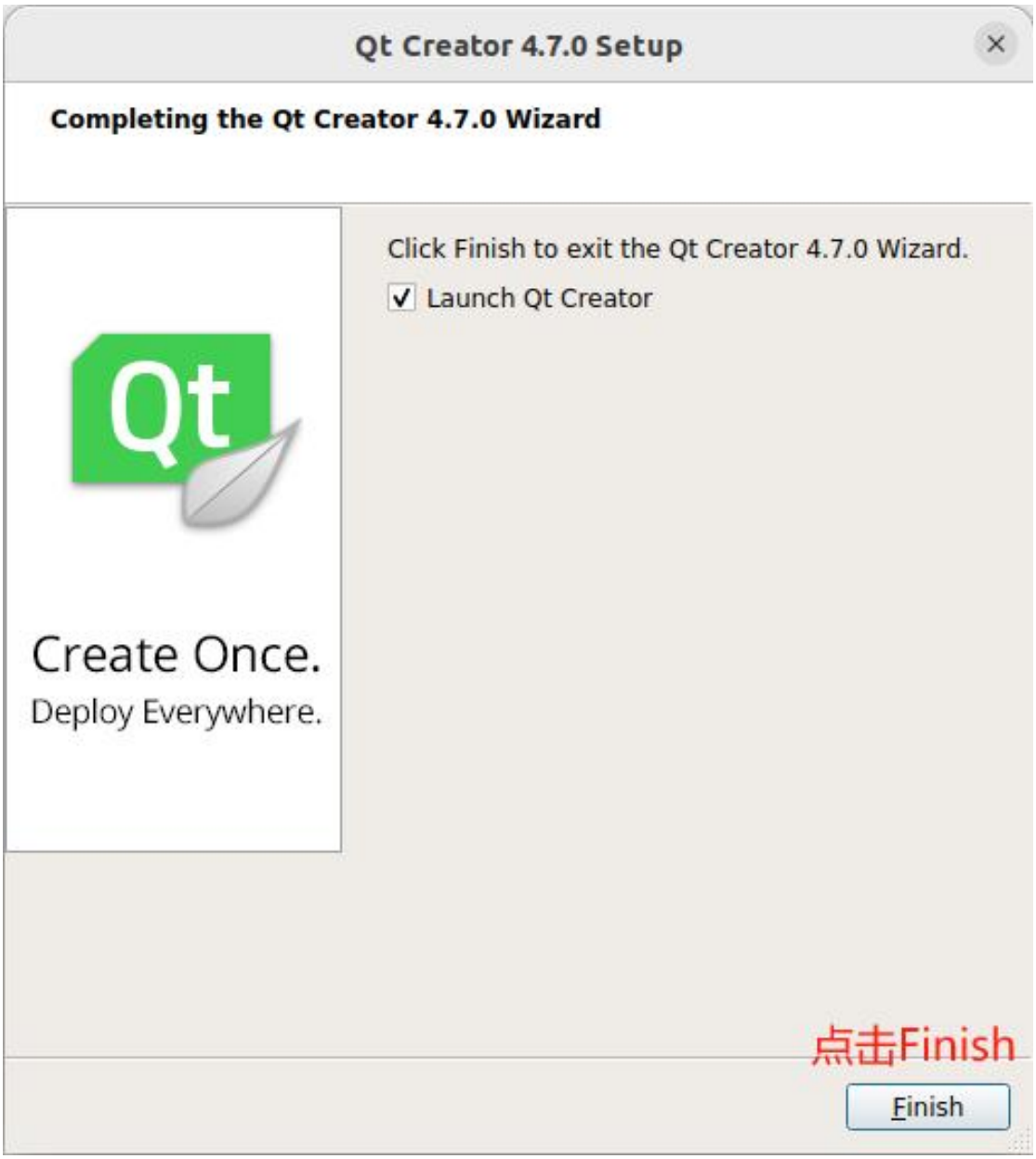

Qt Creator 4.7.0 Setup
Select Components
Please select the components you want to install.
Qt
Tools
Qt
This component will occupy approximately 467.54 MiB on your hard disk drive.
Default
Select All
Deselect All
点击Next
< Back
Next >
Cancel



点击 Install，等待安装完成。



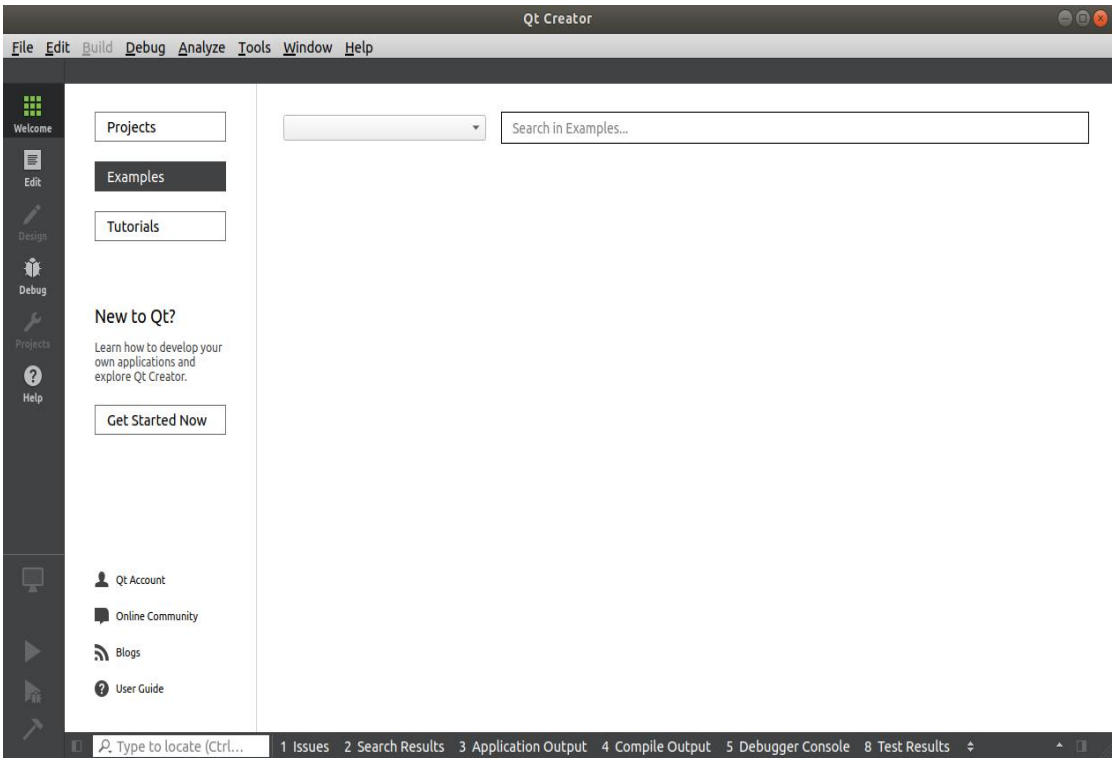
安装完成，点击 Finish。



这时将自动打开 Qt 界面，也可以通过命令行启动，执行以下命令，以后台方式打开 Qt Creator，用户打开时以自己实际安装路径为准。

```
elf@ubuntu:~$ cd /home/elf/qtcreator-4.7.0/bin
elf@ubuntu:~$ ./qtcreator &
```

出现 Qt Creator 工具界面，Qt Creator 安装完毕。



6.3.4.2. Qt Creator 环境配置

Qt 是跨平台的图形开发库，支持众多操作系统，在进行编译前需要对 Qt Creator 的编译环境进行配置。

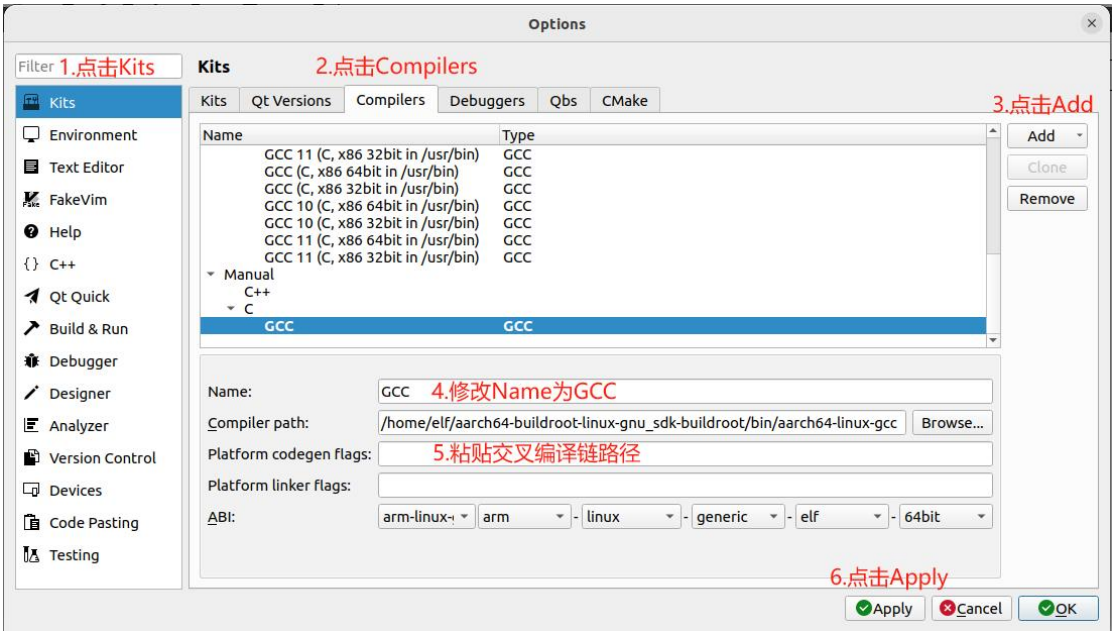
1.交叉编译器配置

⚠ **注意：默认的开发环境已经安装了交叉编译链，如果自己搭建的开发环境，需要参考交叉编译链安装章节安装交叉编译链（默认安装路径为/home/elf/aarch64-buildroot-linux-gnu_sdk-buildroot）**

进入 qtcreator 的安装路径打开 qtcreator。

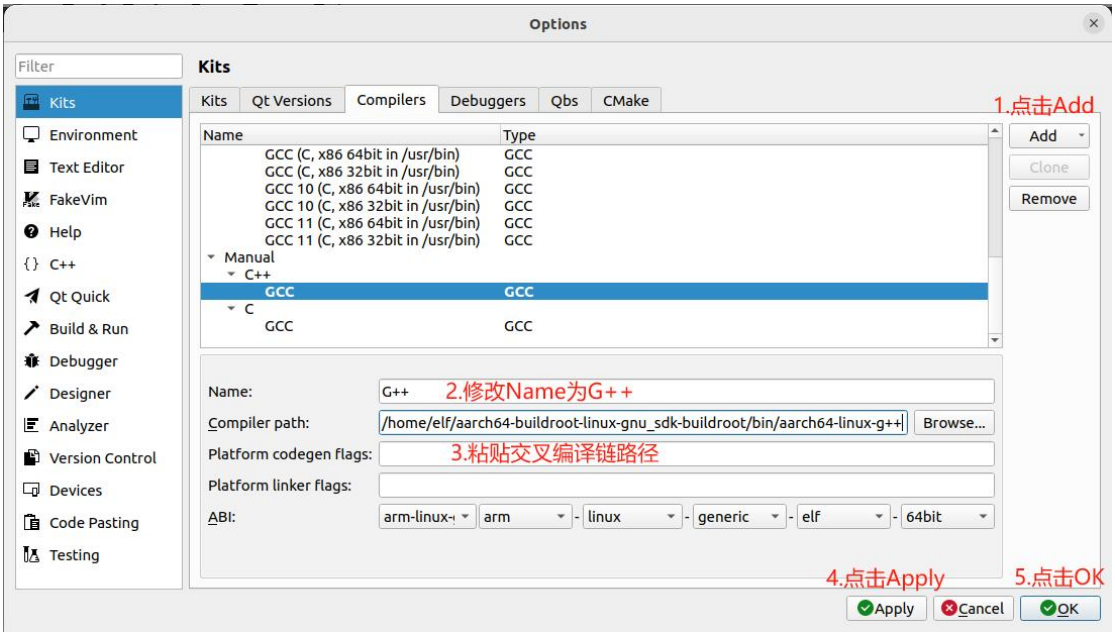
```
elf@ubuntu:~$ cd qtcreator-4.7.0/bin/
elf@ubuntu:~/qtcreator-4.7.0/bin$ ./qtcreator &
```

- 点击 Tools ->Options->Kits->Compilers，点击 Add ->GCC->C。
- Name：GCC。
- Compiler path：/home/elf/aarch64-buildroot-linux-gnu_sdk-buildroot/bin/aarch64-linux-gcc。



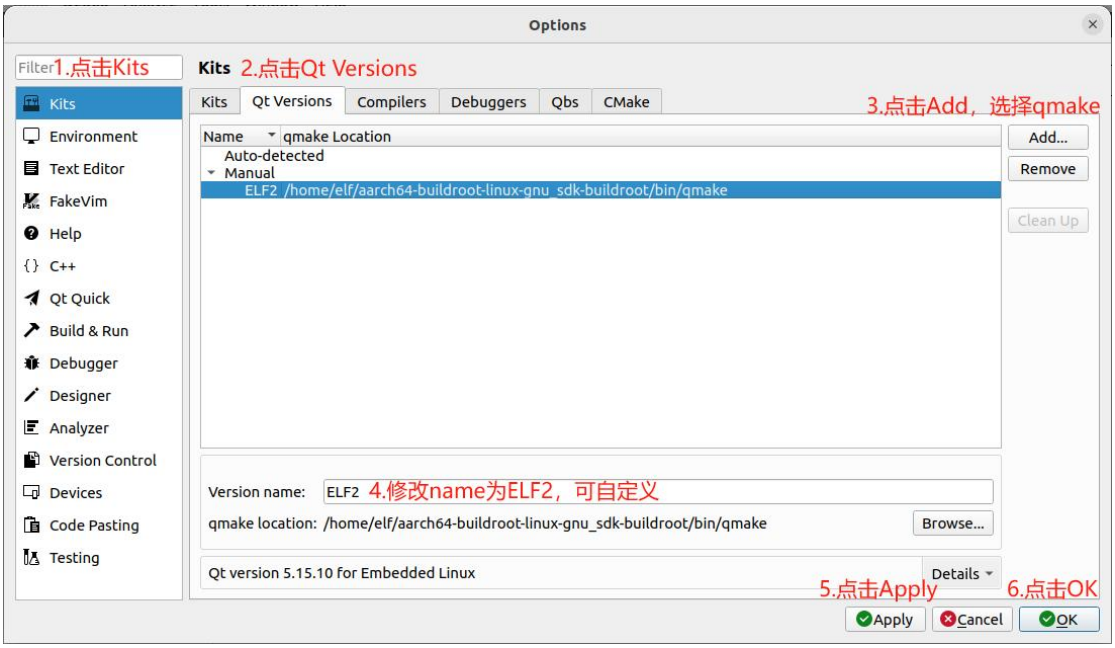
按照同样的方法添加 GCC 编译器。

- 点击 Add->GCC->C++。
- Name：G++。
- Compiler path：/home/elf/aarch64-buildroot-linux-gnu_sdk-buildroot/bin/aarch64-linux-g++。



2.Qt Versions 配置

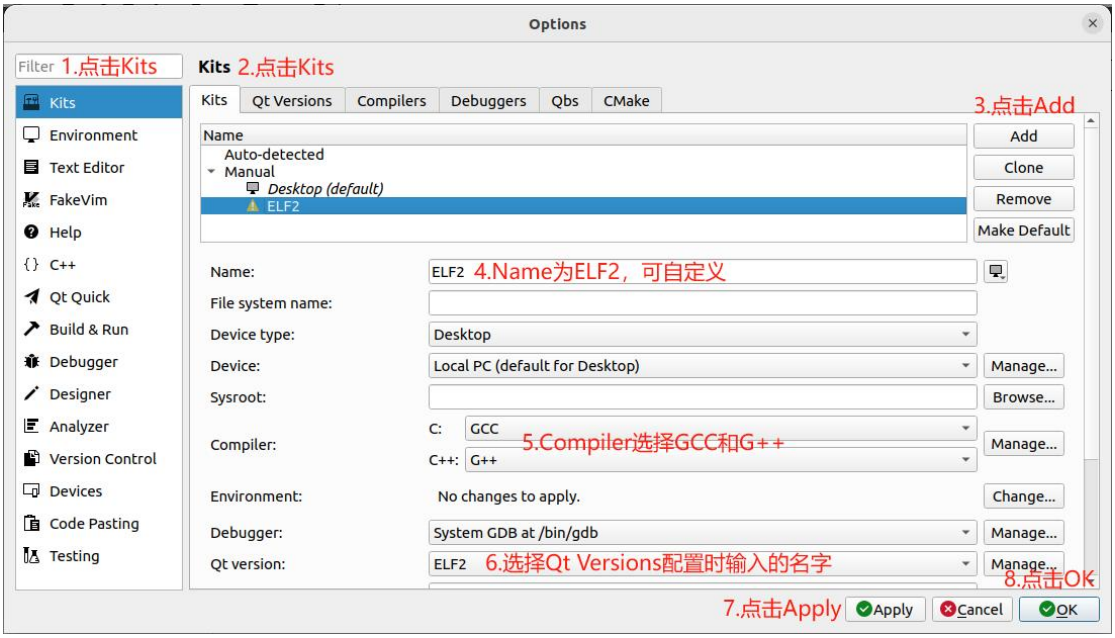
- 点击 Tools ->Options->Kits->Qt Versions。
- 点击 Add，弹出对话框选择/home/elf/aarch64-buildroot-linux-gnu_sdk-buildroot/bin/qmake，点击 Open 添加。
- Version name：ELF2。
- 点击 Apply，点击 OK。



3.Kits 配置

Kits 是一个构建套件，用来构建和选择开发编译环境，对于有多种 QT 库的项目很有用。将之前添加的交叉编译器和 QT Version 添加到 Kits 中，构建适合开发板的编译环境。

- 点击 Tools ->Options->Kits->Kits->Add。
- Name：ELF2。
- Compiler：GCC 和 G++。
- Qt version：ELF2。
- 点击 Apply，点击 OK。



第七章 相关代码编译

Linux 系统是在 buildroot 的基础上构建的桌面操作系统。

本章节主要描述开发板相关源码的编译方法，包括内核源码编译、应用程序编译方法。

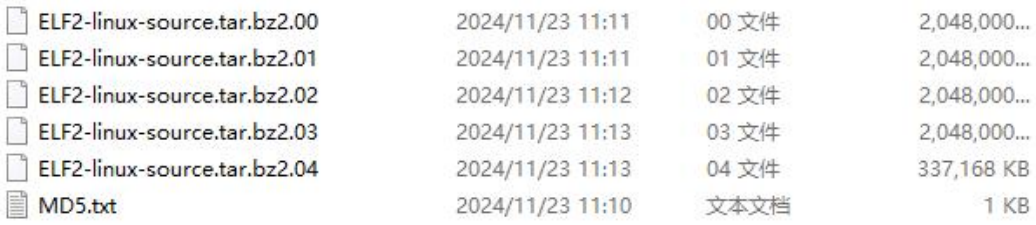
7.1. 编译前准备

7.1.1. 环境说明

- 开发环境操作系统：Ubuntu22.04 64 位版
- 开发板使用 Bootloader 版本：u-boot-2017.09
- 开发板内核版本：linux-5.10.209
- 开发板移植 QT 版本：qt5.15.10

7.1.2. 拷贝源码

程序源码：ELF 2 开发板资料包\02-Linux 源代码\ELF2-linux-source.tar.bz2.0*



在家目录下创建 work 工作目录。

```
elf@ubuntu:~$ mkdir work                                    //按照顺序创建工作目录
```

将用户资料中的源码文件 ELF2-linux-source.tar.bz2.0* 拷贝到虚拟机/home/elf/work 目录。

```
elf@ubuntu:~$ cd work/                                      //切换到工作目录
elf@ubuntu:~/work$ cat ELF2-linux-source.tar.bz2.0* > ELF2-linux-source.tar.bz2
elf@ubuntu:~/work$ tar -xvf ELF2-linux-source.tar.bz2       //在当前位置解压压缩包
```

运行命令后等待完成即可。

7.2. 源码编译

注意：

- **初次解压内核源码后，需要先对源码进行整体编译。**
- **整体编译过后，可根据实际情况再进行单独编译。**
- **该源码编译需要开发环境运行内存 16G 及以上，请不要修改我们提供的 VM 虚拟机镜像配置。**

7.2.1. SDK 源码全编译

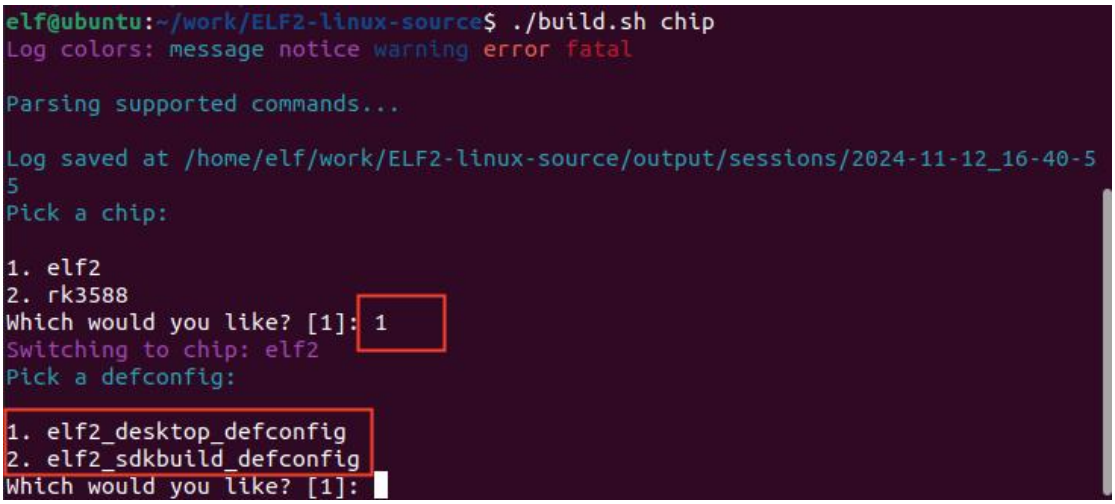
在源码路径内，提供了编译脚本 build.sh，运行该脚本对整个源码进行编译，需要在终端切换到解压出来的源码路径，找到 build.sh 文件。

注意:由于编译 Buildroot 所需时间较长，全编译过程中会跳过编译 buildroot。

```
elf@ubuntu:~/work$ cd ELF2-linux-source/
```

以下操作需要在源码目录下操作，全编译：

```
elf@ubuntu: ~/work/ELF2-linux-source$ ./build.sh chip
```



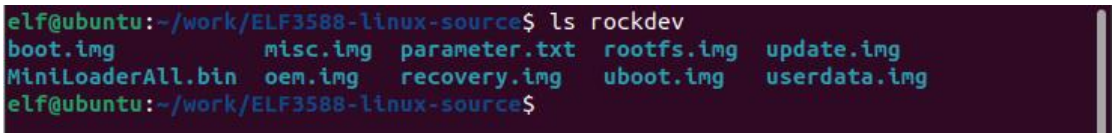
首先选择编译平台，输入 1 选择 ELF 2 板卡；接着选择编译的文件系统：ELF2 支持两种文件系统。若要编译 Ubuntu 22.04 文件系统，请输入 1，选择对应的 elf2_desktop_defconfig 配置文件；若要编译 buildroot 来构建文件系统，请输入 2，选择对应的 elf2_sdkbuild_defconfig 配置文件。

选择之后，使用 build.sh 进行全编译。

```
elf@ubuntu: ~/work/ELF2-linux-source$ ./build.sh
```

编译时间比较久，请耐心等待。

编译成功后，系统镜像将在 rockdev 文件夹下生成。



rockdev 文件夹中的镜像文件均为软链接，目标文件所在路径如下：

update.img	ELF2-linux-source/output/update/Image/update.img
uboot.img	ELF2-linux-source/u-boot/uboot.img
boot.img	ELF2-linux-source/kernel/boot.img
rootfs.img	Ubuntu22.04 文件系统:ELF2-linux-source/ubuntu/jammy-rootfs.img Buildroot 文件系统：ELF2-linux-source/buildroot/output/elf2_fs/images/rootfs.ext2

⚠ **注意：update.img 为打包好用于 OTG 或者 TF 卡完全烧写用，其它文件为分步烧写使用。**

清除编译生成的文件，用户在源码路径下进行操作。

```
elf@ubuntu: ~/work/ELF2-linux-source$ ./build.sh cleanall
```

```
elf@ubuntu:~/work/ELF3588-linux-source$ ./build.sh cleanall
Log colors: message notice warning error fatal

Log saved at /home/elf/work/ELF3588-linux-source/output/sessions/2024-09-30_16-19-36
Using last kernel version(5.10)
make: Entering directory '/home/elf/work/ELF3588-linux-source/kernel'
  CLEAN   certs
  CLEAN   drivers/firmware/efi/libstub
  CLEAN   drivers/misc/lkdtm
  CLEAN   drivers/scsi
  CLEAN   drivers/tty/vt
fatal: not a git repository (or any of the parent directories): .git
  CLEAN   fs/unicode
  CLEAN   kernel
  CLEAN   lib
  CLEAN   net/wireless
  CLEAN   usr
  CLEAN   vmlinux.symvers modules-only.symvers modules.builtin modules.builtin.modinfo
  CLEAN   scripts/basic
  CLEAN   scripts/dtc
  CLEAN   scripts/kconfig
  CLEAN   scripts/mod
  CLEAN   scripts
  CLEAN   include/config include/generated .config .version Module.symvers
make: Leaving directory '/home/elf/work/ELF3588-linux-source/kernel'
make: Entering directory '/home/elf/work/ELF3588-linux-source/u-boot'
  CLEAN   dts/../arch/arm/dts
  CLEAN   dts
  CLEAN   examples/standalone
  CLEAN   tools
  CLEAN   tools/lib tools/common
  CLEAN   spl/arch spl/board spl/cmd spl/common spl/disk spl/drivers spl/dts spl/env spl/fs spl/lib spl/u-boot-spl spl/u-boot-spl-dtb.bin s
pl/u-boot-spl-nodtb.bin spl/u-boot-spl.bin spl/u-boot-spl.dtb spl/u-boot-spl.lds spl/u-boot-spl.map spl/u-boot-spl.sym spl/u-boot.cfg tpl/a
rch tpl/board tpl/common tpl/disk tpl/drivers tpl/dts tpl/fs tpl/u-boot-spl.lds tpl/u-boot-tpl tpl/u-boot-tpl-nodtb.bin tpl/u-boot-tpl.bin
tpl/u-boot-tpl.map tpl/u-boot-tpl.sym tpl/u-boot.cfg
  CLEAN   u-boot u-boot-dtb.bin u-boot-nodtb.bin u-boot-nodtb.bin.digest u-boot-nodtb.bin.gz u-boot.bin u-boot.cfg u-boot.cfg.configs u-boo
t.dtb u-boot.lds u-boot.map u-boot.srec u-boot.sym System.map bl31_0x00040000.bin bl31_0x000f0000.bin bl31_0xff100000.bin rk3588_spl_loader
_v1.17.113.bin tee.bin u-boot-dtb.bin u-boot-nodtb.bin u-boot.bin uboot.img bl31_0x00040000.bin.gz tee.bin.gz u-boot-nodtb.bin.gz .cc
  CLEAN   scripts/basic
  CLEAN   scripts/dtc
  CLEAN   scripts/kconfig
  CLEAN   include/config include/generated spl tpl
  CLEAN   .config .config.old include/autoconf.mk include/autoconf.mk.dep include/config.h
make: Leaving directory '/home/elf/work/ELF3588-linux-source/u-boot'
Running build.sh - cleanall succeeded.
elf@ubuntu:~/work/ELF3588-linux-source$
```

该操作清除所有中间文件，但不影响源文件，包括已经有改动的源文件。

7.2.2. SDK 源码单独编译

7.2.2.1. 单独编译内核

用户在源码路径下进行操作。

```
elf@ubuntu: ~/work/ELF2-linux-source$ ./build.sh kernel
```

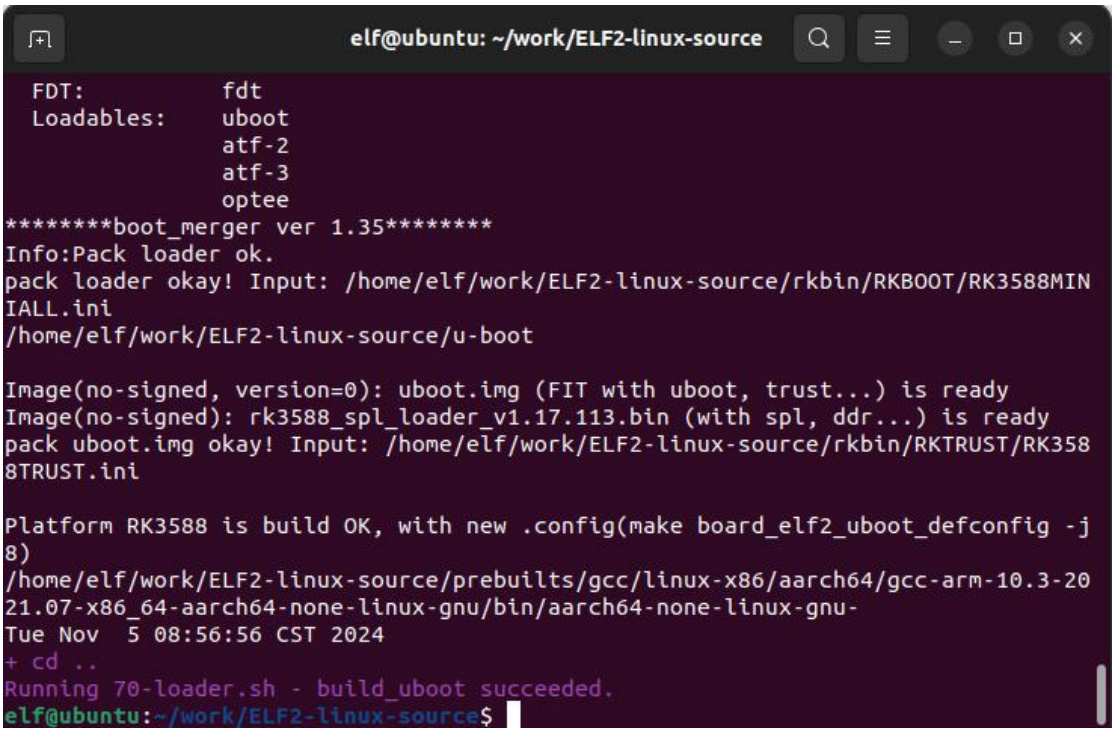
```
  INSTALL drivers/media/tuners/tea5761.ko
  INSTALL drivers/media/tuners/tea5767.ko
  INSTALL drivers/media/tuners/tua9001.ko
  INSTALL drivers/media/tuners/tuner-simple.ko
  INSTALL drivers/media/tuners/tuner-types.ko
  INSTALL drivers/media/tuners/tuner-xc2028.ko
  INSTALL drivers/media/tuners/xc4000.ko
  INSTALL drivers/media/tuners/xc5000.ko
  INSTALL drivers/net/wireless/intel/iwlwifi/iwlwifi.ko
  INSTALL drivers/net/wireless/intel/iwlwifi/mvm/iwlmvm.ko
  INSTALL drivers/net/wireless/rockchip_wlan/rkwifi/bcmdhd/bcmdhd.ko
  INSTALL drivers/net/wireless/rockchip_wlan/rkwifi/bcmdhd/dhd_static_buf.ko
  INSTALL net/ipv4/netfilter/iptable_nat.ko
  INSTALL net/netfilter/nf_nat.ko
  INSTALL net/netfilter/nf_nat_ftp.ko
  INSTALL net/netfilter/nf_nat_tftp.ko
  INSTALL net/netfilter/xt_MASQUERADE.ko
  INSTALL net/netfilter/xt_NETMAP.ko
  INSTALL net/netfilter/xt_REDIRECT.ko
  INSTALL net/netfilter/xt_nat.ko
  DEPMOD  5.10.209
make: Leaving directory '/home/elf/work/ELF3588-linux-source/kernel'
Running 10-kernel.sh - build_kernel succeeded.
elf@ubuntu:~/work/ELF3588-linux-source$
```

编译成功后 update.img 里的内核不更新。请分步烧写 kernel/boot.img 文件。

7.2.2.2. 单独编译 uboot

用户在源码路径下进行操作。

```
elf@ubuntu: ~/work/ELF2-linux-source$ ./build.sh uboot
```



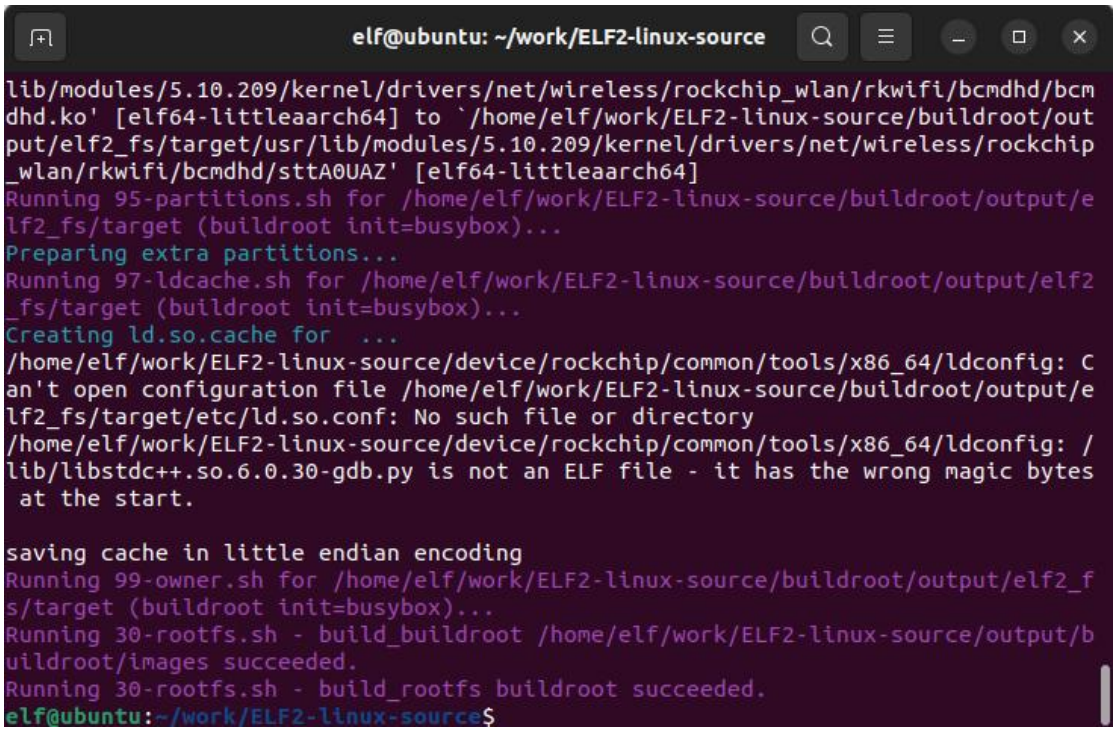
编译成功后 update.img 里的 uboot 不更新。请分步烧写 u-boot/uboot.img 文件。

7.2.2.3. 单独编译 buildroot

用户在源码路径下进行操作。

注意：只要单独编译了 buildroot ，之后的每次全编译都会编译 buildroot。

```
elf@ubuntu: ~/work/ELF2-linux-source$ ./build.sh rootfs
```



编译成功后 update.img 里的文件系统不更新。请分步烧写 buildroot/output/elf2_fs/images/rootfs.ext2 文件。

7.2.2.4. 打包镜像文件

经过单独编译后，update.img 中的镜像并不会自动更新，因此需要重新打包镜像，生成新的 update.img 文件。

```
elf@ubuntu: ~/work/ELF2-linux-source$ ./build.sh updateimg
```

7.2.3. 编译并运行命令行应用

7.2.3.1. Buildroot 系统编译并运行命令行应用

注意：若板卡搭载的是 Buildroot 系统，请参照以下步骤进行应用编译。

elf2 开发环境中提供了一套交叉编译工具，源码包中也包含了一套交叉编译工具。为了方便开发，可以直接使用开发环境中的交叉编译工具进行应用开发，无需解压源码包。接下来，我们将介绍如何使用开发环境中的交叉编译工具进行编译。

本小节使用看门狗测试程序。

1、使用 cd 命令进入到测试源码目录

```
elf@ubuntu:~$ cd /home/elf/work/ELF2-linux-source/app/elf/elf_cmd/cmddemo_wdt/
```

2、添加交叉编译器路径，进行交叉编译

```
elf@ubuntu:~/work/ELF2-linux-source/app/elf/elf_cmd/cmddemo_wdt$ export PATH=/home/elf/aarch64-buildroot-linux-gnu_sdk-buildroot/bin/:$PATH
elf@ubuntu:~/work/ELF2-linux-source/app/elf/elf_cmd/cmddemo_wdt$ aarch64-linux-gcc main.c -o cmddemo_wdt
```

用 file 命令查看生成的文件信息

```
elf@ubuntu:~/work/ELF2-linux-source/app/elf/elf_cmd/cmddemo_wdt$ file cmddemo_wdt
cmddemo_wdt: ELF 64-bit LSB pie executable, ARM aarch64, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-aarch64.so.1, for GNU/Linux 3.7.0, not stripped
```

通过结果可以看到编译生成的是 64 位、ARM 的文件。

3、将编译生成的 cmddemo_wdt 通过 U 盘拷贝到板卡上，比如/root 路径下，下述以 U 盘为例，拷贝到开发板，运行测试。

```
root@elf2-buildroot:~# cp /mnt/udisk/cmddemo_wdt ./
root@elf2-buildroot:~# ./cmddemo_wdt /dev/watchdog settimeout 60 &
 //执行 60s 复位命令
```

参考用户使用手册“看门狗测试”章节测试。

7.2.3.2. Desktop 系统编译并运行命令行应用

注意：若板卡搭载的是 Desktop 系统，请参照以下步骤进行应用编译。

本节以看门狗测试程序（cmddemo_wdt）为例，详细介绍如何在 Ubuntu 开发环境中，通过 chroot 方式调用开发板文件系统的编译器完成命令行应用的编译，并最终部署至开发板运行。

1.挂载开发板文件系统：

```
elf@ubuntu:~/work/ELF2-linux-source$ cd ubuntu/    //进入预编译文件系统所在目录
elf@ubuntu:~/work/ELF2-linux-source/ubuntu$ mkdir rootfs    //创建挂载目录
elf@ubuntu:~/work/ELF2-linux-source/ubuntu$ sudo mount jammy-rootfs.img rootfs/
    //挂载开发板文件系统镜像
[sudo] password for elf:
elf@ubuntu:~/work/ELF2-linux-source/ubuntu$ cd rootfs/    //进入挂载后的文件系统
```

2.在 chroot 环境中编译源码：

```
elf@ubuntu:~/work/ELF2-linux-source/ubuntu/rootfs$ sudo cp /home/elf/work/ELF2-linux-source/app/elf/elf_cmd/cmddemo_wdt/main.c root/     //将测试程序源码拷贝至文件系统内
elf@ubuntu:~/work/ELF2-linux-source/ubuntu/rootfs$ sudo chroot ./     //切换到开发板文件系统环境（chroot）
root@ubuntu:/# cd root/     //进入源码目录
root@ubuntu:~# gcc main.c -o cmddemo_wdt     //编译
```

编译完成后，按 Ctrl + D 组合键退出 chroot 环境，回到 Ubuntu 原始命令行。

3.将程序拷贝至 U 盘：

```
elf@ubuntu:~/work/ELF2-linux-source/ubuntu/rootfs$ sudo cp root/cmddemo_wdt /media/elf/9C68-BC78/     //将生成的可执行文件拷贝到已挂载的 U 盘（请根据实际 U 盘路径修改）
```

4.卸载文件系统：

```
elf@ubuntu:~/work/ELF2-linux-source/ubuntu/rootfs$ cd ..
elf@ubuntu:~/work/ELF2-linux-source/ubuntu$ sudo umount rootfs     //卸载文件系统
```

5.在开发板上运行程序：

```
root@elf2-desktop:~# cp /run/media/sda1/cmddemo_wdt ./     //从 U 盘拷贝程序到开发板当前目录
root@elf2-desktop:~# ./cmddemo_wdt /dev/watchdog settimeout 60 &     //运行看门狗测试程序，设置 60 秒超时复位
```

参考用户使用手册“看门狗测试”章节测试。

7.2.4. 编译并运行 QT 应用程序

7.2.4.1. Buildroot 系统编译并运行 QT 应用

注意：若板卡搭载的是 Buildroot 系统，请参照以下步骤进行应用编译。

- 方法一：通过命令行使用 qmake 进行编译

```
elf@ubuntu:~/work$ export PATH=/home/elf/aarch64-buildroot-linux-gnu_sdk-buildroot/bin/:$PATH
elf@ubuntu:~/work$ cd ELF2-linux-source/app/elf/elf_qt/src/watchdog/
elf@ubuntu:~/work/ELF2-linux-source/app/elf/elf_qt/src/watchdog$ qmake
elf@ubuntu:~/work/ELF2-linux-source/app/elf/elf_qt/src/watchdog$ make
```

在路径/home/elf/work/ELF2-linux-source/app/elf/elf_qt_out/release/bin 目录下会看到新生成的二进制文件 qtdemo_watchdog。

```
elf@ubuntu:~/work$ ls ELF2-linux-source/app/elf/elf_qt_out/release/bin
qtdemo_watchdog
```

将编译生成的可执行文件通过 U 盘拷贝到板卡上，并将依赖的配置文件拷贝到和应用同级目录。

```
root@elf2-buildroot:~# cp /mnt/udisk/qtdemo_watchdog ./
root@elf2-buildroot:~# mkdir data
root@elf2-buildroot:~# cp -r /usr/bin/data/conf.txt data/conf.txt
```

运行测试即可。

```
root@elf2-buildroot:~# ./qtdemo_watchdog
```

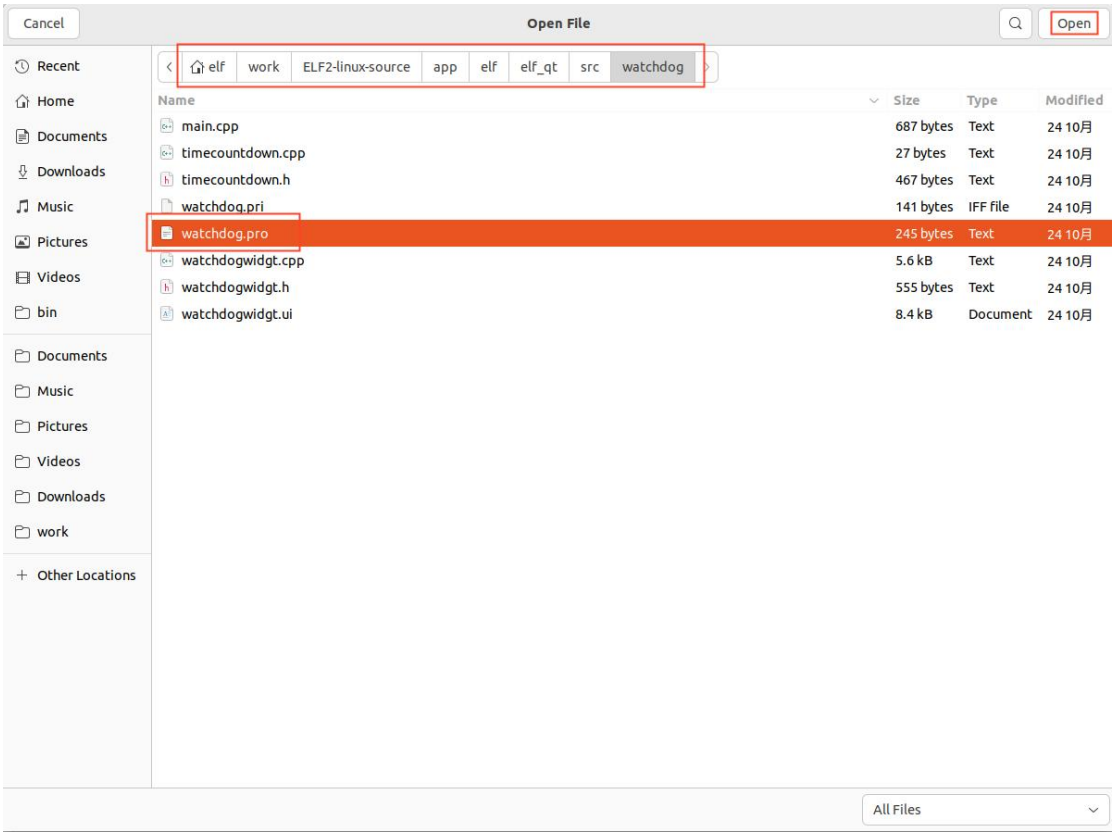
可参考用户使用手册“看门狗测试”章节测试。

- 方法二：通过 Qt Creator 调用 qmake 进行编译

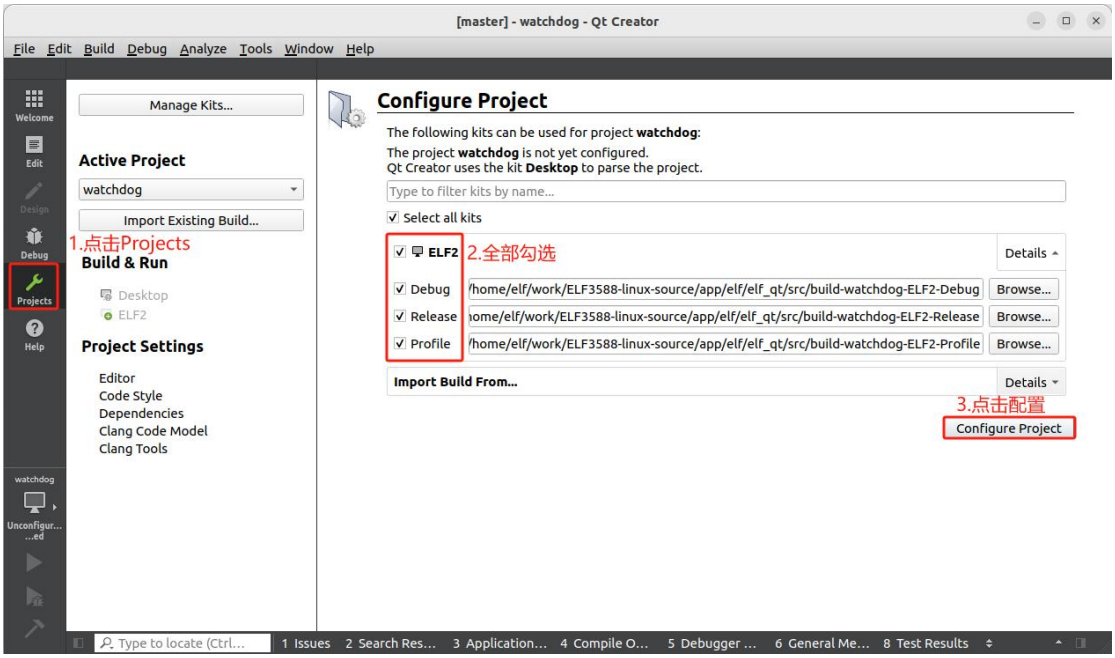
在开发环境打开 Qt Creator（用户根据自己的实际路径打开），

```
elf@ubuntu:~$ cd qtcreator-4.7.0/bin/
elf@ubuntu~/qtcreator-4.7.0/bin$ ./qtcreator &
```

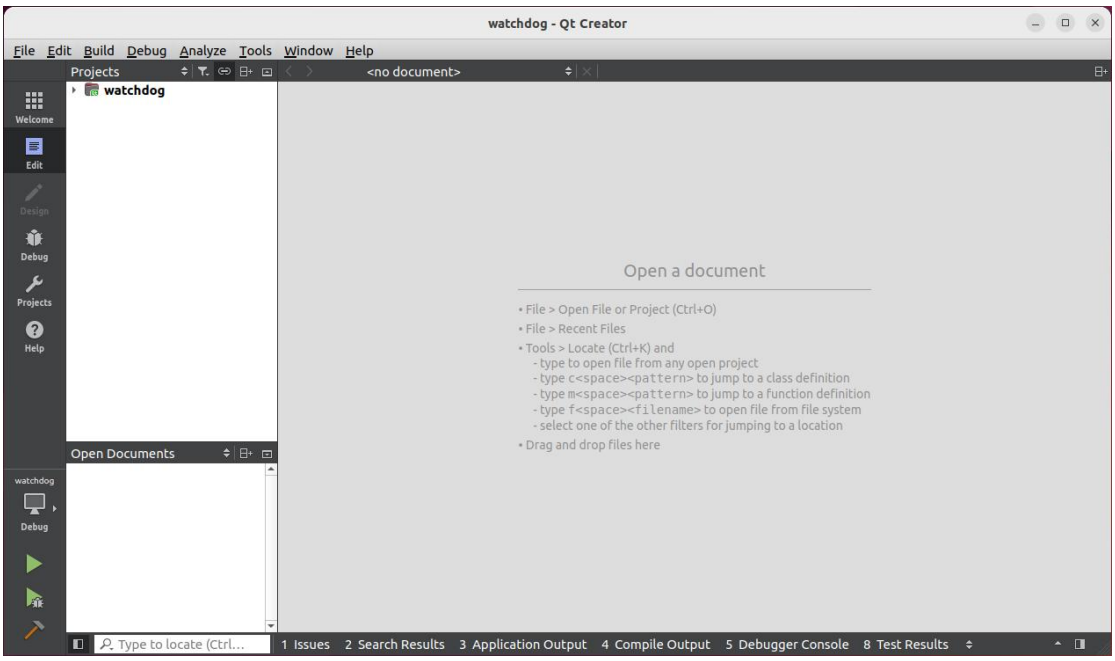
点击 File->Open File or Project，弹出窗口，选择/home/elf/work/ELF2-linux-source/app/elf/elf_qt/src/watchdog/watchdog.pro



打开项目后界面如下：（若没有自动改变页面，请按照截图所示选择）。

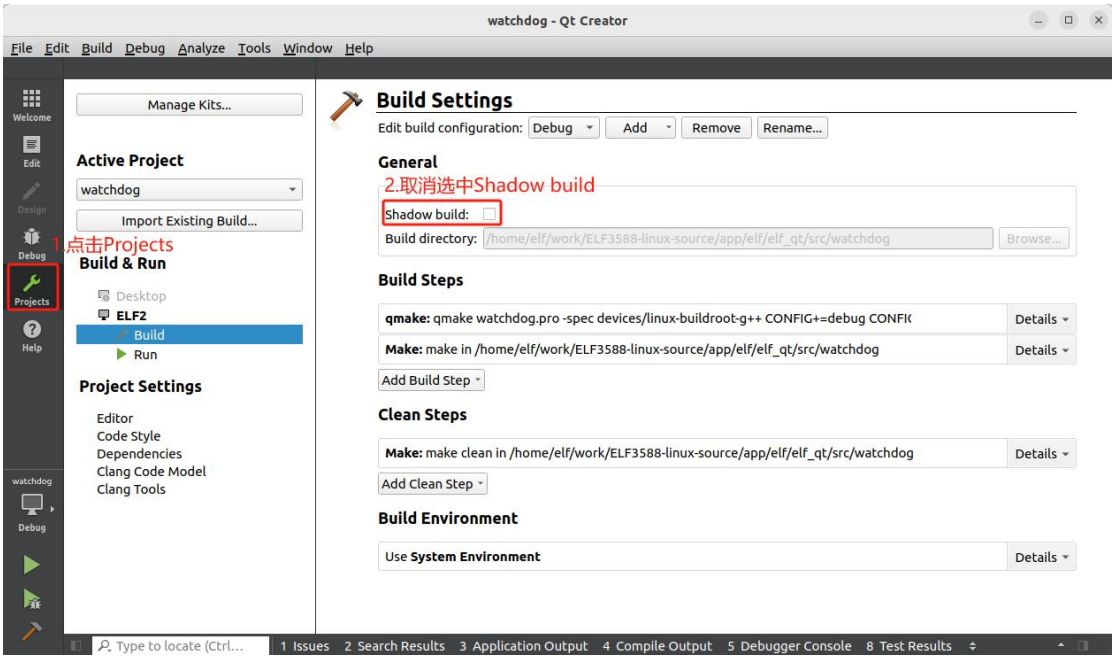


点击 Configure Project 后将使用“Qt Creator 环境配置”章节中构建的编译环境，选择后界面如下。

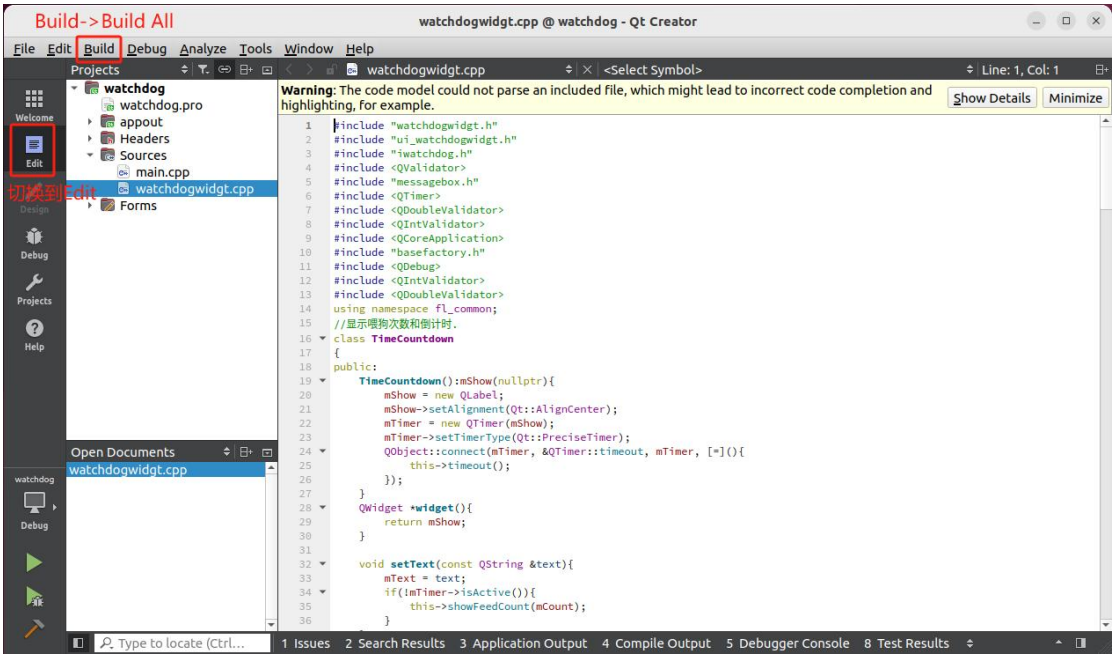


点击 Build->Clean All 进行清空。

点击 Projects 取消选中 Shadow build。



然后点击 Build->Build All 进行编译。



右下角 Build 进度条走完之后代表编译完成，此时在路径 /home/elf/work/ELF2-linux-source/app/elf/elf_qt_out/debug/bin 目录下会看到新生成的二进制文件 qtdemo_watchdog，如下：

```
elf@ubuntu:~/work$ ls ELF2-linux-source/app/elf/elf_qt_out/debug/bin
qtdemo_watchdog
```

将编译生成的可执行文件通过 U 盘拷贝到板卡上，并将依赖的配置文件拷贝到和应用同级目录。

```
root@elf2-buildroot:~# cp /mnt/udisk/qtdemo_watchdog ./
root@elf2-buildroot:~# mkdir data
root@elf2-buildroot:~# cp -r /usr/bin/data/conf.txt data/conf.txt
```

运行测试即可。

```
root@elf2-buildroot:~# ./qtdemo_watchdog
```

可参考用户使用手册“看门狗测试”章节测试。

7.2.4.2. Desktop 系统编译并运行 QT 应用

注意：若板卡搭载的是 Desktop 系统，请参照以下步骤进行应用编译。

本节以 ELF 2 开发板资料包\01-教程资料\应用篇之-Qt 应用编程\Qt 例程源码\00_HelloWorld 例程为例，详细介绍如何在 Ubuntu 开发环境中，通过 chroot 方式调用开发板文件系统的编译器完成 QT 应用的编译，并最终部署至开发板运行。

1.挂载开发板文件系统：

```
elf@ubuntu:~/work/ELF2-linux-source$ cd ubuntu/    //进入预编译文件系统所在目录
elf@ubuntu:~/work/ELF2-linux-source/ubuntu$ mkdir rootfs    //创建挂载目录
elf@ubuntu:~/work/ELF2-linux-source/ubuntu$ sudo mount jammy-rootfs.img rootfs/
    //挂载开发板文件系统镜像
[sudo] password for elf:
elf@ubuntu:~/work/ELF2-linux-source/ubuntu$ cd rootfs/    //进入挂载后的文件系统
```

2.在 chroot 环境中编译源码：

```
elf@ubuntu:~/work/ELF2-linux-source/ubuntu/rootfs$ sudo cp -r /media/elf/C8EC77D
4EC77BAF0/00_HelloWorld/ root/  //将测试程序源码通过 u 盘拷贝至文件系统内（请根
据实际 U 盘路径修改）
```

```
elf@ubuntu:~/work/ELF2-linux-source/ubuntu/rootfs$ sudo chroot ./    //切换到开发板文件系统环境（chroot）
root@ubuntu:/# cd root/00_HelloWorld/    //进入源码目录
root@ubuntu:~/00_HelloWorld# qmake
root@ubuntu:~/00_HelloWorld# make
```

编译完成后，按 Ctrl + D 组合键退出 chroot 环境，回到 Ubuntu 原始命令行。

3.将程序拷贝至 U 盘：

```
elf@ubuntu:~/work/ELF2-linux-source/ubuntu/rootfs$ sudo cp root/00_HelloWorld/HelloWorld /media/elf/C8EC77D4EC77BAF0/    //将生成的可执行文件拷贝到已挂载的 U 盘（请根据实际 U 盘路径修改）
```

4.卸载文件系统：

```
elf@ubuntu:~/work/ELF2-linux-source/ubuntu/rootfs$ cd ..
elf@ubuntu:~/work/ELF2-linux-source/ubuntu$ sudo umount rootfs    //卸载文件系统
```

5.在开发板上运行程序：

```
elf@elf2-desktop:~$ sudo cp /run/media/sda1/HelloWorld ./    //从 U 盘拷贝程序到开发板当前目录
elf@elf2-desktop:~$ export DISPLAY=:0.0
elf@elf2-desktop:~$ sudo chmod 777 HelloWorld
elf@elf2-desktop:~$ ./HelloWorld    //运行测试程序
```

声明

本手册版权归保定飞凌嵌入式技术有限公司所有。未经本公司的书面许可，任何单位和个人无权以任何形式复制、传播、转载本手册的任何部分，违者将被追究法律责任。

保定飞凌嵌入式有限公司所提供的所有服务内容旨在协助用户加速产品的研发进度，在服务过程中所提供的任何程序、文档、测试结果、方案、支持等资料和信息，都仅供参考，用户有权不使用或自行参考修改，本公司不提供任何的完整性、可靠性等保证，若在用户使用过程中因任何原因造成的特别的、偶然的或间接的损失，本公司不承担任何责任。

更多帮助

注意事项与维护

- 请勿带电插拔核心板及外围模块！
- 请遵循所有标注在产品上的警示和指引信息。
- 请保持本产品干燥。如果不慎被任何液体泼溅或浸润，请立刻断电并充分晾干。
- 使用中注意本产品的通风散热，避免温度过高造成元器件损坏。
- 请勿在多尘、脏乱的环境中使用或存放本产品。
- 请勿将本产品应用在冷热交替环境中，避免结露损坏元器件。
- 请勿粗暴对待本产品，跌落、敲打或剧烈晃动都可能损坏线路及元器件。
- 请勿使用有机溶剂或腐蚀性液体清洗本产品。
- 请勿自行修理、拆卸本公司产品，如产品出现故障请及时联系本公司进行维修。
- 擅自修改或使用未经授权的配件可能损坏本产品，由此造成的损坏将不予以保修。



更新通知与学习资源

ElfBoard 产品相关资料会持续完善和更新，本手册内容也会及时跟进。请在使用这些资料时，确保它们为最新版本；

最新的资料更新通知可通过微信公众号获取，课程视频和嵌入式知识科普则可以在抖音、哔哩哔哩等平台观看，以帮助大家更好地了解和使用 ElfBoard 产品。

官方公众号　官方抖音　哔哩哔哩

资料获取

1.请登录 ElfBoard 官网“www.elfboard.com”，进入“资料”专区进行下载；

2.下载前请阅读《用户资料极速下载必读》。

售后服务政策

1.如产品使用过程中出现硬件故障可根据售后服务政策进行维修；

2.售后服务政策：参见官网“www.elfboard.com”首页->技术服务->售后服务政策。

送修地址

1.地　址：河北省保定市高开区向阳北大街 2699 号飞凌嵌入式技术有限公司新楼五层售后维修部

2.联系人：售后维修部

3.电　话：0312-3102650-952、953

4.邮编：071000

5.邮寄须知：建议使用顺丰、圆通或韵达，且不接收任何到付。

技术支持范围

1.本公司产品的软、硬件资源提供情况咨询；

2.本公司产品的软、硬件手册使用过程中遇到的问题。

技术讨论范围

1.源码的修改以及理解；

2.操作系统如何移植；

3.用户自行修改以及开发中遇到的软硬件问题；

注：以上三点虽不属于技术支持范围，但我公司会尽力为用户提供帮助，如仍然没能解决敬请谅解。

技术支持方式

1. 论坛：bbs.elfboard.com
2. QQ 群：474552704

技术支持时间

1.周一至周五：上午 9:00—11:30；
下午 13:30—17:00；

2.公司按照国家法定节假日安排休息，在此期间无法提供技术支持，请将问题发送至论坛 ELF 2 专区或 QQ 群，我们会在工作日尽快给您回复。